# THE OMNI-COMPASS LIVE BENCHMARK ENGINE MANUAL

*How to Read the Live Benchmark Engine:*
*Its Monitors, Gauges and Knobs,*
*Native beside Omni-Compass*

**AJ Dubra**
Owner and Developer

**THE OMNI-COMPASS LLC**
October 2026

**The Omni-Compass Live Benchmark Engine Manual**
How to Read the Live Benchmark Engine: Its Monitors, Gauges and Knobs, Native beside Omni-Compass
October 2026

Copyright © 2026 The Omni-Compass LLC. All rights reserved. No part of this book may be reproduced, stored or transmitted in any form or by any means, except as allowed by the Omni-Compass Evaluation License, without the written permission of The Omni-Compass LLC.

The Omni-Compass LLC reserves the right to change this book, the software it describes and their terms at any time, without notice. Every copy, export and printout of any part of this book carries this page, the LICENSE, the NOTICE and the Disclosures unchanged.

All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC (no filing number is stated, and no grant, registration or approval is claimed). www.omni-compass.com

“Omni-Compass”, the Omni-Compass rose and the related names and marks are trademarks of The Omni-Compass LLC. Other names that appear in this book are the property of their owners and are used only to identify the systems Omni-Compass works with.

**License.** This book and the software it describes are proprietary and are not open source (SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0). They may be used only to evaluate Omni-Compass and to reproduce its published results, in simulation and on systems the reader owns or controls. Any commercial use, commercialization, monetization, production use, redistribution, hosted or managed service, or incorporation into any product or service requires a written Omni-Compass Enterprise License, signed by The Omni-Compass LLC and paid for. The full terms are printed in Appendix F.

**Disclaimer.** The software is provided “as is”, without warranty of any kind. This book is the reading manual of the live benchmark engine page and carries no benchmark results of its own; every published result carries its evidence class. A modelled result is a statement about a model; a software benchmark is a statement about the software it ran on; only a hardware meter speaks for hardware. Nothing in this book is a promise of a particular saving on a particular system: the only number that applies to a system is the one its own paired runs produce, on its own receipt.

Set in Liberation Serif and DejaVu Sans Mono, the text at 11 pt on a 16.5 pt line. Built from the repository by `docs/book/build_book.py`; the book and the code change in the same commit.

The Omni-Compass LLC · www.omni-compass.com

# Contents

**The Problem and the Answer** vii

**Foreword** viii

**Preface** viii

**Prelude: A Night at the Wall** x

**Introduction: The Page, Its Monitors and What This Manual Covers** xi

**The key: GAIN, STEADY and LOSS** xii

**What You Are Looking At** xiv

**Quick Reference: The Card for the Glove Box** xvi

**Disclosures, Declarations and Disclaimers** xvii

**Part One. Monitor Wall, Control Panel and Contents** 1

1. Monitor Wall . . . . . 1
2. Control Panel . . . . . 4
3. Contents: the Page's Order and Its Numbering . . . . . 22

**Part Two. Results** 24

4. Scorecard . . . . . 24
5. Benefit Against Native . . . . . 27
6. Benefit Board . . . . . 28
7. LOSS Register . . . . . 29

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

8. Status Board . . . . . 30

**Part Three. Live Operation** **33**

9. Live Organism . . . . . 33

10. Golden Signals and Event Log . . . . . 35

11. Control Console . . . . . 36

12. Heartbeat Dial . . . . . 39

13. Control Cycle . . . . . 40

14. Phase Plane and Scoreboard . . . . . 41

15. Brain State . . . . . 43

16. Verdict . . . . . 45

17. Dashboards . . . . . 46

**Part Four. Real Runs and Platforms** **50**

18. Kubernetes Run Playback . . . . . 50

19. Repetition Telemetry . . . . . 53

20. Controller Decision Trace . . . . . 55

21. Cloud Providers . . . . . 57

22. Orchestrators and Governors . . . . . 60

23. GPU and CPU Hardware . . . . . 61

24. CI Pipeline . . . . . 62

**Part Five. Proof and Models** **63**

25. Python Runs and Monte Carlo . . . . . 63

26. C++ Twin . . . . . 66

27. Scenarios and Injection Grid . . . . . 67

28. Whole Organism . . . . . 70

29. Measured Effect . . . . . 72

30. Telemetry Analytics . . . . . 74

31. Master Switch Wiring . . . . . 75

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

32. Specification and Mathematics . . . 76
33. Failure Modes and Controls . . . 78
34. Self-Governed Rendering . . . 79

**Part Six. The Page's Appendixes 80**

35. Title Page and Build . . . 80
36. Reading Guide . . . 83
37. Problem and Answer . . . 87
38. Executive Summary and Reading Key . . . 87
39. Monitor List . . . 88
40. The Glossary and Index . . . 88
41. Disclosures, Law and Contact . . . 89

**Part Seven. Reading a Result off the Page 90**

42. The Mechanism of Action, Side by Side . . . 90
43. From a Tile to Its Record . . . 92
44. When a Tile Reads LOSS . . . 93
45. What the Page Does Not Tell You . . . 94

**Part Eight. License and Your Copy 96**

46. License and Commercial Terms . . . 96
47. Verifying Your Copy of the Page . . . 96

**Back Matter 97**

Glossary . . . 97
Appendix A. The Page, Section by Section . . . 100
Appendix B. Every Control . . . 102
Appendix C. The Gauges on the Wall . . . 107
Appendix D. The Colours and the Marks . . . 108
Appendix E. Troubleshooting the Page . . . 109

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Appendix F. The License . . . 110
Appendix G. Licensing: Questions and Answers . . . 122
Appendix H. Third-Party Notices . . . 124
Appendix I. The Enterprise License: Standard Terms . . . 125
Appendix J. Patents and Trademarks . . . 129
Appendix K. Security Policy . . . 131
Appendix L. The Page Word for Word: Every Section, Monitor and Knob . . . 132
Index . . . 272
Contact . . . 282

# List of Figures

Every figure is drawn from the code it pictures, by one of two kits (`docs/book/figures.py`; `docs/book/schematics.py` for the wiring schematics, block diagrams, flowcharts, timing diagrams, state machines, the receipts charted from their own tables and the grids), and printed where the text first needs it; the same drawings stand beside the text on GitHub, in `docs/book/figures/`. A chart of a receipt is read by the reading key: it pictures the table and adds nothing to it.

Figure 1. The seven organisms on the spine . . . 34
Figure 2. The compass band, its cushions, and the smoothing of the force . . . 42
Figure 3. The engine's six states, their couplings and the control law . . . 44
Figure 4. The double well of the alignment and the cubic potential of the stress . . . 45
Figure 5. Block diagram: the verdict's paired trial around a slow knob, read GAIN, STEADY or LOSS . . . 47
Figure 6. Flowchart: the verdict's trial on one step, from the reference to the reading . . . 48
Figure 7. Timing diagram: the hand-back after a reset, a kill and a death . . . 51
Figure 8. One pedal, no brake: the modes and the two switches . . . 54
Figure 9. One decision of the governor, from the senses to the audit . . . 56
Figure 10. State machine: the governor's modes . . . 76

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Figure 11. Block diagram: the compass law as a closed loop on one muscle . . . . . 91

# The Problem and the Answer

**What the whole industry faces.** Every computing system in service, from one server to a cloud region, from a GPU to a building's chillers, a grid feeder or a robot cell, is run by a stack of native controllers: the Kubernetes autoscalers and scheduler, the cloud's node pools, the GPU's power and clock firmware, the database's cache and connection pools, the building's cooling loops, the battery's management system, the servo's loop. Each was designed and tuned alone, against its own setpoint, with margins sized for the worst case it might meet. Three costs follow, on every stack:

1. **Capacity held idle.** Worst-case margins stay in place through the common case. Machines, power caps, caches and cooling run far inside their limits, and the room between the operating point and the limit is paid for and seldom used.
2. **Late reaction.** Reactive loops act on averaged, lagging signals and wait out deliberate cooldowns, so queues and response-time tails form before capacity arrives.
3. **Loops at cross purposes.** The loops share power, cooling, connections and memory. One loop's correction is another loop's disturbance, and no loop sees the whole.

Demand for computing, above all for AI and the cloud, is growing faster than the power, cooling and hardware that can be built for it. The usual answer is to add: more machines, more power, more people watching thresholds. Replacing the native controllers is not an answer most operators will take: they are proven, often certified, and the people on call know them.

**What Omni-Compass does about it.** Omni-Compass is a supervisory governor. It sits on top of those native controllers and never replaces them. It plugs into each setting a native controller already exposes, through a two-way plug (one wire in, the reading; one wire out, the setting), takes a snapshot of every setting before its first write, and governs the whole stack as one body, the organism: every muscle driven as far as its own limit and never past it, and, on the engine that stands, never below native; where it cannot bring a muscle a GAIN, that muscle stays native. It acts before a line is crossed instead of after, reads back every write, puts every give-back to its own trial, records every decision with its reason, walks every setting home if it stops (the lease) and stops at once on the kill switch. Its first rule is do no harm, judged at the organism: never less work or health than native (`DISCLOSURES.md`, section 3). The aim is the one headline: more work, faster, on fewer machines, with less energy, from the hardware and the controllers already in place; in a cloud, more work for the same bill.

**Where the page shows it.** The live benchmark engine page plays every benchmark back from its own SHA-256-checked files, native beside Omni-Compass on top of native, lane by lane, and runs the engine's

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

own runs in the page: the Python's seeded runs, and the same runs through the engine's C++ twin, side by side (the page's section 5b). This book reads that page; the mechanism and every result are in the Owner's Manual.

**What is not claimed.** No new physics, no new model of computation, and no promise for any system Omni-Compass was not run on. It does not make a system immune to failure, delay or drift: it keeps each muscle inside its own limits and shows, row by row, what it did against native. A result on another system is that system's own, from its own paired runs; where it differs, the first presumption is the wiring (`DISCLOSURES.md`, section 4).

# Foreword

A governor that cannot be watched cannot be trusted. Every controller a stack already runs has a dashboard somewhere: the autoscaler's replica count, the card's clock and power, the building's setpoints. What none of them had was one wall where the whole body is seen at once, native beside the same body governed, decision by decision, with the record each number came from one click away. The live benchmark engine is that wall, and this book is how to read it.

I asked for it to be a monitor and not a drawing: every record on it made again by the repository's own code, every raw file checked against its own SHA-256 list before a byte of it is drawn, every word on it the run's own reading and never one drawn from the stream. So the page can be trusted the way a meter is trusted: it shows what the record holds, and when the record has nothing yet (a cloud set still running, a chip run not yet started) it says so and shows nothing in its place.

Read the key first. Then stand at the wall: the page opens on it, the control panel under it, and every reading that went against native is raised on the status board before anything that went for it. If a tile reads LOSS, this book tells you where its record is and what to do: the row stays, its cause is found (our wiring first, native's setup second, the scoring third), and the test runs again. Nothing on the page is pushed toward GAIN; the only thing forced is the mechanism.

# Preface

This is the Live Benchmark Engine Manual, the third book: how to read the live benchmark engine page, the monitor of the engine and of the benchmark runs GitHub ran. It is for the operator at the dashboard: which tile means what, which knob does what, where every number comes from, and how the mechanism of action of Omni-Compass against native shows on the monitors. It explains neither how the engine is built (the Owner's Manual) nor how it is wired onto a stack (the Operator's Manual).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

The book is in eight parts, in the page's own order (the Contents follow), and it can be read in two ways.

| You are | Read |
|---|---|
| The operator at the wall | the Quick Reference card; Parts One and Two; when a tile reads LOSS (Part Seven) |
| CTO, architect, head of platform | the scorecard (chapter 4); the fuse box and its tabs (chapter 2); Part Five |
| CEO, CFO, CIO, board member | the Problem and the Answer; the Disclosures; chapters 4, 5, 6 and 21 |
| Site reliability engineer | Part Two; the golden signals and metrics (chapter 10); the wall stack, `deploy/wall` (chapter 2) |
| Control or plant engineer | the contract and the mechanism knobs (chapter 2); chapters 13 to 16 |
| Auditor, diligence team | Part Six; Part Seven; the pipeline (chapter 24); the page word for word (Appendix L) |
| Procurement and counsel | the Disclosures and the papers bound into the back matter |

**Conventions.** `Code` is a command, file, anchor or switch exactly as typed. "Native" is the system by itself, orange; "Omni-Compass" is Omni-Compass on top of native, blue. Those are the two sides, and the only two: Omni-Compass is never by itself. Violet marks only a knob or profile turned from the factory tune. Red is never a side: it is the stop sign of a line crossed. "A section" is a numbered part of the page, named by its number, its title and its anchor; "a chapter" is a chapter of this book. A knob marked **LIVE** acts on the live benchmark engine; one marked **RECORDED** chooses a record and modifies nothing. Every number carries its evidence class: P a physical meter, L live software, S our own models (modelled); a model is never proof.

**The standards every page of this manual keeps.** The page, and so this book, keeps the following discipline throughout:

1. **Native beside Omni-Compass, always.** Every monitor shows native beside Omni-Compass, the same whole system on the same demand, the same draw of noise, the same injected fault, at the same second. Omni-Compass is never shown as a competitor to the controllers it sits on, and never shown by itself: it always sits on top of native.
2. **Nothing typed in.** Every number is read from a record the repository holds when the page is built or opened; every raw file is matched against its run's own SHA-256 list before it is drawn; the brain, the compass law and the verdict running in the page are the engine's own code, ported and proven equal to the Python and the C++ on every build (`docs/engine/check_live_engine.py`).
3. **The record's own word.** Every word on the page is the reading its record carries under the one key; the page never draws a word from the stream it plays, and never prints "up", "down", "better" or "worse".

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

4. **Every row, LOSS included.** A reading that went against native is raised first, as an alarm, with its source record and the repository's root-cause document where one exists. Nothing is cut from the page because it went against Omni-Compass.
5. **Nothing read across engine versions.** The page names the engine it stands on, every record is rebuilt on the engine that stands, and `--check` fails a page whose engine differs; each three-run table on it names its own engine, and a table stands on an older engine only until its runs on the engine that stands land.
6. **Models are not proof.** Every view says its evidence class. The official Kubernetes and database runs are class L, played from their own checked files. The harness, the shipped controller on the modelled cluster, the GPU card's model, the Linux host's model, the realms table and the grid are class S, our own models, shown beside the real runs and never presented as proof.
7. **Do no harm, in its one meaning.** Every knob on the console acts on both sides alike; what Omni-Compass does with it is held by its guards: no muscle driven below native or past its own limit, nothing taken from one muscle to give to another on the engine that stands, a give-back tried against native first and refused on a LOSS (`DISCLOSURES.md`, section 3).

# Prelude: A Night at the Wall

It is a quarter past two. The on-call engineer has the wall open on one screen and the cluster's own console on the other. In the control panel under the wall the kill switch reads Omni-Compass ON, and the active profile reads FACTORY. On the wall every tile carries its signal: green, GAINING, on nearly all of them. Native in orange on the left of each pair, Omni-Compass in blue on the right, both moving on the same demand. The orange line of the work waiting climbs as the night's batch starts; the blue line climbs too, then bends sooner, because the governor's ticker (the page's section 4a) shows the pedal pushed at the first decision the compass left its center, and two machines recommended before native's autoscaler had seen its first late sample.

A row of the status board's alarms turns red. It reads LOSS: the pod start wait, mean, +13% in one run. The engineer clicks it and lands on the three-run table it came from: run A of the Azure steady set, 33 paired repetitions, the interval −44 to +60 seconds, *no* in the significant column. The red is that run's own word; the three-run reading needs runs B and C as well, and this run's interval includes zero, so it is not yet beyond the noise. The row stays on the page, as every row does. The engineer opens the root-cause document the row links to, `docs/POD_START_WAIT.md`, and reads that a pod's start on a shared host is the test machine's own unpacking of an image, the same on both sides at the same host load. Nothing to do tonight; the record is complete.

At three the batch is done. The blue line of the capacity held eases back a step at a time toward the orange one: the pedal lifted, the lease walking every setting toward native. The engineer scrolls to the page's section 3f, the phase plane, and watches sixty dots drift from the right edge back toward the middle line, native's a few decisions behind. Then back to the wall. The kill switch green, the signals calm, the alarms empty but for the

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

one red row that will stay until its runs B and C land and the three of them are read together. The status board's last tile, Build Freshness, reads twelve minutes.

# Introduction: The Page, Its Monitors and What This Manual Covers

**The problem the page answers.** Omni-Compass's answer to the problem the whole industry faces (the first section of this front matter) is a supervisory governor on top of controllers that each have their own dashboard, and the owner of the stack has to answer one question about it every day: is it doing what the receipts say, right now, on this body, and would I see it if it were not? The receipts are tables; a table is read after the fact. The page is the same record played back in motion, native beside Omni-Compass, with the governor's own decisions and the native controllers' own moves on two tickers side by side, so the mechanism is seen, not inferred.

**What the page is made of.** Four exporters build its records on the engine that stands: the realms harness (one organism and each muscle alone; `docs/engine/export_harness_trace.py`), the shipped controller on the modelled cluster (on top of the autoscaler at the max, and in the cloud's setting; `docs/engine/export_controller_trace.py`), the official runs read back from their raw files and scored again by the repository's own scorer (`docs/engine/export_runs.py`, `tools/live_reps.py`), and the chip and host models with every judged row of the newest receipts (`docs/engine/export_scenarios.py`). A fifth record is the build itself: the engine, the commit, the time, and GitHub's own list of the runs in progress and of every run started for the chips and the clouds (`docs/engine/github_ledger.json`).

**What runs live in the page.** The brain (`omnicompass/core.py`, integrated by the fourth-order Runge–Kutta method), the compass law (`omnicompass/compass_law.py`) and the verdict (`omnicompass/verdict.py`) run in the page itself, ported to the browser and proven equal to the Python and the C++ on every build. They govern a stress-test model of sixty muscles, each on top of its own native controller, and they govern the page's own drawing (the page's section 5j). Everything else is a record played back.

**What this manual covers.** Every section of the page in the page's own order; every gauge, with its side, its aim and its word; every control, with its backend, its effects and its limits; the factory profile against the active one; the two tickers that show the mechanism of action; the trace from any tile to the record it came from; and what to do when a tile reads LOSS. It covers the page as built on omni-v23; a later engine renames itself on the page's title block and this book is realigned in the same push.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**What this manual does not cover.** How the engine is built, its equations beyond the ones the page itself prints, the wiring of Omni-Compass onto any stack, and the benchmark results themselves as a body of evidence: those are the Owner's and the Operator's Manuals and `docs/STATE_OF_PLAY.md`.

# The key: GAIN, STEADY and LOSS

Every judged reading in Omni-Compass is one of three words: **GAIN**, **STEADY** or **LOSS**. Better, same and worse are not used anywhere. The word is decided by what the gauge is for, never by the plus or minus sign of its number: who sets the aim differs from one system to the next, and each gauge's aim is written in its preregistration before the runs.

| Word | What each letter stands for | Its sense in underwriting | What it means in a table |
|---|---|---|---|
| **GAIN** | **G** Growth (asset appreciation); **A** Advantage (risk reduction); **I** Improvement (margin expansion); **N** Net-positive (profitable outcome) | surplus and positive variance | the gauge moved toward its declared aim against native: more work done, a shorter wait, fewer misses or failures, or a resource cut |
| **STEADY** | **S** Stable (zero variance); **T** True to baseline (expected norm); **E** Equilibrium (balanced risk); **A** At par (no net change); **D** Drift-free (within the 0.5% floor); **Y** Yield held (nothing lost) | the par baseline: equilibrium | native and Omni-Compass within 0.5% of each other: the mean of the three runs' changes under the 0.5% floor, or no change at all; a knob native already runs at the top of its range is STEADY, at its limit on native |
| **LOSS** | **L** Leakage (capital drain); **O** Off-track (adverse selection); **S** Setback (claim liability); **S** Subpar (negative variance) | actuarial deficit and exposure | the gauge moved away from its declared aim: less work done, a longer wait, more misses or failures |

How the sign of a number becomes a word:

- **Time waited, missed or failed** (a response time, time over the line, failed requests): a fall is GAIN, a rise is LOSS.
- **Work done** (work inside the line, throughput, a hit rate): a rise is GAIN, a fall is LOSS.
- **A resource spent or held** (energy, the power drawn and its peaks and swings, machines, memory, processor time, a bill, wear on a part): a fall is a **cut**, and a cut is GAIN. A rise is **power given**: what Omni put in to buy the work and the speed shown beside it. Power given is always shown and never

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

hidden, and it is never called a loss (the founder's order of 12 October 2026: the max rides every muscle's edge; it never shaves). A peak is power held, so a peak that rose is power given, in a data centre and in a building alike (17 October).

- **Work bought with each unit of a resource** (work per energy): a rise is a cut in what each unit of work took, a GAIN; a fall is power given, because it falls when more power is given for the same work.
- So a minus can read GAIN (a response time cut 65%, a bill cut 30%) and a plus can read LOSS (failed requests up).
- **The floor, and every change above it** (the founder's orders of 20 and 24 October 2026): a change of half of one percent (0.5%) or more either way is a change, and reads **GAIN** or **LOSS** (or **power given**) by what the gauge is for; STEADY is kept for a change under 0.5%. The change is the mean of the three runs' changes, each in percent of native. Where every run moved the same way with its 95% interval clear of zero, the word reads **in all three of our runs**; otherwise it reads **by the mean of our three runs** and says in how many of the three runs it stood beyond the noise, so a reader sees how firmly it stands. Where runs moved 0.5% or more in opposite directions, the reading is **the runs disagree**. A mean under 0.5% reads **STEADY, under the floor**, with its size.
- **Two numbers, never mixed** (the referee's work order of 25 October): every row shows native's value and Omni-Compass's, the **raw change** (Omni-Compass − native) / native with its true sign, + for a rise and − for a fall whatever the gauge is for, labelled raw, and apart from it the word with its size and no sign. A bill cut 3% reads raw −3.0%, GAIN 3.0%; failed requests up 2% read raw +2.0%, LOSS 2.0%; a pod start wait from 0.22 s to 0.83 s reads raw +275.0%, LOSS 275.0%; a change under the 0.5% floor reads STEADY 0.3%. No raw sign is ever turned to match a word.

Some preregistrations name their outcome with a label written before the runs (for example SUPERIOR WITHIN GUARDRAILS in the card protocol). The label stays as it was frozen, and every table shows it with its word first: TOWARD ITS AIMS reads GAIN; BOTH WAYS reads GAIN AND LOSS; AWAY FROM ITS AIMS reads LOSS; SAME AS NATIVE reads STEADY (the same as native); AT ITS LIMIT ON NATIVE reads STEADY (at its limit on native); SUPERIOR WITHIN GUARDRAILS reads GAIN (within the guardrails); ENERGY IMPROVEMENT WITH SERVICE TRADEOFF reads GAIN AND LOSS (energy cut, service traded for it); SERVICE IMPROVEMENT WITH ENERGY TRADEOFF reads GAIN, power given (service GAIN, energy given for it); NONINFERIOR / INCONCLUSIVE reads STEADY (inside the noise, within the guardrails); NOT ESTABLISHED reads STEADY (not established: inside the noise, a guardrail crossed on the mean). A run that breaks its own validity checks reads INVALID and carries no word.

Every reading is one of these. **GAIN, LOSS or power given in all three of our runs**: A, B and C moved the same way, every 95% interval clear of zero. **GAIN, LOSS or power given by the mean of our three runs**: the runs' mean moved 0.5% or more, with the count of runs that stood beyond the noise beside it. **STEADY, under the floor**: the mean moved less than 0.5%. **STEADY, the same**: no change at all. **The runs disagree**: runs moved 0.5% or more in opposite directions, so the test is unstable there and is looked into. **Shown, not judged**: a gauge declared in advance as shown only, such as the knob's own setting.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Why these three words: they are the words of underwriting and actuarial work, where every outcome is graded as surplus, par or deficit, and each letter spells out the same rule. Omni-Compass is there to add (GAIN), never to take (LOSS), and to hand a muscle back to its own native setting where it cannot add (STEADY). Its first rule is do no harm, in the founder's meaning of 19 October 2026: the organism Omni-Compass governs is never left turning out less than its native controllers make it turn out, nor less healthy, and every muscle works only as far as its own limit (`DISCLOSURES.md`, section 3). So **LOSS should never appear**: where it does, the row stays in the table, its cause is found and fixed (our own wiring first, then native's setup, then the scoring) and the test is run again.

Disclosure and disclaimer: GAIN, STEADY and LOSS, and the letters behind them, are The Omni-Compass LLC's own convention for reading its own results. The underwriting senses explain the words; they are not an actuarial, financial or insurance opinion, and no outside body defined, reviewed or approved them. A reading states what our own runs measured against native under the declared aims, not a promise about any other system; every number stands beside its word, so a reader can judge it by any other convention. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

**How to read a number beside a word.** The word comes first and settles the direction; the number is the size, and it wears the word's sign so the two never pull apart: GAIN 29.6% on pods waiting means 29.6% fewer pods waited than under native (a fall in waiting is the gain, so it is written with the plus of the word, never with the minus of the gauge); LOSS 2.3% on alert minutes means 2.3% more minutes over the line; STEADY 0.3% means a change under the 0.5% floor; power given 3.1% means 3.1% more of a resource put in to buy the work beside it. The gauge's own direction is never hidden: on the page, hover any word or any coloured number and it says what the word means and why the number wears its sign, and where the page knows the gauge, what rose or fell against native (the page's appendix D, "Size, Sign and Raw Change").

# What You Are Looking At

**A live tuning engine, and one tuned engine.** The page is the **live tuning bench**: the interface that operates the engine's controls and displays their effects through the connected backend. Every knob, button and monitor on it turns in real time, native alone, Omni-Compass on top, or both, and **any knob can move every metric, up or down**. Every benchmark is one engine at one profile, the **factory profile**; any other setting shows the engine's range, never a benchmark result (the box that opens the page's appendix G, `#tuningBench`; chapter 41). The words used here for the engine, its profiles and its bench are written once, in `docs/FACTORY_PROFILE.md`, and this book uses them exactly.

**What the live benchmark engine is.** The live benchmark engine is one web page, `docs/engine/OMNI_COMPASS_LIVE_ENGINE.html`, built by the repository's own code

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(`docs/engine/build_live_engine.py`) from the engine that stands and from the repository's own results. It is a monitor of the engine and of the benchmark runs GitHub ran: every number on it is read from a record the repository holds (a three-run table, a run's raw files, the engine's own audit, the realms table, the models' own readings), and every word on it (GAIN, STEADY, LOSS, power given) is that record's own reading under `tools/readings.py`. The page decides nothing about any result and types nothing in. Where it plays the repository's own models (evidence class S) it says so, and a model is never proof.

**What "native" and "Omni-Compass" mean on the page.** *Native* is the **native baseline**: the whole system running by itself, every autoscaler, scheduler, firmware, thermostat, orchestrator and controller it ships with, every muscle wired and moving by its own machinery. It is drawn in orange. *Omni-Compass* (on the page also *Omni-Compass and native*, or *Omni-Compass on native*) always means Omni-Compass on top of native: the same whole system with all of that still running underneath, and the governor on top of all of it, wired in through the two-way plug. It is drawn in blue. There are two sides everywhere on the page, and only two. Omni-Compass is never by itself: it always sits on top of a native controller, on muscles that have their own, so no monitor, table, record or button shows a third. Red is never a side: it is only LOSS and the alarm.

**What a monitor is.** The page is laid out as an operations monitor is: a wall of tiles first (the monitor wall), each tile a small copy of a monitor further down the page, with the control panel under it; then the results; then the monitors themselves, section by section, each with its own knobs in a line at its head ("Controls") and a strip under it; and last the appendixes, where the page keeps its explanations. A knob marked *Live* acts on the live benchmark engine, the engine's own mathematics running in the page; a knob marked *Recorded data* only chooses which record is shown and never modifies it. Every monitor shows native beside Omni-Compass on the same demand, the same seconds, so the one difference left between the two sides is Omni-Compass.

**What a gauge is.** A gauge is one measured quantity of the system (the work waiting, the work done, a response time, the machines in service, the energy drawn, a bill). On the page each gauge is drawn for both sides with its paired reading: the change from native, its 95% interval, and its word. The word is decided by what the gauge is for (the key, above), never by the sign of its number.

**How to read the page, in order.** The page opens on the monitors: the monitor wall (the page's section 0), every monitor live in five blocks, and directly under it the control panel (the page's section 1), with the kill switch, the fuse box, the mechanism's knobs, the runs, the reports and the manuals. Then come the contents; then Part 2, the results (the scorecard, the benefit against native, the benefit board, the LOSS register and the status board); then Part 3, the live operation of the engine (the live organism, the golden signals and the event log, the console, the heartbeat dial, the control cycle, the phase plane and scoreboard, the brain's state, the verdict and the dashboards); then Part 4, the real runs and the platforms (the Kubernetes run playback and its telemetry, the controller's decision trace, the clouds, the orchestrators, the GPU and the CPU, the pipeline); then Part 5, the proof and the models (the Python's runs and the Monte Carlo, the C++ twin, the scenarios and the injection grid, the whole organism, the measured effect, the telemetry analytics, the master switch's wiring, the specification, the failure modes and the page's own rendering); and last the appendixes A to G (the title page and build, the reading guide with every section's explanation, the problem and the answer, the executive

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

summary and the reading key, the monitor list, the glossary and index, and the disclosures). This manual reads the page in the same order, one chapter for each section.

**What this manual is, and is not.** It is the interpretation of the page: how to follow the dashboard, read each gauge, turn each knob, and see the mechanism of action of Omni-Compass against native as the monitors show it. It defines every control, its backend, its effects, its limits, and the distinction between factory and active settings, in the words of `docs/FACTORY_PROFILE.md` (chapter 6). Nothing in it decides a result: the records of authority are the repository's result files, the raw files GitHub archived with their SHA-256 lists, the two other manuals and `DISCLOSURES.md`.

# Quick Reference: The Card for the Glove Box

Keep this card beside the screen that shows the wall. Every line is explained in the chapter named beside it.

**The sides** (chapter 36). Orange is native, the system by itself. Blue is Omni-Compass on top of native. Two sides, and only two: Omni-Compass is never by itself. Violet is a knob or a profile turned from the factory tune, never a side. Red is never a side.

**The three words** (the key). Green GAIN, yellow STEADY, red LOSS (the word itself shaped as a stop sign, white on red). A resource that rose is *power given*, blue. White with a dashed edge is *the runs disagree* or *the runs differ*; grey is *shown, not judged*; neither is a reading. A change of 0.5% or more is a change and carries its word; under 0.5% it reads STEADY.

**The signal words** (chapter 36). On every monitor, section, part, breaker and in the dock: green GAINING (round), yellow STEADY (a yield diamond), red REDLINING (an octagon, a stop sign, blinking), and a dark AWAITING where a monitor's run has not landed, never green.

**The scorecard** (chapter 4). One light and one number for each way to run it against native, and the whole workload's index on each route.

**The kill switch** (section 2.28). The first row of the fuse box, the head of the control panel's third panel (the page's section 1), every monitor's strip, every section's controls and the dock. Green ON: the governor's outputs reach the muscles. Red: engaged, every setting handed back to native at once, the decisions shaded red. White OFF: every Omni-Compass side follows native in a tint of native's colour. "Return to native" in the control panel is the reset, a separate button with the same effect on the page (chapter 2).

**The profile** (chapter 2). The active profile reads FACTORY, CANDIDATE or CUSTOM, with its fingerprint. The factory profile is the released one, omni-v23's, the heartbeat 60 s; nothing is held when the page opens. *Restore factory profile* puts the mechanism and the heartbeat back; *Hold the active profile* holds every knob as

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

set; *Try the candidate* applies the sweep's best beat as a candidate, never a release. The heartbeat has six buttons, 10, 15, 30, 60, 90 and 120 s; the fuse box's tabs add the live model as Prometheus metrics and every press as the request a running governor takes.

**The status board** (chapter 8). Every reading that went against native, worst first, each with its source record and its root-cause document where one exists; filters: All, LOSS, the runs disagree, live now, guards and open items, GitHub.

**Where a number comes from** (chapter 43). Click the tile: its monitor names the table, the run and the file, whose SHA-256 was matched before it was drawn (the tables in `results/live/`, the raw files on the `raw` branch).

**When a tile reads LOSS** (chapter 44). The row stays. Read the three-run reading, not the one run. Open the root-cause document. The cause is found in this order: our wiring, native's setup, the scoring. The test runs again. Nothing is tuned toward GAIN; where Omni-Compass cannot bring a knob a gain, that knob stays native.

**The console** (chapter 11). Every knob acts on both sides alike: demand, noise σ, sense delay +0 to +6, four faults of 40 decisions, the dial, the load profile, the frame load, the time, the day, every knob to its start.

**The age of the page** (chapter 8). The status board's Build Freshness tile. The page is rebuilt by GitHub on every change to the engine, a model or a result, and every three hours (chapter 24).

**What the page is not** (chapter 45). It decides nothing. Class S views are models, not proof. The records of authority are the result files, the raw files and their SHA-256 lists, the manuals and `DISCLOSURES.md`.

# Disclosures, Declarations and Disclaimers

This page is the one place every declaration about Omni-Compass is made. The README, the manual, the book, every guide and every report point here. Where any other page seems to say more than this page, this page governs.

## Rights

1. Omni-Compass, its engine, its mathematics, its software and its documentation are the property of The Omni-Compass LLC. All patents, copyrights and trademarks covering them have been filed in the United States by The Omni-Compass LLC. No filing number is stated here, and no grant, registration or approval is claimed.
2. The software is licensed for evaluation and simulation only (`LICENSE`). Running it in production, on any system beyond evaluation, or in any product or service, requires a signed, paid Omni-Compass Enterprise License.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

3. Names of other companies and products (NVIDIA, Kubernetes, Red Hat OpenShift, Amazon EKS, Google GKE, Microsoft AKS, Karpenter, Lambda and others) are the property of their owners. They identify the systems Omni-Compass was tested with or connects to. No affiliation, endorsement or certification by any of them is stated or implied.
4. All rights reserved. The Omni-Compass LLC reserves the right to change the software, the documentation, the results and the terms at any time, without notice. Every copy, fork, export, report, archive or printout of any part of Omni-Compass carries the README's notice, `LICENSE`, `NOTICE` and this page unchanged. The Omni-Compass LLC, www.omni-compass.com

## What a result is, and what it is not

1. Every result carries its evidence class: **T** a theorem, **V** verified in code, **S** our own models (modelled), **L** live software (real Kubernetes on GitHub's machines), **P** a physical meter (a real card's own power meter). A model is a statement about the model. A software benchmark is a statement about the software and machine it ran on. Only a meter speaks for hardware.
2. The models were written by the same people who wrote the law. They show how the mechanism behaves; they are not an independent test.
3. On Kubernetes in kind every machine stays powered, so energy there is a declared model, not a meter.
4. Every comparison is paired: the same seed, load and clock with and without Omni-Compass, in an order rotated by repetition. Intervals are 95% intervals over the paired repetitions. Ratios across seeds are summarised as geometric means. Labels are assigned by code from rules written and committed before the run (`docs/GPU_PREREGISTRATION.md`, `docs/REALMS_PREREGISTRATION.md`, `docs/K8S_COMPASS_PREREGISTRATION.md`), never by hand.
5. Losses and failed runs are kept beside the wins (`docs/EVIDENCE_LEDGER.md`, `results/live/`, git history). The first real-card run, for example, saved energy (+3.6% work per energy, proven) and made the slowest answers 58.5% slower; it is published as it happened.
6. No result is a promise of a saving on any other system. No dollar figure in this repository is a measured result. Nothing here is a vendor certification, a safety certification, legal advice or investment advice.

## Do no harm, and the rule Omni-Compass keeps on every muscle

**What "do no harm" means** (the founder's words of 19 October 2026; this is its meaning everywhere in Omni-Compass, and it has no other). Omni-Compass has one goal: to strengthen the organism it governs and keep it healthy, working at the best state its own muscles can carry. Whatever is fed into the organism (its work, its data, its energy: its nutrients) passes through its muscles and comes out as its output. Omni-Compass's brain drives the organism toward the most and the best output it can make from what it is fed (the most work, done fastest, with the fewest misses and failures) while it keeps every muscle healthy.

- **Harm is judged at the organism.** Harm is the organism turning out less than its native controllers make it turn out (less work done; more time waited, missed or failed), or the organism made less healthy: a

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

muscle pushed past its own limit, a service pushed over its own line, or anything made to fail. Omni-Compass does neither.

- **Every muscle only as far as it can go.** Omni-Compass can push a muscle only as far as that muscle itself can be pushed: its own limit, read live from the system, never past it. It finds the room each muscle already has and uses all of it. Whether its mathematics can draw a little more out of a muscle than the muscle was built to give is an open question; a measurement answers it before anything is claimed.
- **The muscles work as one body.** Where one muscle's strength cannot be used without straining another, the brain offsets and engages the muscles together to bring the whole organism to its best state. It may use one muscle less, leave it native, or rest it, so that another can carry more, whenever the organism's whole output comes out ahead and every muscle stays healthy. Which muscles to drive and which to leave native is the brain's live decision, at every moment and in every run.
- **Reactive and proactive.** Like an immune system, the brain answers a breach at once, and it acts before a line is crossed.
- **Every row is still shown.** Each muscle's and each gauge's reading stays in every table beside the organism's, LOSS included. Nothing is cut.

**How the engine that stands keeps it.** The engine that stands keeps do no harm in its strictest form. By default it moves each muscle's settings toward more work, faster, up to the muscle's own limit and never past it (the max); on the dial anyone can tune it down toward the same work with less (the tune-down). Either way it keeps a step only where the muscle is never behind native for it: a step that costs the muscle more for each piece of work, or loses more than **2%** in any measure (work done, response time at the median, p95 or p99, time over the service line, or the machine's own cost of running Omni-Compass), is undone. A resource spent that rose is **power given**, shown beside the work, the speed and the wear it bought, never called a loss. Where no move passes that test, Omni-Compass leaves the muscle exactly as it runs alone. It takes nothing from one muscle to give to another: that form never harms the organism, but it leaves unused any gain that needs one muscle to give way to another. Letting one muscle give way so that the organism's whole output comes out ahead, judged on the organism and on every muscle's health, is the next engine version's work. It is built and proven in our own models first (the model gate), every benchmark is run again on it, and no result claims it before its runs exist.

**The factory tune: one tuned engine, and a tuning bench.** The engine in this repository is presented as The Omni-Compass LLC tuned and ships it: self-running, set at the factory. Every muscle in the catalog is wired and runs on the setting found for it (its own card where one was found on development seeds and kept under the strict rule, 364 muscles in `realms/muscle_cards_v5.csv`; the engine's own setting for every other), frozen and fingerprinted by version (`python3 tools/omni_version.py`); no setting is presented as universal. **Every published benchmark result is one engine at its one factory tune**: the engine its table names (`python3 tools/omni_version.py`), run again on each engine that stands. From omni-v17 that tune is held to a declared rule (`docs/THE_TUNE.md`, `tools/tune_study.py`): of the tunes that harm no gauge against native, the one against which no other tune tried reads GAIN on one service gauge and LOSS on none, searched on development seeds and read again on untouched ones; the claim is

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

non-dominance among the tunes searched, never a global optimum. The live benchmark engine page and the control panel are a live tuning engine: every knob, button and monitor turns in real time (the beat, the dial, the pedal, each muscle driven or left native, up or down, in or out, minus or plus) to show the mechanism of action with native alone, with Omni-Compass on top, or both side by side; what it shows at any other setting is the engine's range, never a benchmark result. As on any engine, where the mixture, the intake, the exhaust, the firing or the pistons change what it delivers, **turning any knob of Omni-Compass can change every metric, up or down, in any direction a tuner chooses**: one engine for many vessels, tuned many ways, for many outputs. A licensee may add on, retune, upgrade or tune down any muscle or combination (the dial, the cards, the bands and the service lines are exposed); a changed tune is the licensee's own, and so are its results. The factory tune is published only where it reads no LOSS: a LOSS found in The Omni-Compass LLC's own runs is treated as a fault in the mechanism or its wiring, found and fixed before the result is published, never as a number to live with (the founder's order of 23 October 2026). The one place a LOSS is meant to be seen is the live benchmark engine, when a person turns its knobs away from the factory tune (the dial, the heartbeat, the sizes, the load) to watch the system react and the brain hold or recover: that showing is the proof that the mechanism can be broken only by leaving it, and a changed tune's readings are the licensee's own.

**The factory profile, and the profile that comes next.** The factory tune is one complete configuration profile, the factory profile: every governing rule, gain, margin, decision interval and default setting shipped together, versioned and fingerprinted. Today's is omni-v19's. The best validated factory profile for the declared operating conditions is not yet established: it is chosen only after every mechanism is verified, every knob combination is tested together on development workloads, and the choice is validated on fresh ones; then it is frozen, carried into every implementation (the C++ harness included) and released under its own identity. Until then no other profile is presented as the baseline; on the live benchmark engine a candidate is evaluated, never released, and a custom profile's readings are its own. The words, the mathematics and the bill's arithmetic are written once in `docs/FACTORY_PROFILE.md`.

## The wiring declaration

**THE PUBLISHED BENCHMARKS ARE THE RECORD OF WHAT OMNI-COMPASS DID ON THE SYSTEMS, LOADS AND ENGINE THEY NAME, EACH WITH ITS EVIDENCE CLASS. WHERE A RESULT ON ANY OTHER SYSTEM DIFFERS FROM THEM, OR ANYTHING LOOKS WRONG, THE FIRST PRESUMPTION IS THE WIRING, THE SETTINGS, OR HOW OMNI-COMPASS WAS IMPLEMENTED OR READ, NOT THE LAW. CHECK THOSE FIRST (THE MANUAL'S "WIRED RIGHT OR WIRED WRONG") BEFORE DRAWING ANY CONCLUSION ABOUT OMNI-COMPASS.**

**Omni-Compass acts only through the wires it is given.** It reads the meters it is pointed at and moves the settings it is allowed to move. If a reading is the wrong one, slow, or blind; if a lever is the wrong one, already owned by another controller, or given a range (cover) that is wrong for the machine; if a band or a service line is set for a different workload; then Omni-Compass will do exactly what its law says with the wrong information, and the result will not be the published one.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Therefore:

1. **It cannot be slapped on.** Every installation goes through the levels of the manual in order (chapter 6), starting read-only, and every writing level begins with the wire check (manual, section 4.4) and a watch arm that must equal native.
2. **If your paired receipts differ from the published benchmarks in direction** (service LOSS, or no saving where one was shown), **the first presumption is wiring, not the law.** Confirm the installation with the checks in the manual's section 4.5 (*Wired right or wired wrong*) before drawing any conclusion about Omni-Compass.
3. **This has happened to us, and is published.** On the first real card (NVIDIA A10, 2026-10-02) the governor read being busy as trouble, set its power lid under the card's own working draw and allowed the clock under the card's own working clock; the card served bursts more slowly and the slowest answers were 58.5% slower. The cause was found in the card's own samples and corrected (`docs/GPU_PREREGISTRATION.md`, amendments 6 and 7). That is what a wiring fault looks like, and how it is found.
4. Omni-Compass never writes outside a lever's cover, never fights another writer (it stops and leaves that lever alone), reads back every write, and returns every lever to the value it read before its first write when it stops or when the OFF switch is used. These protect the machine; they do not make a wrong wiring right.

## Safety and responsibility

1. Run watch mode first; run the wire check before any write; keep the master OFF switch (`python3 tools/omni_switch.py off`, the whole harness at once) in reach at every level, and use it at the first sign of anything wrong, a suspected breach included.
2. Omni-Compass is a supervisory governor. The machines' own controls (firmware, autoscalers, safety systems) stay in place and keep their own protections; Omni-Compass sets only values they already accept.
3. Do not connect Omni-Compass to safety-critical systems (vehicles, medical devices, grid protection, life safety) except as a modelled study, and never without the qualified review those systems require.
4. The software is provided "as is", without warranty of any kind, as stated in `LICENSE`. The operator is responsible for its use on their systems.

## Changes to the work and to this page

1. The software, the law's settings, the benchmarks, the results, the documents, the licensing terms and this page may change at any time, without notice. The version in the repository's `main` branch at a given commit is the version that commit describes; `RELEASE_MANIFEST.json` and `python3 verify.py` identify it.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

2. Statements about work in progress, planned features, expected results and future runs are expectations, not promises. Where a result has not yet been measured, the documents say so (`docs/STATE_OF_PLAY.md`, `docs/DOSSIER.md`, section 8).
3. A signed Omni-Compass Enterprise License governs its own terms for its own term.

## Privacy and data

Omni-Compass contains no telemetry. It sends nothing to The Omni-Compass LLC or anyone else, and makes no network connection of its own except to the systems an operator points it at. Its logs and receipts stay on the operator's machines.

## Where everything is

| Question | Page |
|---|---|
| What has been measured, and how strongly | `docs/STATE_OF_PLAY.md`, `docs/DOSSIER.md`, `docs/EVIDENCE_LEDGER.md` |
| The rules written before each run | `docs/GPU_PREREGISTRATION.md`, `docs/REALMS_PREREGISTRATION.md`, `docs/K8S_COMPASS_PREREGISTRATION.md` |
| How to wire it, level by level, and how to confirm it is wired right | `docs/OMNI_COMPASS_MANUAL.md` (chapters 4 and 6), the book `docs/OMNI_COMPASS_MANUAL.pdf` |
| The license, in plain answers, and third-party components | `LICENSE`, `LICENSING_FAQ.md`, `ENTERPRISE_LICENSE.md`, `NOTICE`, `THIRD_PARTY_NOTICES.md`, `PATENTS.md`, `TRADEMARKS.md`, `SECURITY.md`, `.github/CLA.md` |
| The muscles, what each is for and how it is wired | `docs/MUSCLE_CATALOG.md` |
| Proof the code is the code that ran | `python3 verify.py`, `RELEASE_MANIFEST.json`, `results/SEAL.json` |
| The live benchmark engine: every monitor, its feed and its knobs; the third manual, how to read and operate it | `docs/engine/OMNI_COMPASS_LIVE_ENGINE.html`, `docs/engine/monitors.json`, `docs/OMNI_COMPASS_LIVE_ENGINE_MANUAL.md` and `.pdf` (the Live Benchmark Engine Manual's Appendix L, the page word for word: `docs/engine/PAGE_WORD_FOR_WORD.md`) |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Question | Page |
|---|---|
| The whole organism, composed every way it is bought | `results/WHOLE_SYSTEM.md`, `tools/whole_system.py`, `docs/engine/export_whole.py`, `docs/WHOLE_SYSTEM_PREREGISTRATION.md` |
| The Python's runs that the live benchmark engine runs again | `tools/engine_runs.py`, `results/engine_runs/`, `results/heldout_seed_*/` |

## The whole organism, the cloud's back end, and the live benchmark engine

**The composition is a model of measurements.** `results/WHOLE_SYSTEM.md` (`tools/whole_system.py`) composes each layer's own measured ratio, native against Omni-Compass on top of native, into one benefit for a whole stack: the product law R(g) = ∏ℓ Rℓ(g) gauge by gauge, its logarithm a sum with the variances added, each layer's runs A, B and C pooled by the inverse of their variances, and one index over the gauges with its 95% interval under two bounds (the gauges independent; perfectly correlated). No run here has stacked the layers: each factor is one layer's own run with the layers under it native, and the product assumes a layer's ratio does not change when the layers around it also carry Omni-Compass. That assumption is tested where the tables can test it (the governors' four ways), and the stacked run that replaces the composition is preregistered in `docs/WHOLE_SYSTEM_PREREGISTRATION.md`. Until it runs, the composition is class S of class P and L measurements, and every page that shows it says so.

**Every way a stack is bought.** The same law is read four ways (`docs/engine/export_whole.py`): on-site bare metal (the software on the owner's own racks and chips); through the cloud (the cloud's build, its bare metal at the back end and the software; and the cloud as two pieces, the back end left to the provider); in the device (Omni-Compass in the bare metal alone, the card and the machine); and as a service (the software alone, for an owner with neither bare metal of their own nor a cloud account). A piece with no measurement enters as exactly 1 and is named.

**The cloud's back end is not ours to meter.** Through the cloud, the bare metal belongs to the provider: no tenant can read its power meters or write its chips' limits, and no run here reaches it. The cloud piece's benefit is what the provider itself meters and bills, the machines counted every fifteen seconds and the bill at list price, read native against Omni-Compass on top of the cloud's own autoscaler. The bare-metal piece is gauged on hardware we can meter (a card's own power meter; a processor's energy counter) and is carried to the provider's bare metal only as the same mechanism on the same kind of part, a modelled factor, never as a measurement of the provider's machines.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**The cloud is a wire.** A cloud is the wire between the software end (the owner's programs, Kubernetes and the open-source orchestrators, managers and autoscalers that carry the work) and the hardware end (the bare metal: the GPU chips, the Linux racks, the data center). Omni-Compass can live in all three places; the wire makes no benefit of its own. Until 31 October every cloud run here was set on the wire alone (the tune-down on the machines only), so it could at most park machines the cloud already made and cut the wire's bill a little. From 31 October (the founder's order of that day: no hard rule) every cloud set runs the whole engine at the max on the cloud's cluster, so its receipt shows that the engine holds there as on site and the work per dollar beside the bill: more work for the same money, never a smaller bill as the aim. The bare metal's energy and throughput are not under a tenant; they are measured where Omni-Compass sits on them (the card and the machine) and composed with the wire by the law above. A provider running Omni-Compass on its own back end would see that benefit on its own chips and racks; no tenant can meter it, so it is never claimed here as a measurement. The company's expected ranges (the founder's figures of 24 October: about 3 to 4% from the software end, 2 to 3% from the bare metal, about 5% from the two together, 1 to 2% from the wire alone) are an expectation, never a result; where a measurement differs, the measurement stands.

**The organisms.** The seven organisms of the catalog (the spine, the four realms, the four stacked, the tower), the orchestra beside them, the live benchmark engine's own sixty-muscle organism, and the whole organism in its three places (the software end, the cloud's wire, the bare metal) composed above: ten organisms. Each is read two ways, native and Omni-Compass on top of native (Omni-Compass is never by itself, the founder's order of 25 October): twenty scenarios, recorded in `results/scenarios/SCENARIOS_20.json` (class S, our own models, three runs), every cell with no run reading "not yet run". None is a product.

**The live benchmark engine.** `docs/engine/OMNI_COMPASS_LIVE_ENGINE.html` is a monitor of the repository's records and of the engine's own mathematics running in the page, and the live tuning bench: a setting other than the factory profile shows a variation of the mechanism, never a result. Every monitor carries its evidence class (P a physical meter, L live software, S our own models) and its state: live (the engine's port, proven equal to the Python and the C++ on every build by `docs/engine/check_live_engine.py`), a replay of a recorded run from its own files, a reading of the repository's tables, or built and awaiting its run, with no invented number and the run that fills it named. The kill switch on the page acts on what runs in the page, and a recorded result changes its colour, never a value, a word or a row. Pointed at an owner's own Prometheus, the page only reads it: it writes to no system.

**The three manuals.** The Operator's Manual wires and runs Omni-Compass, the Owner's holds the whole mechanism and every result, the Live Benchmark Engine Manual reads the page; each carries this notice, the `LICENSE`, the `NOTICE` and this page.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART ONE

# Monitor Wall, Control Panel and Contents

*The monitor wall, the control panel directly under it, and the page's contents.*

CHAPTER 1

# Monitor Wall

This chapter reads the page's section 0, **Monitor Wall** (`#s-wall`).

## 1.1 What the wall is

The wall (the page's section 0, `#s-wall`) is every monitor of the page in one view: the same canvases the sections draw, copied small four times a second, so nothing is drawn twice from different data. Every tile carries at its top its number and its title in capital letters, what the monitor shows, and under them its section and its state (live, replay, from the records, a model, or awaiting its run), so a tile says where its monitor is and what it represents before it is opened. Click a tile and the page scrolls to the monitor it copies. The page's own governor (the page's section 5j, chapter 34) leaves a monitor resting while it is off screen; every monitor is drawn once as the page opens, so no tile waits for its section to be scrolled to, and while the wall is in view the resting monitors are drawn in turn, one a frame, each by its own drawing code; a tile with the accent border is a monitor on screen now.

Under the wall a line, **Monitor Lights** (`#wallLamps`), counts the lights (section 1.3). The section's controls line reads: click a tile to open its monitor; every tile is read four times a second. Directly under the wall stands the control panel (the page's section 1, chapter 2), whose first panel holds the button that puts back the factory tune.

## 1.2 The numbered monitors and the five blocks

Every monitor of the page (every chart canvas) carries its number in a badge above it. The number is the monitor's place on the wall (the page's section 0), which shows every monitor at once in five blocks, each under its own heading and read left to right, then the next row: 1, **Benefit and Organism** (twelve monitors, three rows of four on a laptop); 2, **Real Runs on GitHub**; 3, **Brain and Mechanism**; 4, **Clouds, Chips, Orchestrators and Scale**; 5, **Proof and Telemetry**. The first four monitors are the recap: the benefit board (monitor 1, *Benefit by Domain*), the whole organism (2, *Stack Composition*), the market map of every three-run table (3, *Market Map*) and the benefit gauge by gauge (4, *Benefit by Gauge Family*). A click on a tile

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

scrolls to the monitor it copies. Two monitors that sit side by side in their section, native beside Omni-Compass, sit side by side on the wall too: a pair never splits across two rows, on the wall or on the page at any width. Each badge names its section, so a monitor and its section read each other both ways. The monitor list (the page's appendix E, chapter 39) gives every monitor by its number; the whole wall is also a Grafana dashboard (`deploy/grafana/omni-compass-wall.json`), its panels numbered as the page numbers its monitors, one row for each block. Appendix L lists every monitor under its number.

## 1.3 The light on every monitor

Each tile and each monitor carries its light as a signal under its number, on the left, standing like a traffic signal: red on top, an octagon like a stop sign, yellow in the middle, a yield diamond, green at the bottom, round; one lamp is lit, and on the page the word beside the number (GAINING, STEADY, REDLINING, OVER THE LINE or AWAITING, section 36.5) says which. **Green, GAINING**: low in its band with room to climb, or a GAIN. **Yellow, STEADY**: inside its band, or level with native (a change under 0.5%). **Red, REDLINING**, blinking red and black: at the top of its band and holding there, or a LOSS row in its record (every row stays); REDLINING comes before the line: a gauge within 3% of the top of its band, or whose slope over its last readings reaches it within 3 readings. **Red, OVER THE LINE**, blinking red and white: past the top of its band. **Dark, AWAITING**: the run that fills it has not landed. **Grey, NO LINE**: a monitor with no line to cross (a bill, a count of runs, a digest check) or no reading yet; neither dark nor grey is ever counted green. A monitor that draws the organism's two sides carries three lights in one button: native's and Omni-Compass's, each read from its own side on the same load, and between them the reading now, Omni-Compass against native by the key (GAIN green, STEADY yellow, LOSS red), each by whether the gauge helps or hurts the muscle and the organism, never by its sign: fewer muscles over their line, fewer pods waiting, or a 95th percentile faster by 0.5% or more is GAIN, and a 95th percentile within 0.5% of native's reads STEADY. So the moment one side turns a colour the other does not is seen as it happens; a live light compares that moment, and the judged readings stay in the record's tables. Click the button for each side's reason; a red or yellow one offers the way back to the factory tune, because a light the knobs turned is a light the factory tune puts back. A line is a health requirement in the words of `docs/FACTORY_PROFILE.md`: a service limit, a resource bound or a hardware limit.

The light reads the series the monitor draws and adds no number: on the live benchmark engine, the muscles against their lines; on a cluster replay, the pods waiting for a place and the 95th percentile against native's at the same moment; on the heart, the decision's own time against half its beat; on the CPU host, the busy share against the 75% line; on the card, the draw against its power limit; on the page's own governor, the time between frames; on the proof, the page's runs against the Python's (green when the digests match, red when they differ); on the governors and orchestrators, their own three-run tables; on a record, the judged rows of the newest tables (a LOSS row lights red and stays). A light fed by our own models says so (class S). Under the wall the line **Monitor Lights** counts the green, yellow, red, dark and grey lights, names every red one, names the two-sided monitors whose native side is red, and gives each live comparison against native once (**Live Comparison Against Native**), beside the engine's own last entry on its side and the **Redline Reflex**'s

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

hand-backs on the live model. The three-reading projection is the light's own: the brain's law acts on the position inside its band and damps on the position's rate of change, with a fail-up at the band's 95% wall; it projects nothing ahead.

What the page's organism is. The sixty muscles replay the repository's own model (`realms/harness.py`, class S) under the climb, whose load steps past what the fixed muscles can carry; at the top of the climb both sides cross their lines, because no controller makes capacity that is not there and this model adds no machine. Native in every benchmark is the system's own automatic controllers as shipped, with nobody turning a knob, the same as Omni-Compass; where a person would step in, the record says so and shows the crossing.

## 1.4 Every monitor's strip and wiring

Under every monitor a strip carries its knobs: the kill switch; *Inspect* (its light's reason, its feed and the query an operator's Prometheus would ask, its wiring, and downloads of its data as CSV, the page's live metrics in the OpenMetrics format and its registry entry as JSON, as Grafana's panel inspector gives them) and *Full screen* (the key `v` over a monitor; Esc to leave); the console's knobs where the monitor is live, its own buttons where it replays, and the factory button.

**Every monitor's wiring.** Under every monitor one line names its wiring (`docs/engine/monitors.json`): the page's own code, the Python it ports or reads, the C++ twin or why there is none, the checks that hold them equal, each file linked at the build's commit, and how it moves; the monitor list carries the same. A live monitor runs the page's port of the engine, held equal to the Python by `docs/engine/check_live_engine.py` and the Python to its C++ twins by the parity tests; a replay plays a record the Python wrote, held equal to its source by `docs/engine/check_records.py`; a record reads a measured table as written, so no C++ applies.

## 1.5 The motion meter

The page watches itself as the viewer sees it. Every monitor in view is photographed small once a second; one whose picture changed in the last three seconds reads ● **moving** on its badge and in its row of the list, one in view for three seconds without a change reads ● **still** (and says so when the engine is paused). A monitor off screen rests, as the page's own governor rests it (chapter 34), and its row keeps what it did when it was last in view; the note under the list counts the monitors seen so far and how many of them moved. Scroll the page once from top to bottom and every row reads what its monitor does.

## 1.6 The keys and the kiosk

The key `?` shows the card **Keyboard Shortcuts**. `v` puts the monitor under the mouse full screen and Esc leaves it; `k` is the kiosk, the wall and its control panel alone (`k` again to leave); `p` pauses or runs; `s` steps one decision; `f` puts back the factory tune; `w` goes to the wall; the number keys 1 to 4 go to the four parts of the main matter, Parts 2 to 5. The kill switch is never on one key: it is its own button, on every monitor, in the dock and in the control panel.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 2

# Control Panel

This chapter reads the page's section 1, **Control Panel** (`#s-health`).

## 2.1 What the section holds

The control panel (the page's section 1, `#s-health`) stands directly under the wall and switches every monitor of the page. It holds three panels, in this order. **Wall Status and Factory Tune** gives the wall's state in one line and the one button that puts back the engine's tune as it ships (section 2.2). **Fuse Box** (`#fuseBox`) switches every monitor from one box: its master rows, the governor request, its seven tabs, the heartbeat, the profile box, the sweep over every beat, the mechanism knobs, the twenty scenarios and the injection grid, one breaker for each monitor, the Prometheus metrics and the governor's commands (sections 2.3 to 2.26). **Operator Controls** (`#ctlPanel`) holds the kill switch, the buttons that run the Python's runs in the page, the jump generator, the variation switchboard, the four reports, the three manuals and the wall as a Grafana dashboard (sections 2.27 to 2.32). The section draws no monitor of its own: the master switch's wiring is the page's section 5g (chapter 31) and the monitor list its appendix E (chapter 39). No value here is typed in: each is computed at runtime from the live benchmark engine or from the repository's records embedded at build time.

## 2.2 Wall Status and Factory Tune

The first panel's head line (`#wallNote`) says how the wall is drawn: how many monitors it holds, in how many blocks, how many are on screen now, and that the rest are drawn in turn while the wall is in view. Beside it a button reads **Back to the factory tune**: one button, wherever the knobs were turned, puts back the engine's tune as it ships (`DISCLOSURES.md`, section 3: the max, Omni-Compass ON, demand ×1, no noise, no added delay, no fault held, every muscle driven, speed 1×) and the page's plant at rest (section 11.2). Native alone is not a tune: it is the kill switch, beside it on the console and at the head of the third panel (section 2.28).

## 2.3 Fuse Box: its rows and its tabs

The fuse box (`#fuseBox`, the second panel of the control panel, the page's section 1) switches every monitor from one box, laid out as a console. Its first row is the master switches, always in view: each acts on the whole page at once, through the same code as the console and the dock, both sides alike. Under it stand seven tabs, one shown at a time:

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Tab | What it holds |
|---|---|
| **Drive** | the plant's conditions and faults, the breakers' two buttons, the factory button, your own system, the keys; the heartbeat; the sweep over every beat and the candidate; the profile's three buttons and the profile box (the active profile, the backend, the actuation, the contract and each deployment route's bill) |
| **Mechanism Knobs** | the engine's own constants (sections 2.11 to 2.23), one group at a time: a row of group buttons above the knobs names each group, from the four pieces to every muscle (sections 2.14 to 2.22), with *All Groups* for all of them at once |
| **Scenarios and Injection Grid** | the ten organisms, each native and with Omni-Compass on native, and the injection grid: every governor, 1 to 500 runs × 1 to 500 organisms on the Python's batch (section 2.24) |
| **Monitor Breakers** | one for every monitor, by section (section 2.24) |
| **Prometheus Metrics** | the live model printed as a Prometheus endpoint, every decision, with the series a running system serves beside each and the PromQL the Grafana wall draws it with (section 2.25) |
| **Commands** | every control beside the request a running governor takes for it, with its role (section 2.25) |
| **All Tabs** | the whole panel at once, as one long box |

The tab chosen, and the group chosen among the mechanism knobs, are kept in the viewer's own browser only, as a convenience; the page opens on *Drive* without them. The fuse box's guide and the mechanism knobs' guide stand in the page's reading guide (appendix B, under *1 Control Panel*), so the controls come first.

Between the master row and the tabs stands one line, **governor request**. After every press it shows the request a running governor would take for it through its control panel (`tools/omni_control.py`), with the role the request needs: a heartbeat button, `POST /v1/beat {"seconds": N}`; Return to native, `POST /v1/reset`; the kill switch, `POST /v1/kill` (and back on, `POST /v1/arm`, two holders of the security role within ten minutes); the dial, `POST /v1/dial`; Restore factory profile, the dial at the max, the factory beat, every muscle driven and autopilot. A control that acts only on this page's own model (the plant's conditions, the faults, the time, the body, the view) says so: it has no request. The page sends nothing anywhere; the line is the request written out, `$OMNI_PANEL` the panel's address and `$OMNI_TOKEN` the operator's token. Every switch is written to the control event log with its decision (the page's section 3c), and the line at the panel's head counts the breakers by their signal word and the monitors rested, and names the Omni-Compass side: on top of native, or native's own under the kill switch. The panel is the bench of `docs/FACTORY_PROFILE.md`: its controls operate the stated backend and show its response.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 2.4 The master rows

The first row: **kill switch** (it reads "Omni-Compass ON · press to hand back to native", and engaged, "ENGAGED · Omni-Compass OFF · native alone"; section 2.28); **tune dial**, *the max* or *tune-down*; **body**, *harness, each muscle alone*, *harness, one organism* or *the live model*; **time**, 1×, 4×, 16×, 60×, *Pause* and *One decision*.

The second row: **demand**, ×0.25 to ×2 the day's own; **noise**, σ 0 to 0.6; **sense delay**, +0 to +6 decisions; **faults, 40 decisions**: *Surge ×2*, *Lose half*, *A sense blind*, *Security hold*; **breakers**: *every breaker ON*, *rest every monitor off screen*; **factory**: *Back to the factory tune*; **your own system** (section 2.25); **keys**: *Keyboard keys* and *Kiosk*.

The dial, the demand, the noise, the sense delay and the faults act on the live stress-test model; turning any of them while the harness plays moves the live benchmark engine to the model, and the control event log says so.

## 2.5 The heartbeat period: six buttons

The third row, **heartbeat period**, is the heartbeat, the governor's full decision: six buttons, **10, 15, 30, 60, 90 and 120 s**, and nothing else (`BEATS`). The factory profile's beat is 60 s; 15 s is the standard of native's own pod autoscaler, which checks every 15 s; the 10 s button shows a beat below that standard. Pressed, a button sets the beat of Omni-Compass on the live stress-test model (moving the live benchmark engine to the model if the harness plays), moves the heartbeat sweep monitor (the page's section 4a) to the same beat, and is logged with its decision.

The live model steps every 15 s of the plant (`STEP_S`), native's own autoscaler check. At a beat of 15 s one full decision lands in every step; from 30 to 120 s one lands every two to eight steps; at 10 s two full decisions land in every other step, both on that step's reading, their pushes compounded (`BEAT_N`). Between full decisions the fast checks act: at the max a muscle past its line has capacity added at once by the reflex (on the tune-down it goes back to native's setting), and otherwise its setting holds while native may still raise its own. The note beside the buttons says what the beat in force does on the live model, for example "60 s now, the factory's: on the live model the full decision every 4 steps of 15 s; the fast checks and native's own 15 s check between".

Its backend is the page's live model (class S); its effect is how often Omni-Compass decides; its limits are the six buttons on the page (the shipped controller itself takes any period from 5 to 120 s, `--interval`), and while the active profile is held the buttons are disabled. A beat other than 60 s makes the active profile a custom profile, unless it is the candidate (section 2.7).

## 2.6 Every beat, three model days each, and the candidate

Beside the heartbeat stands the sweep, the group the page labels **beat sweep · class S** (our own model). **Run every beat (10 to 120 s), three days each** runs the live stress-test model for a whole model day at each of the six beats on three days, the page's own opening seed and the two after it, the same three at every sweep, with

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

the same demand on both arms; when it ends, the model starts the day in hand again from its seed. Nothing runs until the button is pressed: until then the note reads "not run yet in this view".

Under the profile box a table gives each beat, Omni-Compass on native against native, the one pair: a light, the change in work with its word and its sign, the muscle-decisions past the line, the failed work and the capacity held, each with its word. Each day is paired with native's on the same seed and read by `three_run_reading` (section 36.4): the change is the mean of the three days' changes; under 0.5% STEADY; from 0.5% up the word by the gauge's aim; days that moved 0.5% or more in opposite directions disagree. Capacity held that rose is power given. The light is red where work, the muscle-decisions past the line or the failed work reads LOSS, green where work reads GAIN, yellow otherwise. The best beat is the one with the most work for Omni-Compass on native and no LOSS; it is outlined and marked "the best", and 60 s is marked "the factory's". Under the table, **Kubernetes Governor Model at Six Beats** reads the Kubernetes governor's own model at the same beats (the page's section 3d, its ladder record, class S) and names, setting by setting, every gauge that reads LOSS against native there; where a beat reads LOSS, the cause is the engine's, on the next version's list, and the real runs at each beat decide (`docs/PACE_PREREGISTRATION.md`).

**Try the candidate** applies the sweep's best beat as a **candidate profile**, a proposed combination of settings under evaluation: the factory's knobs with the heartbeat at that beat. The log says "a candidate is evaluated, never released". It does nothing before a sweep has run ("no candidate yet: run every beat first") and nothing while the active profile is held. A model's best beat is a candidate for the real runs, never a result; the factory's beat stays until the real runs read it.

## 2.7 The profile: factory, candidate, active and custom

The **profile** group holds three buttons, and the profile box (`#fuseProfile`) under it says which profile governs.

- **Restore factory profile** reapplies the released configuration: every knob of the mechanism and the heartbeat back to the factory's (60 s), the candidate cleared and any hold released. It leaves the master switch as it stands (its log line reads "Omni-Compass still on").
- **Hold the active profile** holds every knob as set: the mechanism's knobs, the heartbeat buttons, the sweep, the heartbeat sweep monitor's beat row and the variation switchboard's beat row are disabled, and the note reads "HELD at decision n: the heartbeat … s and every knob held as set; restore the factory profile to turn them again". Unheld, the note reads "every knob turns; the factory profile is the released one (omni-v23: the heartbeat 60 s, every muscle on its card); no other profile is released".
- **Return to native** hands governance back: the reset, the operator's control (`tools/omni_control.py`, `POST /v1/reset`). On the page every setting goes to native's own controller and every Omni-Compass side follows native, as with the kill switch, but the switch then reads "OFF by the reset · native alone · press to turn ON" and the event log names the reset. A second press of Return to native changes nothing (native already runs alone); the switch turns Omni-Compass on again. The kill switch (section 2.28) is separate: for security and instant. On a real machine,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`docs/FACTORY_PROFILE.md` gives the reset's meaning: hand governance back and restore the recorded native settings, as the adapter supports.

The box's first line is the **active profile**: its kind, FACTORY, CANDIDATE or CUSTOM, the engine, the heartbeat and the profile's fingerprint, "profile" and eight hexadecimal digits. The fingerprint (`profHash`) is the 32-bit FNV-1a hash of every setting of the mechanism, the heartbeat and every muscle's body included: one identity for one set of settings, so two views showing the same fingerprint run the same settings of the mechanism. The kind is FACTORY when no knob of the mechanism is turned and the heartbeat is the factory's; CANDIDATE when no knob is turned and the heartbeat is the candidate's; CUSTOM otherwise. The line is printed green for FACTORY, yellow for CANDIDATE and violet for CUSTOM, violet being the page's colour of a tune turned from the factory, never a side. Its note says what that means: the factory profile is "the released configuration, every muscle on the setting found for it; every published result is this profile's"; a candidate is "a proposed combination under evaluation … it becomes a factory profile only after validation and a new release"; a custom profile is the count of knobs turned, "identified by its fingerprint; its results are its own".

The second line names the **backend**: "this page's live stress-test model (class S, our own model): no meter, so no physical energy reading; the sections marked replay play recorded runs from their own files", or, while the harness plays, the realms harness, recorded and replayed from its record, with the reminder that the live model's knobs act on the live model.

## 2.8 Factory and active settings

The distinction between factory and active settings is the one `docs/FACTORY_PROFILE.md` writes. The **factory profile** is the complete, versioned set of Omni-Compass default settings shipped together: the governing rules, gains, margins, decision interval and every other default. Freezing it fixes the released rules and default tuning; the engine's internal state, its commands and the resources it allocates go on responding to changing conditions, so one fixed factory profile governs a changing workload. The **active profile** is the settings actually governing the current run, named on the profile box with its fingerprint. A **candidate profile** is a proposed combination under evaluation; a **custom profile** is any identifiable configuration that differs from the released factory profile. On the page, every benchmark result played back ran the factory profile of the engine its table names; a candidate or custom profile on the bench is a variation of the mechanism (class S), never a benchmark result, and its results are its own. The page opens on the factory profile and holds nothing: no profile is locked when the page opens.

Three buttons put things back, and they differ. *The mechanism back to the factory* (in the mechanism panel) puts back the mechanism's knobs alone. *Restore factory profile* puts back the mechanism's knobs and the heartbeat, clears the candidate and releases a hold. *Back to the factory tune* (in the control panel's first panel, on the console, in the dock and on every strip) puts back all of that and the page's plant at rest as well (section 11.2).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 2.9 Actuation and the contract, this decision

The box's third line reads Omni-Compass on native's actuation in the decision on screen: how many settings it **requested** (a move of more than 2% of the setting), how many **landed as asked** (inside the muscle's floor and ceiling), how many were **held at the muscle's limit** (clipped to that floor or ceiling), and how many muscles are **still moving toward their setting** (capacity in service more than 2% from its setting, following its lever with the muscle's own response time τ). The line adds that the model's plant refuses nothing, while a real adapter reads every write back and holds a refused one (omni-v12). The line follows the distinction `docs/FACTORY_PROFILE.md` draws (Chapter 3) between the requested, the commanded and the acknowledged settings. While the harness plays, nothing is requested, commanded or read back in that view (the harness replays recorded settings, and the page's section 3e traces one of its decisions), and the line carries a button, *Run the live model*, that moves the page to the live model, where the counts are computed.

The fourth line, **contract, this decision**, writes the same decision in the symbols of `docs/FACTORY_PROFILE.md`, section 3, for Omni-Compass on native:

k = the decision on screen · t[k+1] = t[k] + Δ, Δ = the heartbeat set · θ = the active profile's fingerprint and its kind ureq: the settings changed · ucmd = ΠC): how many were cut to the muscle's own limit uack = R(ucmd): how many of the sixty settings stand at their command, and how many are still moving

Under it the whole contract is printed once, y(k) = h(x(t[k])) + ε(k); (ureq, z(k+1)) = Gθ, y(k), A(k)); ucmd = ΠC); uack = R(ucmd); x(t[k+1]) = Φ(x(t[k]), uack, w(k)); θ(k+1) = θ(k) once released, with what each symbol is on this page: x the sixty muscles' settings and loads, y their readings with the sense delay and the noise σ set on the panel, w the demand, C each muscle's own range, A Omni-Compass's hand (the max or the tune-down). On the harness the line names k, Δ and θ and says the three settings are computed only on the live model; with Omni-Compass off it says nothing is requested and every setting is native's own. The counts are the page's own model (class S): the model's plant refuses nothing, so every cut is the muscle's own range, never a refused write.

## 2.10 The deployment route and its bill

Two buttons choose the deployment route, **On site: software on your own hardware** or **Through the cloud**, and a table gives that route's bill as the sum of its costs. Each row is one cost component, with the native total, the Omni-Compass total and what is measured:

| Route | Cost components |
|---|---|
| On site | energy: kWh × your price per kWh; machines: their cost over their life; operation: people, space and cooling |
| Through the cloud | compute: machines billed × the list price; energy at the back end (the cloud bills no energy to a tenant: inside its price); storage, network and the control plane |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Every native total and every Omni-Compass total reads **unresolved** where no price is recorded, and today none is resolved: the only component metered in money is the cloud's compute, on Azure, at machine-hours times the list price, and the other components carry no price in the repository. Beside a component the page prints its measured change from the whole organism's composition with its word (energy and machines on site, the bill through the cloud), a supporting measurement, never the bill. The formula under the table is the bill's:

$C(r, a) = \Sigma_k\ c(r, a, k)$, $S(r, a) = (C(r, \text{native}) - C(r, a)) / C(r, \text{native})$

where r is the route, a the arm, and c(r, a, k) each cost component inside the declared boundary, counted once; the saving S is defined for a positive native cost and the same workload in both arms. A component without a recorded price leaves its total unresolved, and the saving with it. The whole-workload index on the scorecard is the composition's one number, not a price (chapter 4). The measured changes follow the whole organism's reading and chip knobs (the page's section 5d).

## 2.11 Mechanism Knobs: where they are and what they turn

Inside the fuse box, under the profile box and on its own tab, stands a second set of knobs, **Mechanism Knobs** (`#mechPanel`, folded or unfolded at its head, its groups under **Mechanism Knob Groups**). The plant's knobs change the plant's conditions (demand, noise, sense delay, faults, the load profile); these change the engine's own equations while it runs. Each knob is a constant of the brain (`omnicompass/core.py`), the governor's law (`omnicompass/adapter.py`, `omnicompass/nervous_system.py`), the compass law (`omnicompass/compass_law.py`), Omni-Compass's hand on each muscle, the redline reflex (`omnicompass/immune.py`), the model's native controller, or one muscle's body. Turned, the next decision of the live stress-test model is computed with it, on Omni-Compass's side; native's controller knobs move every native controller, the one under Omni-Compass on native included. Only the live brain reads the panel; every other brain on the page reads the factory's numbers.

## 2.12 How to turn a knob, and how to read it

Each knob is a slider with its value, its equation and its factory value under it, and ↺ beside it puts that one knob back. A knob turned from the factory turns violet, the page's colour of a changed tune, never a side; each group has a button that puts the group back; **The mechanism back to the factory** puts every knob of the panel back, and *Restore factory profile* and *Back to the factory tune* put back these and more (section 2.8). The count beside it reads either "every knob at the factory: the engine as it ships" or how many knobs are turned, "a variation of the mechanism (class S); its readings are its own"; the active profile then reads CUSTOM with a new fingerprint. Every turn is written to the control event log with its decision. Each group prints a live reading of the quantities its knobs act on, every decision, and the line at the top of the panel prints the live model's decision with the muscles past the line, the work served, the work failed and the work waiting this day, native beside Omni-Compass on native. The harness replays recorded positions, so turning any knob

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

moves the live benchmark engine to the stress-test model (the log says so); the knobs act on that model only. While the active profile is held, the panel's knobs are disabled.

## 2.13 Where each group acts

The compass law, Omni-Compass's hand, the redline reflex, native's controller and every muscle's body move the muscles at once. The brain's four pieces move the brain's own state (E, U, I[U], S, B on the brain's monitors) and its grants: at the max every organ may add capacity unless a security hold stands, so the brain's calm acts through the give-backs; set the dial to the tune-down to watch the four pieces move the muscles too.

## 2.14 Piece 1, State and Basin

Source: `omnicompass/core.py`, `cpp/src/core.cpp`. Live reading: E, U, I[U], the push, and whether U is in its basin.

| Knob | Range | Factory | Equation it enters |
|---|---|---|---|
| the pull toward the basin, K[P] | 0 to 30 | 12 | $u = -U + K[P]\cdot(1 - U)$, bounded by U[auth]; push = u/U[auth]; a push above κ's knee holds every give-back |
| the command's authority, U[auth] | 5 to 50 | 25 | the largest command; the larger it is, the smaller the same command reads as push |
| assimilation, a | 0 to 1 | 0.339 | $x \leftarrow (1 - a)\cdot x + a\cdot x[obs]$: 0 hears nothing, 1 forgets everything |
| the double well's depth, μ | 0 to 8 | 3.782 | $\dot{U} = \mu\cdot U\cdot(1 - U^2) - \dot{E}/E[max] - \lambda[U]\cdot U + u$: two basins at U = ±1, a ridge at 0 |
| energy's relaxation, α[E] | 0.5 to 8 | 3.487 | $\dot{E} = -\alpha[E]\cdot E + \beta[int] + \beta[ext] + v$ |
| energy's scale, E[max] | 0.5 to 4 | 2.0 | unity loses Ė/E[max]: the smaller, the harder energy shakes it |
| energy's drag on the integral, $\sigma_1$ | 0 to 1.5 | 0.38 | $\dot{I}[U] = (1 - U) - \sigma_1\cdot E - \delta\cdot S - \lambda[I]\cdot I[U]$ |
| the integral's leak, λ[I] | 0 to 2 | 0.719 | how fast the remembered shortfall fades |
| unity's own leak, λ[U] | 0 to 1 | 0 | a drag on U itself |

What an operator sees: a lower K[P] or a lower a leaves U slower to return to its basin after a surge, and the push on the brain's monitors smaller; a shallower μ flattens the double well, so the ball on U's landscape wanders nearer the ridge; a smaller E[max] lets the same stress tilt the well harder.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 2.15 Piece 2, Basin Potential and Flow

Source: `omnicompass/core.py`. Live reading: S, S[eq] beside its factory value, and σ = S/S[eq], with a note when σ passes 1.

| Knob | Range | Factory | Equation it enters |
|---|---|---|---|
| the inflow, δ | 0.05 to 2 | 0.705 | $\dot{S} = \delta - \alpha[s]\cdot S - \tfrac{3}{4}\cdot\beta[s]\cdot S^2$, so $\Phi(S) = -\delta\cdot S + \tfrac{1}{2}\alpha[s]\cdot S^2 + \tfrac{1}{4}\beta[s]\cdot S^3$ and $\dot{S} = -\Phi'(S)$ |
| the linear drain, α[s] | 0 to 0.6 | 0.12 | the flow rests at S[eq], the root of $\dot{S} = 0$: $S[eq] = (-\alpha[s] + \sqrt{(\alpha[s]^2 + 3\beta[s]\cdot\delta)})/(1.5\cdot\beta[s])$, about 2.37 at the factory |
| the quadratic drain, β[s] | 0.01 to 0.6 | 0.10 | σ = S/S[eq] above 1 holds every give-back |

What an operator sees: a larger δ or smaller drains raise S[eq] and the stance's resting place on its landscape; while σ stands above 1 the panel says every give-back is held, and on the tune-down the give-backs stop.

## 2.16 Piece 3, Bath

Source: `omnicompass/core.py`. Live reading: B, Ḃ, the bath's natural period 2π/ω[B] (about 6.28 at the factory) and its damping ω[B]/Q[B] (0.4 at the factory).

| Knob | Range | Factory | Equation it enters |
|---|---|---|---|
| the bath's frequency, ω[B] | 0.2 to 3 | 1.0 | $\ddot{B} = \gamma[c]\cdot\delta\cdot S - (\omega[B]/Q[B])\cdot\dot{B} - \omega[B]^2\cdot B$; the drive on energy beats at ½ω[B] |
| the bath's quality, Q[B] | 0.5 to 10 | 2.5 | the higher, the less the bath is damped and the longer it rings |
| the flow's coupling into the bath, γ[c] | 0 to 3 | 1.0 | how much of the flow S drives the bath |
| the bath's drive on energy, c | 0 to 3 | 1.0 | v = cos(½ω[B]·t)·c·tanh(z) enters Ė |
| the drive's offset, $\lambda_0$ | −2 to 2 | −0.40 | $z = \lambda_0 + \lambda_1\cdot(U - \tfrac{1}{2}) + \lambda_2\cdot S$ |
| unity's weight in the drive, $\lambda_1$ | 0 to 3 | 1.55 | as above |
| the flow's weight in the drive, $\lambda_2$ | 0 to 3 | 1.15 | as above |

What an operator sees: a higher Q[B] lets B ring longer after a disturbance on the bath's landscape; a larger c feeds that swing back into E, so the stress on Piece 1's reading oscillates.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 2.17 Piece 4, Governor Law

Source: `omnicompass/adapter.py`, `omnicompass/nervous_system.py`, `cpp/src/governor.cpp`. Live reading: ρ*, κ, h, calm, and the organs that may give back now.

| Knob | Range | Factory | Equation it enters |
|---|---|---|---|
| the utilisation target, $\rho_0$ | 0.5 to 1 | 0.914 | $\rho^* = \text{clamp}(\rho_0 - k[I]\cdot I[U] - k[E]\cdot E, \rho[\text{min}], \rho_0)$: how full a muscle may run after a give-back |
| the target's floor, ρ[min] | 0.3 to 0.95 | 0.627 | as above |
| the shortfall's pull on the target, k[I] | 0 to 1 | 0.252 | as above |
| energy's pull on the target, k[E] | 0 to 1 | 0.297 | as above |
| the push that still lets go, κ's knee | 0 to 0.05 | 0.005 | $\kappa = 1$ while push ≤ the knee, else knee/push |
| the unity gate, U[gate] | 0 to 0.95 | 0.481 | $h = (U - U[\text{gate}])/(1 - U[\text{gate}])$; calm $= \kappa\cdot h\cdot(1 - (\sigma - 1)^+)\cdot(1 - \min(v, 1))$ |
| the grants θ[nodes], θ[pods], θ[cpufreq], θ[power], θ[routing], θ[gpu] | 0 to 1 each | 0.7, 0.5, 0.3, 0.3, 0.5, 0.3 | an organ may give back only while calm ≥ its θ, the line clean, the senses seeing |

What an operator sees: raising an organ's θ above the live calm removes that organ from the list that may give back now; on the tune-down its muscles stop handing capacity back. At the max this piece acts only through the give-backs.

## 2.18 Compass Law

Source: `omnicompass/compass_law.py`, `cpp/src/compass_law.cpp`. Live reading, for the muscle chosen in the last group: native's p, Omni-Compass's p, v and force F, k[d] for its τ, and the muscles past the line now on each side.

| Knob | Range | Factory | Equation it enters |
|---|---|---|---|
| the band's low edge (load ratio), lo | 0 to 0.6 | 0.2 | $p = (r - lo)/(hi - lo)$: where each muscle stands in its band |
| the line (load ratio; 1 = full), hi | 0.6 to 1.5 | 1.0 | past it a muscle is over the line |
| the centre, where the two forces balance | 0.2 to 0.8 | 0.5 | below it the force pulls capacity back, above it pushes capacity in |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Knob | Range | Factory | Equation it enters |
|---|---|---|---|
| the cushion before the line | 0 to 0.25 | 0.05 | from 1 − cushion the force is the whole authority A: a muscle is never let past the line |
| the spring, k[p] | 0.1 to 4 | 1 | F = A·tanh((k[p]·(p − centre) + k[d]·v)/A) |
| the authority, A | 0.1 to 2 | 1 | the force's ceiling: smooth (tanh), never hammering |
| the velocity's smoothing, s | 0 to 0.95 | 0.5 | v ← s·v + (1 − s)·Δp/dt |
| the damping, × each muscle's critical k[d] | 0 to 3 | ×1 | k[d] = max(0, 2√(k[p]·τ/dt) − 1)·dt for each muscle's own τ |

What an operator sees: the phase plane's shading (the force field) moves with the centre, the spring and the authority; a smaller cushion lets the right-hand dots run closer to the right edge before the force fails up; less damping lets the Omni-Compass side overshoot and the strip's muscles past the line rise.

## 2.19 Push, Lift-Off and Tune-Down

Source: the page's capacity law on `omnicompass/adapter.py`. Live reading: the settings Omni-Compass wrote this decision, the muscles it holds away from native's setting, and the dial.

| Knob | Range | Factory | Equation it enters | | |
|---|---|---|---|---|---|
| the push: capacity added per unit of force | 0 to 1 | 0.30 | the max: L ← L·(1 + gain·F) while F > 0 and the organ may expand; never below native | | |
| the lift-off's base | 0 to 1 | 0.15 | F < 0: L moves toward native's setting by min(1, base + slope· | F | ) |
| the lift-off's slope | 0 to 2 | 0.5 | as above | | |
| the tune-down's floor, × native | 0.5 to 1 | 0.85 | the dial at the tune-down: never under this share of native's setting | | |
| the tune-down's step per unit of force | 0 to 0.5 | 0.10 | one step back: L = max(floor·L[nat], min(L, L[nat])·(1 + step·F)) | | |
| the dwell before a give-back, decisions | 0 to 10 | 3 | calm decisions in a row before one step back | | |
| the give-back's ceiling in the band | 0 to 1 | 0.40 | a step back only while p is under it | | |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What an operator sees: a larger push gain makes the Omni-Compass side add capacity faster under a surge (capacity held rises: power given), a smaller lift-off holds it longer after the surge; the four tune-down knobs act only with the dial on the tune-down.

## 2.20 Redline Reflex

Source: `omnicompass/immune.py`, its C++ twin `cpp/src/immune.cpp`, and the nervous system's `native_hold` (`omnicompass/nervous_system.py`). Live reading: the reflex on or off, the muscles held native now, the hand-backs and the stays this day, and the chosen muscle's own state (under its line, over it so many decisions in a row, or held native with so many decisions of its hold left).

| Knob | Range | Factory | What it sets |
|---|---|---|---|
| the reflex on (1) or off (0) | 0 or 1 | 1 | off: no muscle is handed to native by the immune rule; the law's push at the wall still acts |
| the law's push at the band's wall | 0.80 to 1 | 0.95 | at or past this place in the band the law pushes at once, before the line |
| redlining: decisions over the line in a row | 1 to 12 | 3 | a muscle redlines when its reading stays over its own line this many decisions in a row |
| stuck: the relief it must show | 0 to 0.30 | 0.05 | stuck while the reading has not fallen by this share of where the redline began, or the knob is at its own limit |
| native stronger: by more than | 0 to 0.30 | 0.05 | native's twin reading lower by more than this share hands the muscle to native; the muscle comes back only when Omni-Compass's side reads lower than native's by the same share and is under its line |
| the hold after a hand-back, decisions | 1 to 60 | 10 | the first hold; it doubles for each hand-back in a row and halves after a quiet hold |
| the longest hold, decisions | 10 to 640 | 160 | the doubling stops here |

The factory numbers are the engine's own (the defaults of `Immune` in `omnicompass/immune.py`); none is written into the rule: every one is a knob on this panel, and each light (GAINING, STEADY, REDLINING, OVER THE LINE) reads the live reading against its own band whatever the knobs are set to. What an operator sees: a shorter streak or a smaller margin hands muscles to native sooner and more often (more hand-backs in the live reading); a longer hold keeps them native longer after a surge. A changed reflex is a variation of the mechanism (class S), its readings its own (section 2.23).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 2.21 Native Controller

Source: the model's native rule, Kubernetes' published autoscaling rule, written by us. Live reading: the settings native's controllers moved this decision, and the chosen muscle's setting and last wish.

| Knob | Range | Factory | Equation it enters |
|---|---|---|---|
| native's target load | 0.3 to 0.95 | 0.70 | desired = current·observed/target |
| native's tolerance | 0 to 0.5 | 0.10 | inside ±tolerance of the target native does nothing |
| native's largest step up, × current | 1 to 5 | ×2 | the most one step may add |
| native's stabilisation window, decisions | 1 to 20 | 5 | a step down only to the highest wish in the window |

What an operator sees: both sides change, because native's controller runs under Omni-Compass on native too. The phase plane marks native's target on native's side while the stress-test model runs.

## 2.22 Muscle Scale, Response and Switch

The model's sixty muscles, twelve to a body part. Pick the muscle, then the scope: **this muscle**, **its body part** or **every muscle**. Live reading: the muscle's units on native and on the Omni-Compass side, its ceiling, floor, size, τ and sensing delay, and whether it is governed.

| Knob | Range | Factory | Equation it enters |
|---|---|---|---|
| scale out: the most units it may hold, × its factory ceiling | ×0.25 to ×4 | ×1 | the ceiling c[max]; Omni-Compass's max rides up to it, never past |
| scale in: the fewest units, × its factory floor | ×0.25 to ×4 | ×1 | the floor c[min]; no controller takes it below |
| scale up: each unit's size | ×0.25 to ×4 | ×1 | capacity in service = units × size, on every arm alike |
| its response time τ | ×0.25 to ×4 | ×1 | c ← c + (L − c)·(1 − e^(−1/τ)); the law's damping follows τ |
| its sensing delay | +0 to +6 decisions | +0 | decisions between what the muscle does and what its controllers read |

The group's switch reads **Omni-Compass governs it** and, pressed, hands the muscles in scope back to native's own controller while every other muscle stays governed; pressed again, Omni-Compass governs them again. The kill switch hands every muscle back at once.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 2.23 A changed tune is not a result

Every knob of this panel shows a variation of the mechanism (class S, our own model). The benchmarks prove the factory profile only; a custom profile is its tuner's, and so are its readings. This is the one place on the page where a person may turn the engine away from its factory profile, break it by hand and watch it break; the factory buttons put it back.

## 2.24 Scenarios and Injection Grid, and the Monitor Breakers

On its **Scenarios and Injection Grid** tab the fuse box carries the twenty scenarios in a compact grid (ten organisms, each native alone and with Omni-Compass on native, each cell one signal word and one number against native); a click on a cell prints its gauges and three runs under the grid, and the full grid stands in the page's section 5c (chapter 27). Under it the injection grid stands compact: the organism, the way, what it is read against, the runs, the organisms stacked and the gauge, the cell's readout and its matrix (sections 27.4 to 27.8). A click on the live benchmark engine's own organism in the Omni-Compass on native column also moves the live benchmark engine to the harness's one organism.

The **Monitor Breakers** tab holds one breaker for every monitor, grouped by section in the page's order, each section with its own signal. Each breaker carries its signal and its word, its number and name, its switch (**ON** draws it live; **REST** stops its drawing, as the page's own governor rests a monitor off screen, and frees the frame for the rest), a way to it, and, on a monitor that draws two sides, which side it shows now (native alone under the kill switch, Omni-Compass on native, or as recorded). *Every breaker ON* and *rest every monitor off screen* act on all of them; the panel's own factory button also turns every breaker ON.

## 2.25 Your own system, Prometheus Metrics and Governor API Commands

The group **your own system** points the page at the operator's own Prometheus: an address, *Connect* and *Disconnect*. Once connected, each monitor's own query (its feed in `docs/engine/monitors.json`, the series `tools/omni_exporter.py` and the governor's `/metrics` expose) is asked every 15 s, and each breaker shows what that system answers. It is read only: nothing is written to the operator's system from the page. The Prometheus must allow the page's address (its `--web.cors.origin` flag, or the page served behind the same address); a page opened inside another site's sandbox may be refused by the browser, and says so.

**The wall beside Omni-Compass, in one command.** `deploy/wall` (its README) stands up Prometheus, Grafana with the Omni-Compass wall loaded as its home dashboard (`deploy/grafana/omni-compass-wall.json`), Alertmanager with the governor's alert rules, node_exporter and the governor's exporter: `cp .env.example .env`, set the Grafana password, `docker compose up -d`, then Grafana at `http://127.0.0.1:3000` and Prometheus at `http://127.0.0.1:9090`, the address to type here. Every port is bound to the machine itself, every

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

image pinned; the stack reads and writes nothing. `--profile alloy` adds Grafana Alloy (`config.alloy`) in the collector's place: it gathers the same targets and writes them into the same Prometheus. Alloy collects and forwards telemetry and decides nothing; it sits beside Omni-Compass, one of the ways the governor's readings travel to the wall.

**The Prometheus Metrics tab** (its panel **Prometheus Metrics Endpoint**) prints the live model in Prometheus's text exposition format, as `GET /metrics` answers on a running system: the decisions, the heartbeat, the master switch, the dial, the active profile (`omni_profile_info{kind, fingerprint, engine}`), the plant's conditions, and for each of the two arms (`arm="native"`, `"on_native"`) the work served, failed and waited, the muscle-decisions past their line, the writes, the cuts to a muscle's range, the setting moves and this decision's actuation (`omni_actuation{arm, stage}`). Under it a table sets each beside the series a running system serves (the governor's exporter, the service's own `http_requests_total` and `http_request_duration_seconds`, node_exporter's `node_cpu_seconds_total`, Kepler's joules, OpenCost's hourly cost) and the PromQL the wall draws it with. Its names are the page's own model (class S); the output passes `promtool check metrics`.

**The Commands tab** (its panel **Governor API Commands**) lists every control with the request a running system takes for it, its role and its note: the kill switch and the arm, Return to native, the dial, the heartbeat, each muscle on or left native (`POST /v1/muscle`, the governor's muscles), the mode (autopilot, watch, off), Restore factory profile, the reads (`GET /v1/status`, `/v1/presses`, `/metrics`); the mechanism's knobs have none (an engine constant changed is another engine, never a press), nor have the plant's conditions and the view.

## 2.26 Every control: its backend, its effects, its limits

| Control | Its backend | Its effects | Its limits |
|---|---|---|---|
| Kill switch | every governor on the page (the live model, the harness's replay, the live controller's replay, the page's render governor) | every setting handed to native's own controller at once; every Omni-Compass side follows native | security only, instant; a recorded result changes colour, never a value; a second press releases it |
| Return to native (the reset) | the same governors | the same hand-back; the switch reads "OFF by the reset" | the operator's control, not a security engagement; a second press changes nothing; the switch turns Omni-Compass on again |
| The dial | the live model | the max (every muscle to its own limit, never below native) or the tune-down (never more than native) | the records keep their own dial |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Control | Its backend | Its effects | Its limits |
|---|---|---|---|
| The body | the live model or the harness record | which body the live monitors draw | the knobs act on the live model only |
| Time | the live model's clock | 1× to 60×, pause, one decision | none on the records |
| Demand, noise, sense delay, faults | the live model's plant | both sides alike, the same draw | demand ×0.25 to ×2; σ 0 to 0.6; +0 to +6 decisions; a fault lasts 40 decisions |
| Heartbeat | Omni-Compass on the live model | how often the full decision lands (section 2.5) | six beats; disabled while held |
| Run every beat, Try the candidate | the live model, three model days a beat | the sweep's table, the best beat; a candidate profile | class S; a candidate is never released |
| Restore factory profile, Hold the active profile | the mechanism's knobs and the heartbeat | the released configuration back; every knob held | the plant's conditions are the factory tune's button |
| Deployment route | the whole organism's composition and the recorded prices | the route's bill as a sum of costs | a component with no price stays unresolved |
| Mechanism Knobs | the live model's equations (sections 2.11 to 2.23) | the next decision computed with the knob | the ranges of the panel; the live model only |
| The breakers | the page's drawing | a monitor drawn live or rested | nothing recorded changes |
| Your own system | the operator's Prometheus, read only | each breaker shows its series | the browser's cross-origin rules |
| Back to the factory tune | every knob of the page | the factory profile and the plant at rest | the kill switch released; native alone is the kill switch |

## 2.27 Operator Controls

The third panel of the control panel, **Operator Controls** (`#ctlPanel`), carries at its head the big kill switch ("Kill switch · Omni-Compass ON"). Under it stand the buttons that run the Python's runs in the page (*Run the Python's 500 runs here*; *Run the Monte Carlo 500: the Python, the C++ and this page, linked*; *Run your own system through it (here or on GitHub)*; *The proof monitor*), the **Jump Generator**, the **Variation Switchboard**, the four reports (*Engineering Report*, *CTO Report*, *Board Report*, *CFO Report*), the three manuals and *The wall as a Grafana dashboard (JSON)*. Its controls line says what acts: the kill switch acts at

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

once on every governor of the page; the buttons run the Python's runs here and fetch the three manuals. The panel closes with the kill switch's own note (section 2.28).

## 2.28 The kill switch and its three states

While Omni-Compass governs on top of native the switch reads ON in green. Pressed, it turns red: at the next decision every governor puts every setting it wrote back, reads each one back and stops (`omnicompass/master.py`, `tools/omni_switch.py`); the word OFF stands white, and every Omni-Compass side of every monitor follows native. Native's own controllers under every muscle never stop and never see the switch. A recorded result never changes with it: its colour does, never a value, a word or a row. The same switch stands at the head of every section, in every monitor's strip, in the fuse box's first row, on the console (*Turn Omni-Compass OFF*) and in the dock; one press anywhere is one press everywhere. It is for security and is instant; the walk back after a crash is the lease, separate.

| State | Colour | What it means on the page | What it means on a real machine |
|---|---|---|---|
| ON | green | the brain's outputs reach every muscle; every monitor shows Omni-Compass acting on top of native | the governors run; every write read back; the lease renewed each beat |
| Kill switch engaged | red | every lever handed to native at once; the decisions shaded red on every time axis; the event log records the engagement and its clearing | `python3 tools/omni_switch.py off`: every governor hands back and exits, and none starts again until `on` |
| OFF | white | the brain keeps reading the native body in shadow, so its monitors keep moving, and nothing it computes leaves it; every Omni-Compass side follows native, drawn in a tint of native's colour, dashed | the governor in watch mode writes nothing (the Operator's Manual, level 1); off, it is not running |

No monitor is switched off by the switch. That is deliberate: the operator sees what the brain would do while it does nothing, which is the watch level of every installation.

## 2.29 What a kill-switch window looks like

Press the kill switch on the console (chapter 11) and every time axis on the page shades the window red from the press to its clearing. Inside it, Omni-Compass's lines fall onto native's; the governor's ticker (the page's section 4a) is empty, for nothing decides on top of native; the phase plane's right-hand dots take the left-hand positions. When the window clears, the lines part again where the governor's first decision lands.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 2.30 The Python's runs at a button, and the Jump Generator

The first button runs the Python's own seeded runs in the page (they also run once on their own as the page opens), the second runs the core's Monte Carlo through the Python, the C++ and the page, linked, and the third opens the panel that runs the operator's own system through it; chapter 25 reads all three.

**Jump Generator.** In the control panel, under the buttons that run the Python's 500, a switch reads **Jump generator · OFF** until it is pressed. **ON**, the page draws set after set from the Python's jump generator (500 runs a set unless 100 or 1,000 is chosen; run k of a set takes the seed base + k·0x9E3779B9 mod $2^{32}$, its stream mulberry32, the same generator `tools/engine_runs.py` and `tools/core_runs.py` write bit for bit), one base after another from the first base shown, which opens on the newest base the Python has run, so the first set reproduces the Python's own. Each set runs twice in the browser: through the engine's runs (the proof monitor) and through the core's Monte Carlo (the Monte Carlo monitor), both in the page's section 5a. Each set's output is its runs, their counts and one SHA-256 digest; where the Python has run the same base its digest stands beside the page's, and they must be equal. The line under the switch counts the sets drawn, the runs of the set in hand and the next base; the switch rests while the page is hidden and runs until pressed again. **OFF** is the page's standing mode, the one it opens in: nothing new is drawn from the generator; the Monte Carlo monitor plays the Python's own 500 (numpy's generator from seed 223387268), the proof monitor recomputes once, as the page opens, the newest set GitHub ran in the Python and checks it digest for digest, then re-checks it one run at a time; the 500 scenarios a seed replay from their record; GitHub's runs play back from their own raw files; and the organism's model day runs on its own seeded stream, the same at every opening. Class S throughout: the engine's own mathematics on draws, never a measurement.

## 2.31 Variation Switchboard

The **Variation Switchboard** is one group of knobs in the control panel that reaches every monitor carrying the variation it names, through that monitor's own setter; the line under it names the monitors that moved, or says that no monitor carries the variation live.

- **Arm**, two buttons, one for each side: *native* is Return to native, the reset (not the kill switch), so every Omni-Compass side follows native; *Omni-Compass and native* releases it, and every monitor shows Omni-Compass on top of native again.
- **Organism**: the spine, the four realms, the four stacked or the tower: the grid's row is marked, and for the spine and the realms the console's fault body part follows.
- **Organisms** and **runs** (1, 10, 100, 500, in line with the Python's batch of 500): the injection grid of the page's section 5c moves to that many runs × organisms stacked (chapter 27), and the scale grid's square is marked where GitHub's grid carries that tier (it runs 1, 10, 100 and 1,000). The organism row sets the injection grid's organism too.
- **Run**: A, B or C on every replay that holds three runs and on the live organism's record.
- **Beat**: 10, 15, 30, 60, 90 or 120 s: the heartbeat sweep monitor moves to that beat, or to the sweep record's nearest one and says which, and, unless the active profile is held, the live model's heartbeat is set to it.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

A monitor that does not carry a variation is left as it is.

## 2.32 Reports, manuals and the wall as a Grafana dashboard

**The reports.** Four buttons in the control panel, *Engineering Report*, *CTO Report*, *Board Report* and *CFO Report*, write a report from the page's own records at the moment they are pressed: the engineer's (every benchmark on the board with its reading, size, rows and record, the Python's runs and their digests, the clouds' finished sets, the lights at that moment), the CTO's (the summary; what stands and what runs, which says that two sides are shown, native and Omni-Compass on top of native, and never Omni-Compass by itself; the board), the board's (the summary, what is proven and what is modelled, the clouds) and the CFO's (the summary, every cloud set's machine-hours and bill at list price by the one key, the rows of power given and the cuts). Each opens the browser's print, to print or save as a PDF, and downloads the same report as a page; the notice stands first and last, every reading is the key's word with the word's sign, and every number names its engine and its run.

Beside them stand the three manuals, each a PDF to download (this one, the Operator's and the Owner's), and *The wall as a Grafana dashboard (JSON)*, which saves the wall as `omni-compass-wall.json` (the repository's `deploy/grafana/omni-compass-wall.json`), its panels on the open-source exporters' series (chapter 39).

## 2.33 The page runs itself

The engine runs itself: its own mathematics decides every beat, and the knobs show the mechanism; they never have to drive it (the page's own words in its reading guide, appendix B). The page opens on the factory profile, with Omni-Compass on, the heartbeat at 60 s, every muscle driven and nothing held, and the live benchmark engine governs from the first decision. Every knob turned is a variation of the mechanism on the live tuning bench (class S); the benchmarks prove the factory profile only, and a changed profile's results are its own.

CHAPTER 3

# Contents: the Page's Order and Its Numbering

This chapter reads the page's **Contents** (`#contents`), directly under the control panel.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 3.1 The page's own contents

The page opens on the monitors. Under the banner (the logo, the name, the engine chip and the legal notice) stand the monitor wall (the page's section 0, `#s-wall`) and, directly under it, the control panel (the page's section 1, `#s-health`). Only then come the page's contents (`#contents`), in three groups: **Wall and Controls** (0 Monitor Wall, 1 Control Panel), **Main Matter** (Parts 2 to 5, each with its lettered sections) and **Appendixes** (A to G). Each row carries its number and its title and links to its anchor. A lettered section (2a, 4d) is a section inside the numbered part that holds it; an appendix carries a capital letter. The page keeps its explanations in appendix B, the reading guide (`#s-guide`): each section's introduction stands there under the section's number and title, so the sections themselves open on their monitors and controls. Native is orange everywhere and Omni-Compass blue, and every monitor, section and part carries its signal and its word: red REDLINING, yellow STEADY, green GAINING.

## 3.2 The sections, as built on omni-v23

Appendix A lists every section with its anchor and the chapter of this book that reads it. In the page's order:

- **Banner** (`#topBanner`): the logo, the name, the engine chip and the legal notice (`#notice`).
- **Wall and Controls:** 0 Monitor Wall (`#s-wall`), every monitor in five blocks: Benefit and Organism; Real Runs on GitHub; Brain and Mechanism; Clouds, Chips, Orchestrators and Scale; Proof and Telemetry; the first four monitors the recap (the benefit board, the whole organism, the market map of every three-run table, the benefit gauge by gauge). 1 Control Panel (`#s-health`), directly under the wall: wall status and factory tune, the fuse box with its tabs and the mechanism's knobs, the kill switch, the runs, the variation switchboard, the reports and the manuals.
- **Contents** (`#contents`).
- **Part 2, Results** (`#p-results`): 2a Scorecard; 2b Benefit Against Native; 2c Benefit Board; 2d LOSS Register; 2e Status Board.
- **Part 3, Live Operation** (`#p-live`): 3a Live Organism; 3b Golden Signals and Event Log; 3c Control Console; 3d Heartbeat Dial; 3e Control Cycle; 3f Phase Plane and Scoreboard; 3g Brain State; 3h Verdict; 3i Dashboards.
- **Part 4, Real Runs and Platforms** (`#p-runs`): 4a Kubernetes Run Playback; 4b Repetition Telemetry; 4c Controller Decision Trace; 4d Cloud Providers; 4e Orchestrators and Governors; 4f GPU and CPU Hardware; 4g CI Pipeline.
- **Part 5, Proof and Models** (`#p-proof`): 5a Python Runs and Monte Carlo; 5b C++ Twin; 5c Scenarios and Injection Grid; 5d Whole Organism; 5e Measured Effect; 5f Telemetry Analytics; 5g Master Switch Wiring; 5h Specification and Mathematics; 5i Failure Modes and Controls; 5j Self-Governed Rendering.
- **Appendixes** (`#p-appendix`): A Title Page and Build; B Reading Guide; C Problem and Answer; D Executive Summary and Reading Key; E Monitor List; F Glossary and Index; G Disclosures, Law and Contact; then the closing legal notice (`#notice-end`).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 3.3 Anchors

Every section has an anchor that does not move when the page is reordered: `#notice`, `#s-wall`, `#s-health` (with the fuse box, `#fuseBox`), `#contents`, `#p-results`, `#s-bottom`, `#s-answer`, `#s-board`, `#s-loss`, `#s-status`, `#p-live`, `#s-engines`, `#s-feed`, `#s-desk`, `#s-dial`, `#s-decision`, `#s-plane`, `#s-brain`, `#s-verdict`, `#s-dash`, `#p-runs`, `#s-ops`, `#s-runs`, `#s-live`, `#s-cloud`, `#s-orch`, `#s-stacks`, `#s-github`, `#p-proof`, `#s-proof`, `#s-cpp`, `#s-scn`, `#s-whole`, `#s-fleet`, `#s-lab`, `#s-master`, `#s-math`, `#s-fail`, `#s-self`, `#p-appendix`, `#title-page`, `#s-guide`, `#s-problem`, `#s-read`, `#s-monlist`, `#s-glossary`, `#s-disclose`, `#notice-end`. A tile on the wall links to the anchor of the monitor it copies; a row of the contents links the same way. When this book names a section it gives its number, its title and its anchor, so the reader finds it on a page whose numbering has moved.

## 3.4 The order

The page is read top down in the order its watchers need it: the wall first, every monitor live, with the control panel under it; then the results (the scorecard, the benefit against native, the benefit board, every LOSS row and the status board); then the live operation of the engine; then the real runs and the platforms they ran on; then the proof and the models; and last the appendixes, which carry the title page and build, the reading guide with every section's explanation, the problem and the answer, the executive summary and the reading key, the monitor list, the glossary and index, and the disclosures. This book follows the same order, chapter for chapter, and carries the problem and the answer and the reading key in its own front matter as well. On a keyboard, `w` goes to the wall and the number keys 1 to 4 go to the four parts of the main matter, Parts 2 to 5.

PART TWO

# Results

*The page's Part 2: what Omni-Compass did against native, every LOSS row included.*

CHAPTER 4

# Scorecard

This chapter reads the page's section 2a, **Scorecard** (`#s-bottom`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 4.1 What it answers

The scorecard (`#s-bottom`), the first section of the page's Part 2, asks one question: is Omni-Compass a benefit? It answers with one light and one number for each way to run a workload, against native alone on the same work, read from the repository's own records. The line at its head (`#blHead`) puts the answer in one sentence: Omni-Compass on native, with its signal word and its change in work against native in the real three-run test. The same number stands in the dock at the foot of the screen ("vs native, more work (3 real runs): on native").

## 4.2 How to read it

Each row is one way to run a workload, against native alone on the same work. The light is the whole verdict: green GAINING, Omni-Compass ahead of native; yellow STEADY, within 0.5% of native (STEADY by the key's floor); red REDLINING, behind native; dark AWAITING, no test of that way in this build. The big number is how much, its size with no sign, the raw change (Omni-Compass − native over native) standing apart with its true sign, labelled raw: on the first row more work done for the same demand, on the bills and the index one number over their gauges, read by the key. A slower service, less work or a failed request turns the light red whatever the number, on every row: on the bills and the index, any table under the row that reads one of them LOSS, in either form, turns it red. Each row names what it stands on: its test, its engine and its evidence class.

## 4.3 Whole-Loop Test by Arm

The first table (`#blArms`) reads a real test, three runs on GitHub's own machines (A, B and C from one commit), class L: the full form, 18 steps of 25 s with the clients stepping 1 2 3 2 3 4 …, native against Omni-Compass on top of native. Its columns are the way to run it, the light, the change against native and what else moved. It has two rows, one for each side.

- **Native Alone** is the yardstick: Kubernetes' published autoscaling rule as The Omni-Compass LLC wrote it, a description of the rule and not Kubernetes' code, on Omni-Compass's own orchestrator and test machines, with no Kubernetes. It is shown, not judged: every other row is read against it, and it carries no reading of its own.
- **Omni-Compass on Native**, Omni-Compass on top of that rule, carries as the big number the mean change in work done (requests answered) over runs A, B and C, each run's own change printed under it with the record's reading. Beside it stand the 95th percentile (how much faster or slower, with its reading), the machines (how many fewer or more), the serving units (how many more or fewer; a rise is power given, the capacity spent to buy the work, never a loss) and the failed or wrong answers over the runs, Omni-Compass against native.
- **The light** is red, REDLINING, when the work's reading or the 95th percentile's is LOSS or the runs disagree, or when Omni-Compass's arm gave more failed or wrong answers than native in any run; green,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

GAINING, when the work reads GAIN; yellow, STEADY, otherwise. A way with no three-run test in the build reads AWAITING, "not yet run".

The note under the table names the record (on this build `results/orchestrator/V21_THREE_ARMS_ABC.json`, of which the page carries only the pair Omni-Compass on native against native, and only those two sides' totals), what it ran, the engine, the commit, the three runs and the file, with the line "Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results."

## 4.4 Whole-Workload Index by Route

The second table (`#blBills`) reads the whole workload, bought two ways: **on site**, your software straight on your own hardware (the software end × the bare metal; no cloud), and **through the cloud**, your software through the cloud onto its hardware (the software end × the cloud's wire × the bare metal at the back end). The big number is the whole workload's index, one number over its gauges, with its word; beside it every gauge with its word and its number. The light follows the index's word, and is red where any table under the route reads a slower service or less work LOSS, in either form; a route with no composition reads AWAITING, "not yet composed". The bills read the repository's own headline, by the index's rule, with the other reading beside it.

The note under it names its source: `tools/whole_system.py` composes every measured layer's own three-run reading into the whole workload (a model of measurements, class S of class L and P; chapter 28). It says what enters today: the bare metal enters as no change, because the machine's own layer is not yet run and the card's one run, made on a controller since replaced, stands outside the headline scope; so both bills are the software end's and, through the cloud, the wire's, and the cloud's own bill is not counted (its tests are one run each, and by the index's rule a table of one run enters as exactly 1), as the cloud-is-a-wire box explains (the page's section 4d, chapter 21). The two bills follow the whole organism's two knobs in the page's section 5d, the reading and the chip's one run in or out. The index here is a composition's one number, not a price; each route's bill as a sum of costs is the control panel's (section 2.10).

## 4.5 All Real Tests Combined

The third table (`#blAll`) gives Omni-Compass on native over every real test together: the Omni index (chapter 6), its light, its number with its word, and its count of GAIN, LOSS and power given rows. The note points to every row of the benefit board (the page's section 2c) and every three-run table (the page's section 5e).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 5

# Benefit Against Native

This chapter reads the page's section 2b, **Benefit Against Native** (`#s-answer`).

## 5.1 What it holds

The section opens on the Omni index, every real test weighed together, read from its record as the page opens (`results/OMNI_INDEX.md`). Where the engine that stands has not yet landed the tables the index is read from, its head says so ("Not Yet Read on omni-v23") and gives the figure of record on the engine its tables stand on, never read as a result of the engine that stands, with the count of its GAIN, LOSS and power given rows and the benefit sheet's word for the same tables. The same line names the LOSS ledger (`docs/LOSS_LEDGER.md`): every LOSS row of the newest tables, the rows Omni-Compass against its own native counted apart, because do no harm is judged by them, each with its cause where one is written (chapter 7).

## 5.2 One real test of the whole loop

Under the index stands one real test of the whole loop, three runs on GitHub's own machines (A, B and C from one commit), class L, on Omni-Compass's own orchestrator (`omni_orchestrator`), the organism run without Kubernetes: native is Kubernetes' published autoscaling rule on the muscles, as written by The Omni-Compass LLC, and the other side is Omni-Compass on top of the same rule. It is one test, not the figure of record. Its table gives each side's signal, then work, the 95th-percentile response time, the machines and the units (the service's own copies), each with its reading, its size with no sign, the raw change apart with its true sign, and every run's own change. Native's row is the baseline in every column. Units that rose are power given: Omni-Compass adds them to buy the work and the speed. The note names the record (on this build `results/orchestrator/V21_THREE_ARMS_ABC.json`), its runs and its commit; the record's other arm, Omni-Compass alone on the muscles, stays in the record and is not carried, because Omni-Compass is never by itself.

## 5.3 Bill by Route

**Bill by Route** reads each route's bill as its native total, then Omni-Compass's, then the saving, 1 − (the Omni-Compass total ÷ the native total), each total the sum of its non-overlapping cost components inside the declared boundary, on a matched workload (`docs/THE_TUNE.md`, section 8). A component's own percent

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

is a supporting measurement, never the bill; where a price or a component is missing the bill reads unresolved, and no price is assumed. Its rows are *On-site: the software on your own bare metal* and *Through the cloud: the cloud's build, its bare metal at the back end, the software*. Beside it stand the totals composed from the tables that stand (`results/WHOLE_SYSTEM.md`, `tools/whole_system.py`), each on its own engine, by the rule that counts only what held in all three runs: a composition across engines (class S over class L tables), never read as one engine's result (chapter 28). The same route's bill as a sum of costs is the fuse box's profile box (section 2.10).

CHAPTER 6

# Benefit Board

This chapter reads the page's section 2c, **Benefit Board** (`#s-board`).

## 6.1 Benefit by Domain

**Benefit by Domain**: every domain, native against Omni-Compass, as the repository's benefit sheet holds it (`results/BENEFIT_SHEET.json`, `docs/BENEFIT_SHEET.md`): the domain, the benchmark, its evidence class, its engine, its benefit (one number and one word), its GAIN rows, power given, STEADY and LOSS counts, and where it stands (landed, running, queued, not yet run). Its bar chart is monitor 1 on the wall: every benchmark's measured effect in one board, by domain and by gauge family, with the Omni index that weighs them together at its head. Each bar carries its table's own word (GAIN green, STEADY yellow, LOSS red, power given blue), and the size beside the word carries the word's sign (GAIN +, LOSS −, STEADY =, power given +). The axis reads 10, 25, 50 and 100% each way on a square-root scale, so a small bar is seen; a bar past 100% runs to the edge and is marked. Every LOSS row stands on the board, is raised on the status board and is listed, with its cause where the LOSS ledger records one, in the page's section 2d.

## 6.2 Benefit by Gauge Family

Every judged row of every newest three-run table (the page's section 5e), sorted into the family of gauge it measures. A CTO reads speed, the service level, failures and work. A CFO reads machines, energy and the bill. A plant engineer reads peaks and swings, wear and accuracy, and misses. Each bar counts the rows that read GAIN, in all three runs or by the mean of the three; the size beside it is the median of those rows' own sizes, each row's size the mean of its runs' raw changes in percent of native (a row whose native stood at zero is counted and not sized); the largest is printed beside the median; power given and LOSS rows are counted beside them, and the runs that disagree or differ are listed, not counted.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 6.3 Trading Floor

Every three-run table of the page's section 5e, read the way a trading floor reads its market. The market map is a squarified treemap: one tile a table, grouped by stack, its area the number of gauges the table judged and its colour its reading: green GAIN, deeper the larger the share of its rows that read GAIN; red wherever any row reads LOSS; yellow STEADY; blue where it gave power; grey with a dashed edge a four-ways comparison that is not Omni-Compass against its native (the orchestrator's own, with no Omni-Compass in either arm; Omni-Compass in place of a controller; two changes at once). A reading cursor walks the map one table every one and a half seconds and names it under the map. The quote board gives every table a quote: its ticker (the table's file) and engine, the gauge that moved most with its mean change across the three runs, and that gauge in runs A, B and C as a candle: the vertical bar is the run's 95% interval, the tick its value, the line the zero of no change. The strip under each quote counts the table's rows by word, and a tape runs every headline across the top. Pointing at a tile or a quote shows its record: the table's title, engine, counts, the headline gauge in each run and the file. Every figure is the table's own; nothing is typed in.

## 6.4 Whole-Board Aggregate

The Omni index (`results/OMNI_INDEX.md`): the geometric mean of the ratios of every measure that enters, where a measure enters only as its three-run reading allows (GAIN, LOSS or power given in all three runs counts; STEADY counts as exactly 1, so nothing inside the noise is claimed either way). Each test is read from its own newest three-run table, no table is pooled across engines, and every row names its table. The index is one number for the board; its row of factors under it is the board's own audit.

CHAPTER 7

# LOSS Register

This chapter reads the page's section 2d, **LOSS Register** (`#s-loss`).

## 7.1 What it lists

The LOSS register lists every LOSS row of every standing three-run table, read from the verdict audit's record (`tools/verdict_audit.py`), the one selection the LOSS ledger (`docs/LOSS_LEDGER.md`) is written from: Omni-Compass against its native first, then the GPU card's own meter (outside the audit's selection), then the orchestrators' four-ways comparisons by what their two arms are. It is recorded data: every row is read from the verdict audit's record of the three-run tables and the GPU card's own record. No row is cut and no number is tuned. Its panels, in order: **Verdict Audit** (every row recomputed from its stored values, one count, by view: the tables, GAIN, power given, STEADY, LOSS, the runs disagree, the runs differ); **LOSS Rows Against Native** (each table with its engine and class, the gauge, every run's native and

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Omni-Compass values with the raw change, and the evidence); **Armada Two-Cluster BARE Table by Run**; the **Discrepancy Report** (what changed against the tables as they were); and the **LOSS Ledger**.

## 7.2 How to read a row of the LOSS Ledger

The LOSS Ledger's columns are the comparison (a red LOSS for Omni-Compass against its own native; an orchestrator's own comparison named by what its two arms are), the table, the gauge with its runs A, B and C and its reading, the cause as the LOSS ledger records it, and where it was fixed and what decides it, exactly as the ledger records them (`tools/loss_ledger.py`). Where the ledger records no cause, the row says so and asserts no knob. The rule on every row: find where the mechanism or its wiring is wrong and correct that in the model gate; a knob with no setting that brings the row STEADY stays native on that stack, so Omni-Compass hurts nothing (do no harm, `DISCLOSURES.md`, section 3); no number is tuned to make a row read well. Chapter 44 is the procedure.

CHAPTER 8

# Status Board

This chapter reads the page's section 2e, **Status Board** (`#s-status`).

## 8.1 The order of the board

The status board (the page's section 2e, `#s-status`) says whether Omni-Compass is engaged, which signals are out of tolerance and where, and raises every LOSS row as an alarm with its source record; each tile and each alarm is computed at runtime from the live benchmark engine or from the repository's records. The board is laid out the way an operations monitor is. The switch and the engine come first. Next come the live organism's signals, native beside Omni-Compass, in the terms of site-reliability practice (latency, traffic, errors and saturation). Then every reading of the real and modelled runs that went against native, worst first. Last come GitHub and the age of the page. A tile's word is its reading against native by the key; a tile with no reading says what it shows.

## 8.2 The tiles

Each tile carries its title, its state, a line of detail and its buttons; every tile is read again four times a second.

| Tile | What it shows | Its buttons |
|---|---|---|
| Omni-Compass Engagement | ON on top of native, or OFF, native alone; the dial | the kill switch; the max or the tune-down |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Tile | What it shows | Its buttons |
|---|---|---|
| Engine That Stands | the engine and its digest; its brain, compass law and verdict run in the page, checked against the Python and the C++ before the page is committed | the mathematics |
| Errors, Muscle-Decisions Past the Line · Class S | the muscle-decisions over their line this day, Omni-Compass against native, read beyond the counts' noise (two standard deviations of their difference): GAIN, STEADY or LOSS; "too early to read" in the day's first 20 decisions | Surge ×2; Lose machines |
| Throughput, Requests Served · Class S (on the harness, Throughput, Work Done · Class S) | the work done (on the live model, the requests served), the same demand on both sides, with the paired reading over the day's decisions | Demand ×1.6; Every knob to its start |
| Latency Stand-In, Queue Depth · Class S (on the harness, Power Drawn (kW), Harness · Class S) | on the live model the queue depth now, on the harness the organism's electrical power now (a rise to do more is power given) | Pause |
| Injected Faults | the faults active and the decisions each has left | Blind a sense; Security hold |
| Three-Run Tables | their LOSS rows (and how many were answered: the rig corrected, run again STEADY) and the rows where the runs disagree | Show the LOSS rows; Show where they disagree |
| Verdict Guard · Class S | how often the engine that stands keeps a step that truly costs 3% more, at the noise the real runs measured (class S): open, the next engine version's work | the verdict; where it can fail |
| CI Pipeline | the runs running and queued when the page was built | every run |
| Cloud Runs | the Azure sets running and their repetitions | the clouds |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Tile | What it shows | Its buttons |
|---|---|---|
| Build Freshness | how long ago the page was built, and from which commit | — |

## 8.3 Alarms and Watch Items by Severity

The table under the tiles, **Alarms and Watch Items by Severity**, lists every alarm and watch item, worst first by kind, each with its item, its reading, its source, the repository's record and a way to it:

1. the live organism, when it has more muscle-decisions past their line than native this day, beyond the noise of the counts (LOSS);
2. every LOSS row of Omni-Compass's own three-run tables, with the root-cause document where the repository has one, and "run again: STEADY" where the row was answered by its own rerun on the corrected rig;
3. every square of the scale-out grid that reads LOSS with the real cluster inside;
4. every LOSS row of an orchestrator's own cost, where Omni-Compass is in neither arm: not a reading of Omni-Compass, standing as its table writes it;
5. every row where the runs disagree or differ;
6. the guards and open items: the verdict can keep a step that costs a little more; two levers act with no trial (a pod's processor limit conveyed from its machine, and a machine added), bounded by rules instead; the resources the tables show and do not judge that rose in all three runs;
7. GitHub: runs on the engine that stands that ended without a result, clouds whose $0 check has not passed, and the benchmarks still awaiting their three-run table on the engine that stands.

The buttons at its head filter it: *All*, *LOSS*, *The runs disagree*, *Live Now*, *Guards and Open Items*, *GitHub*. The line above it counts the LOSS rows, the rows where the runs disagree, the guards and open items, and the items from GitHub.

## 8.4 Reading an alarm row

An alarm row names its gauge and its table, its size with the word's sign over runs A, B and C, the table's file, engine and evidence class, and the repository's root-cause document where one exists (`docs/POD_START_WAIT.md`, `docs/TAIL_LATENCY.md`, and the others). Chapter 44 is what to do with one.

## 8.5 The age of the page

The last tile, Build Freshness, counts from the build's time to now on the viewer's clock and names the commit. A page older than three hours and a few minutes means the rebuild did not run or did not land (chapter 24); the records on it are still true for the build they name.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

PART THREE

# Live Operation

*The page's Part 3: the engine at work, live, native beside Omni-Compass, from the organism to the dashboards.*

CHAPTER 9

# Live Organism

This chapter reads the page's section 3a, **Live Organism** (`#s-engines`).

## 9.1 The two organism rings

The system rendered as an organism: each node is a muscle (one controlled actuator) and each ring its operating range. The left instance runs under its native controllers alone; the right runs the same controllers with Omni-Compass's brain, nerves, heart and immune system supervising them. The body is chosen in the page's section 3e or in the fuse box's first row: *Each Muscle Alone* (native, then on top, one at a time), *One Organism* (the sixty muscles in one body) or the *Stress-Test Model* (the console's sixty muscles). The two monitors are *Native Organism Ring · Class S* and *Omni-Compass Organism Ring · Class S*. Evidence class S: our own models, shown beside the real runs and never presented as proof.

## 9.2 Reading a ring

A ring is a muscle's band drawn as a circle: calm at the inner edge, the line at the outer. The muscle's live value is the ball on the ring; the reference, the knob at native, is marked. Under the max a muscle is moved only where it has room above native; a muscle already at its limit on native is left native by design and its ball sits where native's does. A dashed white ring marks every muscle that differs between the two sides at that moment, a red tail shows where a muscle went in its last 12 decisions, and a click on a muscle follows it. The counters under each ring (past the line now, misses so far, settings off native, the two sides differing now) and the scoreboard (chapter 14) give both sides and the difference. Where the two sides look the same, they are the same.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

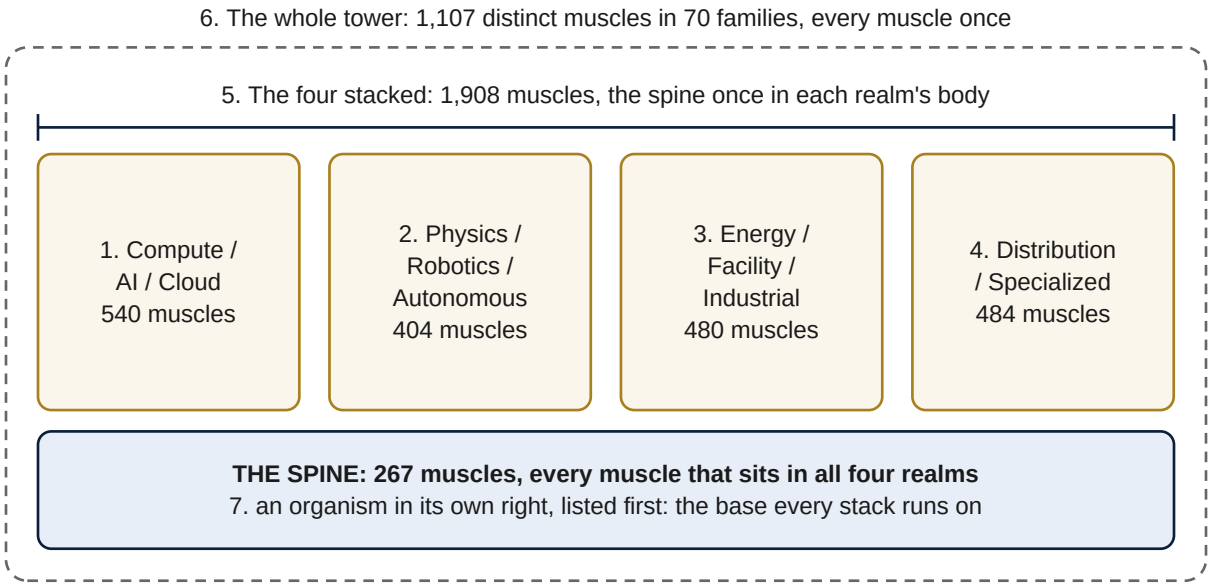



**Figure 1.** The seven organisms. The spine (267 muscles, every muscle that sits in all four realms), the four realms (Compute/AI/Cloud, Physics/Robotics/Autonomous, Energy/Facility/Industrial, Distribution/Specialized), the four stacked, and the whole tower of 1,107 muscles in 70 families. The organism monitor plays one organism of sixty muscles and each muscle alone; the measured effect in full (chapter 29) carries all seven.

## 9.3 Body Scan by Organ

As a scan reads a living body organ by organ, this reads the system playing in the page's section 4a (the benchmark, run and repetition chosen there), organ by organ, half minute by half minute, from its own SHA-256-checked files: native beside Omni-Compass. Each organ names what it reads; where native has no such organ (the governor's heart, the nerves) the cell says so.

## 9.4 Immune System

The immune system is Omni-Compass's; native runs on its own controllers. Every room and every machine pool has a native twin: the same muscle under its own controller, on native's heat and load, the body's memory of healthy. A room that stands warmer than its twin is inflamed. Where its own knob can cool it, it is cured on the spot. Where it cannot, it sends the whole body a signal, its heat debt, like a cytokine calling for help, and the other muscles answer: the machine pools give back what Omni-Compass added until the room stands no warmer than native's. The skeleton (the bone: the power and cooling lines the whole body shares) grants nothing it cannot carry. A cooling setting is walked back, never snapped. On the monitor an inflamed room is marked, its heat debt is a bar, and the pools' give-back is seen on their rings. In the words of

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

`docs/FACTORY_PROFILE.md` this belongs to the defensive system: constraint enforcement, fault detection, containment, recovery and restoration (the bone and the twins, the lease's walk back, the kill switch).

## 9.5 Anatomy by Part

Part by part, in the order of control, the brain first and the whole organism last: the brain, the nervous system, the heart, the spine, the immune system, the muscles, the blood, the power (the nutrients), the intake, the digestion, the exhaust and the organism, each with what native has, what Omni-Compass and native has, and where it lives in the code.

CHAPTER 10

# Golden Signals and Event Log

This chapter reads the page's section 3b, **Golden Signals and Event Log** (`#s-feed`).

## 10.1 Golden-Signal Feed, Last 240 Decisions · Class S

The golden-signal feed of the page's section 3b, `#s-feed`, streams the live organism's four golden signals, native beside Omni-Compass: errors (muscles over the line), throughput (work done) and, because neither body has a response time or a machine's busy share, latency's and saturation's stand-ins: on the live model the work waiting in line and the capacity in service, on the realms harness the spread of p across the muscles and the electrical power drawn (kW). One point per decision streamed from the live benchmark engine, gapless, the last 240 decisions, each signal on its own axis, with each side's last, mean and maximum as a time-series dashboard reports them; a kill-switch window shaded red; native in orange, Omni-Compass in blue, the same demand on both. The feed is the status board's signals at full size and full history.

## 10.2 Engine Event Log

The live benchmark engine's own events as it emits them, newest first, with the wall-clock time and the decision: a threshold breach and its clearing on either side, every injected fault and its end, the kill switch, the dial, a new day, and each organ's hand-back grant withheld or opened again, at that organ's own threshold θ (pods 0.5, nodes 0.7, the clock, the card and power 0.3, routing 0.5 at the factory: `omnicompass/nervous_system.py`). The log is what an operator reads to answer "what happened at 02:14": the decision number ties it to every other monitor.

Its buttons filter it by grade: *All*, *ALERT*, *WARN*, *INFO*, as an operations log grades its events.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 10.3 OpenMetrics Exposition

The live benchmark engine's metrics in the OpenMetrics text format, the exposition format Prometheus scrapes and Grafana charts, refreshed every two seconds. They are the same families, under the same names, as the fuse box's Prometheus Metrics tab prints (section 2.25): the master switch, the decisions, the plant's knobs, work served, failed and waiting, the muscle-decisions past their line, writes, capacity, actuation, band time, actuator moves and reversals per side, and frame time; then the verdict's keep ratio at each true cost (chapter 16), the three-run tables' rows by class, by the engine each table ran on, by the way it compares and by reading, and the build's engine, digest and commit. *Copy the exposition* puts the text on the clipboard, so the page's own metrics can be pasted into a scrape target and charted beside the stack's.

CHAPTER 11

# Control Console

This chapter reads the page's section 3c, **Control Console** (`#s-desk`).

## 11.1 What the console is

Runtime parameter control and fault injection on the live benchmark engine: load scaling, noise injection, sensor latency, capacity loss, a demand surge, a sensor blackout, a security hold, the dial, and the kill switch. Every control is applied to both sides identically; the response is plotted per decision, native beside Omni-Compass. The engine these knobs turn is the engine's own mathematics running in the page: the brain, the compass law and the verdict, proven equal to the Python and the C++ on every build. It governs the stress-test model's sixty muscles, each on top of its own native controller. The knobs move the live benchmark engine in the page's sections 3a, 3b, 3e to 3g, 3i and 5h, and the kill switch and the frame load also move the page's own render governor (the page's section 5j).

## 11.2 Factory Tune

One button, on the console under the kill switch (its knob title **Factory Tune**), on the dock, in the control panel's first panel and in its fuse box, and in every monitor's own knobs, puts back two things at once: the engine's tune as it ships (`DISCLOSURES.md`, section 3: the dial at the max, every muscle driven, Omni-Compass ON, the 60 s beat, the factory profile restored and any hold released, the mechanism's knobs at the factory) and the page's plant at rest (the day as written, demand ×1, noise σ 0, sense delay +0, no fault held, the load profile as written, speed 1×, running, no frame load, and the heartbeat sweep monitor on 60 s, on-site, pods waiting, against native). It says what it put back. Turn anything, break it, watch it break; this button brings all of it back. Native alone is not a tune: it is the kill switch. The control panel's own factory button also turns every breaker ON.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 11.3 Do no harm, at the console

Every control acts on both sides alike; what Omni-Compass does with them is held by its guards: no muscle driven below native or past its own limit, nothing taken from one muscle to give to another on the engine that stands, a give-back tried against native first and refused on a LOSS. Its one meaning is written once, in `DISCLOSURES.md`, section 3, and the page's appendix G carries it.

## 11.4 Every control

Appendix B lists each control in a table. In reading order:

| Control | Marked | What it does, on both sides |
|---|---|---|
| Kill switch (*Turn Omni-Compass OFF / ON*) | LIVE | every lever to native at once; the window shaded red; every Omni-Compass side follows native |
| Factory Tune: every knob back to the factory tune | LIVE | section 11.2 |
| Tune Dial: the max / tune-down | LIVE | the max: every muscle to its own limit, never below native; tune-down: never more than native, capacity handed back only with the organ's grant and the release gate |
| Demand (×0.25 to ×2) | LIVE | scales every muscle's demand from this decision on; ×1 is the day as written |
| Noise Injection (σ 0 to 0.6) | LIVE | multiplies each muscle's demand every decision by 1 + σ·z, z a fresh normal draw, one draw per muscle shared by both sides |
| Sense Delay (+0 to +6 decisions) | LIVE | adds decisions between what a muscle does and what its controller reads, on both sides: native's controllers and Omni-Compass's law read the same late number |
| Faults: the body part; Surge ×2 | LIVE, 40 decisions | doubles that body part's demand |
| Lose half its machines | LIVE, 40 decisions | halves a pool's capacity in service |
| A sense goes blind | LIVE, 40 decisions | forbids any hand-back (blind means hold) |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Control | Marked | What it does, on both sides |
|---|---|---|
| Security hold | LIVE, 40 decisions | forbids any addition |
| Pause, One decision, 1×, 4×, 16×, 60× | LIVE | the console's clock: a decision every 0.9 s at 1×; one decision steps once |
| The same day again, A new day | LIVE | replays the seed with the knobs as set; a new day draws a new seed |
| Every knob to its start | LIVE | returns demand, noise and delay to the day as written |
| Frame Load (as it is, +8 ms, +16 ms) | LIVE | burns the stated time inside every frame of the page, both sides alike, for the page's own render governor to hold (chapter 34) |
| Load Profile (the day as written; a step ×1.6; a ramp ×1 to ×2 over 120 decisions; a sine ±40%, period 60 decisions; the Azure bursts `1 4 1 6 1 9`; the real day on 1 to 9; a sequence of the operator's own), *Inject* | LIVE | the same multiplier on every muscle and both sides; a sequence holds each value for 20 decisions and is scaled to its own mean, so the day's mean load is kept |

## 11.5 What a fault shows

Press *Surge ×2* and watch the page's section 3f: both sides' dots for that body part move up and right; native's controllers each see their own reading one to three decisions late and add in the middle of their range; the compass law pushes at the first decision the position leaves the center, so the blue dots turn back before the right edge while orange ones cross it. Press *A sense goes blind* and the governor's hand-backs stop while its additions go on (blind means hold, never a guess). Press *Security hold* and every addition stops while hand-backs go on. In every case the native side is put through the same fault, so the difference is Omni-Compass alone.

## 11.6 The console's own monitors and the dock

Under the knobs the console draws **System Response per Decision · Class S** (native in the left lane, Omni-Compass in the right on the same scale, every knob turned marked at its decision and a kill-switch window shaded), the **Inter-Frame Interval** of the page against the 60 Hz budget, the **Control Event Log** (newest first: every knob, switch and button pressed, with its decision), and the **Control-Loop Performance Assessment** of the day, native beside Omni-Compass: the threshold breaches this day, the recovery time after the last injected fault, the throughput, the time inside the operating band, the mean absolute deviation from the band's centre, the peak band position, the actuator moves and the direction reversals of p. Each judged row

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

reads by the key's floor (section 36.4); the band statistics are shown, not judged; the moves and reversals are resources, power given where they rose.

The dock at the foot of the screen keeps the console in reach from anywhere on the page: the kill switch, the factory tune, the state of the switch, the scorecard's number against native (Omni-Compass on native's change in work in the real three-run test), the wall's worst signal with its counts, the decision, the muscles past the line on each side, the frame time, demand and noise, a surge, a lost pool, pause, a way to the control panel (`#s-health`) and the console, and a button that folds it.

CHAPTER 12

# Heartbeat Dial

This chapter reads the page's section 3d, **Heartbeat Dial** (`#s-dial`).

## 12.1 The dial

The governor's heartbeat, its full decision, is a dial from omni-v17 on (the page's section 3d, `#s-dial`): the main engine, the factory profile, decides every 60 s, and on the page any of six beats can be set, 10, 15, 30, 60, 90 or 120 s, with the fast checks (the scheduling floor, the pod reflex, the conveyance every 15 s) every 5 s at every beat. Every clock of the governor counts seconds (omni-v16), so turning the dial changes only how often it looks. Each beat stands beside native with what it buys (the reads of the cluster a day) and what it costs (the governor's own processor, and every service row read against native and against the main engine's 60 s), because an operator may want a fast beat for its reads and accept what it costs, or a slow one. At each beat the other knobs may want their own combination; that tuning is the next study, in our own models first.

## 12.2 Beat Ladder: the two tables

Its panel, **Beat Ladder · Class S**, gives every beat in two tables for each setting the models read (**Cloud Setting as Read 22 and 23 October: the Tune-Down, Machines Alone (Struck 31 October)**: the compass law at the tune-down, the machines alone, the cloud's rule when these records were read; **kind Benchmark at the Max, Autoscalers Sensed**: the compass law at the max, every autoscaler sensed). The first, **Against Native**, gives each beat's value and its word against native: the reads of the cluster a day, the governor's own processor, the pods waiting for a place, the 95th percentile, the minutes over the line, the autoscaler short of its target, the machine-hours and the energy per core-hour; native has no governor, so its reads and processor are blank, and the main engine's row is set in bold. The second, **Against the Main Engine's 60 s**, gives each other beat's change against it, with its word. The reads are what a beat spends to look more often, shown as the key reads a resource spent: power given.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 12.3 Reading a row

Read a row across: a beat whose service words read GAIN or STEADY against native and whose cost columns the operator accepts is a beat that can be set. A word of LOSS against native is the engine's at that beat and is listed with every other LOSS row (the page's section 2d, chapter 7). The values are our own models (class S, `results/model_gate/V17_LADDER.json`, `tools/beat_ladder.py`), each beat's mean over seeded days on a plant that moves every 5 s, paired by seed; the real runs at every beat (`docs/PACE_PREREGISTRATION.md`, amendment 3) take their place in the note above the tables when their three runs land. The dial is the tuning bench's own example: the factory profile runs the main engine at 60 s; any other beat is the engine's range on the page, never a benchmark result, until its own three runs land on that beat and are read as that beat's. A beat set on a live system is the operator's own custom profile, and its results are its own.

## 12.4 Where the beat is set

The beat is set on the governor's command line (`--interval`) or with the control panel's heartbeat button (`POST /v1/beat`, `tools/omni_control.py`: one button for each stop of the dial), which hands every setting back and starts the governor again at the new beat. The governor's own screen in Grafana counts the time since its last decision in beats of the setting it runs at and alerts after three (`omni_heartbeat_seconds`, `deploy/grafana/omni-compass.json`, `deploy/prometheus/omni-compass-rules.yaml`). On the page the heartbeat buttons of the control panel set the live model's beat (section 2.5); the dial's tables are records and do not move with them.

CHAPTER 13

# Control Cycle

This chapter reads the page's section 3e, **Control Cycle** (`#s-decision`).

## 13.1 Decision Sequence

**Decision Sequence**: one control cycle traced end to end at reduced speed: the sensed inputs, the brain's decision, and the actuator (the muscle) it wrote. The max: every muscle to its own limit, never below native; the capacity it holds is power given. The tune-down: never more than native, and a step back only with the organ's grant and the release gate. In this model native already sits near its band, so the tune-down finds little to hand back; the scoreboard (chapter 14) reads its runs A, B and C live when the dial is on the tune-down. Class S.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 13.2 How to read it

Follow the particles: in along the wires from the muscles' meters to the inner arc of nine senses; through the brain's six spokes; out through the nerves (the grant), the compass (the force) and the plug (the write). Each stress on the console lasts 40 decisions from the moment it is pressed, and both sides meet the same demand, so a cycle watched during a surge shows the push on the blue side and the native controllers' late, middle-of-the-range moves on the orange one.

## 13.3 Cycle Controls and the muscle's card

The section's panel **Cycle Controls** shares the console's state: **Body**, *Each Muscle Alone*, *One Organism* or *Stress-Test Model*, each native, then on top; **Run**, A · 3000, B · 3001 or C · 3002 (the harness record's seeds); **Play**, pause, 1× to 60×, a new day, and *Run A, B and C*; and, on the stress-test model, **Tune Dial** and **Stress** (a surge on the compute realm, a sense gone blind, a security hold, the kill switch). The card beside the cycle follows one muscle: its band, Omni-Compass's force F (solid blue to add, pale blue to hand back), native's lever and the lever on top, and its own switch: *this muscle*, pressed, hands that one muscle back to native's own controller while the rest stay governed, and *Every muscle back under Omni-Compass* restores them all; the kill switch hands every muscle back at once.

CHAPTER 14

# Phase Plane and Scoreboard

This chapter reads the page's section 3f, **Phase Plane and Scoreboard** (`#s-plane`).

## 14.1 The two phase planes

A phase portrait, the picture control engineers use for a system's state. Across is where each muscle sits in its band: p is 0 when calm, 0.5 at the center and 1 at its line. Up and down is how fast it moves there (v, per decision). A muscle at rest sits on the middle line. One climbing toward its line moves up and right, and past the right edge it is past its line. Native is on the left and Omni-Compass on the right: the same muscles at the same instant, one dot each, with an arrow from where native holds a muscle to where Omni-Compass holds it, and a red ring, drawn as an octagon, round a muscle past its line. In the stress-test model the shading on the right is the force the compass law would apply there: the add force toward the line, the hand-back force toward calm. In the realms harness each muscle answers to its own rule, so no single field is drawn.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

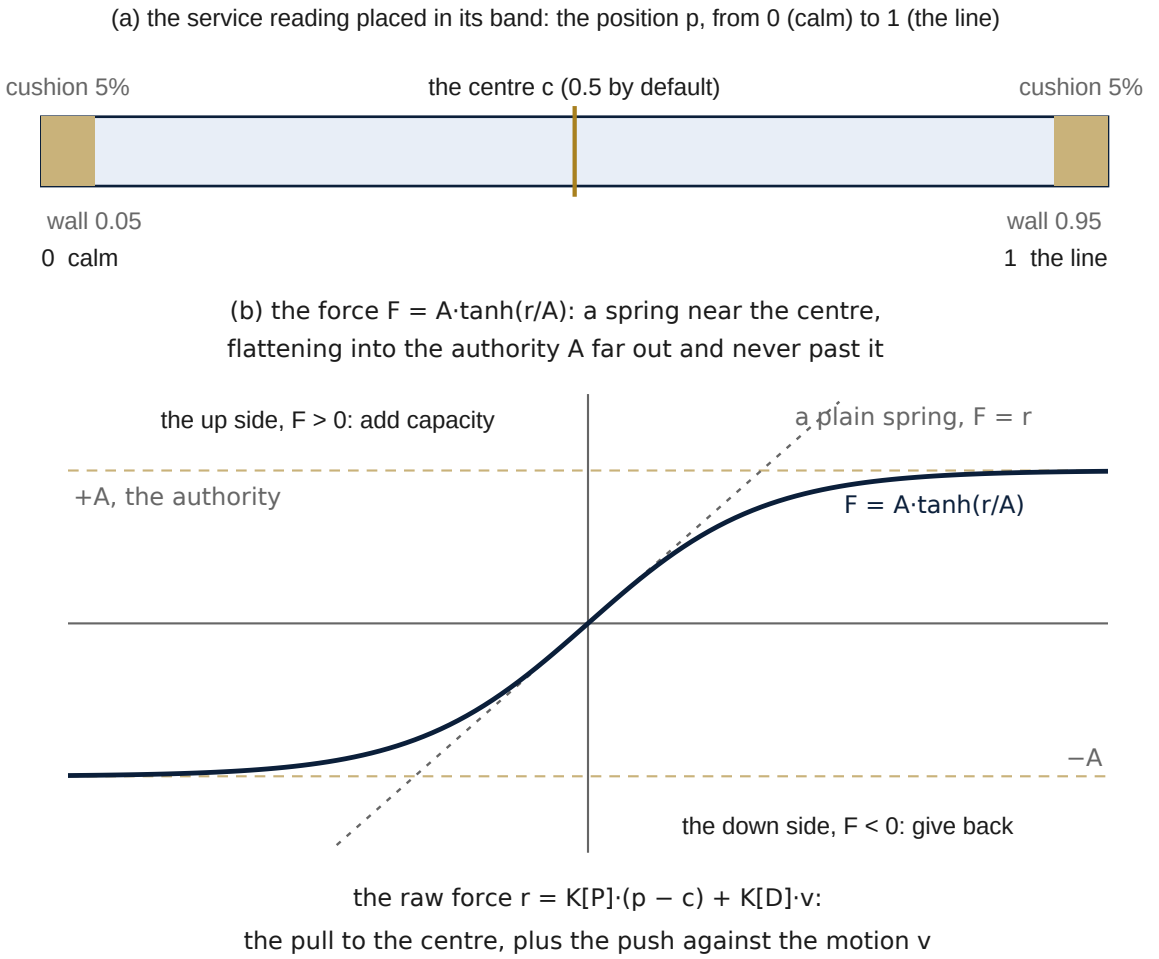



**Figure 2.** The compass band and the two forces. A reading is placed in its band, p = (r − lo)/(hi − lo), calm at 0, the center at ½, the line at 1, with 5% cushions at each end. Two antagonist forces act on the knob: the add force as the reading climbs toward the line, the hand-back force as it falls toward calm, combined as F = A·tanh((kₚ(p − ½) + k[d]·v)/A), smooth, never hammering. It fails up past 0.95 and never hands back while it climbs above the center. The phase plane's shading is this force at every point.

## 14.2 Stat Panels, Last 120 Decisions

Each of the **Stat Panels, Last 120 Decisions · Class S** is one gauge of the live organism, read every decision on both sides, as an operations dashboard shows it: native's value now and Omni-Compass's, the last 120 decisions of both (native solid orange, Omni-Compass blue), and the running paired reading in the tables' words. With the master switch OFF the decisions are shaded red and Omni-Compass's side follows native.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 14.3 Running Scoreboard: how each row is read

Each row of the **Running Scoreboard · Class S** is the paired change, native against Omni-Compass at the same decision, averaged over blocks of ten decisions so neighbouring decisions do not count twice, with its 95% interval (Student's t over the blocks). *This run*: a change under 0.5% of native either way reads STEADY, and from 0.5% up the word follows the gauge's aim, whatever the interval says (work: a rise is GAIN; misses, failed work, waiting: a fall is GAIN; energy or capacity held: a rise is power given, a fall a cut). *Three runs*: the change is the mean of runs A, B and C; a mean under 0.5% reads STEADY; from 0.5% up the word by the gauge's aim; runs that moved 0.5% or more in opposite directions disagree (section 36.4). The spread of positions and the settings are shown, never judged: no rule of the engine aims at them. The spread can widen a little under the tune-down, because the calm muscles it hands back move toward their line while the busy ones stay.

CHAPTER 15

# Brain State

This chapter reads the page's section 3g, **Brain State** (`#s-brain`).

## 15.1 What is drawn

The brain's six state variables rendered on their potential-energy landscapes: a state resting at a minimum is stable; a state driven toward a maximum is under load. Each ball is a live value of the engine's brain, integrated by the Runge–Kutta method every decision. In the stress-test model its push and target feed the body's levers. In the harness views the harness arm acts muscle by muscle with no brain, so the brain drawn here is the page's own, fed the harness's readings. Class S.

## 15.2 The three potentials the page prints

**Double-Well Potential of U, Eq. (2)**, the utility's well, $V(U) = -\mu(U^2/2 - U^4/4) + (\dot{E}/E_{max})\cdot U$: a rising stress E tilts the well toward the basin of collapse, and the ball rolls toward it unless the body is relieved. **Cubic Potential of Stress S, Eqs. (5)–(6)**, the stance's well, $\Phi(S) = \alpha_s S^2/2 + \beta_s S^3/4 - \delta S$: S rolls to its minimum S* and the senses pull it back each decision. **Bath Oscillator B, Eq. (7)**, the tide, $\dot{B} = \gamma[c]\cdot\delta\cdot S - (\omega[B]/Q[B])\cdot\dot{B} - \omega[B]^2\cdot B$: the slow tide of physical load, a damped oscillator driven by the stance.

## 15.3 Nerves and Calm

$calm = \kappa\cdot h\cdot(1 - (\sigma - 1)^+)\cdot(1 - \min(v, 1))$, where κ is the convergence (1 while the push is at or under its knee, else the knee over the push), h the basin health ((U − U[gate])/(1 − U[gate]), held to [0, 1]), σ the stress ratio

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

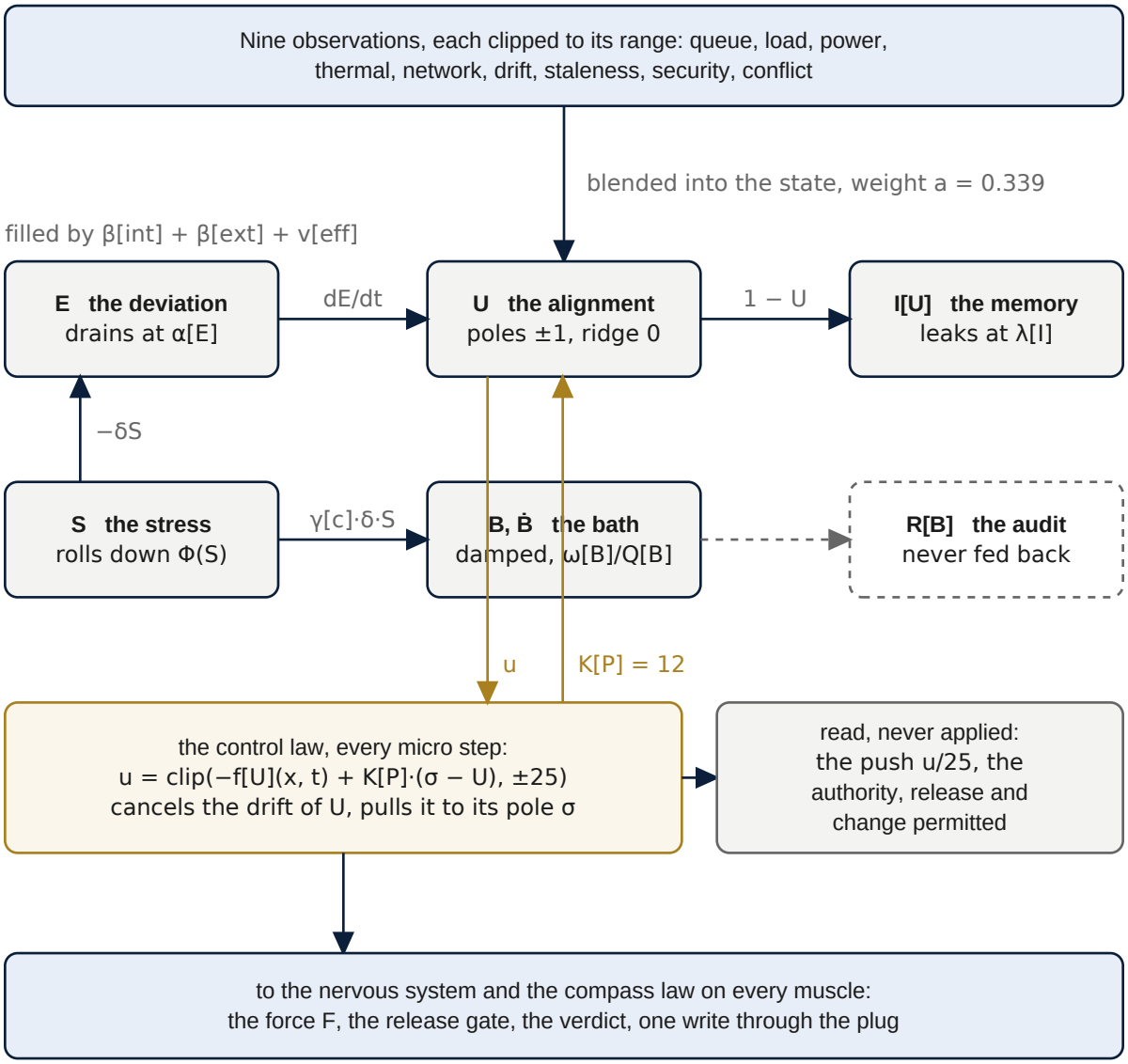



**Figure 3.** The engine's six states. E the stress, U the utility, I[U] the memory of need, S the stance, B the slow tide of physical load and Ḃ its rate, each drawn on its own scale (E, U and I[U] on [0, 1], S on [−1, 1], B on [−0.5, 1], Ḃ on [−0.5, 0.5]). The brain monitor draws each as a ball on its own landscape, live, every decision.

S/S* and ν the unmet need (I[U], never below 0); the monitor draws the four as meters beside calm. An organ may hand capacity back only when calm ≥ its threshold θ, the line is clean and every sense is live; expanding stays allowed except under a security hold. Under the max, lifting off toward native never needs this grant; going below native does. The monitor draws calm against θ for each organ, so the operator sees why a hand-back was refused. The four scalars and the grants are the nervous system of `docs/FACTORY_PROFILE.md`: observation, signal propagation, coordination and authority pathways.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

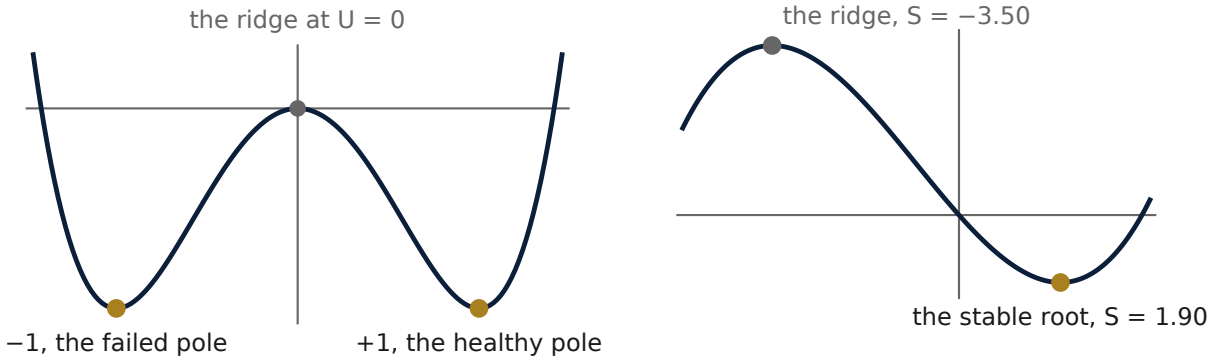
(a) the alignment U, a double well:

$V(U) = \mu(U^4/4 - U^2/2)$

(b) the stress S, a cubic potential:

$\Phi(S) = \alpha_s S^2/2 + \beta_s S^3/4 - \delta S$

($\alpha_s$ 0.12, $\beta_s$ 0.10, $\delta$ 0.5: the engine's defaults)




$dU/dt = \mu U(1 - U^2) + \ldots$

the ball rolls to the nearer pole

$dS/dt = -d\Phi/dS$: S rolls to the stable root;

the landscape is open below the ridge

**Figure 4.** The double well. The utility U rests in one of two basins: the working basin, where the body does its work inside its lines, and the basin of collapse. The stress E tilts the landscape; the brain's push and the body's relief tilt it back. A ball near the ridge between them is a body under load; the compass law and the nerves exist to keep it on the working side.

## 15.4 Native Controller Trace

Native has no brain to draw: each of its controllers reads one reading, its own muscle's, and moves. The orange panel, **Native Controller Trace · Class S**, draws, live from the same decisions, the mean position native's muscles' readings put them at, the share of its muscles past their line, and how many of its own controllers moved their setting this decision. Native has no nerves: nothing grants or refuses a controller, and a blind sense or a security hold changes nothing it does.

CHAPTER 16

# Verdict

This chapter reads the page's section 3h, **Verdict** (`#s-verdict`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 16.1 The change-acceptance test

Before a knob moves one step past native, the step runs as a trial: costs at native first, then at the step. The engine reads the change exactly as a report reads a row, on the log of each cost, with its 95% interval. Each step is kept unless the measurement reads LOSS or its estimated cost exceeds the 2% allowance; a STEADY result (no difference resolvable from the noise) is accepted, so on a high-variance workload a step with a small true cost can pass, and the lab under the monitors measures that false-acceptance rate. Its two monitors are *Trial Costs and 95% Interval* (the panel **Current Trial**) and *Share of Trials Allowed*.

## 16.2 The lab

Three knobs: *what the step truly does to the cost*, the *allowance* (tolerance) and the *noise σ of the log cost*, with the *pieces of work measured each side*. Set what the step truly costs and watch the rule decide: the keep ratio at each true cost is drawn, beside a rule of medians alone for comparison (the medians alone allow a step when the trial's median is within the allowance of the reference's; the engine's rule also refuses any step whose interval sits wholly above zero, and measures on while the interval is wider than the allowance, up to three times the pieces each side, keeping a step only if both estimates are inside the allowance).

The lab opens wired as the shipped Kubernetes controller is wired, on the noise the real runs measured; the **Wiring** buttons choose *Kubernetes, Google 2011 Day*, *Kubernetes, Load Swing 1 to 8*, *GPU Card* or *Model Gate*. *Run one trial* and *Run 300 trials* run the rule at the cost set; a new trial starts on its own every few seconds.

## 16.3 How each muscle wires it

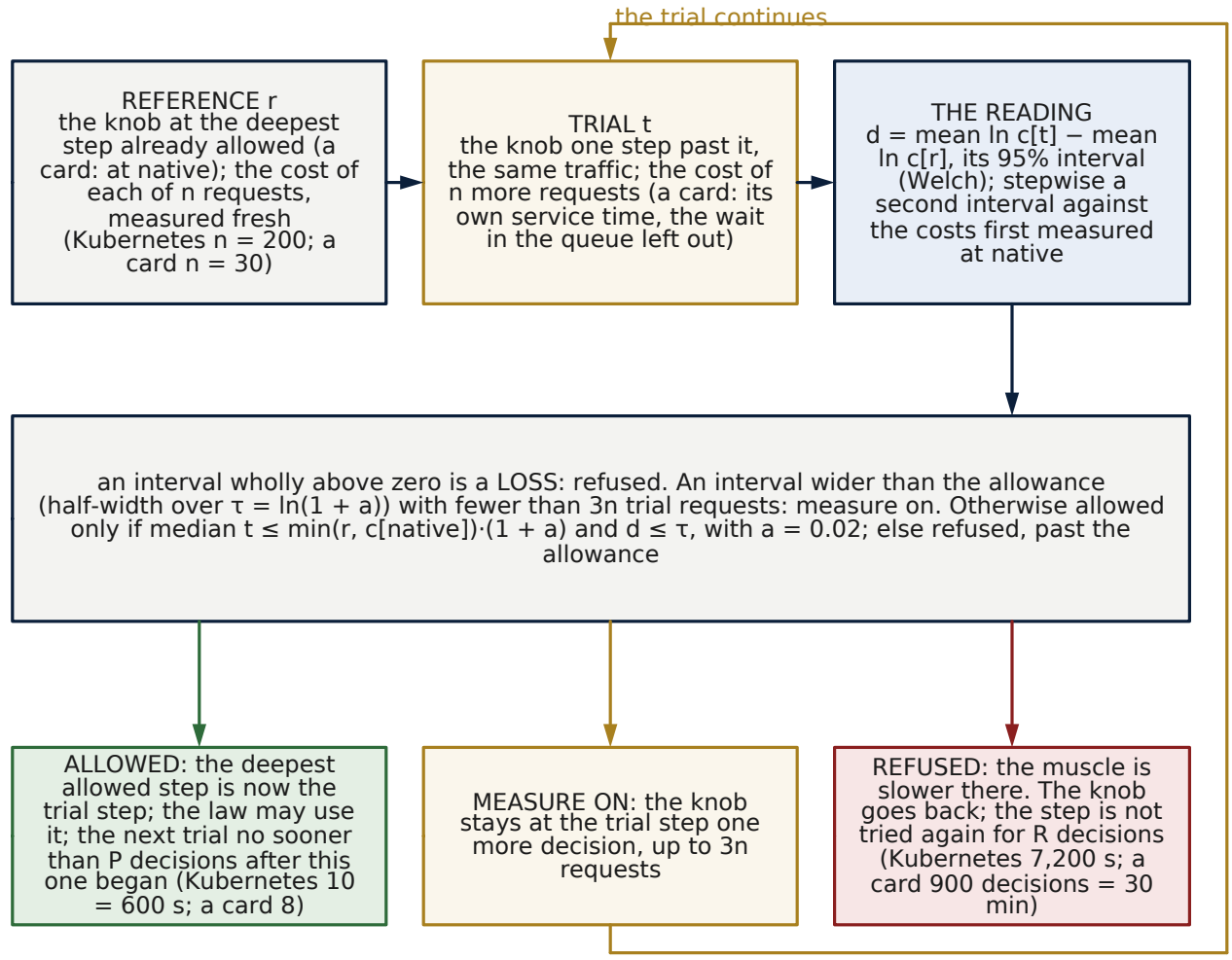
The Kubernetes controller measures 200 requests each side against a 2% allowance (`--verdict-samples 200 --allow 0.02`), the cost being each request's response time; the noise is measured at every build on each request of the newest tables' own runs. The GPU card measures 30 requests; in its model the card's own time per request carries no noise. The model gate measures 60 (σ 0.25). Native has no trial: its knob never moves past its own setting, so there is nothing to judge; every step Omni-Compass takes past native has to pass this test first.

CHAPTER 17

# Dashboards

This chapter reads the page's section 3i, **Dashboards** (`#s-dash`). It draws the same live benchmark engine in nine forms an operations team already reads, each fed by the page's own model as it runs (class S), native in orange beside Omni-Compass in blue, each change with its word by the key. The forms are the industry's; the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.




The verdict stands over every slow knob (the machines, the clock ceiling, a cache's notch). A trial begins only while the service is calm and nothing waits; a phase that sees the service leave calm, or that takes longer than its limit (Kubernetes 1,200 s; a card 30 decisions), is abandoned and the knob goes back, the step neither allowed nor refused. The knob never passes the deepest allowed step, so steps cannot add up past the allowance. On the max, the Kubernetes machine count never gives back a machine the cluster had on its own.

**Figure 5.** Block diagram: the verdict's paired trial around a slow knob, read GAIN, STEADY or LOSS. The reference is the cost of n pieces of work at the deepest step already allowed; the trial is the cost of n pieces at the step. Both on the log of the cost, with the 95% interval of their difference: an interval wholly above zero refuses the step; one inside the allowance keeps it; one wider than the allowance measures on, up to three times n each side. The verdict monitor runs this rule live at any true cost the viewer sets.

numbers are this engine's; no vendor's product is drawn or implied. On a phone each drawn form also writes what it shows as plain lines under the drawing.

## 17.1 The nine forms

The section's **Form** buttons choose one of nine forms:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

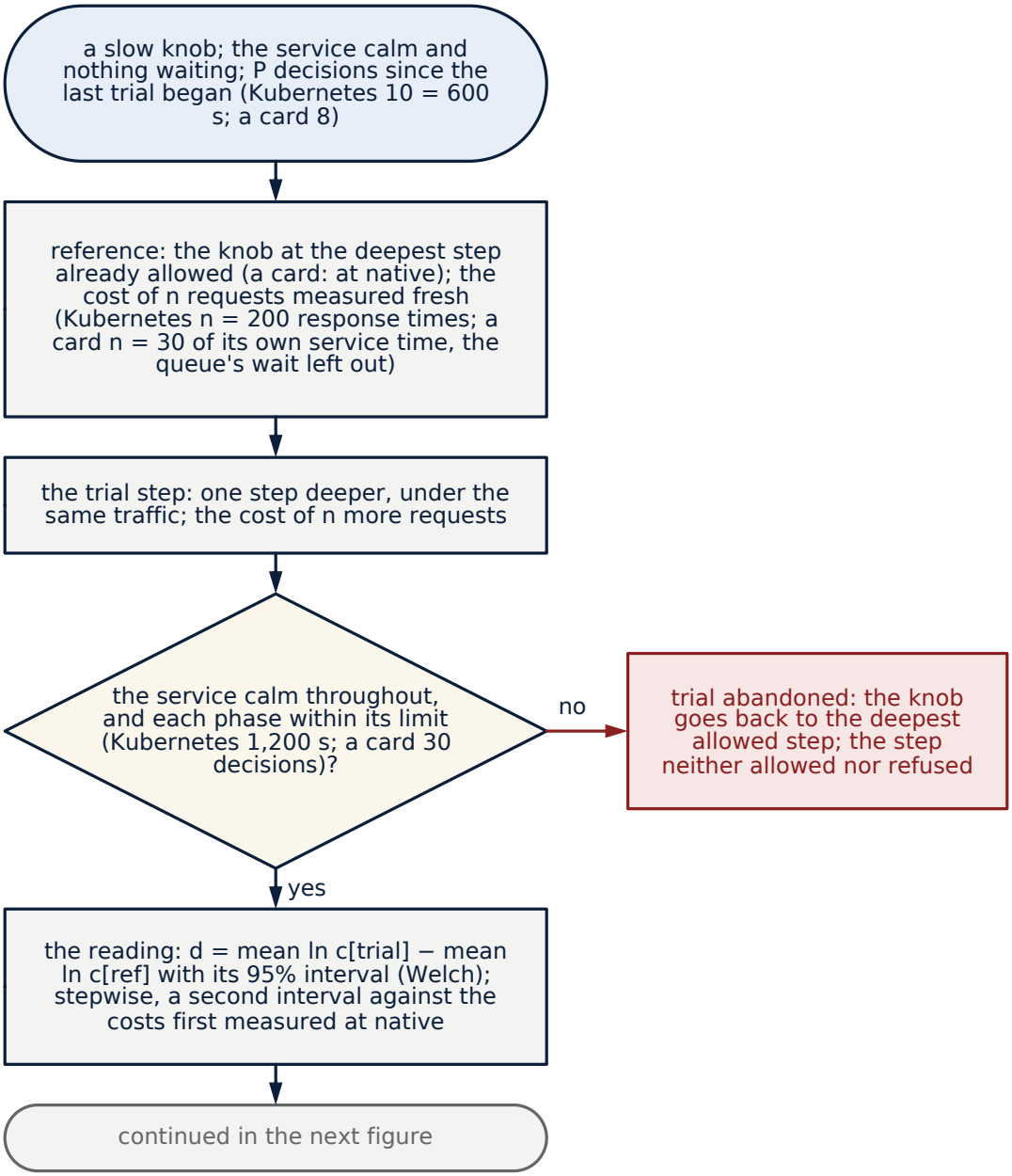



**Figure 6.** Flowchart: the give-back trial. A trial starts only with the organ's grant and a clean line; it runs to its full measurement and is never ended by the calm it causes; its reading against native decides, GAIN or STEADY keeps the step, LOSS refuses it and the setting stays; the release gate then passes or holds the write. The ticker's state column reads left native until a trial starts and acting while one runs.

| Form | What it shows |
|---|---|
| Board of Panels | stat panels (muscles past their line, mean load, writes this decision, work so far), two time series (muscles past their line; capacity held) and a bar gauge per body part |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Form | What it shows |
|---|---|
| Host Map | every muscle a cell, one row per body part, shaded by its load against its line; a cell past its line is the stop sign; native's map beside Omni-Compass's |
| Graph of One Series | one series over the decisions with the PromQL a running system's wall uses for it, and the change by its word |
| Event Stream | every decision as a line, newest first: each side's requests, cuts to a muscle's range and muscles past their line |
| Cluster Console | each body part a node, each muscle a workload, load and state on both sides; each row's button leaves the muscle native or drives it again and shows the request a running governor takes (`POST /v1/muscle`) |
| One-Line Diagram | the demand feeding five buses through twelve breakers each: closed, a muscle Omni-Compass drives; open, left native; tripped, past its line |
| Wall of Numbers | the decision, the heartbeat, the switch, muscles past their line, work waiting and failed, writes, native beside Omni-Compass |
| Cost by Body Part | the capacity held, body part by body part, the quantity a bill multiplies by its price; no price is recorded for the model, so no money is printed |
| Two-Way Pipeline, Omni-Compass's own form | demand, muscles, senses, readings, the brain, the constraints, the plug and the read-back, each edge with this decision's count; native's controllers and the kill switch beside it; the exporter carrying it all to Prometheus, Alloy and the wall |

## 17.2 Level, range and series, and the two tables

The **level** gathers each form by muscle, by body part or for the whole organism (*Each Muscle*, *Each Body Part*, *Whole Organism*); the **range** sets 20, 60 or 240 decisions; the **series** chooses what the graph draws. Under the forms the section's table, **Roles in the Control Stack**, gives the words for every tool a reader may know (an observer, a collector, a console, a native controller, an automatic optimizer, a supervisory governor), who decides in each, well-known examples and where Omni-Compass sits beside each; `docs/FACTORY_PROFILE.md` carries the same table, and `docs/THE_FIELD.md` maps the products by name. A second table, **First Five Products to Benchmark**, names the first five benchmarks against products that decide and act on their own (ScaleOps, StormForge Optimize Live, IBM Turbonomic, CAST AI and Zeus), each with its rig and its cost as read on 24 October, and each marked not yet run.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART FOUR

# Real Runs and Platforms

*The page's Part 4: the real runs played back, and the platforms they ran on.*

CHAPTER 18

# Kubernetes Run Playback

This chapter reads the page's section 4a, **Kubernetes Run Playback** (`#s-ops`).

## 18.1 What plays

A real-time operations dashboard over the benchmark runs themselves (class L), not a model. Every series streams half minute by half minute out of the raw files GitHub archived, each file's SHA-256 matched before a byte is drawn: native (the whole cluster with everything it ships with) against Omni-Compass, the same load, the same second. Pick a benchmark, a run (A, B or C) and a repetition at the head of the section or in the page's section 4b; the tiles, the candles, the heatmap, the bars and the tickers all move together with it. The **Data Pipeline** is its own monitor: every stage from the GitHub Actions run to the page (the run, the raw files archived, the SHA-256 check, the official scorer, the three-run tables, the build, the page) drawn whole, the stage being read lit and the data moving between them.

## 18.2 Governor Heartbeat and Lease

A supervisory governor is only alive while its loop turns: once every decision it reads the whole body, decides, writes, and proves it is still there. That turn is the heart. Each beat pumps every reading in through the plug and the decision out (`omni_controller/controller.py`, its main loop) and renews the governor's lease (`master.heartbeat()`, `omnicompass/master.py`). If the heart stops, its process gone or its lease older than its limit (the larger of 120 s and five beats), the watchdog (`tools/omni_switch.py watchdog`) walks every setting back to native: a stopped heart leaves the body exactly as native runs it. Native has no heart of its own: each of its controllers keeps its own clock, the pod autoscaler one sync every 15 s, and nothing proves any of them is still there.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

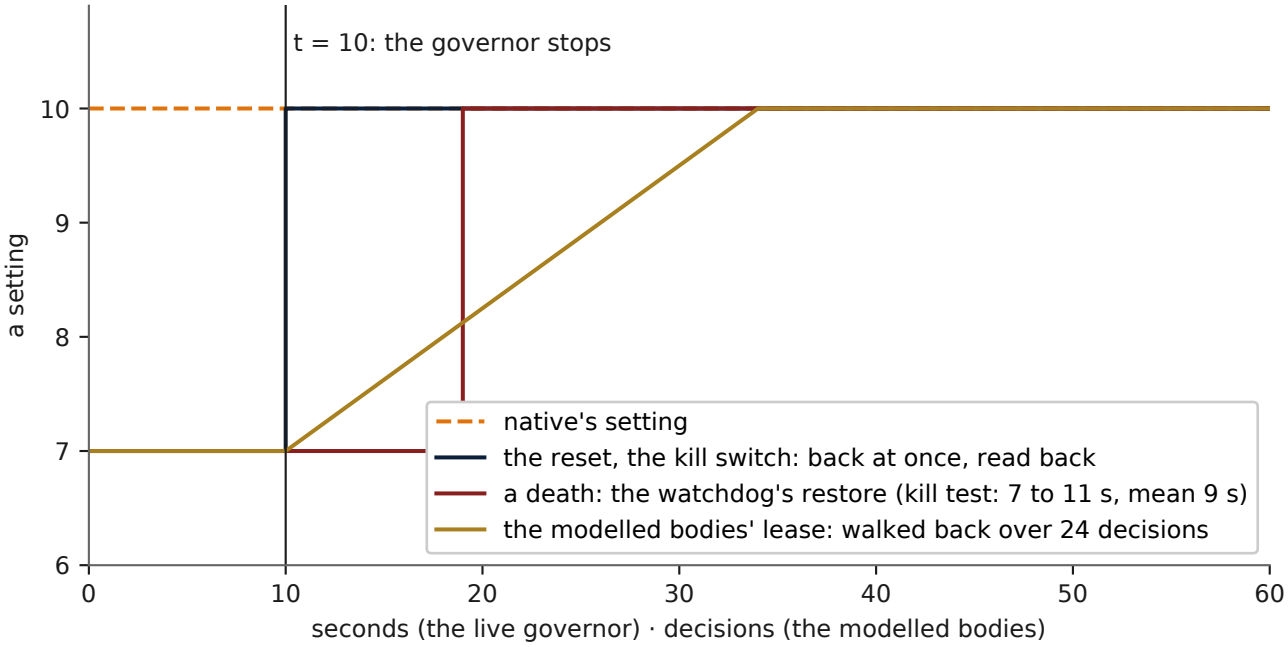



**Figure 7.** Timing diagram: the lease. The governor renews its lease on every beat; the watchdog reads the lease on its own clock. When the governor dies without handing back, the lease ages past its limit (the larger of 120 s and five beats) and the watchdog walks every setting back to native, a step at a time, reading each back. On the page, the heart monitor shows the beat and how close the lease came to its limit; a stopped heart leaves the body exactly as native runs it.

## 18.3 A healthy heart, never pushed

The beat runs at a fixed, calm rate (the factory profile's full decision every 60 s since omni-v17, omni-v16's every 15 s, the fast checks every 5 s between beats; the runs replayed here ran on the engine each table names, omni-v12 and omni-v21, each at the beat its run's own files record) and is never driven faster by the load it governs. Every clock of the engine counts seconds, so the beat's pace sets how often the body is read, never what a rule means: a give-back still waits its trial's minutes, the cushion still keeps its twenty minutes of memory, and a machine still goes back at most one a minute. What it costs is counted inside the body's own load (the row *Omni's own CPU*), so the organism never pays for its brain without knowing it. A decision that fails is recorded and skipped: it writes nothing, the body keeps the last settings that landed, and the next beat comes at its normal time. Two gauges read its health: the share of each beat spent deciding, and how close the lease came to its limit. The page calls a heart under strain when a decision takes more than half its beat, or a gap between beats reaches 80% of the lease. Both lines are the page's reading, not rules of the engine; the engine's own guard is the lease and the watchdog.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 18.4 Heartbeat Sweep

Under the heart stands the heartbeat sweep (the panel `#beatPanel`, monitor *Heartbeat Sweep · Class S*). The beat is a setting of the run, not a constant: the factory profile sets 60 s since omni-v17 (omni-v16 set 15 s, the pod autoscaler's own pace), and an operator may choose another for a system's own reasons (a slower beat reads the cluster less often; a faster one answers sooner). The monitor shows what each choice does. Its five rows of buttons pick the record to read:

- **Beat:** 5, 10, 15, 20, 30, 60, 75, 90 or 120 s.
- **Setting:** on-site, where the governor sizes the pool on top of the pod autoscaler, or in the cloud, the machines alone (the cloud's rule when the sweep was read on 22 October, struck on 31 October).
- **Engine:** omni-v16, every clock in seconds, or omni-v15, its rules counted in beats.
- **Against:** what each reading is set against: native, the same engine at 15 s (omni-v16's factory beat), or the same engine at 60 s (omni-v17's). The sweep's record is omni-v16's and omni-v15's; the dial's ladder (the page's section 3d, chapter 12) reads omni-v17. The monitor opens on 60 s, the factory's beat on the engine that stands.
- **Gauge:** *Pods Waiting* (pods waiting for a place), *Machines* (machine-hours), *Energy per Core-Hour*, *95th Percentile* and *99th Percentile* response, *Alert Minutes* (the minutes over the line), and the engine's own cost: *Engine CPU per Day*, *Cluster Reads per Day*, *Writes per Day* and *Fast-Check Rescues per Day*, the machines its 5-second check added only after a pod already waited.

The tiles give the chosen beat whole. They show what it is read against and every gauge's mean change with its 95% interval and its word (GAIN, STEADY, LOSS or power given, by the one key); the number carries the word's sign (GAIN +, LOSS −, STEADY =), never the gauge's raw direction. The last tiles give the engine's own cost a day; native, which runs no governor, costs nothing there. The rescues tile says why a slow beat waits more: the full decision sizes the pool ahead of the load, and read less often, the load outruns it between decisions, so the 5-second check adds the machine only once a pod already waits.

The chart gives the chosen gauge at every beat. Each bar is the mean change over the seeds, coloured by its word, with its whisker the 95% interval. A fall cannot pass −100%, so an interval reaching past it is cut there and marked ▼; the tiles print it whole. The beat chosen is framed, and a cursor reads the beats in turn and names the row it holds.

Every number is our own model (evidence class S), never a benchmark result. The shipped controller of each engine runs on the modelled cluster's 5-second plant, eight seeds of a day each, paired by seed and read by the model gate's own rule (`tools/heartbeat_sweep.py`; the record, `results/model_gate/HEARTBEAT_SWEEP.md`). Every beat on this monitor is read in the model only: a real run reads the one beat it was started with, the governor's `--interval`, and the gauges `omni_decisions_total` and `omni_own_cpu_cores` of `tools/omni_exporter.py` show the beat it runs at and what it costs.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 18.5 Response Time Candlesticks, Latency Heatmap and Throughput per 30 s Bin

**Response Time Candlesticks**, one candlestick per 30-second bin (open, high, low, close of the responses in the bin), native beside Omni-Compass; **Latency Heatmap**, every response binned by time and by duration, the darker the fuller; **Throughput per 30 s Bin**, requests served per 30-second bin. Read the three together: a GAIN in the 99th percentile shows as the candles' upper wicks shortening on the blue side while the heatmap's upper band thins and the throughput bars stand level or higher.

## 18.6 Governor Decision Ticker

From the repetition's own `audit.jsonl`: the minute and second of the decision, the compass position (0 calm, 1 at the line), and the force on the pedal (push: toward more; lift: handing the setting back toward native). Then the machines it recommends, and the give-back verdict's state: *left native* means every setting stays native's; *acting* means a trial on its own measurement. Every line of the ticker is a line of the governor's audit, and the audit is the record of authority for what it did.

## 18.7 Native Controller Ticker

Native has no governor; its own controllers' moves are the second ticker, read from the native arm's `capture.csv`: the replicas its pod autoscaler holds, the pods waiting for a place and the machines ready, every change in the same repetition, newest first. With Omni-Compass OFF the first ticker is empty, for nothing decides on top of native. The two tickers side by side are the mechanism of action in its plainest form (chapter 42).

## 18.8 Paired Repetitions and Benchmark Status by Block

**Paired Repetitions**: each repetition of the run as one small pair of lines, the one streaming now outlined; click a pair to stream it. **Benchmark Status by Block**: landed, running, queued, or not yet run, for every Kubernetes benchmark of the campaign. The run's receipt and each repetition's score stand in the page's section 4b (section 19.3).

CHAPTER 19

# Repetition Telemetry

This chapter reads the page's section 4b, **Repetition Telemetry** (`#s-runs`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

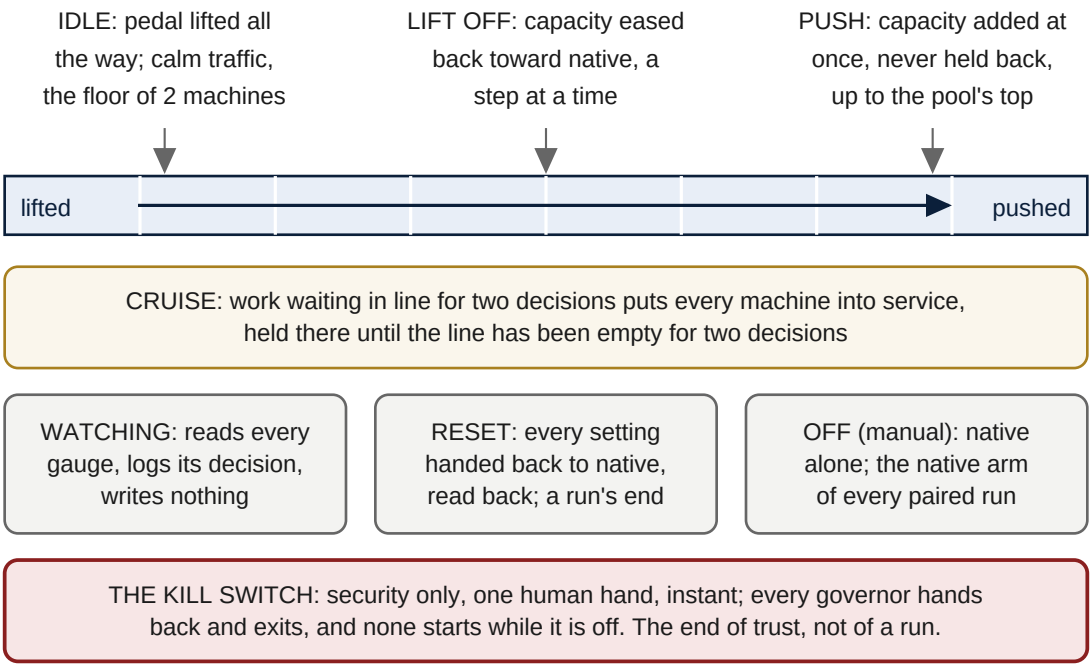



**Figure 8.** One pedal, no brake. Pushing the pedal adds capacity at once; lifting it eases capacity back toward native a step at a time, never below idle, the floor of two machines. Cruise holds every machine in service while a known queue waits. Watching reads and writes nothing. On the governor's ticker, push and lift are the force column, and the machines recommended are what the pedal bought or handed back.

## 19.1 What it is

The benchmark runs themselves at one-second resolution: each three-run table's runs A, B and C read back from the raw files GitHub archived (`results/live/raw`, on the `raw` branch), native and Omni-Compass, repetition by repetition, played second by second. Every file the page reads has its SHA-256 computed and matched against the run's own `SHA256SUMS_ALL.txt`, and every Kubernetes repetition is scored by `tools/live_reps.py`; where a run holds its own scored summary, the scorer's mean over its repetitions equals it. Class L: real Kubernetes on kind, real databases and a real broker.

## 19.2 How to use it

Choose the benchmark, the run and the repetition; the page's section 4a dashboard follows. Read a gauge's two lines against each other for the whole repetition, then read the receipt of the run under it: the receipt's word is the reading; the lines are why. A repetition whose job failed and ran again is marked; one that failed three times is listed and left out.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 19.3 Run Receipt, Repetition Score and Benchmark Landing Status

Under the lanes, **Run Receipt** is the receipt of this run as the repository wrote it, and **Repetition Score** is this repetition scored by the repository's own scorer, gauge by gauge, native beside Omni-Compass with the run's reading over all its repetitions, which must equal the run's own summary over its repetitions (the count of matches is printed). The page holds its own scoring of every repetition against the run's own scored summary (`LIVE_REPS.json`) and prints the largest relative difference: at machine precision (about 1e-16) for every run, the governors that run a group of two clusters included, whose every file the scorer reads at its own place in the repetition's folder. **Benchmark Landing Status** gives each benchmark's group, its newest three-run table and where it stands now.

CHAPTER 20

# Controller Decision Trace

This chapter reads the page's section 4c, **Controller Decision Trace** (`#s-live`).

## 20.1 The one place the brain drives the muscles decision by decision

The shipped controller, `omni_controller/controller.py`, on the modelled cluster of `pilot/selfpilot.py`, wired as the repository wires it. Each decision runs the chain left to right: the senses, the six-state brain, the nerves and the machine organ's grant, the compass law on the service (the cloud setting), the verdict's trial, the release gate, the plug. Every value below the chain is the controller's own audit record of that decision, recorded by `docs/engine/export_controller_trace.py` on runs A, B and C (six hours each, a decision every 60 s, the fast checks at every 15-second tick between; class S). Two buttons choose the setting, *On Top of the HPA at the Max* and *Cloud at the Tune-Down, Machines Only*, and three choose the run; the Linux host and the card are the page's section 4f.

## 20.2 Audited Decision Chain

The panel **Audited Decision Chain** steps through the chain; the columns, in the audit's order: the stress E, the utility U, the calm, κ, h, σ, ν (the four scalars of the nerves), the gate and whether it passed, whether machines were in service (a blind reading holds the decision), the recommendation, the machines seen, the density ρ, the compass position and velocity, the force, the verdict's word, the writes, the queue, and the law that decided. A row whose gate did not pass wrote nothing; a row whose verdict reads LOSS kept the setting at native; a row with a write names the lever and the value read back.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

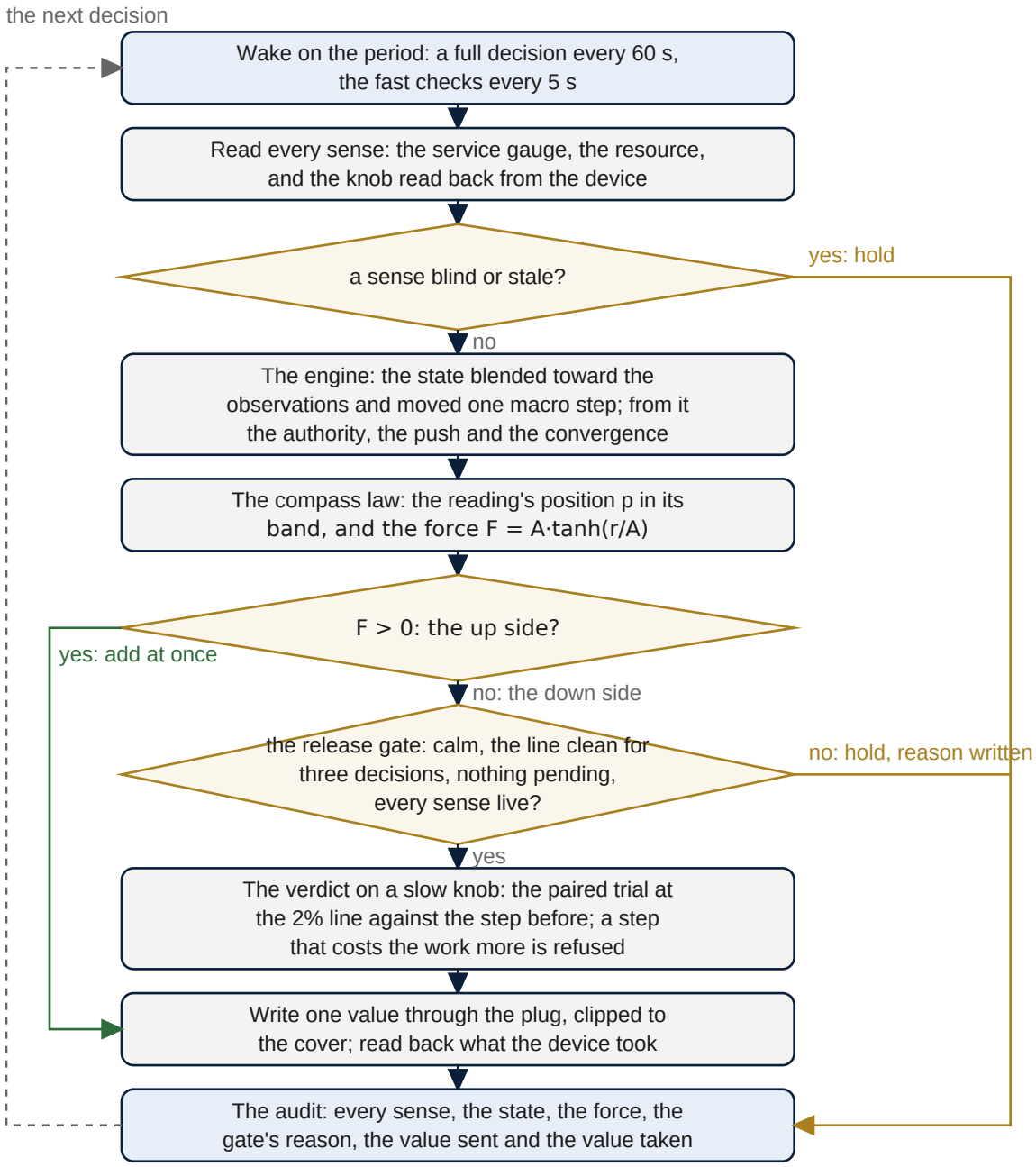



**Figure 9.** One decision of the governor. Read the body (the senses), place every reading in its band, let the brain's state evolve, grant or refuse through the nerves, apply the compass law to each muscle (push or lift), run the verdict's trial on a step past native, pass the release gate, write through the plug and read it back, renew the lease. On the decision-trace monitor each box is one column of the audit, and *open the same controller, decision by decision* steps through them.

## 20.3 Three-Run Score

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

**Three-Run Score · Class S**: the paired change per 15-second tick, averaged over ten-minute blocks, with its 95% interval (Student's t) in each run; the three runs read by the key (section 36.4). Machines and power: a fall is a cut, a GAIN; a rise is power given. Our own models (class S): a model's small numbers are the model's.

## 20.4 Kubernetes Three-Run Tables and Governor Steering

Beside the model's three runs, **Kubernetes Three-Run Tables · Class L** sets what the real thing reads (class L: real Kubernetes, three runs each): every Kubernetes three-run table on the page, each on its newest engine, with its rows that read GAIN in all three of our runs, power given, LOSS in all three and STEADY. Under them, **Governor Steering on Kubernetes · Class L** reads how steadily the governor steered on real Kubernetes: its own moves on every table `results/STEERING.json` holds, each on its own engine, beside native's own controllers.

CHAPTER 21

# Cloud Providers

This chapter reads the page's section 4d, **Cloud Providers** (`#s-cloud`).

## 21.1 The cloud is a wire

The cloud providers' section (the page's section 4d) opens with a box: "THE CLOUD IS A WIRE. A CLOUD RUN ALONE CAN SHOW ONLY A LITTLE." A cloud is the wire between the software end (the owner's programs, Kubernetes and the open-source orchestrators, managers and autoscalers that carry the work) and the hardware end (the bare metal: the GPU chips, the Linux racks, the data center, on-site or off-site). Omni-Compass can live in all three places: on the software end, on the cloud's wire, and on the bare metal. On the wire alone (every cloud run on the page so far: Omni-Compass at the tune-down on the machines only, the pods exactly native's, the cloud's rule until 31 October), all it could do was park machines the cloud's own autoscaler already made, after its own trial, so the most those runs can show is a small cut in the bill of the wire: the wire makes no benefit of its own. From 31 October every cloud set runs the whole engine at the max on the cloud's cluster, so its receipt shows that the engine holds there as on site and how much of the software end's gain comes through the wire, read as work per dollar beside the bill. It cannot show the energy and throughput gained at the bare metal, because the provider's machines are not a tenant's to meter. The larger benefit lives at the two ends. No tenant can meter a provider's back end, so the page claims that benefit only as the same mechanism measured on hardware the repository can meter (the page's section 4f), never as a measurement of the provider's machines.

The box then gives the company's own expectation, not a measured result (24 October): about 3 to 4% from the front end, 2 to 3% from the back end, about 5% from the two ends together, and 1 to 2% from the wire

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

alone, as a smaller bill or more work for the same bill. Under it a table reads the measured readings of each place, gauge by gauge (work, speed, machines, energy, the bill): the software end, the bare metal, the cloud's wire alone, on site with the two ends together, and through the cloud with all three, each cell its word and its number with its interval, composed by our own code (class S) from the layers measured in it (class L and P), and the cloud's wire, one run each, reading "not read"; where a measurement differs from the expectation, the measurement stands. The same box and table stand at the head of the whole organism (the page's section 5d), and both follow that section's reading and chip knobs.

## 21.2 More work for the same bill

In a cloud Omni-Compass shows two things: that the engine holds on the cloud's machines as it does on site, and the work per dollar, more work for the same bill, never a smaller bill as the aim (the founder's words of 31 October). From 31 October every cloud set runs the whole engine at the max on top of the cloud's own autoscaler (the founder's order of that day: no hard rule; until then every cloud set ran at the tune-down on the machines alone, `OMNI_MACHINES_ONLY=1`, `--give-back-only`, the order of 14 October, and those sets stand as their own record). The governor senses the service, moves its pod target inside the autoscaler's cover, conveys the processor, and parks a machine the cloud already made, after its own trial, waking it on a breach; it asks the cloud for no machine itself. Every gauge is read against native by the one key, and the cloud's judged number is the work per dollar: the bill may rise where the work rises more (the founder's words of 31 October: "anybody will pay 1% more for 10% more product"), a rise shown beside the work it bought, and a tune whose work per dollar falls against native is the tune falling out wrong, fixed in our models. Azure's queue holds its sets at the max until they carry the load whose work grows with the speed (the work meter), so each receipt can read the work per dollar. Azure runs first and is the measure of the other two (Google Cloud, Amazon Web Services): the same tuning, mechanism and harness. Google's and Amazon's sets are not yet run: their queues are switched off (24 October), kept ready as Azure's sets run, and the page reads each of their rows "not yet run" until each queue is switched on.

## 21.3 Cloud Run Ledger, Cloud Bill and Gate Monitors, and the Cloud-Setting Controller

**Cloud Run Ledger** gives each provider's four sets (the steady set, the bursts `1 4 1 6 1 9` and `1 6 1 8 1 6`, the real day of Google 2011 on 1 to 9) and their status: measured to the end, stopped before a result, or running or queued. **Cloud Bill and Gate Monitors** draws one monitor a provider, the same harness, mechanism and tuning in all three: each bar pair is a finished set, native's machine-hours (orange) beside Omni-Compass's (blue), the run's own word by the one key with its size, and the bill at list price under it; one run is never a set's reading. A provider with no finished set shows its empty frame and the reason its hourly gate holds, in its own job's words; nothing is drawn for a run that has not run. **Azure Bill by Run** gives the bill on Azure run by run (machine-hours billed native · Omni-Compass; the bill; the 95th percentile; the failed requests), each with the engine it ran on. **Cloud-Setting Controller · Class S** plays the shipped controller in the cloud setting, live: native's machines against Omni-Compass's on the repository's modelled cluster, the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

cloud's own autoscaler running in both arms, its run and speed following the console's clock; the bill counts every machine that exists, in service, booting or parked.

## 21.4 The stopping rule

From 19 October every Azure set runs up to 100 paired repetitions a run, the paid cap, under the preregistered stopping rule (`tools/stop_rule.py`). After 10, 20, 40 and 70 repetitions the bill's change is read with its 99% interval. The run stops once the bill is read (GAIN, power given, or STEADY within ±2% of native's bill), and never while the speed or the failed requests read a LOSS. Those runs carry "to the cap, stopping rule" in their title and count as their own A, B and C. So a cloud set that has run for a day and a half and shows no result on the board is not a fault: its bill has not yet been read at a look, and `peek` (`.github/workflows/peek.yml`) prints its finished repetitions, paired, while it runs.

## 21.5 Reading a cloud receipt

The rows an operator reads first: machines billed (machine-hours) and the compute bill at list price, both aiming lower; then the response time's mean, 95th and 99th percentiles, the time over the line, the failed requests and the alert minutes, all aiming lower and all required STEADY or GAIN; then the rows shown and not judged: the settings commands the governor wrote on its own (native is 0), the share of its additions decided inside the line, Omni-Compass's own processor time. A bill not yet read at a look (GAIN, power given, or STEADY within ±2% of native's bill, by the stopping rule) runs on to its next look.

## 21.6 When a cloud is not ready

The conductor starts a cloud's set only after that cloud's newest check passed (`.github/workflows/cloud-check.yml`, `tools/cloud_key_check.py`: the key whole, the login, the plan, the machine type, every quota), at no cost. The monitor shows a failed check as the cloud's status, in the check's own words: a Google key stored cut short (the stored secret shorter than a whole key file) is stored again whole (`docs/GOOGLE_SETUP.md`, section 4); an Amazon account on the Free plan, whose organization policy denies every machine read, is moved to the Paid plan (`docs/AMAZON_SETUP.md`, section 1a); an Azure key stored in another form than the whole block from `{` to `}` (a client secret's value alone, or a storage account's access key, for two) is stored again: one value to each of four boxes, or the whole block under a new name, which leaves a running set's login untouched (`docs/AZURE_SETUP.md`, section 7; Azure joined the check on 22 October). Nothing is built and nothing is billed until the check passes.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 22

# Orchestrators and Governors

This chapter reads the page's section 4e, **Orchestrators and Governors** (`#s-orch`).

## 22.1 What they are

The governors and orchestrators that sit between the work and the cluster: batch schedulers and job queues (Kueue, Volcano, the KAI scheduler, Armada), multi-cluster governors (Karmada, Open Cluster Management), the default Kubernetes scheduler and its bin-packing profile, the co-location and rightsizing controllers (Koordinator, Crane, CruiseKube, Goldilocks), the GPU and edge orchestrators (KubeRay, KubeEdge, Nephio) and Gödel. Each runs live on real Kubernetes as its project ships it, native and with Omni-Compass on top of it; Omni-Compass never replaces the orchestrator.

## 22.2 Road to a Result, Orchestration Stack and the Catalog

**Road to a Result**, every governor at once: its register row, its preregistration, its rig on GitHub, runs A, B and C, and its three-run table as it lands. **Orchestration Stack**, live: Omni-Compass on top, the governor native as shipped, Kubernetes and its nodes beneath, the work dots moving at the live engine's demand (a picture of the flow, not a measurement, replaced by the governor's readings the moment its three runs land). **Governor and Orchestrator Catalog**: what each governs, native as shipped, Omni-Compass on top, and what the repository holds today. Click a governor (a tile, a row of the road or of the catalog) to hold it in every view.

## 22.3 Four-Way Comparison

Each governor runs with the organism alone (nativebare), the orchestrator as shipped (native), Omni-Compass on top of the organism alone (compassbare) and Omni-Compass on top of the orchestrator (compass), all four in every repetition, each comparison paired within it and read by the one key (`docs/BATCH_QUEUE_PREREGISTRATION.md`, amendment 6). On the organism alone Omni-Compass sits on top of Kubernetes' own scheduler. The monitor shows the four as four columns of one table per governor; the comparisons an operator reads first are native against compass (what Omni-Compass adds on top of the orchestrator) and nativebare against compassbare (what it adds with no orchestrator at all). In the official-run playback (the page's section 4a) each of a governor's five tables plays its own two arms of the run's repetitions, named on the lanes, the candles, the heat map and the repetition's table by the table's own words, the left lane its reference and the right lane its candidate; Omni-Compass's colour is drawn only on an arm that

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

carries Omni-Compass, so the orchestrator's own cost (the orchestrator as shipped against the organism alone) plays in native's colours on both lanes.

## 22.4 Orchestra Organism

The eighth organism: every governor's catalog muscles, each governor's a body of its own and all of them together one body, native and with Omni-Compass on top (modelled, class S; `tools/run_orchestra.py`, `results/orchestra/ORCHESTRA.md`). It is a grouping of catalog rows like the seven, never an engine file.

CHAPTER 23

# GPU and CPU Hardware

This chapter reads the page's section 4f, **GPU and CPU Hardware** (`#s-stacks`).

## 23.1 Metered Hardware Run Slots

The silicon itself, the chip at the far end of every stack: there the max would ride each card's own limit, and every watt it takes is power given, shown beside the work it buys. **Metered Hardware Run Slots** holds the slots for the metered GPU and CPU runs (class P: the card's own meter on Lambda; the package's own energy counter, `tools/run_cpu_power.py`), every chip run GitHub has started, newest first (**Chip Run Ledger**), and then the models of one GPU card and one Linux host (class S), native beside Omni-Compass, each run twice on the same load and seed, native first.

## 23.2 Metered A100 Run and CPU Host Model

The engine that stands on a real card, its own meter recording native beside Omni-Compass, plays second by second (class P): an NVIDIA A100 (SXM4, 40 GB) on Lambda on 24 October (workflow run 40079645681, omni-v18's card governor at commit e3b05dc), ten paired repetitions of 600 s an arm, native (the card as shipped), Omni-Compass watching only (the switch OFF: it reads and writes nothing) and Omni-Compass governing the power limit and the clock ceiling, every `nvidia-smi` sample and every decision of the governor's own audit (`results/gpu/run-20261024T091235Z`, `docs/engine/export_gpu_run.py`). On this card native already runs both knobs at their top (the power limit 400 W of 400 W, the clock unlocked to 1,410 MHz), so the factory tune's governor asked for exactly that in its 3,050 decisions and wrote nothing: at its limit on native, every service gauge STEADY. The runs on the engine that stands and their three-run tables are counted in the hardware run slots and listed run by run in the Chip Run Ledger; the 8-card run has not been run (its one launch was a priced dry run). The A10 run of 2 October (the card controller of commit c908054, replaced since) stays as its own record. Its table is

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

re-read by the one key. The CPU host is built and waits for its machine: the CPU power harness (`tools/run_cpu_power.py`) puts Omni-Compass on top of the Linux kernel's own frequency governor and meters the processor's package energy (RAPL); no GitHub runner exposes that counter or a writable ceiling, so until a Linux machine that does is attached the monitor plays the harness on its own modelled machine (class S), marked AWAITING ITS MACHINE, a model, not a measurement.

## 23.3 Chip Model Settings

Buttons choose the stack (*GPU Card*; *Linux Host*; and two that lead to the page's sections 4c and 4d, Kubernetes and the cloud), the work on the card (*AI Token Generation*; *Compute-Bound*) and the card's native power setting (*Shipped Limit, 150 W*; *Operator Cap, 105 W*). The table, **Chip Model Settings · Class S**, lists every setting with three runs each: the seconds held off native, the writes, the power (W) or the host's busy share native · Omni-Compass, and the response (ms) native · Omni-Compass. Click a setting to play it; under it stand **Omni-Compass Audit Trail · Class S** and **Three-Run Score · Class S**.

## 23.4 8-Card GPU Node, Awaiting Its Run

In the same section a node of eight NVIDIA cards stands built before its run (`scripts/gpu_8card.sh`, `docs/GPU_RUN_GUIDE.md`, section D): eight card slots, each native lane drawn from the repository's one-card model (class S, the same model on every card until the run, never eight measurements), each Omni-Compass lane empty and marked awaiting its run. Nothing on it is invented: on a running node the same panel reads the NVIDIA DCGM exporter's series card by card, and the 8-card run fills it when it lands.

## 23.5 Reading a row with nothing held off native

A row with no second held off native is Omni-Compass deciding to stay native: its trial of one step read LOSS and was refused, or the knob is already at its limit on native. That row is the engine's own rule at work and is not a fault. The slots marked not yet run are written as the repository holds them and fill in, played back from their raw files like every Kubernetes run, the moment their three runs land.

CHAPTER 24

# CI Pipeline

This chapter reads the page's section 4g, **CI Pipeline** (`#s-github`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 24.1 When the page is rebuilt

GitHub rebuilds the page (`.github/workflows/live-engine.yml`, `docs/engine/build_live_engine.py`) whenever the engine, a model or a result changes, and every three hours. Every record is made again by the repository's own code on the engine that stands, checked against the Python and the C++, and committed. The page's own file is outside its own push filter, so a rebuild never starts another rebuild; a front-page run that did not succeed starts no rebuild (`tests/test_workflow_triggers.py` holds both).

## 24.2 GitHub Actions in Flight and Running and Queued Workflows

**Running and Queued Workflows**: every workflow run in progress or queued at the build, as GitHub's own list at that moment, and the first three of them in each state. Above the table, **GitHub Actions in Flight** draws GitHub Actions as a monitor: the runs in flight by workflow (running solid blue, waiting pale), the Azure set to the paid cap with its repetitions measured so far, and each cloud's hourly gate with the reason its own job wrote. The hardware and cloud runs, with their terminal status, are in the page's sections 4d and 4f; every run on the engine that stands that terminated without a measurement is raised on the status board.

## 24.3 Reading the pipeline against the conductor

The conductor (`scripts/dispatch_grid.py`, every 15 minutes) starts every run of the campaign on the engine that stands, A of every benchmark, then B, then C, a few a tick, and never starts one twice; it starts nothing while an Azure run, a grid report or a verify waits for a runner. So a pipeline monitor showing one Azure run queued and nothing else starting is the conductor holding, by its rule, not a fault. A long run on a free runner (the organisms at 1,000 copies) lists as in progress for its whole shard.

PART FIVE

# Proof and Models

*The page's Part 5: the engine's own runs, its twin, its models and its specification.*

CHAPTER 25

# Python Runs and Monte Carlo

This chapter reads the page's section 5a, **Python Runs and Monte Carlo** (`#s-proof`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 25.1 Three-Way Run Check

The proof monitor, **Three-Way Run Check** (the page's section 5a, `#s-proof`, in the panel **Seeded Engine Runs and Replayed Scenarios**), reads every run three ways: the Python's set as GitHub ran it (`tools/engine_runs.py`, its run linked), the same draws through the C++ harness on the engine's C++ twins (`cpp/tools/engine_runs.cpp`), and this page's own port, one SHA-256 digest each beside the digest of the draws themselves; the three must be one. Once that first check ends, the page checks the set again without end: one run at a time, about twice a second, each run recomputed from its own seed by the page's engine and held against the Python's receipt; the run being checked is boxed on all three lanes and its result printed under the digests.

## 25.2 Running the Python's runs again

In the proof (the page's section 5a, `#s-proof`), beside "Run a new set here", the button "Run another 500" runs the next set from the base after the last one, and "Run three more sets" runs three in a row; the table under the monitor lists every set run on the page with its digest and, where the Python has run the same base, the Python's record and whether the two digests match; where it has not, the line names the engine-runs workflow on GitHub with the runs and the base that make the Python run the same set.

## 25.3 Held-Out Scenario Replay

Under the proof monitor, in **Held-Out Scenario Replay**, the Python's 500 scenarios a seed replay from their own record, native beside Omni-Compass, scenario by scenario, with every trace hash; its words stand in its own table.

## 25.4 Engine Core Monte Carlo

The Monte Carlo monitor, **Engine Core Monte Carlo** (the page's section 5a, `#mcPanel`), draws the Python's Monte Carlo of the engine's core. Every line is one run of the engine's core (`omnicompass/core.py`: six states, the fourth-order Runge–Kutta step 10 times a macro step of 0.1 s, 20 macro steps), its U from step 0 to step 20, drawn up to a cursor that sweeps the 20 steps in 10 seconds and holds 3. The left lane is native: the same plant with no command (u = 0), left to its own double well. The right lane is Omni-Compass on top: the bounded command toward the nearest basin ($|u| \le 25$, held across the four stages), or, by the knob, toward the far basin, or with the double well switched off ($\mu = 0$). The green bands are the basins, $|U \mp 1| \le 0.10$. A run is conveyed when it has stayed in one basin 5 macro steps in a row and certified at 10; each lane's counters read at the cursor (in a basin, conveyed, certified, of 500), a run's head turns green once certified, and the bars under each lane count the macro step each run was conveyed and certified. The set the page opens on is the Python's own 500 (`benchmarks/core_evidence.py`, numpy's generator from seed 223387268), whose counts must be its record's (`results/core_evidence.json`: Omni-Compass on top conveyed 500 and certified 500, at macro steps 4 to 6; native alone conveyed 476 and certified 427). A set the page draws from the jump generator is computed by the page's own port of the core, line for line, and its digest must be the one

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`tools/core_runs.py` writes for the same base (`tools/core_runs.py --verify` runs the page's port in node and compares; the engine-runs workflow writes the Python's record of every base it runs). The table under the monitor lists every set, its counts on both sides and whether its digest matches the Python's. The monitor's light reads the set shown: green when Omni-Compass on top certified more runs than native alone, yellow when the same, red when fewer or when the page's digest differs from the Python's or the C++'s record of the same base.

## 25.5 The linked run and the Custom System Run

Two buttons link the runs. **Run the Monte Carlo 500: the Python, the C++ and this page, linked** (in the control panel and on the monitor) stands three boxes side by side: 1, the Python, the newest set `tools/core_runs.py` ran from the jump generator, with its digest and its record; 2, the C++, the C++ twin of the core (`cpp/tools/core_runs.cpp` on the sealed `cpp/src/core.cpp`) on the very draws the Python wrote (the injection, its own SHA-256), with its digest, how many of the 2,000 run-ways agree to 1e-9 and the largest difference; 3, this page, its own port of the core run on the same base when the button is pressed. The line under them reads **ALL THREE THE SAME** when the three digests are one (every U at every macro step, four ways, to six decimals), and says so if they differ; the three are sealed together (`tools/seal.py`, the thirteenth twin) and `verify.py` runs the C++ against the Python on every push. **Run your own system through it** opens the panel **Custom System Run**: a system is your own stack written as ranges of the core, a name and, for any of the six starting states and the 22 parameters, a range or one number, the rest the engine's own (`systems/README.md`). It opens on the engine's own ranges, every one editable; **Load the example** puts in `systems/example_heavy_swing.json` (heavy outside demand, a strong swing, little headroom; not any real stack). **Check it** reads the block as the Python does and names anything refused; **Run it here** draws 500 runs in the browser through the page's port (on the base the Python ran that system on, so its digest compares, else on the linked run's base) and draws them in the monitor beside native; **Download it as a file** saves it for `systems/`; **Run it on GitHub** checks it, copies it and opens the **monte-carlo** workflow (`.github/workflows/monte-carlo.yml`): press **Run workflow**, paste the block into **system**, set the runs and the base, and run. On GitHub's own machine, at no cost, the Python draws the runs over your ranges, the C++ twin runs the same draws, the page's port is run on them in node, and the three digests must be one; the record, with the run's link, goes to `results/core_runs/` and joins the monitor's table at the page's next build. A system file committed under `systems/` runs on every push that changes it. GitHub offers **Run workflow** for a workflow on the repository's main branch, so the button's GitHub half works once omni-v18 reaches main; the page half works now. The block is read as numbers only, never run as code. Class S: the engine's own mathematics on your ranges, never a measurement of your stack; wiring Omni-Compass onto the stack itself is the plug of the Operator's Manual.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 26

# C++ Twin

This chapter reads the page's section 5b, **C++ Twin** (`#s-cpp`).

## 26.1 The Python's runs through the C++ engine: five tables

The engine is written twice, in Python and in C++, line for line where a twin exists. Every time the page is built, the Python's own runs are fed into the C++ as its input and the C++ answer is set beside the Python's, number by number. Where they are the same to the last digit, the two are one engine. The section holds five tables.

| Table | What it shows | How to read it |
|---|---|---|
| Twin Parity Checks | each Python file and its C++ twin, the inputs the check fed both, the largest difference, and the reading | "the same" on every row is a whole engine in two languages; "DIFFERS" names the row to look at |
| C++ Engine Run Receipts | the record of the Python's seeded runs the page shows (the jump generator's 500), its draws (the Python's injection) fed to the C++ harness at the page's build (`cpp/tools/engine_runs.cpp`): the record, its runs and base seed, the Python's digest, the C++ engine's digest, how many receipts are identical to the last bit, the largest difference, the reading, and the GitHub run that made the record | the Python's runs are the input and the C++ engine's receipts the output; the two digests one string and every receipt identical is the same engine run twice, in two languages |
| Compass Law Parity on Page Runs | every compass law the page's own runs wired while the page was built, grouped by the file that wired it, each law's readings in the order they came, through the C++ law and through the Python again | the C++ against the Python, and the Python again against the run's own forces: 0 (identical) on both is the run's law reproduced exactly |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Table | What it shows | How to read it |
|---|---|---|
| Organism Law Parity at the Tune-Down | each organism, its muscles, the muscles that run the compass law at the tune-down, the readings fed through the C++, and any muscle that differs | at the max, the default brain, the edge and the bone decide each muscle, and they have no C++ twin yet; the tune-down is where every muscle's law is the compass law, so that is where it is fed through |
| Python-Only Components | every part of the engine with no C++ twin yet, by file | the list is the work still to twin; nothing on it is hidden from the other tables |

A box under the organisms' table finds any muscle by its catalog id: it names each organism the muscle's law ran in, its readings, and whether the C++ and the Python gave the same answer. A muscle with no lever at the tune-down stays native there, and the box says so.

CHAPTER 27

# Scenarios and Injection Grid

This chapter reads the page's section 5c, **Scenarios and Injection Grid** (`#s-scn`).

## 27.1 Twenty Scenarios

The grid of the page's section 5c, `#s-scn`, reads ten organisms two ways each: the spine; the four realms (Compute/AI/Cloud, Physics/Robotics/Autonomous, Energy/Facility/Industrial, Distribution/Specialized); the four stacked; the whole tower; the orchestra; the live benchmark engine's own organism of sixty muscles; and the whole organism in its three places. Each runs **native alone** and **Omni-Compass on native** (the factory profile, on top of every native controller), on the same seeds, so the same demand meets both. Omni-Compass is never by itself, so there is no third way. That makes twenty cells, each read by the key against native, from the record `results/scenarios/SCENARIOS_20.json` (written out as `results/scenarios/SCENARIOS_20.md`, by `tools/scenarios_20.py`; class S, our own models, three runs A, B and C). A cell with no run reads "not yet run", never a number. The same grid stands, compact, in the control panel (section 2.24). The earlier record of thirty scenarios is kept as history in `docs/history/SCENARIOS_30_2026-10-24.md`; the page does not read it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 27.2 One cell

Each cell carries one signal word and one number against native. The native column reads STEADY and 0%, the yardstick. In the Omni-Compass on native column the organism's gauges are work, the time over the line and the power drawn: a cell is red, REDLINING, when any of them reads LOSS, green, GAINING, when any reads GAIN and none LOSS, and yellow, STEADY, otherwise; its number is the gauge that decided the light (work, where work decides it), with the word's sign (+ a GAIN, − a LOSS, = STEADY). The whole organism's cell reads its composition's index over the gauges (chapter 28). Hover a cell for each gauge's reading and each run's change.

## 27.3 Its detail and its three runs

Click a cell for its detail under the grid: the organism, what it is, its muscles and its class, then a table of its one pair, Omni-Compass on native against native, on work, the time over the line and the power drawn, each with its word, its reading and the change of each run A, B and C. Every run under 0.5% reads STEADY; from 0.5% up the word by what the gauge is for; runs pointing different ways, the runs disagree. The note names the record's file, its engine and its commit. A click on the live benchmark engine's organism in the Omni-Compass on native column also puts the live benchmark engine on that body, the harness's one organism.

## 27.4 Injection Grid

Under the twenty scenarios the section carries the injection grid (`#s-inj`; the record `results/injection_grid/INJECTION_GRID_<base>.json` and its `.md`, written by `tools/injection_grid.py`; class S). Its line under the heading, **The batch**, names the batch it stands on: its base, its 500 injections and the jump generator's rule (seed_k = base + k·0x9E3779B9 mod $2^{32}$), the digest of the Python's runs of that batch and its file, and the C++ engine's digest of the same seeds ("identical", with the count of runs that agree). The same batch drives the Python's runs in the page's section 5a. An injection is one seed: every plant of every organism draws its demand, its weather and its disturbances from it, under the climb, for 240 decisions of 15 s, and the same injection reaches every organism and every way.

The same line prints two checks the page makes as it opens. First it re-hashes the record's cells the way the Python hashed them and prints "identical digest to the Python's" or "not identical to the Python's". Then it recomputes every cell's mean from the runs' means, joined and stacked as the Python joins and stacks them, and prints the number of cell gauges checked. It reports "every one agrees" with the largest difference, or the count that differ by more than 0.001 points and where the largest one is. The engine, the commit, when it ran and on how many processes close the line.

## 27.5 The knobs

- **Organism**: the nine organisms with a plant, by their scenario number (the spine 1 to the live benchmark engine's own organism 9), each with its muscles. The whole organism (10) is a composition of measured tables and takes no injection; its reading stands in the twenty scenarios above.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Way**: *Omni-Compass on native*; *a governor on the organism* (each of the fourteen: the Kubernetes scheduler with its bin-packing profile, Kueue, Volcano, KAI, Koordinator, CruiseKube, Crane, KubeRay, Gödel, Armada, Karmada, Open Cluster Management, KubeEdge and Nephio); *Omni-Compass on top of a governor*. The orchestra already holds every governor's muscles, so on it only Omni-Compass on native is offered.
- **Against**: native, and, for Omni-Compass on top of a governor, the governor on the organism too, so Omni-Compass's own effect on that governed stack is read alone.
- **Runs**: 1, 10, 100 or 500. **Organisms stacked**: 1, 10, 100 or 500. Every cell is the one batch cut down: 100 runs are the first 100 of the batch's 500, 10 the first 10, 1 the first; and each run carries a stack of 500 organisms, of which 100 organisms are the first 100 copies, 10 the first 10, 1 the first (copy c of run k on injection k + c, mod 500). A click on 100 shows those 100 and no others. A cell whose runs × organisms pass 500 uses an injection more than once, and its readout says so.
- **Gauge**: work, the share of muscles over the line, or the power per muscle. Every gauge is read per muscle. Two ways on the same muscles read the same either way, and a governor's way, which carries its own body's muscles beside the organism's, is read like for like.

The switchboard's *Organisms* and *Runs* (section 2.31) and its organism row set the same knobs.

## 27.6 What a cell shows

The readout on the left prints the cell's word and its number with the word's sign, the gauge, the way and what it is read against. Under it come the reading in the key's words for its run count ("in all 500 of our runs", "by the mean of our 10 runs", "in our one run") and how many runs are beyond the noise. It then gives the range the runs moved over in plain directions ("from up 2.6% to up 3.4%") and the mean as this page recomputes it. The scenario number, the organism, the cell's tier, the injections it draws on and its reference follow. The cell's three gauges stand at the end, each with its word.

Under the readout, the strip draws each run's change against its reference as a dot. The band is the 0.5% floor. A dot is green toward the gauge's aim, red away from it and amber for power given, by the floor alone; the dashed line is the mean. The cell's word adds the noise: each run's 95% interval over its 24 blocks of ten decisions (Student's t), read by the key. Hover a dot for its run, its injections and its seed.

On the right, the matrix shows every runs × organisms cell of the chosen way and gauge, each with its signal, its number and its word. A click on a cell selects it.

## 27.7 Ways ranked, organisms on the same injections, one injection

- **Ways Ranked Against Native**: every way on this organism in this cell, ranked against native by work, then by the share over the line, with power beside. Its line names the first way, the rank of Omni-Compass on native, and the ranks held by the ways with Omni-Compass on top; the rows with Omni-Compass are drawn in its colour. A click on a way selects it. In our models a governor's knobs run under their own native controllers, so a governor's way reads native's work index by construction. What it moves is the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

share over the line and the power of its knobs, and its rank says so. The governors as their projects ship them, governing real workloads, are read on real Kubernetes in the page's section 4e (class L): fourteen named, twelve with a rig built, eleven run the four ways.

- **Organisms on the Same Injections**: the chosen way and cell on every organism, so each organism's reaction to the same injections is read side by side. A click on an organism selects it.
- **Single Injection Across Organisms**: pick k (0 to 499) with the arrows or the box. The line prints its seed and the rule that made it. The table gives each organism's mean over that run's 240 decisions per muscle, the reference to the way, with the change in plain directions.

## 27.8 Exports, import and the reference

- **This cell, every run (CSV)**: each run k, its first and last injection, its first seed and its change on the three gauges.
- **This organism, every way and cell (CSV)**: every pair, tier and gauge with its mean, word, reading, runs beyond the noise, smallest and largest run.
- **The batch's 500 injections (CSV)**: k and its seed.
- **The whole record (JSON)**: the record as the page holds it.

Each file opens with the notice and the batch's line (its base, the Python's and the C++ engine's digests, the engine, the commit). **Import** reads another record of `tools/injection_grid.py` (another batch). Every view and both checks follow it, the batch line says it is imported and not this build's record, and *Back to this build's record* returns. Every cell has a **reference**, `IG/base/organism/way|against/NxM/gauge` (for example `IG/1488855732/tower/on_native|native/500x1/work`). *Copy this cell's reference* copies it, and typing it in the box and pressing *Go* reaches that cell on any copy of the page that holds the batch. Another batch is run with `python3 tools/engine_runs.py --runs 500 --base B` (the Python, then the C++ engine on the same seeds) and `python3 tools/injection_grid.py --base B`. On GitHub, every set the engine-runs workflow commits on main starts the injection-grid workflow (eight shards, then the record and the page's rebuild), and a dispatch runs any base.

One batch is one run in the key's sense; three batches make a three-run reading. Class S, our own models.

CHAPTER 28

# Whole Organism

This chapter reads the page's section 5d, **Whole Organism** (`#s-whole`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 28.1 Every way a stack is bought

The whole organism (the page's section 5d, `#s-whole`) opens with the same box as the clouds' section, the cloud is a wire, and its table of the three places Omni-Compass lives (section 21.1); the three places compose into one organism and one bill below it, read two ways, native and Omni-Compass on top of native.

Omni-Compass is wired into every piece of a stack through its own two-way plug: the software (the cluster, its add-ons, its governors and orchestrators, and the services on it: the pools, the consumers, the caches), the bare metal under it (the chip's power limit and clock; the machine's kernel and its own meter) and, where the stack is rented, the cloud that bills the machines. The whole-organism monitor (the page's section 5d, `#s-whole`) has a button for each way a stack is bought (**Scenario**): *On-Site Bare Metal*, the software on the owner's own bare metal (software × bare metal, no cloud); *Through the Cloud*, the cloud's build, its bare metal at the back end and the software (cloud × bare metal × software), with the cloud as two pieces beside it; *In the Device*, the bare metal alone, for one who buys the chip or the machine and no software stack on it; and *As a Service*, the software alone, for one who runs it on machines someone else meters. The cloud is a piece of the stack, bought and billed, and what Omni-Compass does there composes with what it does on every piece beneath it.

A tenant cannot meter the provider's bare metal at the back of a cloud: the cloud's piece is the bill and the machines the provider meters, and the bare-metal piece is gauged on hardware the repository meters and carried to the provider's only as a modelled factor, said so on the monitor in the box **CLOUD BACK-END METERING DISCLOSURE** (`DISCLOSURES.md`, section 9). A piece with no measured run yet is named and left out of the whole, never filled with a guess.

## 28.2 Stack Composition

The stack, native beside Omni-Compass, with the work flowing down through the pieces: the service, the cluster, the governors, the machines, the chip, and, through the cloud, the provider's bill. The dots move at the live engine's demand; the picture is of the flow, not a measurement; each piece's measured factor replaces the picture's weight the moment its three runs land.

## 28.3 Composed Benefit by Gauge

The benefits, gauge by gauge, added together over every measured piece: work (more), speed (a shorter 95th percentile), machines (fewer), energy (less), the bill (smaller), and the one number, the index over the gauges. Two readings are offered by the buttons (**Reading**): *Every Piece, Intervals Carried* (every factor with its own 95% interval, the whole their product) and *Strict Reading, Index Rule* (only a factor that reads GAIN or power given in all three of its runs counts; STEADY counts as exactly 1). A third pair of buttons, **Chip Run**, puts the chip's newest finished run *Beside the Count* or *In the Count*.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 28.4 Composition by Piece and Layer

Piece by piece and layer by layer: each factor with its 95% interval, the whole their product. Read the table left to right: the piece and its layer, the five gauges, the index, and the receipt (the table's own row) each factor is read from. A factor's interval that includes 1 is STEADY and multiplies the whole by exactly 1 in the strict reading.

## 28.5 Bills and Receipts

What native spent (the cost; runs A, B and C), what Omni-Compass spent (the expense; A, B and C), and the benefit (pooled, with its 95% interval), in the units each receipt bills in: machine-hours, dollars at list price, watt-hours of the declared model, the meter's watt-hours where a meter exists. The cost is native's, the expense is Omni-Compass's, and the benefit is their difference read by the key: a smaller bill is GAIN, a larger one is power given, shown beside the work it bought.

## 28.6 Tables Outside the Count

The governors' tables on this page, the models beside them, the separability test (whether the pieces' factors may be multiplied: the factors are independent where each piece's run holds every other piece at native), and where the missing pieces are: every slot not yet measured is named, and the whole is read without it rather than with a guess in its place.

CHAPTER 29

# Measured Effect

This chapter reads the page's section 5e, **Measured Effect** (`#s-fleet`).

## 29.1 What it holds

The measured effect in full (the page's section 5e, `#s-fleet`) is everything the benefit board stands on, from the organisms down to the single muscle: the seven organisms under their three loads, all 1,107 muscles of the catalog one cell each, every registered benchmark still to run, the scale-out grid of copies and runs, and every three-run table in the repository with every row as it is written. Its first monitor, *Seven Organisms Under Three Loads · Class S*, draws the seven organisms of the catalog under three loads (the spine first; the four realms; the four stacked; the whole tower), each native and with Omni-Compass on top, misses against native; the whole stack's two arms are the whole organism's composition (chapter 28) until the stacked run measures them in one run. Its second, *Catalog Muscle Matrix · Class S*, draws every muscle of the catalog as one matrix,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

each cell its word against native; click a cell to read the muscle (its name, family, realm, template, knob and its word under each load).

## 29.2 Open Register Slots

The register's slots, in order (`docs/REGISTER.md`): every muscle in them already runs, modelled, in the 1,107-muscle catalog and the seven organisms, native and with Omni-Compass on top; a slot is the same test on the outside world's own software, each queued until its three runs land. A slot shows its status in the register's own words, what is measured (the lines of its preregistration) and the known limits said before the result.

## 29.3 Scale-Out Grid

Seven organisms × 1, 10, 100 and 1,000 copies × 1, 10, 100 and 1,000 runs: modelled (**Misses · Class S**) and with the real cluster inside (**95th Percentile · Class L Inside the Model**), both values shown in every square. The 1,000 tier runs on GitHub's free runners only; paid machines stay at 100. Two buttons choose *the engine that stands: its runs as they land* or *the road: the newest grid with its squares landed*. A square not yet landed says so; a square beyond a free runner's memory or time is listed as pending runner capacity, never rented. The grid's monitor draws the engine that stands' own campaign as GitHub runs it: each square filled by its runs started (pale) and measured (solid), as GitHub was read at the build, with the age of that reading in its heading; the reading cursor walks every started square and names it under the grid. Once the grid's receipt is written, each square takes its word. The road draws the newest grid with its squares landed, under its own engine's name, never read across.

## 29.4 Standing Three-Run Tables

Every three-run table in the repository, native against Omni-Compass, every row as written, each benchmark's newest engine's table; open a row to read every judged row that is not STEADY. The table's own rule is printed with it; rows shown and not judged are left out of the counts; a tuning case is shown and never counted.

## 29.5 Realms Table

From `results/realms_v5/REALMS_V5.json`, the realms runner at the max, its engine and commit named above it (`tools/run_realms_v5.py`, catalog `realms/catalog_v5.csv`), ten paired seeds per muscle and per organism, the lease after 80% of every run. Misses: the plant decisions over their line, summed over the ten seeds. A muscle reads GAIN when one of its aims (more work, faster, fewer misses) moved beyond the noise under some load with none moving the other way; STEADY at its limit on native when its knob already sits at the top of its range; STEADY, the same as native, when it wrote and nothing moved beyond the noise. Energy is power given where it rose. The seven organisms are listed spine first, then

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

the four realms, the four stacked and the whole tower, each under its own load, the climb and the work waiting in line (the three buttons). Evidence class S: a model's energy is not a meter's.

## 29.6 Families by Stack

Each muscle alone, native and then with Omni-Compass on top, grouped by the family of its knob and the stack it lives on; *Find a muscle* searches the 1,107 by name. A family reads GAIN when more of its rows read GAIN than LOSS; the row of a muscle at its limit on native is counted as STEADY.

CHAPTER 30

# Telemetry Analytics

This chapter reads the page's section 5f, **Telemetry Analytics** (`#s-lab`).

## 30.1 Telemetry Tensor Cube

The engine's own record is a tensor: two sides × every decision × sixty muscles, each entry a muscle's compass position (0 calm, 0.5 the center of its band, 1 its line), recorded from `realms/harness.py` (class S). It fills in decision by decision as the page's section 3a plays: the tensor, the difference matrix and the principal motions replay that record on the live benchmark engine's clock, and no knob of the console changes them. Read it as a sheet of heat: a column (one decision) that is light on the blue side and dark on the orange side is a body Omni-Compass held nearer calm at that moment.

## 30.2 Tensor Difference Matrix and Principal Modes

The difference between the sides, muscle by decision, with the sign of the word; and the body's principal motions, the tensor's eigen-decomposition: the few patterns of movement that account for most of the body's motion, each with the share of the variance it carries. The line across each bar is the decision on screen: the share of that decision's own departure from the run's mean that lies in the mode. A first motion that is the whole body rising and falling together is a body breathing with its demand; a motion that is one room against the rest is that room's inflammation.

## 30.3 Service Level and Error Budget

The official runs (class L) read as a site-reliability engineer reads a service: the service level this repetition, the error budget burned, and every gauge of this repetition against native, each replaying the official run chosen in the page's section 4a (**Service Level and Error Budget**, then **Repetition Gauges Against Native**).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 30.4 Omni-Compass Narration

Omni-Compass speaks for itself: every line is read from its own records at the moment on screen (the official run playing in the page's section 4a, its governor's own audit, and the engine's harness on the console's clock) and says what it is doing, beside what native does. *Speak aloud* reads it out on the viewer's own browser voice as the run plays.

## 30.5 Ask Omni-Compass

Ask in your own words, or pick a question (what are you doing right now; where do you read LOSS, and why; explain yourself to a buyer; show me your largest GAIN). Omni-Compass answers in the first person, for its own whole without it. The answers are drawn from the page's records and are not reviewed by any outside party.

CHAPTER 31

# Master Switch Wiring

This chapter reads the page's section 5g, **Master Switch Wiring** (`#s-master`).

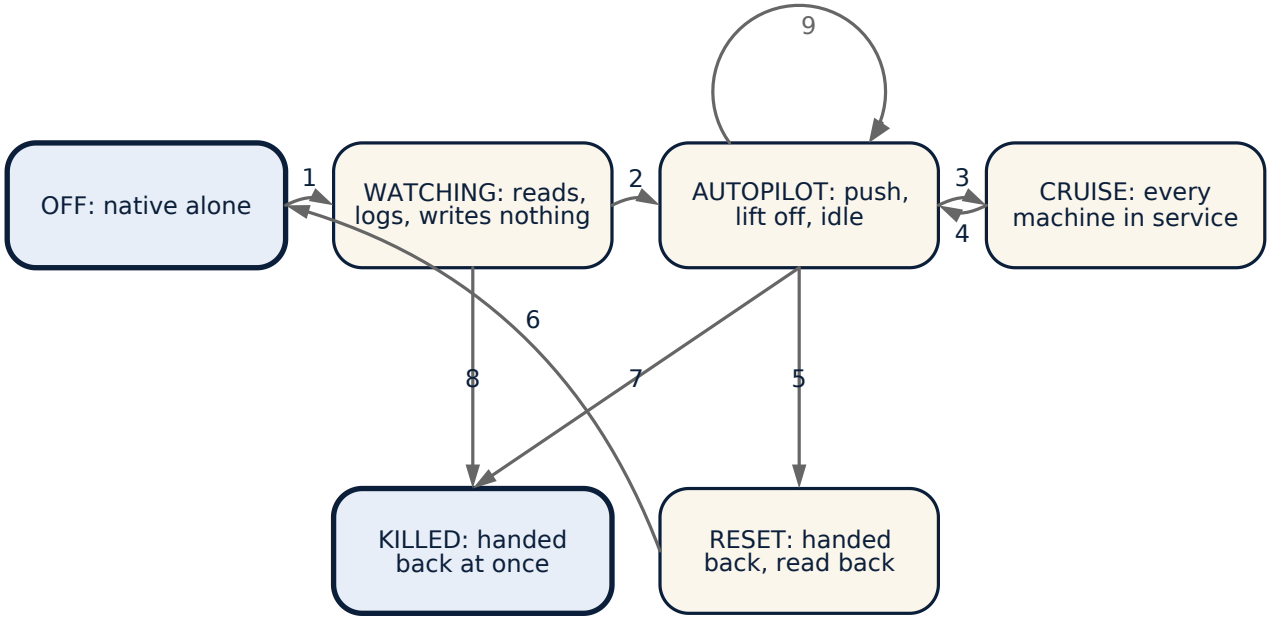
## 31.1 One switch for the whole organism

The master switch (the page's section 5g, `#s-master`; its kill switch heads the page's section 1) is wired at the top of the organism (the monitor *Master Switch and Kill Switch*): switch, brain, organism, every muscle, as the repository builds it (`omnicompass/master.py`, `tools/omni_switch.py`). It holds the brain's outputs. Every governor on the page is registered with it and obeys it at once; on a real machine every governor exits at the switch.

## 31.2 The table under the switch

The table's columns: the governor; where it lives (the module, the section); its state; what it does at the switch. Every governor the page runs (the stress-test model's, the harness's, the live controller's replay, the page's own render governor) is a row, so there is no governor on the page that the switch does not reach.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.




1 level 1 · 2 the wire check passed; level 2 and up · 3 pods waited 2 decisions · 4 the line empty 2 decisions · 5 the reset file: the end of a run · 6 exit 0 · 7 the kill switch, from any mode · 8 the kill switch · 9 idle at once when demand is at zero

**Figure 10.** State machine: the governor's modes. OFF is native alone; WATCHING reads and logs and writes nothing; AUTOPILOT pushes, lifts off and idles; CRUISE holds every machine in service while a known queue waits; RESET hands every setting back and reads it back. The kill switch leaves every mode for OFF at once; the lease walks a crashed governor's settings back to native. On the page the switch's three states are ON (green), the kill switch engaged (red) and OFF (white, the brain reading in shadow).

CHAPTER 32

# Specification and Mathematics

This chapter reads the page's section 5h, **Specification and Mathematics** (`#s-math`).

## 32.1 Every part and its equation

The component specification: every part of the engine, its equation in Unicode as the repository writes it, and the source module that implements it. Items 1 to 5 and 8 are the engine's code and run in every view; items 6 and 7 as written (the compass law on each muscle, the plug's lever) drive the stress-test model; in the harness views the max and the plants' own overrides take their place; the live controller runs the compass law on the service, as the page's section 4c shows.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Item | Part | What the page prints | Module |
|---|---|---|---|
| 1 | Senses | the muscles' meters read and normalised to nine observations: q, load, power, thermal, network, drift, stale, security, conflict | `adapter.observe_vector` |
| 2 | Assimilate | $x \leftarrow (1 - a)\cdot x + a\cdot x[obs]$, $a = 0.339$: the engine is pulled toward what the senses read | `adapter.assimilate` |
| 3 | Evolve | equations (1) to (7), the Runge–Kutta method, ten micro steps of 0.01, the controller evaluated and not applied | `core.macro_step` |
| 4 | Allocate | $push = clip(-f[U] + 12\cdot(1 - U), \pm 25)/25$ and $\rho^* = clamp(\rho_0 - k[I]\cdot I[U] - k[E]\cdot E, \rho_{min}, \rho_0)$ | `adapter.Governor.step` |
| 5 | Nerves | $\kappa$, h, $\sigma$, $\nu$ into calm; calm against each organ's $\theta$ grants contraction (the spine to nodes, $\theta$ 0.7; compute to pods, 0.5; physics to the processor's clock, 0.3; energy to power, 0.3; distribution to routing, 0.5) | `nervous_system` |
| 6 | Compass | $p = (r - lo)/(hi - lo)$; $F = A\cdot\tanh((k_P(p - ½) + k[d]\cdot v)/A)$; fails up past 0.95; never hands back while climbing above the center; k[d] at three times critical damping from the muscle's own $\tau$, the push | `compass_law.CompassLaw` |
| 7 | Plug | one wire in, one wire out: each write clipped to the cover; the snapshot taken once at the start is what reset restores; each write read back; the kill switch hands every lever to native at once | `compass_law.Plug`, `master.py` |
| 8 | Verdict | the paired trial of chapter 16 | `verdict.py` (its C++ twin `cpp/src/verdict.cpp`, chapter 26) |
| 9 | The whole organism | each layer measured against its own native, the layers composed by the product law with every interval carried in logarithms, two readings of the same numbers, the separability assumption | `tools/whole_system.py`, `docs/engine/export_whole.py` |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Item | Part | What the page prints | Module |
|---|---|---|---|
| | | tested on the governors' four ways | |

## 32.2 What the specification is for

It lets the reader of the page check that what moves on the monitors is what the code says: the six spokes at the center of the cycle (E, U, I[U], S, B, Ḃ) are item 3's state, the inner arc of nine segments is item 1's senses, the force field of the phase plane is item 6's law. The Owner's Manual carries the derivations; this page carries the statements.

## 32.3 Stress-Test Model and Reads and Sets by Part

Under the specification, **Stress-Test Model · Class S** writes out the model drawn for the page: one decision of muscle i in the order it runs (capacity following its lever with its response τ; the work served, the waiting work and the failed work; the reading, read one to three decisions late; native's rule; the compass position and force; the max and the tune-down; the plug; the kill switch), with both arms on the same demand. A table, **Reads and Sets by Part**, names every part, what it reads, what it sets and its source: native's controller under each muscle, the max of the harness, the brain, the nerves, the immune system, the compass law, the release gate, the plug and the verdict. Two notes close the section: **Engine Code Provenance** (what is the engine's code, ported, and checked against the Python and the C++ on every build) and **Realms Harness and Stress-Test Model**, the two bodies.

CHAPTER 33

# Failure Modes and Controls

This chapter reads the page's section 5i, **Failure Modes and Controls** (`#s-fail`).

## 33.1 Failure-Mode Register

Every known way Omni-Compass can fall short, the control that mitigates it, that control's coverage and the residual risk where it does not reach, and its implementation in the code; every failure mode the live benchmark engine can be put through has a fault-injection control on the console, named in the row. The register is `docs/SAFETY_CASE.md` and `docs/THREAT_MODEL.md` read as a table.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 33.2 Unjudged Resource Costs

Under the register, the rows the three-run tables report but do not judge: the governor's own commands, its processor time, the pending pods in a snapshot, the energy per core-hour. They are printed so the cost of the governor is never hidden; they carry no word because their preregistrations declared them shown only.

CHAPTER 34

# Self-Governed Rendering

This chapter reads the page's section 5j, **Self-Governed Rendering** (`#s-self`).

## 34.1 Omni-Compass inside the page

The engine that runs the page has Omni-Compass in it. Native is the browser's own loop: every chart on screen drawn on every frame, and when the frame overruns, whole frames dropped. Omni-Compass is the same loop with the engine's own compass law governing it. Each chart is a muscle whose knob is how many frames it waits between draws, and the reading is the time between frames. Below the band's center the pedal lifts and every chart is handed back to native, drawn every frame. Toward the line it pushes: the charts farthest from where you are looking wait a frame or more, so the frame itself stays inside its line. Evidence class L: live software, on the viewer's own machine.

## 34.2 Reading it

The monitor, *Inter-Frame Interval by Window*, shows the frame time on both sides, the frames dropped by native's loop, and the frames waited by each chart under Omni-Compass; the table **Paired Window Measurement** pairs the two sides window by window. On a laptop with the wall in view, the charts on screen draw every frame and the rest wait; scroll, and the waiting moves with the eye. The kill switch on the console reaches this governor too: engage it and the page draws as the browser alone would.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART SIX

# The Page's Appendixes

*The page's appendixes A to G, where it keeps its build, its explanations, its key and its disclosures.*

CHAPTER 35

# Title Page and Build

This chapter reads the page's appendix A, **Title Page and Build** (`#title-page`).

## 35.1 The title page

The title page (the page's appendix A, `#title-page`) carries the build: the engine that stands and its digest, the commit and the time the repository built the page, the evidence classes (P a meter, L live software, S our models, each marked), the arms ("native · Omni-Compass and native, never Omni-Compass by itself"), and a line that names the engine every table's runs ran on and how many benchmarks are being run again on the engine that stands (`#bEngRuns`). Its opening paragraph says the same in full: two sides everywhere, and only two, native and Omni-Compass and native; Omni-Compass is never by itself, it always sits on top of a native controller; what omni-v18 and omni-v19 added to run it otherwise is still in the engine and is not shown on the page. Under the build lines stand **Product Scope**, the **Colour Key** and, folded, **Coverage, Signals and Result Words** (**Benchmark Coverage**, **Live Signals on the Band**, **Result Words**); its last line points to the two disclosures that head the page's appendix G (chapter 41): this page is a live tuning engine, and the benchmarks are one tuned engine; and real runs, a real engine, nothing hardwired. Where the records stand on a newer engine than the page's words were written for, a banner under the wall says so and says to read the newer engine wherever a sentence names the older one as the engine that stands.

Beside the title, on a desktop, stand the owner's two illustrations, one above the other (on a phone they sit under the title): the stacks Omni-Compass sits on top of as he pictures them, and under it the Omni-Compass as the unified control authority. Each says under it that it is an illustration, not a result, and the first says that the product names and marks in it belong to their owners, with no partnership, sponsorship or endorsement stated or implied. The engine draws over them: a ring at every decision of the live benchmark engine; on the rose two hands, native's (orange, the longer) and Omni-Compass's (blue), each at the mean place of that side's muscles in their bands, calm at north and the line at south; on the control core a glow that grows with the settings Omni-Compass wrote in that decision; and under each, that decision's counts. The marks are the engine's state (the page's section 3a), never a number of the picture's own.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 35.2 What the page is

The live benchmark engine is a monitor of the Omni-Compass engine and of the benchmark runs GitHub ran. A monitor shows what its instrument reads; a drawing shows what its author believes. The page is the former by construction: every record it replays is made again by the repository's own code on the checkout it is built from, and `--check` fails a page whose engine is not the one `python3 tools/omni_version.py` reads. So the page never runs ahead of the repository: when a new engine stands, the build stamps it on the page and rebuilds every record on it, and the page says in a banner that its prose was written for the older engine until a person realigns the words.

It is also a live tuning engine, a tuning bench. Its buttons turn the engine's knobs in real time: the heartbeat (six buttons in the control panel, from 10 to 120 s), the dial between the max and the tune-down, the pedal, each muscle driven or left native, up or down, in or out, minus or plus. Turning them shows the mechanism of action and its variations with native alone, with Omni-Compass on top of native, or both side by side. As on any engine, where the mixture, the intake, the exhaust, the firing or the pistons change what a car's engine delivers, **turning any knob of Omni-Compass can change every metric, up or down, in any direction a tuner chooses**: one engine for many vessels, tuned many ways, for many outputs. The records the page replays are something else: the main engine's one tune, the factory tune (its factory profile), as the benchmarks ran it. That tune is chosen by a declared rule: of the tunes that harm no gauge against native, the one against which no other tried tune reads GAIN on one service gauge and LOSS on none, searched on development seeds and read again on untouched ones (`docs/THE_TUNE.md`). What the page shows at any other setting demonstrates the engine's range and is never a benchmark result; a tune other than the factory tune belongs to its tuner, and so do its results. The box that opens the page's appendix G says which is which (`#tuningBench`). The beat on the dial (the page's section 3d) is one such knob: the factory profile runs it at 60 s, and every other beat is shown beside native with what it buys and what it costs, as range, never as a result. Each muscle's own setting is part of the same factory profile: its card where one was found under the same rule, the engine's setting for the rest (`realms/muscle_cards_v5.csv`).

In the words of `docs/FACTORY_PROFILE.md`, the page is the **live tuning bench**: the interface that operates the engine's controls and displays their effects through the connected backend. The **factory profile** is the complete, versioned set of Omni-Compass default settings shipped together; on the engine that stands it is omni-v23's (`OMNI_V23.json`): the governor's full decision every 60 s, the fast checks every 5 s, every muscle on the card found for it and on the engine's own setting otherwise. The **active profile** is the settings actually governing the current run. An adjustment on the bench produces an identifiable active configuration, a **custom profile**; it becomes a new factory profile only after validation and a new release declaration. The control panel shows which profile is active and its fingerprint (section 2.7).

## 35.3 The five records

The page carries five kinds of record, and every monitor says which it plays.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Record | Made by | What it holds | Class |
|---|---|---|---|
| The harness | `docs/engine/export_harness_trace.py` | the realms harness (`realms/harness.py`, the max of `realms/compass_arm_v5.py` with the bone, the twins and the immune system): one organism of sixty muscles, and each muscle alone, native and with Omni-Compass on top, decision by decision | S |
| The controller | `docs/engine/export_controller_trace.py` | the shipped Kubernetes governor (`omni_controller/controller.py`) on the modelled cluster, on top of the autoscaler at the max and in the cloud's setting, every tick and every audited decision, runs A, B and C of six hours | S |
| The runs | `docs/engine/export_runs.py` | the official runs as recorded: every three-run table's runs A, B and C read back from their raw files, every file's SHA-256 matched against the run's own list, every Kubernetes repetition scored by `tools/live_reps.py` and checked against the run's result | L |
| The scenarios | `docs/engine/export_scenarios.py` | the GPU card (`realms/gpu_card.py`), the Linux host (the conveyance of `tools/model_gate.py`), every muscle and organism of the realms table (`results/realms_v5`), and every judged row of the newest receipts (`results/live/`) | S, with the receipts' own classes |
| The build | `docs/engine/build_live_engine.py` | the engine, its digest and the commit; the time; GitHub's runs in progress and queued; every run started for the chips and the clouds (`docs/engine/github_ledger.json`) | the record of the build |

Beside them the page embeds the records its newer sections read, each named where it is used: the real three-run test of Omni-Compass on native against native (the scorecard and the benefit against native, chapters

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

4 and 5), the twenty scenarios (`results/scenarios/SCENARIOS_20.json`, chapter 27), the whole organism's composition (written by `tools/whole_system.py`, chapter 28) and the glossary (`docs/engine/glossary.json`, chapter 40).

## 35.4 What runs live

Three parts of the governing engine run in the page itself: the brain (`omnicompass/core.py`), the compass law (`omnicompass/compass_law.py`) and the verdict (`omnicompass/verdict.py`), each ported to the browser and proven equal to the Python and the C++ on every build by `docs/engine/check_live_engine.py`. They drive the stress-test model of sixty muscles (the page's sections 3a, 3c, 3e to 3g) and the page's own render loop (the page's section 5j); the page's own runs of the Python's seeded runs and of the core's Monte Carlo (chapter 25) run the same ported code. Every other monitor plays a record.

## 35.5 What the page decides

Nothing. Every word on it is the reading its record carries under the one key (`tools/readings.py`), applied once, in the repository, when the table was written. The page never reads a word off the stream it draws. When a monitor has no record yet (a cloud set still running, a chip run not yet started), it says what the repository holds and shows nothing in its place: the empty slot is itself a true reading of the record.

## 35.6 The appraisal

Every view of the page is reproduced from the repository's own code by an independent path and compared value by value (`docs/engine/APPRAISAL.md`; rerun with `docs/engine/check_live_engine.py` and `docs/engine/check_records.py`). The brain's mathematics agree to machine precision; the harness views and the live controller are identical to the repository's own runs on every tick and every plant. Chapter 47 says how to rerun the appraisal on your copy.

CHAPTER 36

# Reading Guide

This chapter reads the page's appendix B, **Reading Guide** (`#s-guide`).

## 36.1 What the guide holds

The page keeps the explanations of its sections in one place, its appendix B, the reading guide (`#s-guide`), so that every section opens on its monitors and controls. Each section's introduction stands there under the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

section's number and title, in the page's order: 0 Monitor Wall, 1 Control Panel, then 2a to 5j. It opens with the **Monitor Wall Legend** (each monitor's signal and word, what the control panel holds, the five blocks, how a tile is drawn) and the **Organism Ring Guide** (two arms, one demand; what a dot's position means; the engine's code behind the default body). Under *0 Monitor Wall* it sets out the light count, the redline reflex and every light's meaning; under *1 Control Panel*, the fuse box and its seven tabs, the mechanism knobs and the manuals; under each lettered section, what that section shows and where it stands in the record. The legend's rules are read in the sections that follow: the sides, the words, the 0.5% rule, the signal words and their shapes, and the marks.

## 36.2 The sides

There are two sides, and only two: native and Omni-Compass on top of native. Omni-Compass is never by itself: it always sits on top of a native controller. Native is orange (`--steel`, #ff8a1f) and Omni-Compass blue, everywhere: every line, bar, tile and number that belongs to one side wears that side's colour, in every chart and table, and the word "native" itself is orange. Red is never a side; it is only LOSS and the alarm. Violet (`--tuned`) is never a side either: it marks only a knob or a profile turned from the factory tune (sections 2.7 and 2.12). Where a monitor draws the two sides on one axis, native is the left lane or the upper row; where it draws them as two instances (the organism, the phase plane), native is on the left. With the master switch OFF, every Omni-Compass side is drawn in a lighter tint of native's colour, dashed, so both stay in view.

## 36.3 The words

GAIN is green, STEADY is yellow, LOSS is red. Every reading, in every table, chart, bar and tile, wears one of those three colours, or blue for power given, and no other. Power given wears blue, Omni-Compass's own colour, because it is what Omni-Compass put in. A LOSS word is printed white on red in the shape of a stop sign. *The runs disagree* and *the runs differ* are white with a dashed edge, and *shown, not judged* is grey, because none of them is a reading. The size printed beside a word is how far the gauge moved from native and carries the word's own sign (GAIN +, LOSS −, STEADY =), never the gauge's raw direction.

## 36.4 The 0.5% rule

The page reads every change by the key's one floor, half of one percent (`tools/readings.py`). A three-run reading follows `three_run_reading`: the change is the mean of the three runs' changes, each in percent of native; a mean of 0.5% or more reads GAIN, LOSS or power given by the gauge's aim, "in all three of our runs" when every run moved the same way clear of zero, and otherwise "by the mean of our three runs" with the count of runs beyond the noise; a mean under 0.5% reads STEADY, under the floor; runs that moved 0.5% or more in opposite directions disagree. The same rule reads the sweep's three model days (section 2.6) and the three-run column of the page's scoreboards. One run's reading, on the scoreboard and on the control-loop table, is under 0.5% either way STEADY and from 0.5% up the word, whatever the interval says. The lights that compare the two sides at one moment read STEADY only inside ±0.5% of native's value;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

where a light counts (the muscles over their line, the pods waiting), a difference of one is a change. Two values that print the same are never told apart: they read STEADY.

## 36.5 The signal words and their shapes

Every monitor, every section heading and part title whose section holds a monitor, every breaker of the fuse box, the wall's tiles, the scorecard's rows, the twenty scenarios' cells and the dock carry a signal and its word, read on the band:

| Signal | Shape | Word | What it means on the page |
|---|---|---|---|
| green | round | GAINING | low in its band with room to climb, or Omni-Compass ahead of native now (a GAIN) |
| yellow | a yield diamond | STEADY | inside its band, or level with native (a change under 0.5%) |
| red | an octagon, a stop sign, blinking red and black | REDLINING | at the top of its band and holding there, or Omni-Compass behind native now (a LOSS) |
| red | an octagon, blinking red and white | OVER THE LINE | past the top of its band: the alarm |
| dark | no lamp lit | AWAITING | the run that fills this monitor has not landed yet; its light stays dark until it does, never green |
| grey | no lamp lit | NO LINE | a monitor with no line to cross (a bill, a count of runs, a digest check) or no reading yet; never counted green |

These four live words belong to the moving monitors. A result fixed after the fact, in a table, reads GAIN, STEADY or LOSS (with power given for a resource spent that rose): GAINING and GAIN go together, STEADY and STEADY, REDLINING and LOSS.

Every signal shows all four live words at once, the lit one bright and the rest dim, on native's side and on Omni-Compass's side alike; a monitor that draws both sides carries a full signal for each, with the result now against native (GAIN, STEADY, LOSS) beside them. Omni-Compass's redline reflex is built never to let a muscle redline: at the top of a muscle's band it pushes at once, before the line. Where a muscle stays over its line three decisions in a row and is stuck (at its own limit, or not relieved by 5% of where the redline began) and native's own twin reads lower by more than 5%, it hands the muscle back to native for 10 decisions, the hold doubling for each hand-back in a row up to 160; it takes the muscle back only when its side is under the line and lower than native's by 5%. Where native is not stronger it stays, because redlining less than native is still a benefit against native. The red is still wired on its side, so that if a muscle does redline there (a fault, a bug), it is seen at once. That is the kill switch's purpose: when something redlines that Omni-Compass itself

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

cannot stop, one press hands every muscle back to native in the same instant. The engine that stands, omni-v23, carries this rule in its immune system, knob by knob (`omnicompass/immune.py`, its C++ twin `cpp/src/immune.cpp`, proven alike on 200 seeded scripts, and the nervous system's `native_hold`; on Kubernetes each service's autoscaler target, native's twin the operator's own target), beside the SLO reflex and the pod reflex of the Kubernetes governor (`omni_controller/controller.py`); the page's live model (class S) applies the same rule, with the engine's own numbers, to every muscle it draws.

Each light reads what its gauge is for, never the direction of its needle: a wait that falls is GAINING, work that falls is not.

The number of every monitor carries the colour key: Omni-Compass blue behind the number of a monitor that shows Omni-Compass on top of native, native orange behind one that shows native alone, both (orange first) behind one that shows the two side by side, with black type. The key is fixed: no switch recolours it.

The signal stands as a traffic signal does, red on top, yellow in the middle, green at the bottom, one lamp lit. Red appears only in a stop sign's shape: a red light is an octagon, a red word (REDLINING, LOSS) is a stop sign's shape printed white on red, and on the canvases any whole circle drawn in red is drawn as an octagon. A section's or a part's signal shows the worst of its monitors' lights (red over yellow over green; AWAITING only where every monitor awaits); hovering it gives the count of each.

## 36.6 The sides are never a reading

Blue does not mean good and orange does not mean bad: they name who is being read. Where the two sides look the same, they are the same, and that is STEADY. Where Omni-Compass's blue line sits closer to a muscle's line than native's orange one, that is the max at work: every muscle to its own limit, never past it, and the capacity it holds there is power given, shown beside the work and the speed it bought.

## 36.7 The marks

A kill-switch window is shaded red across every time axis. A monitor on screen now carries an accent border on its tile. A tile with no reading says what it shows (a count, a state, an age) and carries no colour of a word. A row shown and not judged (the knob's own setting, the governor's own commands, Omni-Compass's own processor time) is grey and counted in no total. A ring that opens and fades at the head of a line is the decision that has just landed on it, and the small box beside the head prints that line's newest value, as a quote board prints the last price. A box drawn round a row, a cell or a bar of a record monitor is its reading cursor: it walks the record one row for each decision of the live benchmark engine, and the row it holds is named under the monitor; the record itself changes only when GitHub's next run rebuilds the page. An age printed beside a reading of GitHub ("read 06:16 UTC, 41 min 07 s ago") counts up every second.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 37

# Problem and Answer

This chapter reads the page's appendix C, **Problem and Answer** (`#s-problem`).

## 37.1 What it holds

The problem and the answer (the page's appendix C, `#s-problem`) prints in one paragraph what every stack faces, what Omni-Compass does about it, where the proof is, what is not claimed and what is protected, and unfolds (**Problem, Answer, Proof, Limits and Protection**) into the whole section: **Industry Problem**, with its outside sources named and quoted only for scale; **Omni-Compass Answer**; **Costs and Responses**, how it addresses each cost and how each is judged; where the page shows each answer; what it does not do; and how it is held to account. This book carries the same matter in its front matter, "The Problem and the Answer".

CHAPTER 38

# Executive Summary and Reading Key

This chapter reads the page's appendix D, **Executive Summary and Reading Key** (`#s-read`).

## 38.1 What it holds

The executive summary and reading key (the page's appendix D, `#s-read`) opens with the Omni index, every real test weighed together, and one paragraph on the two sides, the three words and the three classes of evidence, and unfolds (**Summary, Key, Terms and Examples**) into the **Executive Summary** (every number read from the records on the page when it opens), the **Reading Key** (GAIN, STEADY, LOSS and power given, the colours, **Size, Sign and Raw Change**, and **Sign and the Gauge's Aim**: why a minus can be a GAIN and a plus a LOSS) and the terms the page uses, each defined once and shown when pointed at. This book carries the key in its front matter, word for word as `tools/readings.py` writes it.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 39

# Monitor List

This chapter reads the page's appendix E, **Monitor List** (`#s-monlist`).

## 39.1 Every monitor by its number

The monitor list (the page's appendix E, `#s-monlist`) is the list that goes with the wall, **Monitors by Number**: every monitor by its number, in the same order, with what it shows, its section, its layer, its evidence class and state (live, replay, record, model or awaiting its run), the open-source exporter and series that would carry it on a running system, its knobs, its wiring (the page's code, the Python, the C++ and the checks that hold them equal) and the motion meter's reading, and, where it waits, the run that fills it. Click a number to go to its monitor, a section to go to its section. The feeds name the open-source exporters an operator's Prometheus and Grafana already run (node_exporter, kube-state-metrics, the NVIDIA DCGM exporter, Kepler, OpenCost, the governor's own `tools/omni_exporter.py`); the button in the control panel downloads the whole wall as a Grafana dashboard on those series (`deploy/grafana/omni-compass-wall.json`, written from `docs/engine/monitors.json`), beside the governor's own screen (`deploy/grafana/omni-compass.json`). A monitor the page draws from the repository's tables, or from its own clock, says so: no series carries it.

CHAPTER 40

# The Glossary and Index

This chapter reads the page's appendix F, **Glossary and Index** (`#s-glossary`).

## 40.1 The four tabs

The glossary and index (the page's appendix F, `#s-glossary`) says what every word on the page means, written from the repository's own pages (each definition names its source file and quotes it), the organisms by their realms and families, and every muscle of the catalog (`realms/catalog_v5.csv`) with the organisms it sits in; the index sends each term to the sections where it is used. It is built by

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`tools/live_glossary.py` into `docs/engine/glossary.json`, and a term whose source no longer says what it is cited for stops the build. Four tabs open its four parts:

- **Terms**: every term with its group, its definition, its other spellings, the sections that use it, and its source with the quoted words; letters A to Z jump through them.
- **Organisms and Scenarios**: the ten organisms and the twenty scenarios, each organism with its muscles, its class, its meaning, its families, its governors where it has them, and its two scenarios, each with its signal word and number from the twenty scenarios' record (click one for its gauges and its three runs, chapter 27). The two arms are named in their colours: native orange, Omni-Compass on native blue.
- **Muscles by Realm and Family**: every one of the catalog's 1,107 muscles once, the spine first and then each realm's own muscles, by family; open a family for its wiring (who runs it natively, what Omni-Compass reads and writes, how it is restored) and its muscles, each with its id, its name, what it is, what the max does with it, its card and the organisms it sits in.
- **Index**: every term, organism and arm, and the sections of the page that print it, in the page's own numbering.

## 40.2 How to find a thing

Type in the search box: the glossary answers with every term, organism and muscle whose words match, the muscles capped at the first 200 with the rest named by count. Click a letter to jump among the terms, a realm among the muscles, and an index entry to its definition. This book's own glossary stands in its back matter; where the two differ in wording, each names its source.

CHAPTER 41

# Disclosures, Law and Contact

This chapter reads the page's appendix G, **Disclosures, Law and Contact** (`#s-disclose`).

## 41.1 What the section holds

The page closes on its appendix G, **Disclosures, Law and Contact** (`#s-disclose`), before the closing legal notice (`#notice-end`). It opens with the live tuning engine box (`#tuningBench`): **THIS PAGE IS A LIVE TUNING ENGINE. THE BENCHMARKS ARE ONE TUNED ENGINE.** Every knob, button and monitor on the page can be turned in real time (the heartbeat at 10, 15, 30, 60, 90 or 120 s, the dial between the max and the tune-down, the pedal, each muscle driven or left native), and turning any knob can change every metric, either way; what the page shows at any setting other than the factory tune demonstrates that range and is never a benchmark result. Every benchmark result is one engine at its one factory tune, chosen by the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

declared rule of `docs/THE_TUNE.md`; a tune other than the factory tune is the tuner's own, and its results are its own. No profile on the page is locked as the baseline (`docs/FACTORY_PROFILE.md`).

### 41.2 The disclosures under it

Under the box stand the disclosure of real runs (**REAL RUNS, A REAL ENGINE, NOTHING HARDWIRED**: every run played back is read from the raw files of runs executed on GitHub's own machines, each file checked against its run's SHA-256 list before a number is drawn; every model view runs the engine's own code, checked against the Python and the C++ on every build), the cloud's back end (a tenant cannot meter a provider's machines), the road to the engine that stands (**Engine Lineage**), what the page is (**Page Scope**), do no harm in its one meaning as `DISCLOSURES.md`, section 3, writes it, the governing law, and the company's contact lines. The legal notice opens the page in its banner and closes it under this section; chapter 46 gives the license's terms.

PART SEVEN

# Reading a Result off the Page

*The mechanism of action side by side, the trace from a tile to its record, what to do when a tile reads LOSS, and what the page does not tell you.*

CHAPTER 42

# The Mechanism of Action, Side by Side

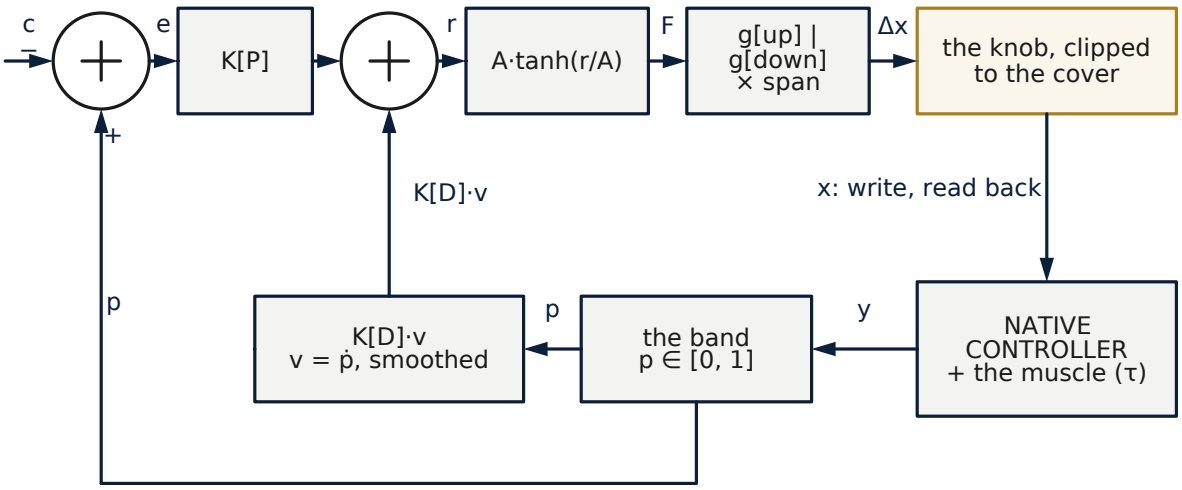
### 42.1 What native does

Each native controller reads one reading, its own muscle's, one to three decisions late (the pod autoscaler one sync every 15 s), and moves its one setting toward the middle of its range, because protecting its one thing is its whole job. It has no heart (nothing proves it is still there), no brain (no state, no memory of need, no stress, no tide), and no nerves (nothing grants or refuses it, and a blind sense or a security hold changes nothing it does). Its moves are the second ticker of the page's section 4a and the orange panel of the page's section 3g.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 42.2 What Omni-Compass does, on top

Once every decision it reads the whole body through the plug (the senses), places every reading in its band, lets the brain's state evolve, grants or refuses through the nerves, applies the compass law to each muscle (push toward more as the reading climbs, lift toward native as it falls, smooth, never hammering), runs the verdict's trial on any step past native, passes the release gate, writes through the plug and reads it back, and renews its lease. Every one of those is a column of the audit, a line of the first ticker and a box of the cycle monitor. The native controllers keep running underneath, untouched; Omni-Compass moves only the settings they already expose.




c the centre of the band; y the reading; p the reading's position in the band; e = p − c; v = ṗ smoothed; r the raw force; F the force after the saturation; Δx the step; x the knob. Fail up: p ≥ 0.95 → F = A at once; the down side held until p is back inside the band. No take-back while the reading climbs above the centre (F < 0 with v > 0 and p > c → F = 0). K[D] is set from the muscle's own response time τ (critical damping), never tuned by hand.

**Figure 11.** Block diagram: the compass control loop. The muscle's reading comes in on the sense terminal, is placed in its band, and the compass law computes a force; the force is a setting written out on the motor terminal, inside the knob's cover, through the plug, and read back; the native controller sits inside the loop, still reading its own gauge and still moving its own setting; the kill switch opens the motor terminal and the lease restores the snapshot. On the page the two tickers show the two loops, native's own and the governor's on top of it, at the same seconds.

## 42.3 Where to see the difference

On the official run playback (chapter 18): the first ticker shows the pedal pushed at the compass position's first move off the center and the machines recommended before native's first late sample; the second ticker shows native's autoscaler adding replicas a sync or two later. On the phase plane (chapter 14): the blue dots turn back before the right edge where the orange ones cross it. On the scoreboard: the work waiting lower, the decisions past their line fewer, the capacity held higher during the climb (power given) and eased back after it (the lift).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

On the brain (chapter 15): calm falling under a surge and the grant refused until the line is clean again, which native has no equivalent of.

## 42.4 What the difference is not

It is not a competitor's dashboard. Omni-Compass never replaces the controller it sits on; every move native makes is still made. A monitor where the two sides are identical is a muscle Omni-Compass left native, at its limit or with nothing to add, and that reading is STEADY and correct.

CHAPTER 43

# From a Tile to Its Record

## 43.1 The chain

Every number on the wall can be walked back to the file it came from.

1. **The tile** names its gauge and its word, and links to the monitor it copies.
2. **The monitor** names the benchmark, the run (A, B or C) and the repetition, or the table and the row, and the evidence class.
3. **The three-run table** is in `results/live/` (each named `V<n>_*.md` for the engine it ran on; each table names its engine), with its runs' ids and its rule printed at its head.
4. **The run's raw files** are on the `raw` branch at `results/live/raw/run-<id>/`, with `SHA256SUMS_ALL.txt`; the page matched every file it drew against that list. `scripts/raw_archive.sh open /run-<id>/` places them where the table names them.
5. **The governor's audit** is the run's `audit.jsonl`; native's own controllers' moves are its `capture.csv`.
6. **The scorer** is `tools/live_reps.py`; the page's per-repetition score equals the run's own summary over its repetitions.

## 43.2 The engine's name and the profile

Every table and every monitor names the engine it ran on; the page's title block names the engine the page stands on (omni-v23). A table on an older engine stands only until its runs on the engine that stands land, and the board says which it is. Nothing is read across engines. Every result the page plays ran the factory profile of the engine its table names; the control panel names the profile governing the live model now, with its fingerprint (section 2.7), and a candidate or custom profile there is never a result of record.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 44

# When a Tile Reads LOSS

## 44.1 The rule

Theoretically the word LOSS should never appear: the engine that stands moves a knob only where a paired trial on the muscle itself shows the muscle never behind for it, and where it cannot bring a knob a gain, that knob stays native. When LOSS does appear, the row stays, and the rule governs: if Omni-Compass hurts something, no mechanism of Omni-Compass is pushed through that muscle. The cause is found in this order, our own wiring first, native's setup second, the scoring third; the mechanism or its wiring is corrected; the test runs again on the next engine version, and the earlier table stands as its engine's record.

## 44.2 Read the three-run reading, not the one run

A red row from one run of a table still running (the Azure steady set at 33 of up to 100 repetitions) is that run's own word for that run and not yet a three-run reading: a three-run word needs all three runs, and it reads "in all three of our runs" only when every run moved the same way clear of zero. Wait for the look.

## 44.3 Open the root-cause document

Where the repository has found the cause it is linked from the tile: `docs/POD_START_WAIT.md` (a pod's start on one shared host is the test machine's own unpacking of an image, the same on both sides at the same host load), `docs/TAIL_LATENCY.md` (where a 95th or 99th percentile reads higher, and why), `docs/BATCH_QUEUE_PREREGISTRATION.md` amendment 11 (the conveyance on a shared host beside another workload's room, fixed in omni-v13). A tile with a root-cause document is a known cause with its fix landed or on the next version's list.

## 44.4 Which knob, and what to turn

The LOSS row names its gauge; the governor's audit names the knob that moved in that window and why. Three outcomes follow, and only these:

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| What the audit shows | What it means | What is done |
|---|---|---|
| a knob moved and the gauge went against its aim while the muscle stayed inside its limit | the wire or the rule is wrong for this muscle | the mechanism is corrected in the model gate first, lands as the next engine version with no LOSS, and the benchmark runs again |
| a knob moved and the muscle went past its own limit, or another muscle's room was taken | a guard did not hold | the guard is corrected (as omni-v13 did for the conveyance on a shared host) and lands the same way |
| no knob moved, and the gauge still differs | the test machine, native's setup or the scoring | the rig or the scorer is fixed; a test machine's limit is never reported as Omni-Compass's result |

Nothing is ever tuned to make a row read well; no row is steered. What is held to the letter is the mechanism: the code does what its rule says, the wire is fastened right, the harness measures as a deployment runs. On the page, each muscle of the stress-test model and of the replay carries its own switch on its card (the page's section 3e), and the mechanism panel's last group carries the same switch for a muscle, its body part or every muscle (section 2.22): the muscles in scope handed back to native while the rest stay governed, to see their part in the result. On a real machine the same choice is the muscle's wiring (the Operator's Manual's wiring of each muscle).

## 44.5 On your own system

The Operator's Manual's "wired right or wired wrong" (the Operator's Manual's section 4.5) and its drill book (the Operator's Manual's chapter 28) are the procedure; `DISCLOSURES.md`, section 4, is the rule: where a result elsewhere differs from the published record, the first presumption is the wiring, the settings or the reading, not the law.

CHAPTER 45

# What the Page Does Not Tell You

## 45.1 It decides nothing

The page is a monitor. Every word on it was decided in the repository when its table was written. A reading you disagree with is taken up with the table and its runs, never with the page.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 45.2 Class S is not proof

The harness, the shipped controller on the modelled cluster, the GPU card, the Linux host, the realms table, the grid's modelled half and the orchestra are our own models. They show the mechanism; they do not measure a machine. A model's energy is not a meter's. The page says the class on every view, and the manuals say it again.

## 45.3 Slots not yet run are empty on purpose

The metered chip runs, the CPU host's meter, the clouds' sets still running and the register's third-party slots are written as the repository holds them and marked not yet run, and a monitor whose run has not landed reads AWAITING, never green. An empty slot is a true reading of the record. Nothing is drawn in its place.

## 45.4 The page's age

A page is true for the build it names. The status board's Build Freshness tile is its age; the title block is its commit. A reading landed after the build is on the next page.

## 45.5 The page's own rendering is measured on your machine

The self-governed rendering is class L on the viewer's own device; its numbers are the viewer's browser's and prove nothing about any server.

## 45.6 Ask Omni-Compass is composed

Its answers are drawn from the page's records by a language model on the viewer's own account, are not reviewed by any outside party, and are not a record. The records are the tables and the raw files.

## 45.7 A changed profile is not a result

The live tuning bench shows the governing mechanism so that the profile can be moved, demonstrated and studied. A candidate or custom profile on the bench is a variation of the mechanism (class S); the page's live model has no meter, so it gives no physical energy reading; and each route's bill stays unresolved where no price is recorded. The page tells you what the factory profile did on the systems its records name, and what the mechanism does when a knob is turned; it does not tell you what a changed profile would do on a real system until that profile is validated and released (`docs/FACTORY_PROFILE.md`).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART EIGHT

# License and Your Copy

*The terms the page and this manual are licensed under, and how to verify a copy of the page.*

CHAPTER 46

# License and Commercial Terms

The live benchmark engine page, this manual and every part of the repository are the property of The Omni-Compass LLC; all rights reserved; all patents, copyrights and trademarks filed in the USA. Use is evaluation and simulation only under `LICENSE` (`LicenseRef-OmniCompass-Evaluation-1.0`); any commercial use, commercialization, monetization, production use, redistribution or hosted service requires a signed, paid Omni-Compass Enterprise License (`ENTERPRISE_LICENSE.md`). The license is governed by the laws of the State of Delaware and the federal laws of the United States, and the state courts of Delaware and the United States District Court for the District of Delaware have exclusive jurisdiction. Serving the page to others is a hosted service and needs the Enterprise License. The license, the licensing questions and answers, the third-party notices, the Enterprise License's standard terms, the patents and trademarks and the security policy are bound into this book in full (Appendices F to K). Licensing, pilots and evaluations: the Contact page.

CHAPTER 47

# Verifying Your Copy of the Page

## 47.1 The engine

`python3 tools/omni_version.py` prints the engine the checkout stands on and its digest; the page's `<html data-engine>` must name the same. `python3 docs/engine/build_live_engine.py --check` rebuilds the page in memory and exits 1 if any record differs from the page; nothing is written.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 47.2 The mathematics

`python3 docs/engine/check_live_engine.py` runs the page's brain, compass law and verdict against the Python and the C++ (`docs/engine/cpp_core_probe.cpp`) and prints the largest difference, which is at machine precision.

## 47.3 The records

`python3 docs/engine/check_records.py` reproduces every record the page carries by an independent path and compares value by value; `docs/engine/APPRAISAL.md` is the standing result.

## 47.4 The wiring

`python3 -m unittest tests.test_live_monitors` holds every monitor to its wiring: every page function a monitor names is in the page's script; every Python, C++ and check file it names is in the repository; every monitor sits in a numbered section; every section of the page closes what it opens; and every panel of the Grafana wall sits under the row of its block of the wall, its section named in its description, numbered as the page numbers it.

## 47.5 The whole

`python3 verify.py` runs every check of the repository, this page's included, and ends `VERIFICATION: PASS`; `python3 tools/release_manifest.py` writes `RELEASE_MANIFEST.json` at the commit and must read 0 FAIL. A copy whose page, engine or records differ from what these print is not a copy of the record.

# Back Matter

BACK MATTER

# Glossary

- **Active profile.** The settings actually governing the current run (`docs/FACTORY_PROFILE.md`); on the page, the first line of the control panel's profile box, FACTORY, CANDIDATE or CUSTOM, with its fingerprint.
- **Actuation.** Each decision's requested settings, those that landed as asked, those held at a muscle's limit, and those still moving toward their setting; the profile box's third line.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Alarm.** A judged row of a real or modelled run that read LOSS, a run that ended without a measurement, or a watch item, raised on the status board worst first with its source and its root-cause document.
- **Anchor.** The part of a page address after `#` (`#s-wall`) that names a section and does not move when sections are renumbered.
- **AWAITING.** The signal word of a monitor whose run has not landed: dark, never green.
- **Band.** A reading's safe range, from calm (p = 0) through its center (p = 0.5) to its line (p = 1), with 5% cushions.
- **Beat.** One turn of the governor's loop: read, decide, write, renew the lease. The heart.
- **Breaker.** One monitor's switch in the fuse box: ON draws it live; REST stops its drawing.
- **Calm.** $\kappa \cdot h \cdot (1 - (\sigma - 1)^{+}) \cdot (1 - \min(v, 1))$: the nerves' one number, against which each organ's threshold $\theta$ grants a hand-back.
- **Candidate profile.** A proposed combination of settings under evaluation; on the page, the sweep's best beat applied by *Try the candidate*, never a release.
- **Class (P, L, S).** The evidence class of a number: a physical meter, live software, our own models.
- **Compass position (p).** Where a reading sits in its band; 0 calm, 1 at the line.
- **Console.** The controls of the page's section 3c: every runtime control of the live benchmark engine, acting on both sides alike.
- **Control panel.** The control panel, the page's section 1 (`#s-health`), directly under the wall: wall status and factory tune, the fuse box, and the kill switch with the runs, the variation switchboard, the reports and the manuals.
- **Custom profile.** An identifiable configuration that differs from the released factory profile; its fingerprint names it, and its results are its own.
- **Decision.** One turn of the governor; the unit of time on every live monitor.
- **Defensive system.** Constraint enforcement, fault detection, containment, recovery and restoration: the bone and the twins, the lease's walk back, the kill switch.
- **Dial.** The max (every muscle to its own limit, never below native; the default) or the tune-down (never more than native).
- **Engine that stands.** The engine version on main, named on the page's title block and in `python3 tools/omni_version.py`; omni-v23 for this edition.
- **Factory profile.** The complete, versioned set of Omni-Compass default settings shipped together; on the engine that stands, omni-v23's, the heartbeat 60 s, every muscle on its card. Every published result is this profile's.
- **Fingerprint (of a profile).** The page's FNV-1a hash of every setting of the mechanism, the heartbeat and every muscle's body included: one identity for one set of settings.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Fuse box.** The second panel of the control panel (`#fuseBox`) that switches every monitor from one place: the master rows, the heartbeat, the sweep, the profile, each route's bill, the mechanism knobs, the twenty scenarios, the injection grid and the breakers.
- **Golden signals.** Latency, traffic, errors and saturation: on the page, the golden-signal feed of the page's section 3b.
- **Governing engine.** The executable mathematical rules that turn observations and internal state into governing decisions.
- **Harness.** The machinery that supplies workloads, executes comparisons, checks requirements and records results; on the page, the realms harness's record (`realms/harness.py`).
- **Health.** The explicit operating requirements: correctness, service limits, resource bounds and every applicable hardware limit.
- **Heart.** The governor's beat and its lease, drawn on the official run playback.
- **Immune system.** Omni-Compass's: native twins, the heat debt, the cure on the spot, the pools' give-back, the bone's grant.
- **Kill switch.** The instant hand-back of every lever to native, for security; red on the switch and on every time axis.
- **Lease.** The governor's proof of life; when it ages past its limit the watchdog walks every setting back to native.
- **Live / Recorded.** A knob marked Live acts on the live benchmark engine; one marked Recorded data chooses a record and modifies nothing.
- **Live tuning bench.** The interface that operates the engine's controls and displays their effects through the connected backend: the live benchmark engine page and its control panel.
- **Master switch.** The one switch at the top of the organism: ON (green), the kill switch engaged (red), OFF (white).
- **Monitor.** One canvas of the page that draws a record or the live benchmark engine; its small copy on the wall is a tile.
- **Muscle.** A specific controllable mechanism, with its own actuator, observations, limits and read-back.
- **Native.** The whole system by itself with everything it ships with: the native baseline; orange.
- **Native baseline.** The system's documented configuration and behaviour with no Omni-Compass governing it: every muscle under its own native controller, as its vendor ships it.
- **Nervous system.** Observation, signal propagation, coordination and authority pathways.
- **Omni-Compass.** Omni-Compass on top of native; blue. Never by itself: it always sits on top of a native controller, so the page has two sides and only two.
- **Organism.** The complete declared system whose useful output, health and resource costs are assessed together.
- **Pedal.** Push for more, lift for less; the force column of the governor's ticker.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Power given.** A resource spent or held that rose: what Omni-Compass put in to buy the work and speed beside it; blue.
- **Reading guide.** The page's appendix B (`#s-guide`): every section's introduction under its number and title.
- **REDLINING.** The red signal word, an octagon, blinking: at the top of its band and holding there, or Omni-Compass behind native now (a LOSS); OVER THE LINE is past the top of its band.
- **Scorecard.** The scorecard, the page's section 2a: one light and one number for each way to run a workload against native, and the whole workload's index on each route.
- **Shown, not judged.** A gauge declared in advance as shown only; grey; counted in no total.
- **STEADY.** The yellow signal word, a yield diamond: inside its band, or level with native (a change under 0.5%).
- **Stopping rule.** The cloud sets' preregistered looks at 10, 20, 40 and 70 repetitions with 99% intervals (`tools/stop_rule.py`).
- **GAINING.** The green signal word, round: low in its band with room to climb, or Omni-Compass ahead of native now (a GAIN).
- **Ticker.** The governor's decisions (first ticker) and native's own controllers' moves (second ticker), newest first.
- **Tile.** One small monitor on the wall; click it to go to the monitor it copies.
- **Twenty scenarios.** Ten organisms, each native and with Omni-Compass on native, each read against native.
- **Verdict.** The paired trial every step past native must pass: kept unless it reads LOSS or exceeds the 2% allowance.
- **Violet.** A knob or a profile turned from the factory tune (`--tuned`); never a side and never a reading.
- **Wall.** Every monitor of the page in one view (the page's section 0), numbered 1 upward by its place in the wall's five blocks, each block read left to right, then the next row.

APPENDIX

# Appendix A. The Page, Section by Section

| Page section | Anchor | Title | Chapter of this book |
|---|---|---|---|
| — | `#topBanner`, `#notice` | Banner: logo, name, engine chip and legal notice | Notice and Disclaimer |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Page section | Anchor | Title | Chapter of this book |
|---|---|---|---|
| 0 | `#s-wall` | Monitor Wall | 1 |
| 1 | `#s-health` | Control Panel (with the fuse box, `#fuseBox`) | 2; 36.1 |
| — | `#contents` | Contents | 3 |
| 2 | `#p-results` | Results | 4 to 8 |
| 2a | `#s-bottom` | Scorecard | 4 |
| 2b | `#s-answer` | Benefit Against Native | 5 |
| 2c | `#s-board` | Benefit Board | 6 |
| 2d | `#s-loss` | LOSS Register | 7, 44 |
| 2e | `#s-status` | Status Board | 8 |
| 3 | `#p-live` | Live Operation | 9 to 17 |
| 3a | `#s-engines` | Live Organism | 9 |
| 3b | `#s-feed` | Golden Signals and Event Log | 10 |
| 3c | `#s-desk` | Control Console | 11 |
| 3d | `#s-dial` | Heartbeat Dial | 12 |
| 3e | `#s-decision` | Control Cycle | 13 |
| 3f | `#s-plane` | Phase Plane and Scoreboard | 14 |
| 3g | `#s-brain` | Brain State | 15 |
| 3h | `#s-verdict` | Verdict | 16 |
| 3i | `#s-dash` | Dashboards | 17 |
| 4 | `#p-runs` | Real Runs and Platforms | 18 to 24 |
| 4a | `#s-ops` | Kubernetes Run Playback | 18 |
| 4b | `#s-runs` | Repetition Telemetry | 19 |
| 4c | `#s-live` | Controller Decision Trace | 20 |
| 4d | `#s-cloud` | Cloud Providers | 21 |
| 4e | `#s-orch` | Orchestrators and Governors | 22 |
| 4f | `#s-stacks` | GPU and CPU Hardware | 23 |
| 4g | `#s-github` | CI Pipeline | 24 |
| 5 | `#p-proof` | Proof and Models | 25 to 34 |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Page section | Anchor | Title | Chapter of this book |
|---|---|---|---|
| 5a | `#s-proof` | Python Runs and Monte Carlo | 25 |
| 5b | `#s-cpp` | C++ Twin | 26 |
| 5c | `#s-scn` | Scenarios and Injection Grid | 27 |
| 5d | `#s-whole` | Whole Organism | 28 |
| 5e | `#s-fleet` | Measured Effect | 29 |
| 5f | `#s-lab` | Telemetry Analytics | 30 |
| 5g | `#s-master` | Master Switch Wiring | 31 |
| 5h | `#s-math` | Specification and Mathematics | 32 |
| 5i | `#s-fail` | Failure Modes and Controls | 33 |
| 5j | `#s-self` | Self-Governed Rendering | 34 |
| — | `#p-appendix` | Appendixes | 35 to 41 |
| A | `#title-page` | Title Page and Build | 35, 43 |
| B | `#s-guide` | Reading Guide | 36 |
| C | `#s-problem` | Problem and Answer | 37; The Problem and the Answer |
| D | `#s-read` | Executive Summary and Reading Key | 38; the key |
| E | `#s-monlist` | Monitor List | 39 |
| F | `#s-glossary` | Glossary and Index | 40 |
| G | `#s-disclose` | Disclosures, Law and Contact | 41, 46 |
| — | `#notice-end` | Closing legal notice | Notice and Disclaimer |

APPENDIX

# Appendix B. Every Control

| Control | Where | Acts on | Leaves alone |
|---|---|---|---|
| Kill switch | the control panel; the head of the page's section 1; every section's | every lever of every governor on the page, to native at once; shades | the records: nothing recorded is modified |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Control | Where | Acts on | Leaves alone |
|---|---|---|---|
| | controls; every monitor's strip; the console; the dock | every time axis red; a second press releases it | |
| Return to native (the reset) | the fuse box's profile row; the variation switchboard's native arm | the same hand-back, named the reset; the switch reads "OFF by the reset" | the records: nothing recorded is modified |
| Turn Omni-Compass OFF / ON | the console | the brain's outputs: OFF, read in shadow, nothing leaves; ON, outputs reach the muscles | the native side, which never stops |
| The max / Tune-down | the control panel; the console; the page's section 3e | the dial of the live benchmark engine's governor | the records; the official runs' own dial, named in each |
| The body | the control panel; the page's section 3e | which body the live monitors draw | the records |
| Heartbeat: 10, 15, 30, 60, 90, 120 s | the control panel | the live model's full decision period | the dial's tables (the page's section 3d), which are records |
| Run every beat (10 to 120 s), three days each | the control panel | three model days at each beat on the live model; the sweep's table | the live day in hand, restored after |
| Try the candidate | the control panel | the heartbeat set to the sweep's best beat, a candidate profile | every other knob |
| Restore factory profile | the control panel | the mechanism's knobs and the heartbeat back to the factory's; a hold released; the candidate cleared | the master switch; the plant's conditions |
| Hold the active profile | the control panel | every knob of the mechanism, the heartbeat buttons, the sweep and the beat rows held | the plant's conditions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Control | Where | Acts on | Leaves alone |
|---|---|---|---|
| On site / Through the cloud | the control panel | which route's bill the profile box shows | the composition |
| Mechanism Knobs; The mechanism back to the factory | the fuse box (`#mechPanel`) | the live model's equations, on Omni-Compass's side (sections 2.11 to 2.23) | the harness record; the records |
| Every breaker ON; rest every monitor off screen; ON / REST | the control panel | whether each monitor is drawn live | the records |
| Your own system: Connect / Disconnect | the control panel | each monitor's query asked of the operator's Prometheus, read only | the operator's system: nothing is written |
| Demand × | the control panel; the console; the dock | every muscle's demand, both sides, from this decision on | the seed |
| Noise σ | the control panel; the console; the dock | each muscle's demand each decision, 1 + σ·z, one draw shared by both sides | the seed's day |
| Sense delay (+0 to +6 decisions) | the control panel; the console | the decisions between a muscle's act and its controller's reading, both sides | the demand |
| Frame load (as it is, +8 ms, +16 ms) | the console; the page's section 5j | the time burned inside every frame of the page | the engine |
| Load profile, Inject | the console | the same multiplier on every muscle and both sides | the day's mean load, kept |
| Surge ×2 / Lose half its machines / A sense goes blind / Security hold | the control panel; the console; the page's section 3e; the status board's tiles | one body part for 40 decisions, both sides | every other part |
| Pause / One decision / 1× 4× 16× 60× | the control panel; the console; the page's section 3e | the console's clock | nothing else |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Control | Where | Acts on | Leaves alone |
|---|---|---|---|
| The same day again / A new day | the console; the page's section 3e | the seed replayed or redrawn, with the knobs as set | the knobs |
| Every knob to its start | the console | demand, noise and delay, back to the day as written | the dial and the switch |
| Back to the factory tune | the control panel's first panel and its fuse box; the console; the dock; every monitor's strip; the key `f` | the factory profile and the plant at rest (section 11.2) | the records |
| Arm, Organism, Organisms, Runs, Run, Beat (the variation switchboard) | the page's section 1 | every monitor that carries the variation (section 2.31) | every monitor that does not |
| A / B / C | the page's sections 4a, 4b, 4c, 4f | which run is played | the other runs |
| Benchmark, run, repetition | the page's sections 4a, 4b | which record is played | the record itself |
| This muscle; Every muscle back under Omni-Compass | the page's section 3e, on the muscle's card; the mechanism panel's last group | one muscle of the stress-test model run by native's own controller alone, or in a replay shown by native's own record, while every other muscle stays governed | the body's recorded totals in a replay; the kill switch, which hands every muscle back at once |
| Its own load / The climb / Work waiting in line | the page's section 5e | which load the realms table is read under | the table |
| On-Site Bare Metal / Through the Cloud / In the Device / As a Service | the page's section 5d | which way the stack is bought in the composition | the receipts |
| Every Piece, Intervals Carried / Strict Reading, Index Rule; Chip Run: Beside the Count / In the Count | the page's section 5d | which rule Composed Benefit by Gauge counts by, and whether the chip's run enters; the scorecard's bills and the | the factors |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Control | Where | Acts on | Leaves alone |
|---|---|---|---|
| | | route's bill follow | |
| All / LOSS / The runs disagree / Live Now / Guards and Open Items / GitHub; All / ALERT / WARN / INFO | the page's sections 2e, 3b | which rows are shown | the rows |
| Copy the exposition | the page's section 3b | the clipboard | the page |
| Inspect; Full screen | every monitor's strip | the monitor's reason, feed, wiring and downloads; the monitor full screen | the monitor's data |
| Run the Python's 500 runs here; a new set (base seed, count); print and download the receipts | the page's sections 1 and 5a | the page's own run of the Python's seeded runs, its digest against the Python's | the Python's record, `results/engine_runs/` |
| Jump generator · OFF / ON | the page's section 1 | set after set from the Python's jump generator, through the proof monitor and the Monte Carlo monitor | the records the page plays when it is OFF |
| Speak aloud / Ask Omni-Compass / Stop | the page's section 5f | the viewer's browser voice; a question composed on the viewer's own account | the records |
| The glossary's search, letters and four parts | the page's appendix F | which terms, organisms or muscles are shown | the glossary's record |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix C. The Gauges on the Wall

| Gauge | Side | Aim | GAIN reads | Power given reads | Where it comes from |
|---|---|---|---|---|---|
| The work waiting | both | time waited: lower | the blue line under the orange | — | the live benchmark engine; on the official runs, pending pods and the queue |
| The work done | both | work done: higher | the blue line over the orange | — | requests served, jobs finished, work inside the line |
| The decisions past their line | both | missed: lower | fewer blue past the line | — | the compass positions over 1 |
| The capacity held | both | a resource held | a fall (a cut) | a rise | machines, replicas, clock in service |
| Response time, mean, 95th, 99th | both | time waited: lower | a fall | — | the runs' own samples, binned |
| Time over the response line | both | missed: lower | a fall | — | the share of samples over the line |
| Failed requests | both | failed: lower | a fall | — | the load generator's record |
| Alert minutes | both | time over the line: lower | a fall | — | whole minutes whose 95th percentile, failed requests counted over, was over the line |
| Machines billed, the compute bill | both | a resource spent | a fall | a rise | the cloud's own count of machines, priced at list |
| Energy (declared model, the meter, Kepler) | both | a resource spent | a fall | a rise | the declared model; the RAPL counters where a meter exists (P); Kepler shown, not judged |
| HPA replicas, pods started | both | a resource held | a fall | a rise | native's autoscaler's own record in both arms |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Gauge | Side | Aim | GAIN reads | Power given reads | Where it comes from |
|---|---|---|---|---|---|
| Pod start wait, total and mean | both | time waited: lower | a fall | — | the pods' own start times; the test machine's limit never read as the governor's |
| The governor's commands, its additions inside the line, its own processor time | Omni-Compass | shown, not judged | — | — | the audit; native is 0 |
| Frame time, frames dropped, frames waited | both | time waited: lower | a fall | — | the viewer's browser (L, the viewer's machine) |

APPENDIX

# Appendix D. The Colours and the Marks

| Mark | Meaning |
|---|---|
| Orange | native |
| Blue | Omni-Compass; on a reading, power given |
| Violet | a knob or a profile turned from the factory tune (the mechanism panel's knobs, a CUSTOM profile); never a side |
| Green | GAIN |
| Yellow | STEADY |
| Red | LOSS; the kill switch and its window; the master switch engaged; only ever in a stop sign's shape |
| White, dashed edge | the runs disagree; the runs differ; not a reading |
| Grey | shown, not judged; NO LINE, a monitor with no line to cross or no reading yet |
| Green, round | GAINING |
| Yellow, a yield diamond | STEADY |
| Red octagon, blinking red and black | REDLINING |
| Red octagon, blinking red and white | OVER THE LINE |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Mark | Meaning |
|---|---|
| Dark, dashed word | AWAITING |
| A tint of orange, dashed | Omni-Compass's side following native with the switch OFF |
| Accent border on a tile | the monitor it copies is on screen now |
| Red shading across a time axis | a kill-switch window |
| A bar on a ring | a room's heat debt (the immune system) |

APPENDIX

# Appendix E. Troubleshooting the Page

| Symptom | Cause | What to do |
|---|---|---|
| The page is slow or stutters | the browser's own loop would draw every chart on every frame | the page's section 5j governs it: charts off screen wait; rest monitors at the breakers; close other tabs; the kill switch shows native's loop for comparison |
| A tile is grey or AWAITING and says "not yet run" | the record does not exist yet | nothing: an empty slot is a true reading; the board says where it stands |
| A monitor says "stopped before a result" | a cloud or chip run ended without a measurement | the status board carries it; the run's own log says why (a key cut short, a plan, a quota); fix at the cloud, never on the page |
| The age of the page is over three hours | the rebuild did not run or did not land | `.github/workflows/live-engine.yml` on GitHub; `python3 docs/engine/build_live_engine.py` locally |
| A number on the page differs from a table | the page was built before the table changed | the Build Freshness tile; the table governs; the next build carries it |
| The two sides are identical on a monitor | the muscle is at its limit on native, or Omni-Compass left it native | STEADY, correct; section 42.4 |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Symptom | Cause | What to do |
|---|---|---|
| The first ticker is empty | the switch is OFF or the kill switch engaged | nothing decides on top of native while it is; turn it ON |
| The heartbeat, the sweep or the mechanism's knobs do nothing | the active profile is held | *Restore factory profile* in the control panel |
| The active profile reads CUSTOM | a knob of the mechanism or the heartbeat was turned | *Restore factory profile*, or *Back to the factory tune* for the whole page |
| *Try the candidate* does nothing | no sweep has run in this view, or the profile is held | *Run every beat*, then try again |
| A route's bill reads "unresolved" | no price for that component is recorded in the repository | nothing on the page: the measured changes beside it are supporting measurements |
| The page refuses to build | the checkout's engine is not the one the page names | a new engine is written into the page's words by a person first; `python3 tools/omni_version.py` |

APPENDIX

# Appendix F. The License

The license under which this book and the software are made available, as it stands in the file `LICENSE` at the root of the repository.

```
Omni-Compass Evaluation License, version 1.0 (revised 11 October 2026)
SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0

Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
All patents, copyrights and trademarks filed in the USA. www.omni-comp
    ↪ ass.com

THIS IS NOT AN OPEN-SOURCE LICENSE. The Software is proprietary. It is
    ↪  published so that it can be evaluated, reproduced
and audited, and for no other purpose. It is licensed, not sold. Evalu
    ↪ ation and simulation are free of charge; every
other use, and in particular every commercialization or monetization o
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
    ↪ f any kind, requires a signed, paid Omni-Compass
Enterprise License.

ACCEPTANCE. By downloading, copying, forking, installing, running, sca
    ↪ nning or otherwise using any part of the Software,
You accept this license. If You do not accept it, You may not use the

    ↪ Software. If You use the Software on behalf of an
organization, You represent that You have authority to bind it, and "Y
    ↪ ou" includes that organization and its
Affiliates.

1. DEFINITIONS

   1.1 "Licensor" means The Omni-Compass LLC.
   1.2 "Software" means everything Licensor publishes or makes availab
    ↪ le under this license, in any form: the source
       code and any object code; the Omni-Compass engine, the compass
    ↪ law, the governors, controllers, brains, shields,

       plugs and wiring; the harnesses, tools, scripts, workflows, con
    ↪ figurations, presets, catalogs, organisms and realms;
       the documentation, the manual, the book, the specifications, th
    ↪ e preregistrations and the receipts; the benchmarks,
       data, tables, reports and Results published by Licensor; and ev
    ↪ ery update, version and copy of any of them.

   1.3 "Mechanism" means the methods, processes, control laws, algorit
    ↪ hms, architectures and systems embodied in or
       described by the Software, including everything described in th
    ↪ e patent applications Licensor has filed.
   1.4 "Evaluation" means internal use by You, free of charge, solely
    ↪ to test, study and assess the Software in

       contemplation of an Enterprise License: reading it; running its
    ↪  tests, verifiers, benchmarks and simulations; and
       running it in observe, watch, dry-run or shadow mode, or in a t
    ↪ est environment, on systems You own or control,
       where it writes to no system that serves production traffic, re
    ↪ al users, customers, patients or passengers, and to
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    no power system, vehicle, machine or physical process outside a
 ↪  test bench.
1.5 "Simulation" means running the Software against modelled, recor
 ↪ ded or synthetic systems, including the organisms,
    realms, traces and simulators published with the Software.
1.6 "Production Use" means any use that is not Evaluation or Simula

 ↪ tion, including any use in which the Software, the
    Mechanism or any output of either controls, governs, tunes, siz
 ↪ es, schedules or otherwise influences a system that
    serves production workloads or real users, or any physical proc
 ↪ ess outside a test bench.
1.7 "Commercial Use" means any use for, or in connection with, comm

 ↪ ercial advantage, revenue, cost reduction in
    operations, or monetary or other compensation, including consul
 ↪ ting, integration, managed, outsourcing, training,
    benchmarking-for-hire and advisory services, and any use by or
 ↪ for a for-profit operation beyond Evaluation.
1.8 "Hosted Service" means making the Software, the Mechanism or an

 ↪ y of their functionality available to any third
    party over a network or as a service, including software-as-a-s
 ↪ ervice, platform-as-a-service, a managed or
    outsourced service, a cloud marketplace offering, or a feature
 ↪ of another product or service.
1.9 "Derivative" means any modification, adaptation, translation, p

 ↪ ort, extension or re-implementation of the Software
    or of the Mechanism, and any work that incorporates, is based o
 ↪ n, or is built using the Software as a reference.
1.10 "Results" means the measurements, tables, receipts, logs, repo
 ↪ rts and other outputs produced by running the
    Software.

1.11 "Enterprise License" means a written agreement for the Softwar
 ↪ e signed by Licensor and a licensee, under which
    the licensee pays the fees it sets out; it may include order fo
 ↪ rms, riders and statements of work.
1.12 "Marks" means the names "Omni-Compass" and "OmniCompass", the
 ↪ Omni-Compass compass rose with its ring of Greek
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
      letters, and every related name, logo and trade dress of Licens
    ↪ or.
   1.13 "Affiliate" means any entity that controls, is controlled by,
    ↪ or is under common control with You, where control
      means ownership of more than half of the voting interests or th
    ↪ e power to direct management.

   1.14 "You" means the individual or entity exercising rights under t
    ↪ his license, with its Affiliates where the
      Acceptance paragraph above includes them.

2. WHAT THIS LICENSE ALLOWS

   2.1 Copyright license. Subject to Your compliance with this license
    ↪ , Licensor grants You a personal, worldwide,
      royalty-free, non-exclusive, non-transferable, non-sublicensabl
    ↪ e and revocable license to download, copy, read,
      run and modify the Software solely for Evaluation and Simulatio
    ↪ n, and to reproduce and publish Results as section

      4 allows.
   2.2 Patent license. Subject to the same conditions, Licensor grants
    ↪  You a license of the same scope, under the patent
      rights Licensor holds or will hold in the filed applications an
    ↪ d any patent issuing from them, solely to the
      extent needed to carry out Evaluation and Simulation with the S

    ↪ oftware. No patent license is granted for any
      Production Use, Commercial Use or Hosted Service, for any Deriv
    ↪ ative used outside Evaluation or Simulation, or for
      any implementation of the Mechanism other than the Software its
    ↪ elf.
   2.3 Forks. You may keep a copy or a fork of the Software on a code-

    ↪ hosting service, publicly or privately, only for
      Evaluation and Simulation, with this license and every notice i
    ↪ ntact. A fork confers no right beyond this license.
   2.4 Reservation. Every right not expressly granted is reserved by L
    ↪ icensor. No right, title, license or interest is
      granted by implication, estoppel, exhaustion or otherwise.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
3. WHAT THIS LICENSE DOES NOT ALLOW

   Unless an Enterprise License expressly permits it, You may not, and
    ↪  may not let or help anyone else:
   (a) make any Production Use or Commercial Use of the Software, the

    ↪ Mechanism or any Derivative;
   (b) commercialize or monetize the Software, the Mechanism, any Deri
    ↪ vative or any Result in any way, directly or
       indirectly, including by charging for access, services, support
    ↪ , results, savings or outcomes they produce;
   (c) offer the Software, the Mechanism or any Derivative as, or as p

    ↪ art of, a Hosted Service;
   (d) sell, resell, license, sublicense, lease, rent, lend, distribut
    ↪ e, publish as a package or binary, or otherwise
       transfer the Software or any Derivative, except forks under sec
    ↪ tion 2.3;
   (e) incorporate the Software, any part of it or any Derivative into

    ↪  any product, service, platform, appliance, image,
       firmware or offering;
   (f) use the Software, the Mechanism, the documentation or the Resul
    ↪ ts to design, build, train, tune, benchmark for
       sale, or improve any product or service that competes with Omni
    ↪ -Compass, or to re-implement the Mechanism;

   (g) use the Software or the Results to train, fine-tune or evaluate
    ↪  any machine-learning model intended to reproduce,
       imitate or substitute for the Mechanism;
   (h) remove, alter or obscure any notice, the SPDX identifiers, the
    ↪ patent and trademark notices, this license, the
       NOTICE or DISCLOSURES.md, or circumvent, disable or work around

    ↪  any limit, switch or safeguard in the Software,
       including the kill switch and the hand-back to native settings;
   (i) connect the Software so that it writes to a safety-critical sys
    ↪ tem, as section 10 describes;
   (j) use the Marks, except to refer to Omni-Compass accurately as TR
    ↪ ADEMARKS.md allows; or
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
   (k) use the Software in violation of any law, or in breach of secti
    ↪ on 11.

4. REPRODUCING AND PUBLISHING RESULTS

   You may reproduce the benchmarks published with the Software and pu
    ↪ blish Your own Results, provided that You: (a)
   name the Software's engine version and the exact commit You ran; (b
    ↪ ) report every measured row, the unfavourable ones
   included, with the evidence class the Software assigns to it; (c) m
    ↪ ake Your complete raw files and configuration
   available to Licensor on request; (d) do not state or imply any end
    ↪ orsement, certification or affiliation by
   Licensor; and (e) carry the notice at the top of README.md. Publish
    ↪ ing Results is not Commercial Use so long as no
   compensation is received for it, and it grants no right beyond this
    ↪  license.

5. THE OMNI-COMPASS ENTERPRISE LICENSE

   5.1 Every Production Use, Commercial Use, Hosted Service, distribut
    ↪ ion, incorporation, OEM or embedded use, and every
      other commercialization or monetization of the Software, the Me
    ↪ chanism or any Derivative requires an Enterprise
      License, signed before the use begins and paid as it provides.
    ↪ Moving from Evaluation to Production Use requires
      an Enterprise License before the first write to any production
    ↪ system.
   5.2 An Enterprise License sets its own scope, fees, term, support,
    ↪ warranties, indemnities and limits
      (ENTERPRISE_LICENSE.md summarizes the standard terms; the signe
    ↪ d agreement alone is binding). For its licensee and
      its scope, a signed Enterprise License governs where it differs
    ↪  from this license.
   5.3 Enterprise licensing: The Omni-Compass LLC, main office infrast
    ↪ ructure, toll-free direct 1-888-339-9551, system route asset 1-8
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ 88-DEX-YLL-1, landline 321-746-4411 registered in Florida, www.o
    ↪ mni-compass.com.

6. OWNERSHIP AND INTELLECTUAL PROPERTY

   6.1 Licensor and its licensors own the Software, the Mechanism, the

    ↪  Marks and all intellectual property rights in
       them, including in every Derivative to the extent it embodies t
    ↪ hem. Nothing in this license transfers ownership.
   6.2 All patents, copyrights and trademarks covering the Omni-Compas
    ↪ s engine, its mathematics and its software have
       been filed in the United States by The Omni-Compass LLC. No fil

    ↪ ing number is stated here, and no grant,
       registration or approval is claimed; the status of every filing
    ↪  is held by the office where it was filed. Every
       person who downloads, copies, forks, scans or runs the Software
    ↪  is on notice of these filings and of this
       license (PATENTS.md, TRADEMARKS.md).

   6.3 Any use outside this license, without an Enterprise License, is
    ↪  unlicensed and infringes Licensor's copyright and
       other rights, in addition to breaching this license.

7. PATENT AND RIGHTS DEFENSE

   If You or any of Your Affiliates bring or join any claim, countercl
    ↪ aim or proceeding alleging that the Software or
   the Mechanism infringes, misappropriates or violates any patent, co
    ↪ pyright or other intellectual property right, or
   challenge the validity or enforceability of any of Licensor's right
    ↪ s in them, every license granted to You under this

   license ends automatically on the date the claim or challenge is fi
    ↪ led.

8. FEEDBACK AND CONTRIBUTIONS

   8.1 If You give Licensor comments, suggestions, ideas, reports or o
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ther feedback about the Software, You grant
       Licensor a perpetual, irrevocable, worldwide, royalty-free righ
    ↪ t to use, disclose and exploit it for any purpose,
       without obligation of any kind.
   8.2 Contributions are accepted only under the contributor license a
    ↪ greement (.github/CLA.md).

9. CHANGES

   Licensor reserves the right to change the Software, its documentati
    ↪ on, its results and these terms at any time,
   without notice. A changed version of these terms applies to every c
    ↪ opy of the Software obtained, and to every use
   made, after it is published in the Software's repository; continued
    ↪  use after publication is acceptance of the
   changed terms. Licensor may discontinue the Software, or this licen
    ↪ se for future copies, at any time.

10. SAFETY

   The Software is not designed, tested or certified for safety-critic
    ↪ al use. Under this license it may not write to
   any system whose failure could cause death, injury, environmental d
    ↪ amage or significant property damage, including
   vehicles and aircraft, medical devices, power-grid protection and d
    ↪ ispatch, industrial safety systems and life-safety
   systems; such systems may be studied only in Simulation. Any live c
    ↪ onnection to a physical system requires an
   Enterprise License, a written safety case accepted by the system's
    ↪ owner, and every review that system's regulator or
   owner requires. The machines' own controls and protections stay in
    ↪ place; the kill switch must remain operable at all
   times.

11. EXPORT CONTROL, SANCTIONS AND LAWFUL USE
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
   You will comply with every export-control, sanctions, anti-corrupti
    ↪ on and other law that applies to the Software and
   to Your use of it, including the U.S. Export Administration Regulat
    ↪ ions and the sanctions administered by the U.S.
   Treasury's Office of Foreign Assets Control. You represent that You
    ↪  are not located in, organized under the laws of,

   or ordinarily resident in a country or region subject to comprehens
    ↪ ive U.S. sanctions, and that You are not on, or
   owned or controlled by a party on, any U.S. restricted-party list.
    ↪ You will not use the Software for any end use
   prohibited by those laws, including the design, development or prod
    ↪ uction of weapons of mass destruction.

12. DATA AND PRIVACY

   The Software contains no telemetry. It sends nothing to Licensor an
    ↪ d makes no network connection of its own except to
   the systems You point it at. Your data and Your Results stay on You

    ↪ r systems and are Yours, subject to section 4. If
   You send Licensor any material (for example in an issue or an e-mai
    ↪ l), You represent that You may lawfully do so and
   that it contains no personal data or confidential information of an
    ↪ y third party.

13. THIRD-PARTY COMPONENTS

   Files that carry a different license notice (for example tests/thir
    ↪ d_party/) and third-party data sets (for example
   the PlanetLab traces under fleet/traces/ and k8s_controlplane/trace
    ↪ s/) remain under their own terms

   (THIRD_PARTY_NOTICES.md). Third-party software the Software runs on
    ↪  or connects to (Kubernetes, KEDA, Karpenter,
   NVIDIA drivers, databases and others) is licensed by its owners und
    ↪ er their own terms.

14. TERM AND TERMINATION
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
   14.1 This license lasts until it ends under this section or section
    ↪  7.
   14.2 It ends automatically, without notice, if You breach it. If Yo
    ↪ ur breach is Your first, was inadvertent, and is
        not a breach of sections 3(a) to 3(g) or section 10, Your lice

    ↪ nse is reinstated if You cure the breach within
        thirty days after You learn of it or after Licensor notifies Y
    ↪ ou, whichever comes first.
   14.3 Licensor may end this license for any person at any time by wr
    ↪ itten notice, and for all persons for future
        copies under section 9.

   14.4 When this license ends, You must stop all use of the Software,
    ↪  delete or destroy every copy and Derivative in Your
        possession or control, and, on request, certify in writing tha
    ↪ t You have done so.
   14.5 Sections 1, 3, 4(b) to 4(e), 6, 7, 8, 11, 12, 15 to 18 and 19
    ↪ survive the end of this license.

15. NO WARRANTY

   TO THE FULLEST EXTENT PERMITTED BY LAW, THE SOFTWARE, THE MECHANISM
    ↪  AND THE RESULTS ARE PROVIDED "AS IS" AND "AS
   AVAILABLE", WITH ALL FAULTS AND WITHOUT WARRANTY OR CONDITION OF AN

    ↪ Y KIND, EXPRESS, IMPLIED OR STATUTORY, INCLUDING ANY
   WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITL
    ↪ E, NON-INFRINGEMENT, ACCURACY, QUIET ENJOYMENT, OR
   THAT THE SOFTWARE WILL BE ERROR-FREE OR UNINTERRUPTED. NO RESULT IS
    ↪  A PROMISE OF ANY OUTCOME ON ANY OTHER SYSTEM;
   MODELLED RESULTS (EVIDENCE CLASS S) ARE STATEMENTS ABOUT THE MODEL,

    ↪  NOT MEASUREMENTS (DISCLOSURES.md). YOU USE THE
   SOFTWARE AT YOUR OWN RISK AND ARE RESPONSIBLE FOR ITS USE ON YOUR S
    ↪ YSTEMS.

16. LIMITATION OF LIABILITY
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
   TO THE FULLEST EXTENT PERMITTED BY LAW, LICENSOR, ITS MEMBERS, MANA
    ↪ GERS, EMPLOYEES, CONTRACTORS AND LICENSORS SHALL NOT
   BE LIABLE FOR ANY INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL, EXE
    ↪ MPLARY OR PUNITIVE DAMAGES, OR FOR ANY LOSS OF
   PROFITS, REVENUE, SAVINGS, DATA, USE OR GOODWILL, OR ANY COST OF SU
    ↪ BSTITUTE GOODS OR SERVICES, ARISING OUT OF OR

   RELATING TO THE SOFTWARE, THE MECHANISM, THE RESULTS OR THIS LICENS
    ↪ E, UNDER ANY THEORY OF LIABILITY, EVEN IF ADVISED OF
   THEIR POSSIBILITY. LICENSOR'S TOTAL LIABILITY FOR ALL CLAIMS SHALL
    ↪ NOT EXCEED ONE HUNDRED U.S. DOLLARS (USD 100). THESE
   LIMITS APPLY EVEN IF A REMEDY FAILS OF ITS ESSENTIAL PURPOSE.

17. INDEMNITY

   You will defend, indemnify and hold harmless Licensor and its membe
    ↪ rs, managers, employees and contractors from and
   against every claim, loss, liability, damage, cost and expense (inc
    ↪ luding reasonable attorneys' fees) arising out of

   or relating to (a) Your use of the Software outside this license, (
    ↪ b) any Production Use, Commercial Use or Hosted
   Service without an Enterprise License, (c) Your Derivatives, (d) Yo
    ↪ ur Results as You publish them, or (e) Your
   violation of any law or third-party right.

18. U.S. GOVERNMENT END USERS

   The Software is "commercial computer software" and "commercial comp
    ↪ uter software documentation" as those terms are
   used in 48 C.F.R. 2.101, 12.212 and 227.7202. A U.S. Government end
    ↪  user acquires only the rights set out in this

   license, consistent with 48 C.F.R. 12.212 and 227.7202-1 through 22
    ↪ 7.7202-4.

19. GENERAL

   19.1 Governing law. This license is governed by the laws of the Sta
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
 ↪ te of Delaware and the federal laws of the United
     States, without regard to conflict-of-laws rules. The United N
 ↪ ations Convention on Contracts for the
     International Sale of Goods does not apply.
19.2 Venue. The state courts of the State of Delaware and the Unite
 ↪ d States District Court for the District of

     Delaware have exclusive jurisdiction over any dispute arising
 ↪ out of or relating to this license, and You consent
     to their personal jurisdiction. Licensor may seek
     injunctive or other equitable relief in any court of competent
 ↪  jurisdiction, without posting a bond, to protect
     its intellectual property or confidential information; a breac

 ↪ h of sections 3 or 10 would cause Licensor
     irreparable harm.
19.3 Enforcement costs. In any action to enforce this license, the
 ↪ prevailing party may recover its reasonable
     attorneys' fees and costs.
19.4 Assignment. You may not assign or transfer this license or any

 ↪  right under it, by operation of law or otherwise;
     any attempt is void. Licensor may assign this license freely.
19.5 Notices. Notices to Licensor go to The Omni-Compass LLC by tel
 ↪ ephone at its main office infrastructure (toll-free direct 1-888
 ↪ -339-9551; system route asset 1-888-DEX-YLL-1, landline 321-746-
 ↪ 4411 registered in Florida) or through www.omni-compass.com. Lic

 ↪ ensor may give notice to
     You through the Software's repository or at any address You ha
 ↪ ve given it.
19.6 Severability and waiver. If any provision is held unenforceabl
 ↪ e, it is enforced to the maximum extent permitted and
     the rest of this license remains in effect. No failure or dela

 ↪ y in enforcing any provision is a waiver of it.
19.7 Entire license. This license, with the NOTICE and DISCLOSURES.
 ↪ md, is the entire agreement between You and
     Licensor about its subject; for a licensee and its scope, a si
 ↪ gned Enterprise License governs where it differs.
19.8 Interpretation. "Including" means "including without limitatio
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ n". Headings are for convenience only. The
        English text of this license controls over any translation.
   19.9 Copies. Every copy, fork, export, report, archive or printout
    ↪ of any part of the Software must carry this
        LICENSE, the NOTICE and DISCLOSURES.md unchanged.

All rights reserved. Revision history: version 1.0 (evaluation and sim
    ↪ ulation only; first published 2026); revised
10 October 2026 (the notice in one wording everywhere); revised 11 Oct
    ↪ ober 2026 (definitions, patent license for
evaluation, hosted-service and machine-learning restrictions, results
    ↪ publication, patent and rights defense, safety,

export control, data, U.S. Government end users, term, cure, indemnity
    ↪ , Delaware governing law and venue).

Enterprise licensing: The Omni-Compass LLC, main office infrastructure
    ↪ , toll-free direct 1-888-339-9551, system route asset 1-888-DEX-
    ↪ YLL-1, landline 321-746-4411 registered in Florida, www.omni-com

    ↪ pass.com
```

APPENDIX

# Appendix G. Licensing: Questions and Answers

Plain answers to the questions buyers, engineers and their lawyers ask. The binding terms are in `LICENSE`; where this page and `LICENSE` differ, `LICENSE` governs. Every declaration is in `DISCLOSURES.md`.

**Is Omni-Compass open source?** No. The source is published so that it can be evaluated, reproduced and audited. It is licensed under the Omni-Compass Evaluation License 1.0 (`LicenseRef-OmniCompass-Evaluation-1.0`), which permits evaluation and simulation only.

**What may I do without a commercial license?** Evaluation and simulation, free of charge (`LICENSE`, sections 1.4, 1.5 and 2): read the code; run `verify.py`, the simulations and the benchmarks; reproduce the published results and publish your own, every row included (`LICENSE`, section 4); run Omni-Compass in watch, dry-run or shadow mode, or in a test environment, on systems you own or control, where it writes to

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

nothing that serves production traffic or real users and to no physical process outside a test bench. A limited patent license comes with it, for evaluation only (`LICENSE`, section 2.2).

**What needs the Omni-Compass Enterprise License?** Every commercialization or monetization of any kind; production use; operating any system beyond evaluation; offering it, or its function, as a hosted, managed or software-as-a-service offering; redistributing it; incorporating it, or any part or derivative of it, into a product, service, appliance or firmware; using it or its results to build or train a competing product (`LICENSE`, sections 3 and 5). The Enterprise License is a written agreement signed by The Omni-Compass LLC and paid for; its standard terms are summarized in `ENTERPRISE_LICENSE.md` (production, hosted service, OEM and embedded, managed service provider, research).

**May I run it as a service for others?** Not without an Enterprise License with a hosted-service rider (`LICENSE`, section 3(c); `ENTERPRISE_LICENSE.md`, section 1).

**May I connect it to a grid, a plant, a vehicle or a medical device?** Not under the evaluation license: such systems may be studied only in simulation. A live connection to any physical system needs an Enterprise License and a written safety case accepted by the system's owner (`LICENSE`, section 10).

**How is the Enterprise License priced?** Against the measured gain on the customer's own paired receipts (native against Omni-Compass on the same system, the same load and the same clock). Terms are set in each signed agreement; no price stated anywhere in this repository is an offer.

**Are patents involved?** All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC (`PATENTS.md`). The evaluation license grants no patent license beyond evaluation.

**May I use the name or the compass rose?** Only to refer to Omni-Compass accurately (`TRADEMARKS.md`).

**May I contribute?** Contributions require the contributor license agreement (`.github/CLA.md`): every contribution is assigned to The Omni-Compass LLC, with a patent license from the contributor.

**Does Omni-Compass send data anywhere?** No. The software contains no telemetry and makes no network connection of its own except to the systems an operator points it at (a Kubernetes API, `nvidia-smi`, a meter command).

**What do scanners report?** The license is declared in machine-readable form for Black Duck, FOSSA, Snyk and REUSE (`pyproject.toml`, `REUSE.toml`, `.fossa.yml`, `.snyk`, `sbom/`); third-party components are listed in `THIRD_PARTY_NOTICES.md`.

**Can the terms change?** Yes. The Omni-Compass LLC reserves the right to change the software, its documentation, its results and these terms at any time, without notice; a changed version applies to copies obtained and uses made after it is published (`LICENSE`, section 9). A signed Enterprise License governs its own term.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Which law governs?** The laws of the State of Delaware and the federal laws of the United States, with exclusive venue in the Delaware state courts and the United States District Court for the District of Delaware (`LICENSE`, section 19). The Omni-Compass LLC is based in Florida.

**Who do I contact?** The Omni-Compass LLC, main office infrastructure, toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1, landline 321-746-4411 registered in Florida, www.omni-compass.com.

APPENDIX

# Appendix H. Third-Party Notices

Omni-Compass uses the following third-party components. Each remains under its own license; nothing in the Omni-Compass Evaluation License changes those terms. Installed versions are those resolved from `requirements.txt`.

| Component | Use | License |
|---|---|---|
| NumPy | numerical arrays | BSD-3-Clause |
| Matplotlib | charts in the reports | Matplotlib License (PSF-based, BSD-compatible) |
| ReportLab (open-source edition) | the printable book and reports | BSD-3-Clause |
| Numba | compiled inner loops of the motion-axis plant | BSD-2-Clause |
| PyTorch (on GPU machines only, not installed by `requirements.txt`) | the GPU workload of the benchmark | BSD-3-Clause |
| `tests/third_party/hpa_independent.py` | an independently written HPA reference used unmodified in tests | its own terms, stated in the file |
| Liberation Serif and Liberation Sans fonts (embedded in the PDF book) | typesetting | SIL Open Font License 1.1 |
| DejaVu Sans and DejaVu Sans Mono fonts (embedded in the PDF book) | typesetting | Bitstream Vera / DejaVu license (free) |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Component | Use | License |
|---|---|---|
| PlanetLab / CoMon VM CPU traces, March 2011 (`fleet/traces/`, `k8s_controlplane/traces/`; source in their `PROVENANCE.json`) | held-out demand for the modelled fleet | their own terms (public research data); no Omni-Compass copyright is claimed in them |
| Google cluster-usage traces 2011 (`clusterdata-2011-2`), from which `results/traces/google2011/` is derived (`docs/TRACES_PREREGISTRATION.md`) | the public demand trace on the real cluster | Google's published terms for the traces; no Omni-Compass copyright is claimed in the trace itself |

Kubernetes, kind, `kubectl`, the NVIDIA driver and `nvidia-smi` are not distributed with Omni-Compass; it calls them where an operator has installed them. Their names are the property of their owners (`DISCLOSURES.md`, section 1).

APPENDIX

# Appendix I. The Enterprise License: Standard Terms

This page summarizes the standard terms of the Omni-Compass Enterprise License, the paid license that every production use, commercial use, hosted service, distribution, OEM or embedded use, and every other commercialization or monetization of Omni-Compass requires (`LICENSE`, section 5). It is a summary for evaluation and planning. It is not an offer, and it binds no one: only an Enterprise License signed by The Omni-Compass LLC and the licensee, with its order form, grants any right, and that signed agreement controls over this page in every respect.

## I.1 What can be licensed

| License | What it allows |
|---|---|
| **Production** | The licensee runs Omni-Compass on top of its own systems (clusters, fleets, cards, sites, plants), on premises or in its own cloud accounts, within the scope in the order form |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| License | What it allows |
|---|---|
| **Hosted service** | The licensee offers Omni-Compass's function to its own customers as a hosted, managed or software-as-a-service offering, under a hosted-service rider (per customer, per managed unit or revenue share) |
| **OEM and embedded** | The licensee ships Omni-Compass inside its own product, appliance, firmware or platform, under an OEM rider (per unit or per installed base) |
| **Managed service provider** | The licensee operates Omni-Compass for named third parties under a managed-service rider |
| **Research** | A university or public laboratory uses Omni-Compass beyond evaluation for non-commercial research and publication, under a research rider |

**Scope** is set in each order form: the licensed systems and their size (machines, cores, cards, sites, muscles or managed units), the fields of use, the territory, the named sites or accounts, and the engine versions covered.

## I.2 Fees

Fees are set in each order form: a subscription, a usage-based fee, a share of the measured value captured, or a combination, with any minimum commitment. Where fees track value, the value is measured on the licensee's own paired receipts (native against Omni-Compass on the same system, the same load and the same clock), by the method the order form names. Payment terms, taxes and late charges are as the order form states. No price, rate or benchmark in this repository is an offer.

## I.3 What the licensee receives

1. The licensed engine at a named, sealed version (its fingerprint, `tools/omni_version.py`), and each later version the subscription covers.
2. The wiring kits and the plug for the licensed systems, the manual and the book, and the receipts tooling.
3. Support and maintenance at the tier in the order form: response targets, named contacts, and corrections of defects that reproduce on the licensed version.
4. Installation through the manual's levels: read-only first, the wire check, a watch arm equal to native, then writing levels one at a time, each with its receipt.

## I.4 Warranties

1. **Conformance.** For the warranty period in the order form, the licensed version performs materially as its documentation describes. The remedy is correction or, failing that, a refund of the fees paid for the nonconforming version for the affected period.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

2. **The safeguards.** The kill switch hands every setting back to the value read before Omni-Compass's first write; the shield keeps every write inside the cover the operator granted; Omni-Compass never fights another writer.
3. **No other warranty.** No outcome, saving or result on the licensee's systems is warranted. Published results are evidence about the systems they ran on (`DISCLOSURES.md`, section 2).

## I.5 Intellectual property

1. The Omni-Compass LLC keeps all right, title and interest in Omni-Compass, the Mechanism and the marks. The licensee receives the rights its order form lists, for its term, and no others.
2. **Patent license.** The Enterprise License includes a license under The Omni-Compass LLC's patent rights in the filed applications, and in any patent issuing from them, for the licensed scope.
3. **Infringement indemnity.** The Omni-Compass LLC defends the licensee against a third-party claim that the licensed version, used as licensed, infringes that party's intellectual property, and pays the damages finally awarded or the settlement it agrees, subject to prompt notice, control of the defense and the usual exclusions (modifications, combinations, unlicensed use, versions superseded by an offered fix). Remedies may include a license, a modification or a refund.
4. **Confidential filings.** The patent application number and other confidential information are disclosed to a licensee under the agreement's confidentiality terms.

## I.6 Limits of liability

Each party's liability is limited to the fees paid or payable in the twelve months before the claim, and neither party is liable for indirect, incidental, special, consequential or punitive damages, except for the exclusions the signed agreement states (for example, breach of confidentiality, the infringement indemnity, and the licensee's breach of the license scope).

## I.7 Confidentiality, data and security

1. **Confidentiality.** Mutual, for the term and a period after it, covering non-public information either party gives the other.
2. **Data.** Omni-Compass contains no telemetry. The licensee's data, readings, settings and receipts stay on the licensee's systems and remain the licensee's. Where the licensee chooses to share data for support, a data-processing agreement applies.
3. **Security.** A software bill of materials ships with every version (`sbom/`); vulnerabilities are handled under `SECURITY.md`; the licensee may run the verifier (`python3 verify.py`) and the seal checks on every delivered version.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## I.8 Safety and operation

1. The licensee remains the operator of its systems. The machines' own controls and protections stay in place; Omni-Compass sets only values they already accept.
2. Any connection to a physical system (grid, plant, vehicle, robot, medical device, facility) requires a written safety case accepted by the system's owner and every review its regulator requires; safety-critical systems are licensed only on those terms.
3. The kill switch stays operable at all times.

## I.9 Compliance, audit and insurance

1. Each party complies with export-control, sanctions and anti-corruption law.
2. The licensee reports usage against the licensed scope as the order form provides, and The Omni-Compass LLC may verify it on reasonable notice, no more than once a year.
3. Insurance, where the order form requires it, is carried by each party at the limits stated there.

## I.10 Term, renewal and transition

1. Term and renewal are as the order form states.
2. On termination or expiry, the licensee stops use, every setting Omni-Compass changed is handed back to native (the hand-back is built into the engine), and every copy is returned or destroyed; transition assistance is available at the rates in the order form.
3. Source-code escrow for the licensed version is available under a three-party escrow agreement.

## I.11 Governing law and precedence

The signed agreement states its governing law and venue; the standard is the law of the State of Delaware, with venue in the Delaware state courts and the United States District Court for the District of Delaware. Its order of precedence is: the order form, then any rider, then the Enterprise License terms, then the documentation. This page and the evaluation license (`LICENSE`) are superseded by the signed agreement for the licensee and its scope.

## I.12 How to obtain one

The Omni-Compass LLC, main office infrastructure, toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1, landline 321-746-4411 registered in Florida, www.omni-compass.com. A license starts with an evaluation on the licensee's own systems (`LICENSE`, section 1.4): watch mode, the wire check, and paired receipts on its own load.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix J. Patents and Trademarks

## J.1 Patents

**All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC.**

The filings cover, among other things, the Omni-Compass engine (the six-state closed law and its bounded control command held through the integration step), the compass law (the service position pulled to the middle of its band by a bounded push and pull, with fail-up at the wall), the antagonist up/down wiring of each muscle, the plug contract (one restore point, read-back, the one-writer rule), the two-wire governance of a processor (clock ceiling and power limit), the two-way nervous system that grants and withdraws authority per organ, and one governor over many machines and realms on one clock.

### *J.1.1 What has been filed*

| Application | a nonprovisional utility patent application under 35 U.S.C. 111(a), filed with the United States Patent and Trademark Office |
|---|---|
| Title of the invention | **The Omni-Compass** |
| Inventor | Alan John Dubra, Kissimmee, Florida, United States |
| Drawing sheets | 15 |
| Application data sheet signed | 20 September 2026 (form PTO/AIA/14, 37 CFR 1.76) |
| Publication | normal eighteen-month publication requested |
| Application number | withheld here on purpose; the filing is verifiable with the USPTO and is shown to a licensee under a signed Omni-Compass Enterprise License |

The application data sheet names no assignee; the application stands in the inventor's name. Any assignment to The Omni-Compass LLC is recorded with the USPTO separately (37 CFR part 3).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

No patent license is granted by publishing this repository, by the evaluation license, or by any act of The Omni-Compass LLC other than a signed, paid Omni-Compass Enterprise License. Use outside the evaluation license is infringement. Everyone who downloads, forks, scans or runs this software is on notice of these filings.

## J.2 Trademarks

**All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC.**

### *J.2.1 The marks*

"Omni-Compass", "OmniCompass", the Omni-Compass compass rose with its ring of Greek letters, and the related names, logos and trade dress (the "Marks") are trademarks of The Omni-Compass LLC; trademark applications have been filed in the United States. Neither the evaluation license (`LICENSE`) nor publication of this repository grants any license to the Marks. A license to use them is granted only in a signed Omni-Compass Enterprise License, and only as it states.

### *J.2.2 What is allowed without a license*

- Referring to Omni-Compass by name, accurately and in plain text, to say what it is, that you evaluated it, or that a result was produced with it, with the engine version and commit (`LICENSE`, section 4).
- Linking to the repository or to www.omni-compass.com.

### *J.2.3 What is not allowed*

- Using any Mark, or anything confusingly similar, in the name of a product, service, company, domain, account, package, repository, fork or derivative.
- Using the logo or the compass rose, or altering any Mark.
- Suggesting endorsement, sponsorship, certification, partnership or affiliation by The Omni-Compass LLC.
- Using a Mark in advertising, comparative claims or sales material without a signed license.

### *J.2.4 Other companies' marks*

Kubernetes is a trademark of The Linux Foundation; NVIDIA is a trademark of NVIDIA Corporation; KEDA and Karpenter are projects of their maintainers; other names belong to their owners. Their mention describes compatibility or a system Omni-Compass was tested with; it states no affiliation, endorsement or certification (`DISCLOSURES.md`, section 1).

### *J.2.5 Reporting misuse*

The Omni-Compass LLC, main office infrastructure, toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1, landline 321-746-4411 registered in Florida, www.omni-compass.com.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix K. Security Policy

## K.1 Reporting a vulnerability

Report a suspected vulnerability privately to The Omni-Compass LLC by telephone at its main office infrastructure (toll-free direct 1-888-339-9551; system route asset 1-888-DEX-YLL-1, landline 321-746-4411 registered in Florida) or through www.omni-compass.com, never in a public issue, pull request or discussion. Include the version and commit, what you saw, how to reproduce it, and its likely impact. We acknowledge reports, keep the reporter informed, and coordinate disclosure: a fix or a mitigation is published before the details are, and the reporter is credited if they wish. Please give us a reasonable time to fix before any disclosure, do not access data that is not yours, and do not disrupt any system; good-faith research that follows this policy will not be pursued by us.

## K.2 What is in scope

The engine, the controllers, the plugs and wiring, the harnesses, the workflows and the verifier in this repository. Third-party software Omni-Compass runs on or connects to (Kubernetes, KEDA, Karpenter, NVIDIA drivers, databases) is reported to its own maintainers.

## K.3 How Omni-Compass is built to fail safe

- The governor issues control actions only through the shield (`omnicompass/shield.py`, `cpp/src/shield.cpp`), inside the cover the operator granted, and never fights another writer.
- It snapshots every setting once, before its first write, and the kill switch (`python3 tools/omni_switch.py off`) hands every setting back; a dead-man lease hands them back if the governor stops.
- It runs under least privilege: on Kubernetes, a service account with `kubectl auth can-i` receipts for what it can and cannot do (`deploy/kind/rbac-omni.yaml`).
- The watchdog acts only on registry records its own user wrote that no other user can change, and never signals a process number the system gave to another process (`tools/omni_switch.py`). Put the switch and the registry in a directory only the governors' user can write (`OMNI_MASTER_OFF`, `OMNI_REGISTRY`); `status` warns when they are not.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- It contains no telemetry and makes no network connection of its own except to the systems an operator points it at.
- The threats of misuse, the guards against each and what is still open are set out in `docs/THREAT_MODEL.md`.
- Every release carries a software bill of materials (`sbom/`) and a seal of the engine and its twins (`results/SEAL.json`, `python3 verify.py`).
- The capture script (`fleet/capture/kube_capture.sh`) is read-only; no component writes to a live cluster unless the operator starts it in a writing mode.

APPENDIX

# Appendix L. The Page Word for Word: Every Section, Monitor and Knob

This appendix is the live benchmark engine page itself, written from `docs/engine/OMNI_COMPASS_LIVE_ENGINE.html` at every build of this book (`docs/book/build_live_manual.py`): every section in the page's own order with its number, title, summary and controls line; every monitor under the number its badge carries, with its evidence class, its state, the open-source feed that would carry it on a running system and its knobs; every knob; and every LOSS row with whose it is, its cause and the knob that answers it. Nothing here is typed twice: what the page says, this appendix says, so the book and the page read each other both ways. This edition was written from the page built 2026-10-31 16:17 UTC on omni-v22 (digest 0e23a0bc561b900a) from commit 0371e899990a.

Each part below is one section of the page, in the page's order and under its number: what the page says it is, its controls, its panels, and every monitor in it under its own number, with what it shows, its evidence and state, the feed that would carry it live, its knobs, how to read it and what to do with it.

## L.1 Monitor Wall

**Controls.** Click a tile to open its monitor. Every tile is read four times a second.

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

## L.2 Control Panel

**Controls.** Live: the kill switch acts at once on every governor of this page; the buttons run the Python's runs here and fetch the three manuals.

**The page, in its own words.**

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

**Wall Status and Factory Tune**

**Fuse Box**

**Mechanism Knobs**

A tune other than the factory's is a variation of the mechanism (class S, our own model): the benchmarks prove the factory tune only, and a changed tune's results are its own.

**Operator Controls**

ON: the page draws set after set from the Python's jump generator, 500 runs a set unless another size is chosen, one base after another (run k of a set takes the seed base + k·0x9E3779B9 mod $2^{32}$, its stream mulberry32, the generator tools/engine_runs.py and tools/core_runs.py write bit for bit). Each set runs twice in this browser: through the engine's runs (288 decisions of the brain, 80 steps of the compass law and one give-back trial a run; the proof monitor, section L.28) and through the core's Monte Carlo (the fourth-order Runge–Kutta step, 20 macro steps of 10 micro steps a run, native beside Omni-Compass; the Monte Carlo monitor, section L.28). Each set's output is its 500 runs, their counts and one SHA-256 digest; where the Python has run the same base its record's digest stands beside this page's, and they must be equal. It runs until the switch is pressed again, and rests while the page is hidden. Class S: the engine's own mathematics on draws, never a measurement.

OFF (the page's standing mode, as it opens): nothing new is drawn from the generator. The page plays its records: the Monte Carlo monitor plays the Python's own 500 (benchmarks/core_evidence.py, numpy's generator from seed 223387268, the record results/core_evidence.json); the proof monitor recomputes, once as the page opens, the newest set GitHub ran in the Python and checks it digest for digest, then re-checks it one run at a time; the 500 scenarios a seed replay from their record with every trace hash; GitHub's runs play back from their own raw files; the organism's model day (sections L.10, L.12 and L.14 to L.17) runs on its own seeded stream, the same at every opening. A button above or in section L.28 still runs one set at a time.

Each knob sets every monitor that carries its variation and names them here; a monitor that does not carry it is left as it is.

Kill Switch. While Omni-Compass governs on top of native the switch reads ON in green. Pressed, it turns red: at the next decision every governor puts every setting it wrote back, reads each one back and stops (omnicompass/master.py, tools/omni_switch.py); the word OFF stands white, and every Omni-Compass side of every monitor follows native, drawn in native's colour. Native's own controllers under every muscle never stop and never see the switch. A recorded result never changes with it: its colour does, never a value, a word or a row. The same switch stands at the head of every section and in every monitor's own knobs; one press anywhere is one press everywhere. It is for security and is instant; the walk back after a crash is the lease, separate.

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## L.3 Part 2 of the page: Results

## L.4 Scorecard

**The page, in its own words.**

The light is the whole verdict: green GAINING, Omni-Compass ahead of native; yellow STEADY, within 0.5% of native (STEADY by the key's floor); red REDLINING, behind native. The big number is how much, its size with no sign (the raw change, Omni-Compass − native over native, stands apart with its true sign, labelled raw): on the first row more work done for the same demand, on the bills and the index one number over their gauges, read by the key (GAIN, STEADY, LOSS by what the gauge is for, never by its sign; STEADY only under 0.5%, the mean of the three runs' changes). A slower service, less work or a failed request turns the light red whatever the number, on every row: on the bills and the index, any table under the row that reads one of them LOSS, in either form, turns it red. The bills read the repository's own headline, by the index's rule, with the other reading beside it. Each row names what it stands on: its test, its engine and its evidence class.

**Whole-Loop Test by Arm**

**Whole-Workload Index by Route · Class S**

**All Real Tests Combined**

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

## L.5 Benefit Against Native

**The page, in its own words.**

**Bill by Route**

**Its monitors.** This section draws no canvas: it is words and their sources, written in the page itself.

## L.6 Benefit Board

**Controls.** Recorded data: every receipt as the repository holds it. The kill switch recolours a recorded result and never rewrites it; console controls never modify it.

**The page, in its own words.**

Every bar is its table's own word (GAIN green, STEADY yellow, LOSS red, power given blue) beside its size; every LOSS row of the benchmarks on this board stands on it, and every LOSS row of every three-run table, the governors' four-ways tables and the service variants included, is raised on the status board (section L.8) and listed, with its cause where the LOSS ledger records one, in section L.7.

**Benefit by Domain**

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Benefit by Gauge Family**

Every judged row of every newest three-run table (section L.32), sorted into the family of gauge it measures. Each bar counts the rows that read GAIN, in all three runs or by the mean of the three; the size beside it is the median of those rows' own sizes, each row's size the mean of its runs' raw changes in percent of native (a row whose native stood at zero has no percent: it is counted and not sized). Power given and LOSS rows are counted beside them.

**Trading Floor**

Every three-run table of section L.32, as GitHub's runs wrote them. The market map: one tile a table, grouped by stack, its area the gauges it judged and its colour its reading: green GAIN, the deeper the more of its rows; red where any row reads LOSS; yellow STEADY; blue where it gave power; grey with a dashed edge a four-ways comparison that is not Omni-Compass against its native (the orchestrator's own, with no Omni-Compass in either arm; Omni-Compass in place of a controller; two changes at once). The board: one quote a table, its ticker and engine, the gauge that moved most with its mean change, and that gauge in each of the three runs (A, B and C) as a candle, the bar its 95% interval and the tick its value against the zero line; the strip under it counts the table's rows by word, and the tape runs every headline across the top. Point at a tile or a quote for its record. Every figure is the table's own.

**Whole-Board Aggregate**

**Its monitors.**

### *L.6.1 Monitor 1: Benefit by Domain*

Every domain's three-run table as one bar: GAIN, STEADY, LOSS and power given, native against Omni-Compass.

- **Evidence and state.** Class L/S; record.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).
- **Wired.** This page's `drawBenefit, fxCursor` → the Python `tools/benefit_sheet.py`, `tools/omni_index.py`, `tools/readings.py` → the C++ none: the repository's measured tables, read as written; no engine runs in the page for them; held equal by `tests/test_benefit_sheet.py`, `tests/test_readings.py`.
- **Moves.** The reading cursor walks the bars, one benchmark at a time.
- **Knobs.** Filter: which rows, words, families or organisms are shown; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** Bars and tiles in the traffic light of the one key: GAIN green to one side, LOSS red to the other, STEADY yellow on zero, power given blue. A figure is a composition or a table's own word, never a single run's number.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **What to do with it.** Read the word before the size, and the interval before the word: an interval across zero is STEADY whatever its centre. A LOSS on the board is answered on the status board, with its cause and its knob.

### *L.6.2 Monitor 4: Benefit by Gauge Family*

The same tables by gauge family: work, speed, misses, resources.

- **Evidence and state.** Class L/S; record.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).
- **Wired.** This page's `drawFam` → the Python `tools/benefit_sheet.py`, `tools/readings.py` → the C++ none: the repository's measured tables, read as written; no engine runs in the page for them; held equal by `tests/test_benefit_sheet.py`, `tests/test_readings.py`.
- **Moves.** The reading cursor walks the families, one at a time.
- **Knobs.** Filter: which rows, words, families or organisms are shown; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** Bars and tiles in the traffic light of the one key: GAIN green to one side, LOSS red to the other, STEADY yellow on zero, power given blue. A figure is a composition or a table's own word, never a single run's number.
- **What to do with it.** Read the word before the size, and the interval before the word: an interval across zero is STEADY whatever its centre. A LOSS on the board is answered on the status board, with its cause and its knob.

### *L.6.3 Monitor 3: Market Map*

Every three-run table on the page as a tile of a squarified treemap, grouped by stack: its area the gauges it judged, its colour its reading (GAIN green, LOSS red, STEADY yellow, power given the accent, an orchestrator's own cost grey and dashed).

- **Evidence and state.** Class L/S; record.
- **Feed.** the build (the repository's three-run tables, no live feed: a recording rule would carry each table's counts).
- **Wired.** This page's `drawTradeMap` → the Python `tools/confirm_abc.py`, `tools/readings.py`, `docs/engine/build_live_engine.py` → the C++ none: the repository's measured tables, read as written; no engine runs in the page for them; held equal by `tests/test_confirm_abc.py`, `tests/test_readings.py`, `docs/engine/build_live_engine.py`.
- **Moves.** The scan cursor walks every tile, one every 1.5 s.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **How to read it.** Bars and tiles in the traffic light of the one key: GAIN green to one side, LOSS red to the other, STEADY yellow on zero, power given blue. A figure is a composition or a table's own word, never a single run's number.
- **What to do with it.** Read the word before the size, and the interval before the word: an interval across zero is STEADY whatever its centre. A LOSS on the board is answered on the status board, with its cause and its knob.

### *L.6.4 Monitor 20: Quote Board*

Every three-run table as a quote: its ticker and engine, the gauge that moved most with its mean change, that gauge in runs A, B and C as a candle (the 95% interval and the value), its rows counted by word, and a tape of every headline.

- **Evidence and state.** Class L/S; record.
- **Feed.** the build (the repository's three-run tables, no live feed: a recording rule would carry each table's counts).
- **Wired.** This page's `drawQuotes` → the Python `tools/confirm_abc.py`, `tools/readings.py`, `docs/engine/build_live_engine.py` → the C++ none: the repository's measured tables, read as written; no engine runs in the page for them; held equal by `tests/test_confirm_abc.py`, `tests/test_readings.py`, `docs/engine/build_live_engine.py`.
- **Moves.** The tape of the headlines runs across the top at 55 px a second.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** Bars and tiles in the traffic light of the one key: GAIN green to one side, LOSS red to the other, STEADY yellow on zero, power given blue. A figure is a composition or a table's own word, never a single run's number.
- **What to do with it.** Read the word before the size, and the interval before the word: an interval across zero is STEADY whatever its centre. A LOSS on the board is answered on the status board, with its cause and its knob.

## L.7 LOSS Register

**Controls.** Recorded data: every row is read from the verdict audit's record of the three-run tables and the GPU card's own record.

**The page, in its own words.**

Every LOSS row of every standing three-run table, read from the verdict audit's record, the one selection the LOSS ledger (`docs/LOSS_LEDGER.md`) is written from: Omni-Compass against its native first, then the GPU card's own meter (outside the audit's selection), then the orchestrators' four-ways comparisons by what

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

their two arms are. Each row carries the cause, the fix and what decides it exactly as the ledger records them (tools/loss_ledger.py); where the ledger records none, the row says so and asserts no knob.

**Verdict Audit**

**LOSS Rows Against Native**

**Armada Two-Cluster BARE Table by Run**

**Discrepancy Report**

**LOSS Ledger**

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

## L.8 Status Board

**Controls.** Live: the filters act at once, and every tile is read again four times a second.

**The page, in its own words.**

The board shows whether Omni-Compass is engaged, which signals are out of tolerance and where, and every LOSS row raised as an alarm with its source record. Each tile and each alarm is computed at runtime from the live benchmark engine or from the repository's records.

**Alarms and Watch Items by Severity**

A tile's word is its reading against native by the key (appendix D); a tile with no reading says what it shows.

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

## L.9 Part 3 of the page: Live Operation

## L.10 Live Organism

**Controls.** Live: the rings, the immune system and the anatomy respond to the console.

**The page, in its own words.**

Evidence class S: our own models, shown beside the real runs and never presented as proof.

**Native**

**Omni-Compass**

**Body Scan by Organ**

The table reads the system playing in section L.20 (the benchmark, run and repetition chosen there) organ by organ, half minute by half minute, from its own SHA-256-checked files: native beside Omni-Compass. Where native has no such organ the cell says so.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Immune System · Class S**

On the ring a pulsing red circle marks the inflamed muscle, and red signals run from it to every machine pool that answers it.

**Anatomy by Part**

**Its monitors.**

### *L.10.1 Monitor 5: Native Organism Ring · Class S*

Sixty muscles of the stress model, each on its own native controller, at their positions in the band.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); kube-state-metrics; series `omni_recommended`, `kube_horizontalpodautoscaler_status_current_replicas`.
- **Wired.** This page's `drawRing` → the Python `omnicompass/adapter.py`, `omnicompass/core.py`, `omnicompass/nervous_system.py`, `omnicompass/compass_law.py`, `realms/harness.py`, `docs/engine/export_harness_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `docs/engine/check_live_engine.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_twins_parity.py`, `docs/engine/check_records.py`.
- **Moves.** Every decision: each muscle moves to its position in the band.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused; Body: which body the organism runs: the harness, the organism, every muscle alone; Muscle Switch: pick one muscle on either ring: its card, and its own switch, which hands that one muscle back to native (Omni-Compass OFF on it) or puts Omni-Compass on it again; the kill switch hands every muscle back at once.
- **How to read it.** The organism the brain governs, muscle by muscle: positions in the compass band, the brain's grants and pushes, the immune system's heat debt; native's own controllers under every muscle.
- **What to do with it.** Turn the console's knobs (demand, noise, sensor delay, faults) and watch both arms take the same input; the difference between the lanes is what Omni-Compass did.

### *L.10.2 Monitor 6: Omni-Compass Organism Ring · Class S*

The same sixty muscles with Omni-Compass on top of their native controllers, the brain's grants and pushes.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_recommended`, `omni_recommended_moves_total`, `omni_writes_total`; on a running system: `omni_recommended`; `increase(omni_writes_total[1h])`.
- **Wired.** This page's `drawRing` → the Python `omnicompass/adapter.py`, `omnicompass/core.py`, `omnicompass/nervous_system.py`, `omnicompass/compass_law.py`, `realms/harness.py`, `docs/engine/export_harness_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `docs/engine/check_live_engine.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_twins_parity.py`, `docs/engine/check_records.py`.
- **Moves.** Every decision: each muscle moves to its position, the brain's grants and pushes pulse.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused; Body: which body the organism runs: the harness, the organism, every muscle alone; Muscle Switch: pick one muscle on either ring: its card, and its own switch, which hands that one muscle back to native (Omni-Compass OFF on it) or puts Omni-Compass on it again; the kill switch hands every muscle back at once.
- **How to read it.** The organism the brain governs, muscle by muscle: positions in the compass band, the brain's grants and pushes, the immune system's heat debt; native's own controllers under every muscle.
- **What to do with it.** Turn the console's knobs (demand, noise, sensor delay, faults) and watch both arms take the same input; the difference between the lanes is what Omni-Compass did.

### *L.10.3 Monitor 11: Immune System · Class S*

Where the two sides differ, and the body's heat debt: a room warmer than its native twin, inflamed, given back.

- **Evidence and state.** Class S; live.
- **Feed.** Prometheus Alertmanager; tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `ALERTS`, `omni_compass_decisions_total{zone="wall"}`; on a running system: `sum(ALERTS{alertstate="firing"})`; `increase(omni_compass_decisions_total{zone="wall"}[1h])`.
- **Wired.** This page's `drawImm` → the Python `omnicompass/adapter.py`, `omnicompass/core.py`, `omnicompass/nervous_system.py`, `omnicompass/compass_law.py`, `realms/harness.py`, `docs/engine/export_harness_trace.py`, `realms/compass_arm_v5.py` → the C++

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `docs/engine/check_live_engine.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_twins_parity.py`, `docs/engine/check_records.py`.

- **Moves.** Every decision: each side's pools, warmest room and muscles past the line take a point.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.
- **How to read it.** The organism the brain governs, muscle by muscle: positions in the compass band, the brain's grants and pushes, the immune system's heat debt; native's own controllers under every muscle.
- **What to do with it.** Turn the console's knobs (demand, noise, sensor delay, faults) and watch both arms take the same input; the difference between the lanes is what Omni-Compass did.

### *L.10.4 Monitor 35: Body Parts on the Band · Class S*

Each body part's muscles (the spine and the four realms) at their positions in the band, native above and Omni-Compass below, every decision.

- **Evidence and state.** Class S; live.
- **Feed.** the realms harness's own record (realms/harness.py, each body part's muscles at their positions in the band, recorded by docs/engine/export_harness_trace.py); no Prometheus feed.
- **Wired.** This page's `drawSwarm` → the Python `omnicompass/adapter.py`, `omnicompass/core.py`, `omnicompass/nervous_system.py`, `omnicompass/compass_law.py`, `realms/harness.py`, `docs/engine/export_harness_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `docs/engine/check_live_engine.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_twins_parity.py`, `docs/engine/check_records.py`.
- **Moves.** Every decision: each muscle's dot moves to its position in the band.
- **Knobs.** Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The organism the brain governs, muscle by muscle: positions in the compass band, the brain's grants and pushes, the immune system's heat debt; native's own controllers under every muscle.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **What to do with it.** Turn the console's knobs (demand, noise, sensor delay, faults) and watch both arms take the same input; the difference between the lanes is what Omni-Compass did.

## L.11 Golden Signals and Event Log

**Controls.** Live: one point a decision from the live benchmark engine; the event log's filters act at once; the metrics refresh every two seconds.

**The page, in its own words.**

The live organism's four golden signals, native beside Omni-Compass, one point a decision, our own model (class S): errors (muscles over their line), throughput (work done), and, because neither body has a response time or a machine's busy share, latency's and saturation's stand-ins: on the live model the work waiting in line and the capacity in service, on the realms harness the spread of p across the muscles and the electrical power drawn (kW). The query under a panel is the analogous series on a running system, never this panel's data.

**Golden-Signal Feed, Last 240 Decisions · Class S**

One point per decision streamed from the live benchmark engine, gapless, each signal on its own axis, with each arm's last, mean and maximum as a time-series dashboard reports them; a kill-switch window is shaded red. Native in orange, Omni-Compass in blue, the same demand on both.

**Engine Event Log**

The live benchmark engine's own events as it emits them, with the wall-clock time and the decision. They are a threshold breach and its clearing on either arm, every injected fault and its end, the kill switch, the dial, a new day, and each organ's hand-back grant withheld or opened again, at that organ's own threshold θ (pods 0.5, nodes 0.7, the clock, the card and power 0.3, routing 0.5 at the factory: omnicompass/nervous_system.py), as the brain gives it. ALERT, WARN and INFO as an operations log grades them.

**OpenMetrics Exposition**

The live benchmark engine's metrics in the OpenMetrics text format, the CNCF exposition format Prometheus scrapes and Grafana charts, refreshed every two seconds: the same families, under the same names, as the control panel's Metrics tab prints in Prometheus's text format (section L.2), from one exposition. They are the master switch, the decisions, the plant's knobs, work served, failed and waiting, the muscle-decisions past their line, writes, capacity, actuation, band time, actuator moves and reversals per arm, and frame time; then the verdict's keep ratio at each true cost, the three-run tables' rows by class, by the engine each table ran on (each that engine's own, never read as the engine that stands's), by the way it compares and by reading, as each table writes it, and the build's engine, digest and commit. Every counter keeps its own arm's value and only rises within a day; the OFF window is marked by omni_master_switch 0 (Omni-Compass's arm then runs at native's own settings), never by putting native's totals in its place.

**Its monitors.**

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

### *L.11.1 Monitor 7: Errors per Decision · Class S*

Errors each decision on the live organism, counted as the muscles over their line, native beside Omni-Compass.

- **Evidence and state.** Class S; live.
- **Feed.** the service's own request counter (Prometheus client), kube-state-metrics for pod restarts; series `http_requests_total{code=~"5.."}`, `kube_pod_container_status_restarts_total`; on a running system: `sum(rate(http_requests_total{code=~"5.."}[5m]))`.
- **Wired.** This page's `drawFeed` → the Python `omnicompass/adapter.py`, `omnicompass/core.py`, `omnicompass/nervous_system.py`, `omnicompass/compass_law.py`, `realms/harness.py`, `docs/engine/export_harness_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `docs/engine/check_live_engine.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_twins_parity.py`, `docs/engine/check_records.py`.
- **Moves.** Every decision of the live benchmark engine: each side's line takes a point.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.
- **How to read it.** The four golden signals of a service, one point a decision of the live benchmark engine: native in orange, Omni-Compass in blue, the same demand on both. A red band is a kill-switch window.
- **What to do with it.** Watch latency and errors first: a rise there is the only thing that can read LOSS. Saturation and throughput are the work and the capacity held: capacity is power given when it rises.

### *L.11.2 Monitor 8: Throughput per Decision · Class S*

Work done a decision (traffic), native beside Omni-Compass.

- **Evidence and state.** Class S; live.
- **Feed.** the service's own request counter (Prometheus client); series `http_requests_total`; on a running system: `sum(rate(http_requests_total[5m]))`.
- **Wired.** This page's `drawFeed` → the Python `omnicompass/adapter.py`, `omnicompass/core.py`, `omnicompass/nervous_system.py`, `omnicompass/compass_law.py`, `realms/harness.py`, `docs/engine/export_harness_trace.py` → the C++ `cpp/src/governor.cpp`,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `docs/engine/check_live_engine.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_twins_parity.py`, `docs/engine/check_records.py`.

- **Moves.** Every decision of the live benchmark engine: each side's line takes a point.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.
- **How to read it.** The four golden signals of a service, one point a decision of the live benchmark engine: native in orange, Omni-Compass in blue, the same demand on both. A red band is a kill-switch window.
- **What to do with it.** Watch latency and errors first: a rise there is the only thing that can read LOSS. Saturation and throughput are the work and the capacity held: capacity is power given when it rises.

### *L.11.3 Monitor 9: Latency Stand-In per Decision · Class S*

Latency's stand-in a decision, native beside Omni-Compass: the work waiting in line on the live model, the spread of p across the muscles on the harness; neither body has a response time.

- **Evidence and state.** Class S; live.
- **Feed.** the service's own duration histogram; series `http_request_duration_seconds_bucket`; on a running system: `histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))`.
- **Wired.** This page's `drawFeed` → the Python `omnicompass/adapter.py`, `omnicompass/core.py`, `omnicompass/nervous_system.py`, `omnicompass/compass_law.py`, `realms/harness.py`, `docs/engine/export_harness_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `docs/engine/check_live_engine.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_twins_parity.py`, `docs/engine/check_records.py`.
- **Moves.** Every decision of the live benchmark engine: each side's line takes a point.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **How to read it.** The four golden signals of a service, one point a decision of the live benchmark engine: native in orange, Omni-Compass in blue, the same demand on both. A red band is a kill-switch window.
- **What to do with it.** Watch latency and errors first: a rise there is the only thing that can read LOSS. Saturation and throughput are the work and the capacity held: capacity is power given when it rises.

### *L.11.4 Monitor 10: Saturation Stand-In per Decision · Class S*

Saturation's stand-in a decision, native beside Omni-Compass: capacity in service on the live model, electrical power drawn (kW) on the harness.

- **Evidence and state.** Class S; live.
- **Feed.** node_exporter; series `node_cpu_seconds_total`; on a running system: `1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m]))`.
- **Wired.** This page's `drawFeed` → the Python `omnicompass/adapter.py`, `omnicompass/core.py`, `omnicompass/nervous_system.py`, `omnicompass/compass_law.py`, `realms/harness.py`, `docs/engine/export_harness_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `docs/engine/check_live_engine.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_twins_parity.py`, `docs/engine/check_records.py`.
- **Moves.** Every decision of the live benchmark engine: each side's line takes a point.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.
- **How to read it.** The four golden signals of a service, one point a decision of the live benchmark engine: native in orange, Omni-Compass in blue, the same demand on both. A red band is a kill-switch window.
- **What to do with it.** Watch latency and errors first: a rise there is the only thing that can read LOSS. Saturation and throughput are the work and the capacity held: capacity is power given when it rises.

## L.12 Control Console

**The page, in its own words.**

Do no harm. What Omni-Compass does with these controls is held by its guards: no muscle driven below native or past its own limit, nothing taken from one muscle to give to another, a give-back tried against native first and refused on a LOSS. Its one meaning is written once, in appendix G, as `DISCLOSURES.md` section 3 writes it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

**System Response per Decision · Class S**

**Inter-Frame Interval**

**Control Event Log**

**Control-Loop Performance Assessment**

**Its monitors.**

### *L.12.1 Monitor 60: System Response per Decision · Class S*

The last 120 decisions of the live benchmark engine: the muscles past their line and the work, native beside Omni-Compass, every knob of the console turned marked.

- **Evidence and state.** Class S; live.
- **Feed.** this page's own clock (no exporter: the page measures itself).
- **Wired.** This page's `drawDeskCv` → the Python `omnicompass/adapter.py`, `omnicompass/core.py`, `omnicompass/nervous_system.py`, `omnicompass/compass_law.py`, `realms/harness.py`, `docs/engine/export_harness_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `docs/engine/check_live_engine.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_twins_parity.py`, `docs/engine/check_records.py`.
- **Moves.** Every decision: both lanes slide one decision, a ring opens at each head with its newest value; every knob turned is marked.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused; Pause: hold the engine's clock; Frame Load: +0, +8 or +16 ms burned inside every frame of this page.
- **How to read it.** The live benchmark engine's response to every knob, native beside Omni-Compass, and the page's own frame budget.
- **What to do with it.** Every knob acts on both arms alike; the difference between them is Omni-Compass's doing alone.

### *L.12.2 Monitor 61: Inter-Frame Interval*

The time between frames of this page against its line under the frame load knob.

- **Evidence and state.** Class S; live.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Feed.** this page's own clock (no exporter: the page measures itself).
- **Wired.** This page's `drawFrameCv` → the Python none: the page's own frame clock, measured in the browser; there is nothing to port → the C++ none: the page's own frame clock, measured in the browser; there is nothing to port.
- **Moves.** Every frame of this page.
- **Knobs.** Frame Load: +0, +8 or +16 ms burned inside every frame of this page.
- **How to read it.** The live benchmark engine's response to every knob, native beside Omni-Compass, and the page's own frame budget.
- **What to do with it.** Every knob acts on both arms alike; the difference between them is Omni-Compass's doing alone.

## L.13 Heartbeat Dial

**Controls.** Recorded data: every beat of the Kubernetes governor read in our own models (class S, results/model_gate/V17_LADDER.json, tools/beat_ladder.py); the real runs at every beat (`docs/PACE_PREREGISTRATION.md`, amendment 3) join their place when their three runs land. The beat itself is set on the governor's command line (--interval) or with the control panel's heartbeat button (POST /v1/beat, tools/omni_control.py: one button for each stop of the dial), which hands every setting back and starts the governor again at the new beat.

**The page, in its own words.**

The governor's heartbeat, its full decision, is a dial from omni-v17 on: the main engine decides every 60 s, and any of six beats can be set on this page, 10, 15, 30, 60, 90 or 120 s, with the fast checks (the scheduling floor, the pod reflex, the conveyance every 15 s) every 5 s at every beat. Every clock of the governor counts seconds (omni-v16), so turning the dial changes only how often it looks. The governor's own screen counts the time since its last decision in beats of the setting it runs at and alerts after three (omni_heartbeat_seconds, deploy/grafana/omni-compass.json, deploy/prometheus/omni-compass-rules.yaml).

**Beat Ladder · Class S**

Each value is the beat's mean over seeded days on a plant that moves every 5 s, the same seeds and plant at every beat, on the engine its row names; the word beside it is the record's own reading against native (GAIN, STEADY, LOSS, power given: the key, appendix D), decided by the seeds' mean change and the 0.5% floor, with whether the seeds' paired interval stands clear of zero written beside it: a word inside the seeds' noise gives the direction of the mean, not a change the seeds stand behind. Native has no governor, so its reads and processor are blank. The reads are what a beat spends to look more often, shown as the key reads a resource spent: power given. A LOSS here is the engine's at that beat, a model's reading (class S): it stands in this table and on the next version's list; section L.7 lists the LOSS rows of the three-run tables of real software, never these.

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

# L.14 Control Cycle

**Controls.** Live: the cycle and the controls below share the console's state.

**The page, in its own words.**

Evidence class S: our own models, shown beside the real runs and never presented as proof.

**Decision Sequence**

**Cycle Controls**

The max: every muscle to its own limit, never below native; the capacity it holds is power given. Tune-down: never more than native, and a step back only with the organ’s grant and the release gate. In this model native already sits near its band, so the tune-down finds little to hand back; the scoreboard (section L.15) reads its runs A, B and C live when the dial is on the tune-down. The spread of positions can widen a little, because the calm muscles it hands back move toward their line while the busy ones stay.

Each stress lasts 40 decisions from the moment you press it. Both arms meet the same demand.

**Its monitors.**

## *L.14.1 Monitor 25: Compass Band of the Followed Muscle*

The compass band with its cushions, the two forces and one decision, step by step.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_compass_position`, `omni_compass_force`; on a running system: `omni_compass_position`; `omni_compass_force`.
- **Wired.** This page's `drawBand` → the Python `omnicompass/compass_law.py` → the C++ `cpp/tools/engine_runs.cpp`; held equal by `docs/engine/check_live_engine.py`, `verify.py`.
- **Moves.** One control cycle traced step by step on the followed muscle, every decision.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.
- **How to read it.** The brain's six state variables integrated by the fourth-order Runge–Kutta step every decision, the compass band with its cushions and its two forces, the organs' grants, native's own controller beside each grant.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **What to do with it.** The brain never forces: it reacts to what the body shows. A grant to expand is not a write; a write happens only where the muscle's own trial allows it.

## L.15 Phase Plane and Scoreboard

**Controls.** Live: the phase planes, the strip charts, the stat panels and the scoreboard respond to the console.

**The page, in its own words.**

Evidence class S: our own models, shown beside the real runs and never presented as proof.

**Native**

**Omni-Compass**

**Stat Panels, Last 120 Decisions · Class S**

Each panel is one gauge of the live organism, read every decision on both sides. It gives native's value now and Omni-Compass's, the last 120 decisions of both (native solid orange, Omni-Compass blue), and the running paired reading in the tables' words. With the master switch OFF the decisions are shaded red and Omni-Compass's side follows native: its value equals native's and its line is drawn in native's colour, dashed on top of native's, so both stay in view.

**Running Scoreboard · Class S**

Each row is read on the paired change, native against Omni-Compass at the same decision, averaged over blocks of ten decisions so neighbouring decisions do not count twice, with its 95% interval (Student’s t over the blocks). This run (tools/readings.py, run_word): a change under 0.5% of native reads STEADY; 0.5% or more, the word follows the gauge’s aim (work: a rise is GAIN; misses, failed work, waiting: a fall is GAIN; energy or capacity held: a rise is power given, a fall a cut), its 95% interval printed beside it. Runs A, B and C (tools/readings.py, three_run_reading): the change is the mean of the three runs' changes; under 0.5% it reads STEADY, under the floor; 0.5% or more reads GAIN, LOSS or power given by the gauge's aim, “in all three of our runs” when every run went the same way clear of zero, otherwise “by the mean of our three runs” with the count of runs beyond the noise; runs that moved 0.5% or more in opposite directions disagree. The spread of positions and the settings are shown, never judged: no rule of the engine aims at them. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

**Its monitors.**

### *L.15.1 Monitor 27: Native Phase Plane*

Native's muscles in position and velocity.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_compass_position`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Wired.** This page's `drawPhase` → the Python `omnicompass/adapter.py`, `omnicompass/core.py`, `omnicompass/nervous_system.py`, `omnicompass/compass_law.py`, `realms/harness.py`, `docs/engine/export_harness_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `docs/engine/check_live_engine.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_twins_parity.py`, `docs/engine/check_records.py`.
- **Moves.** Every decision: each muscle's point moves in position and velocity.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.
- **How to read it.** The brain's six state variables integrated by the fourth-order Runge–Kutta step every decision, the compass band with its cushions and its two forces, the organs' grants, native's own controller beside each grant.
- **What to do with it.** The brain never forces: it reacts to what the body shows. A grant to expand is not a write; a write happens only where the muscle's own trial allows it.

### *L.15.2 Monitor 28: Omni-Compass Phase Plane*

Omni-Compass's muscles in position and velocity, the band's walls drawn.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_compass_position`.
- **Wired.** This page's `drawPhase` → the Python `omnicompass/adapter.py`, `omnicompass/core.py`, `omnicompass/nervous_system.py`, `omnicompass/compass_law.py`, `realms/harness.py`, `docs/engine/export_harness_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `docs/engine/check_live_engine.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_twins_parity.py`, `docs/engine/check_records.py`.
- **Moves.** Every decision: each muscle's point moves in position and velocity.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.

- **How to read it.** The brain's six state variables integrated by the fourth-order Runge–Kutta step every decision, the compass band with its cushions and its two forces, the organs' grants, native's own controller beside each grant.
- **What to do with it.** The brain never forces: it reacts to what the body shows. A grant to expand is not a write; a write happens only where the muscle's own trial allows it.

### *L.15.3 Monitor 26: Scoreboard Strip · Class S*

The body's capacity in service and its demand (in the harness, its electrical power), the muscles past their line and the spread of positions over the day, native beside Omni-Compass.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_compass_decisions_total`, `omni_writes_total`; on a running system: `increase(omni_compass_decisions_total{zone="wall"}[1h])`; `increase(omni_writes_total[1h])`.
- **Wired.** This page's `drawStrip` → the Python `omnicompass/adapter.py`, `omnicompass/core.py`, `omnicompass/nervous_system.py`, `omnicompass/compass_law.py`, `realms/harness.py`, `docs/engine/export_harness_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `docs/engine/check_live_engine.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_twins_parity.py`, `docs/engine/check_records.py`.
- **Moves.** Every decision of the live benchmark engine (900 ms at 1×): the line takes a point, a ring opens at its head and its newest value is printed beside it.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.
- **How to read it.** The brain's six state variables integrated by the fourth-order Runge–Kutta step every decision, the compass band with its cushions and its two forces, the organs' grants, native's own controller beside each grant.
- **What to do with it.** The brain never forces: it reacts to what the body shows. A grant to expand is not a write; a write happens only where the muscle's own trial allows it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

# L.16 Brain State

In the harness views the harness arm (realms/compass_arm_v5.py) acts muscle by muscle with no brain, so the brain drawn here is the page's own, fed the harness's readings. With each muscle alone there is no body, and its power and thermal readings are held at 0.3.

**Controls.** Live: the wells, the native trace and the nerve meters respond to the console.

**The page, in its own words.**

Evidence class S: our own models, shown beside the real runs and never presented as proof.

**Double-Well Potential of U, Eq. (2)**

$V(U) = -\mu(U^2/2 - U^4/4) + (\dot{E}/E_{max})\cdot U$, eq. (2). A rising E tilts the well toward the basin of collapse.

**Cubic Potential of Stress S, Eqs. (5)–(6)**

$\Phi(S) = \alpha_s S^2/2 + \beta_s S^3/4 - \delta S$, eqs. (5)–(6). S rolls to S*; the senses pull it back each decision.

**Bath Oscillator B, Eq. (7)**

$\dot{B} = \gamma[c]\ \delta\ S - (\omega[B]/Q[B])\ \dot{B} - \omega[B]^2\ B$, eq. (7). B is the slow tide of physical load.

**Native Controller Trace · Class S**

Native has no brain to draw: each of its controllers reads one reading, its own muscle's, and moves. Drawn live from the same decisions as the wells above: the mean position its muscles' readings put them at (0 calm, 1 at the line), the share of its muscles past their line, and how many of its own controllers moved their setting this decision. Class S, our own models.

**Nerves and Calm**

Native has no nerves: nothing grants or refuses a controller, and a blind sense or a security hold changes nothing it does.

$calm = \kappa\cdot h\cdot(1 - (\sigma - 1)^+)\cdot(1 - \min(\nu, 1))$. An organ may hand capacity back only when calm ≥ its θ, the line is clean and every sense is live; expanding stays allowed except under a security hold. Under the max, lifting off toward native never needs this grant; going below native does.

**Its monitors.**

## *L.16.1 Monitor 29: Double-Well Potential of U*

The brain's double-well potential and where its state sits.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_verdict_state`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Wired.** This page's `drawBrain, drawWell` → the Python `omnicompass/core.py`, `omnicompass/adapter.py` → the C++ `cpp/src/core.cpp`, `cpp/src/governor.cpp`; held equal by `docs/engine/check_live_engine.py`, `tests/test_cpp_governor_parity.py`.
- **Moves.** Every decision the ball rolls to the brain's state and a ring opens where it lands.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.
- **How to read it.** The brain's six state variables integrated by the fourth-order Runge–Kutta step every decision, the compass band with its cushions and its two forces, the organs' grants, native's own controller beside each grant.
- **What to do with it.** The brain never forces: it reacts to what the body shows. A grant to expand is not a write; a write happens only where the muscle's own trial allows it.

### *L.16.2 Monitor 30: Cubic Potential of Stress S*

Stress S on its cubic potential $\Phi(S) = \alpha_s S^2/2 + \beta_s S^3/4 - \delta S$, rolling toward S*; the senses pull it back each decision (eqs. (5)–(6)).

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_compass_position`, `omni_compass_force`.
- **Wired.** This page's `drawBrain, drawWell` → the Python `omnicompass/core.py`, `omnicompass/adapter.py` → the C++ `cpp/src/core.cpp`, `cpp/src/governor.cpp`; held equal by `docs/engine/check_live_engine.py`, `tests/test_cpp_governor_parity.py`.
- **Moves.** Every decision the ball rolls to the brain's state and a ring opens where it lands.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **How to read it.** The brain's six state variables integrated by the fourth-order Runge–Kutta step every decision, the compass band with its cushions and its two forces, the organs' grants, native's own controller beside each grant.
- **What to do with it.** The brain never forces: it reacts to what the body shows. A grant to expand is not a write; a write happens only where the muscle's own trial allows it.

### *L.16.3 Monitor 31: Bath Oscillator B*

The bath oscillator B, the slow tide of physical load, over the last 120 decisions (eq. (7)), its spring at the latest B.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_recommended`.
- **Wired.** This page's `drawBrain` → the Python `omnicompass/core.py`, `omnicompass/adapter.py` → the C++ `cpp/src/core.cpp`, `cpp/src/governor.cpp`; held equal by `docs/engine/check_live_engine.py`, `tests/test_cpp_governor_parity.py`.
- **Moves.** Every decision the bath's line takes a point and a ring opens at its head.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.
- **How to read it.** The brain's six state variables integrated by the fourth-order Runge–Kutta step every decision, the compass band with its cushions and its two forces, the organs' grants, native's own controller beside each grant.
- **What to do with it.** The brain never forces: it reacts to what the body shows. A grant to expand is not a write; a write happens only where the muscle's own trial allows it.

### *L.16.4 Monitor 32: Native Controller Trace · Class S*

What native's controller would do at the same decision, beside the brain's grant.

- **Evidence and state.** Class S; live.
- **Feed.** kube-state-metrics; series `kube_horizontalpodautoscaler_status_desired_replicas`.
- **Wired.** This page's `drawNat6c` → the Python `realms/harness.py`, `docs/engine/export_harness_trace.py` → the C++ none: native's own controllers, as the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

harness's record or the page's own model holds them; held equal by `docs/engine/check_records.py`.

- **Moves.** Every decision of the live benchmark engine.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.
- **How to read it.** The brain's six state variables integrated by the fourth-order Runge–Kutta step every decision, the compass band with its cushions and its two forces, the organs' grants, native's own controller beside each grant.
- **What to do with it.** The brain never forces: it reacts to what the body shows. A grant to expand is not a write; a write happens only where the muscle's own trial allows it.

## L.17 Verdict

**The page, in its own words.**

The verdict is the change-acceptance test. Each step is trialled against native and kept unless the measurement reads LOSS or its estimated cost exceeds the 2% allowance. A STEADY result (no difference resolvable from the noise) is also accepted, so on high-variance workloads a step with a small true cost can pass; the lab below measures that false-acceptance rate.

**Current Trial**

**Share of Trials Allowed**

Native: its knob never moves past its own setting, so there is no trial and nothing to judge; every step Omni-Compass takes past native has to pass this test first. The medians alone allow a step when the trial's median is within the allowance of the reference's. The engine's rule (omni-v12's, unchanged) also refuses any step whose interval sits wholly above zero. It measures on while the interval is wider than the allowance, up to three times the pieces measured each side. It keeps a step only if both estimates are inside the allowance. Ported from omnicompass/verdict.py; the costs are modelled (class S).

Wiring per Muscle. The Kubernetes controller measures 200 requests each side against a 2% allowance (omni_controller/controller.py --verdict-samples 200 --allow 0.02), the cost being each request's response time. The noise is measured at every build on each request of the newest tables' own runs. The σ 0.05 and 0.25 used before 19 October did not come from these files and are withdrawn. The GPU card measures 30 requests (gpu_compass.py --verdict-samples 30); in its model the card's own time per request carries no noise. The model gate measures 60 (tools/model_gate.py, σ 0.25). Keep Rate by Cost. Computed live in this page: trials

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

at each cost with this lab's own rule, 200 requests a side to start (the trial side measured on to 600 while its interval is wider than the allowance), a 2% allowance and the measured σ (modelled, class S). With noise this wide, 200 requests a side do not refuse a small LOSS on their own; with no noise any LOSS it can read is refused, 0.1% included. A larger sample would refuse more. Above the verdict the max never goes below native at all, so in the default it never runs (section L.22: no trial on top of the HPA).

**Its monitors.**

### *L.17.1 Monitor 33: Trial Costs and 95% Interval*

A give-back trial judged against native: its reference, its step, its interval and the word.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_verdict_state`; on a running system: `omni_verdict_state`.
- **Wired.** This page's `drawVerdict, vTick, judge` → the Python `omnicompass/verdict.py`, `omnicompass/student_t.py` → the C++ `cpp/tools/engine_runs.cpp`; held equal by `docs/engine/check_live_engine.py`, `verify.py`.
- **Moves.** A cost measured every 28 ms until the verdict reads; a new trial starts 2.6 s after.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.
- **How to read it.** A give-back trial judged against native: its reference, its step, its interval, the word; then the keep ratio at every true cost.
- **What to do with it.** A step is kept only when its interval stays inside the allowance; a refused step leaves the muscle where native had it.

### *L.17.2 Monitor 34: Share of Trials Allowed*

The keep ratio of the verdict at each true cost, from trials run in this page's idle time.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_verdict_state`.
- **Wired.** This page's `drawVerdict2, runTrialSync` → the Python `omnicompass/verdict.py`, `omnicompass/student_t.py` → the C++

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`cpp/tools/engine_runs.cpp`; held equal by `docs/engine/check_live_engine.py`, `verify.py`.

- **Moves.** Trials run in the browser's idle time; the shares fill as they run.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native).
- **How to read it.** A give-back trial judged against native: its reference, its step, its interval, the word; then the keep ratio at every true cost.
- **What to do with it.** A step is kept only when its interval stays inside the allowance; a refused step leaves the muscle where native had it.

## L.18 Dashboards

**The page, in its own words.**

Each form is fed by this page's own model as it runs (class S), native in orange beside Omni-Compass in blue, each change with its word by the key; the forms are the industry's, the numbers are this engine's, and no vendor's product is drawn or implied. The level sets what the host map and the cluster console gather (each muscle, each body part, or the whole organism) and whether the cost form reads body part by body part or the whole organism; the board, the graph, the event stream, the one-line diagram, the wall of numbers and the pipeline always read the whole organism.

**Roles in the Control Stack**

**First Five Products to Benchmark**

Each takes its own preregistration (the knobs it owns, the muscles left native, its learning window), a trial run that is never counted, and its three runs; a vendor's licence is read for evaluation and publication first. Every other product, the observers, the consoles and the native controllers, with its sources: `docs/THE_FIELD.md`. A vendor's savings figure is that vendor's own claim.

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

## L.19 Part 4 of the page: Real Runs and Platforms

## L.20 Kubernetes Run Playback

Class L: the benchmark runs themselves, streamed half minute by half minute from the raw files GitHub archived, every file SHA-256-checked.

**The page, in its own words.**

**Data Pipeline**

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### Governor Heartbeat and Lease

A supervisory governor is only alive while its loop turns: once every decision it reads the whole body, decides, writes, and proves it is still there. That turn is the heart. Each beat pumps every reading in through the plug and the decision out (omni_controller/controller.py, its main loop), and renews the governor's lease (master.heartbeat(), omnicompass/master.py). If the heart stops, its process gone or its lease older than its limit (the larger of 120 s and five beats), the watchdog (tools/omni_switch.py watchdog) walks every setting back to native: a stopped heart leaves the body exactly as native runs it. Native has no heart of its own: each of its controllers keeps its own clock (the pod autoscaler every 15 s, the firmware, the thermostats) and no beat joins them. On the engine that stands the governor's heart beats every 60 s at the factory, a full read of the state once a minute (from omni-v17); 15 s, the pod autoscaler's own pace, is a beat a person may pick on the dial (3d), as are 10, 30, 90 and 120 s. omni-v16 beat every 15 s (`docs/OMNI_V16.md`).

The beat runs at a fixed rate, never driven faster by the load it governs: on the engine that stands one decision every 60 s at the factory, or the beat a person picks on the dial, the fast checks every 5 s between beats. The runs replayed here ran on the engine each table names, omni-v12 and omni-v21, each at the beat its run's own files record. Every clock of the engine counts seconds, so the beat's pace sets how often the body is read, never what a rule means: a give-back still waits its trial's minutes, the cushion still keeps its twenty minutes of memory, and a machine still goes back at most one a minute. What it costs is counted inside the body's own load. A decision that fails is recorded and skipped: it writes nothing, the body keeps the last settings that landed, and the next beat comes at its normal time (safe_step). Two gauges below read its health: the share of each beat spent deciding, and how close the lease came to its limit. The page calls a heart under strain when a decision takes more than half its beat, or a gap between beats reaches 80% of the lease. Past half its beat, the governor would have less than half its period left to act before the next reading. Both lines are this page's reading, not rules of the engine; the engine's own guard is the lease and the watchdog.

### Heartbeat Sweep · Class S

The governor's full read of the state, its heartbeat, is a setting of the run: every clock of the engine counts seconds, so the beat sets only how often the body is read, never what a rule means (omni-v16, `docs/OMNI_V16.md`). The factory sets 60 s since omni-v17, the main engine (omni-v16 set 15 s, the pod autoscaler's own pace; this sweep reads omni-v16 and omni-v15, and the dial's own ladder reads omni-v17 at every beat in section L.13). A slower beat reads the cluster less, a faster one answers sooner; this monitor shows what each beat does. Each bar is our own model (evidence class S, never a benchmark result): the shipped controller on the modelled cluster's 5-second plant, eight seeds of a day each, paired by seed with native and read by the one key (tools/heartbeat_sweep.py, `results/model_gate/HEARTBEAT_SWEEP.md`). Every beat here is read in the model only: a real run reads the one beat it was started with, the governor's --interval.

### Response Time Candlesticks

### Latency Heatmap

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

**Throughput per 30 s Bin**

**Paired Repetitions**

**Governor Decision Ticker**

**Native Controller Ticker**

From the repetition's own audit.jsonl: the minute and second of the decision, the compass position (0 calm, 1 at the line), and the force on the pedal (push: toward more; lift: handing the setting back toward native). Then the machines it recommends, and the give-back verdict's state: "left native" means every setting stays native's; "acting" means a trial on its own measurement. Native has no governor; its own controllers' moves are the second ticker, read from the native arm's capture.csv: the replicas its pod autoscaler holds, the pods waiting for a place and the machines ready. With Omni-Compass OFF the first ticker is empty, for nothing decides on top of native.

**Benchmark Status by Block**

**Its monitors.**

### *L.20.1 Monitor 13: Data Pipeline*

Every stage from the GitHub Actions run to this page: the run, the raw files archived, the SHA-256 check, the official scorer, the three-run tables, the build and the page, the stage being read lit.

- **Evidence and state.** Class L; replay.
- **Feed.** GitHub Actions API (read at the build); live: github-actions-exporter.
- **Wired.** This page's `drawPipe` → the Python `scripts/raw_archive.sh`, `tools/live_reps.py`, `tools/confirm_abc.py`, `docs/engine/build_live_engine.py` → the C++ none: the stages are GitHub's runs and the repository's own tools; held equal by `docs/engine/build_live_engine.py`.
- **Moves.** The stage being read lights in turn and the data moves between the stages.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Speed: 1×, 4×, 16× or 60× of the engine's clock; Gauge: which gauge of the run is shown.
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v23; an older engine's run is its own record.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

### *L.20.2 Monitor 14: Governor Heartbeat and Lease*

Native's own 15 s clock above, every beat of the governor's decision loop in the repetition below, native beside Omni-Compass.

- **Evidence and state.** Class L; replay.
- **Feed.** kube-state-metrics; the load generator's own histogram (the rig's loadgen); series `kube_horizontalpodautoscaler_status_current_replicas`, `kube_node_status_allocatable`; on a running system: `sum(kube_horizontalpodautoscaler_status_current_replicas)`.
- **Wired.** This page's `drawHeart` → the Python `docs/engine/export_runs.py`, `omni_controller/controller.py` → the C++ none: the live software's own measurements, played back from their SHA-256-checked files; held equal by `docs/engine/build_live_engine.py`.
- **Moves.** The chosen repetition replayed decision by decision; native's own 15 s clock beats above.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Speed: 1×, 4×, 16× or 60× of the engine's clock; Gauge: which gauge of the run is shown; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v23; an older engine's run is its own record.

### *L.20.3 Monitor 36: Heartbeat Sweep · Class S*

The governor's beat as a knob: what a decision every 5 to 120 s does against native, or against the same engine at 15 s or 60 s, on-site and in the cloud, gauge by gauge with its 95% interval, and what each beat costs the engine; our own model.

- **Evidence and state.** Class S; model.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_decisions_total`, `omni_decision_seconds`, `omni_last_decision_timestamp_seconds`, `omni_own_cpu_cores`; on a running system: `60 * rate(omni_decisions_total[5m])`; `omni_own_cpu_cores`; `time() - omni_last_decision_timestamp_seconds`.
- **Wired.** This page's `drawBeat, beatSetup` → the Python `tools/heartbeat_sweep.py`, `tools/model_gate.py`, `omni_controller/controller.py` → the C++ none here: the sweep runs the shipped Python controller on the model cluster; the C++ governor twin is held to the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Python's at 60 s and at 15, 10 and 5 s (tests/test_cpp_governor_parity.py); held equal by `results/model_gate/HEARTBEAT_SWEEP.md`, `docs/engine/build_live_engine.py`.

- **Moves.** A cursor reads the beats in turn and names the row it holds; every button redraws the chart from the record.
- **Knobs.** Beat: the governor's full read of the state and its decision: every 5, 10, 15, 20, 30, 60, 75, 90 or 120 s in the heartbeat sweep's record; on a real governor its --interval (60 s at the factory since omni-v17, 15 s on omni-v16); Scenario: on-site bare metal, through the cloud, in the device, as a service; Engine: which engine's record the monitor reads: the engine that stands or the one before it; Baseline: what each reading is set against: native, or the same engine at another setting; Gauge: which gauge of the run is shown; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v23; an older engine's run is its own record.

### *L.20.4 Monitor 15: Response Time Candlesticks*

The response time of the chosen run as candles, one a half minute, native above and Omni-Compass below, on one logarithmic scale.

- **Evidence and state.** Class L; replay.
- **Feed.** the load generator's own histogram (the rig's loadgen); series `http_request_duration_seconds_bucket`; on a running system: `histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[1m])))`.
- **Wired.** This page's `drawCandle` → the Python `docs/engine/export_runs.py`, `tools/live_reps.py`, `scripts/raw_archive.sh` → the C++ none: the live software's own measurements, played back from their SHA-256-checked files; held equal by `docs/engine/build_live_engine.py`.
- **Moves.** The chosen run replayed half minute by half minute.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Gauge: which gauge of the run is shown; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v23; an older engine's run is its own record.

### *L.20.5 Monitor 16: Latency Heatmap*

The responses of each half minute counted in eight response-time buckets, native above and Omni-Compass below.

- **Evidence and state.** Class L; replay.
- **Feed.** the load generator's own histogram (the rig's loadgen).
- **Wired.** This page's `drawHeat` → the Python `docs/engine/export_runs.py`, `tools/live_reps.py`, `scripts/raw_archive.sh` → the C++ none: the live software's own measurements, played back from their SHA-256-checked files; held equal by `docs/engine/build_live_engine.py`.
- **Moves.** The chosen run replayed half minute by half minute.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Gauge: which gauge of the run is shown; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v23; an older engine's run is its own record.

### *L.20.6 Monitor 17: Throughput per 30 s Bin*

The requests answered each half minute, native and Omni-Compass side by side, the failed requests marked.

- **Evidence and state.** Class L; replay.
- **Feed.** the load generator's own histogram (the rig's loadgen).
- **Wired.** This page's `drawBars` → the Python `docs/engine/export_runs.py`, `tools/live_reps.py`, `scripts/raw_archive.sh` → the C++ none: the live software's own measurements, played back from their SHA-256-checked files; held equal by `docs/engine/build_live_engine.py`.
- **Moves.** The chosen run replayed half minute by half minute.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Gauge: which gauge of the run is shown; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v23; an older engine's run is its own record.

### *L.20.7 Monitor 18: Paired Repetitions*

Each repetition's reading in the chosen gauge with its interval.

- **Evidence and state.** Class L; replay.
- **Feed.** the load generator's own histogram (the rig's loadgen).
- **Wired.** This page's `drawReps` → the Python `docs/engine/export_runs.py`, `tools/live_reps.py`, `scripts/raw_archive.sh` → the C++ none: the live software's own measurements, played back from their SHA-256-checked files; held equal by `docs/engine/build_live_engine.py`.
- **Moves.** Each repetition's reading filled in as the run replays.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Gauge: which gauge of the run is shown; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v23; an older engine's run is its own record.

## L.21 Repetition Telemetry

Class L: real Kubernetes on kind, real databases and a real broker; each three-run table's runs A, B and C, read back from the raw files GitHub archived (results/live/raw), every file's SHA-256 matched against the run's own SHA256SUMS_ALL.txt.

**The page, in its own words.**

**Native**

**Omni-Compass**

**Run Receipt**

**Repetition Score**

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

**Benchmark Landing Status**

**Its monitors.**

### *L.21.1 Monitor 19: Repetition Telemetry per 30 s Bin*

One repetition half minute by half minute from its archived capture: the 95th percentile of the response time, the replicas, the host's processor and the governor's heartbeat, native beside Omni-Compass.

- **Evidence and state.** Class L; replay.
- **Feed.** kube-state-metrics; metrics-server; Kepler; series `kube_node_status_condition`, `kube_pod_status_phase`, `kepler_node_package_joules_total`; on a running system: `sum(kube_pod_status_phase{phase="Pending"})`; `rate(kepler_node_package_joules_total[1m])`.
- **Wired.** This page's `drawRuStrip` → the Python `docs/engine/export_runs.py`, `tools/live_reps.py`, `scripts/raw_archive.sh` → the C++ none: the live software's own measurements, played back from their SHA-256-checked files; held equal by `docs/engine/build_live_engine.py`.
- **Moves.** The archived capture replayed half minute by half minute; a ring opens at each head with its newest value.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Repetition: which repetition of the run is replayed; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v23; an older engine's run is its own record.

## L.22 Controller Decision Trace

The shipped controller, omni_controller/controller.py, runs decision by decision on the modelled cluster of pilot/selfpilot.py, wired as the repository wires it. Each decision runs the chain left to right: the senses, the six-state brain, the nerves and the machine organ’s grant, the compass law on the service (cloud setting), the verdict’s trial, the release gate, the plug. Every value below is the controller’s own audit record of that decision, recorded by docs/engine/export_controller_trace.py on runs A, B and C (six hours each, a decision every 60 s, class S).

**Controls.** Recorded data: the shipped controller replayed decision by decision from its own audit (class S).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

**The page, in its own words.**

**Native**

**Omni-Compass**

**Audited Decision Chain**

**Three-Run Score · Class S**

The paired change per 15 s tick, averaged over ten-minute blocks, with its 95% interval (Student’s t) in each run. Runs A, B and C (tools/readings.py, three_run_reading): the change is the mean of the three runs' changes; under 0.5% it reads STEADY, under the floor; 0.5% or more reads GAIN, LOSS or power given by the gauge's aim, “in all three of our runs” when every run went the same way clear of zero, otherwise “by the mean of our three runs” with the count of runs beyond the noise; runs that moved 0.5% or more in opposite directions disagree. Machines and power: a fall is a cut, a GAIN; a rise is power given. Our own models (class S): a model’s small numbers are the model’s.

**Kubernetes Three-Run Tables · Class L**

**Governor Steering on Kubernetes · Class L**

**Its monitors.**

### *L.22.1 Monitor 21: Native Cluster Trace · Class S*

The controller every Kubernetes and cloud set runs, replayed decision by decision from its own audit: native's cluster.

- **Evidence and state.** Class S; replay.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_nodes_observed`; on a running system: `omni_nodes_observed`.
- **Wired.** This page's `drawCluster` → the Python `omni_controller/controller.py`, `pilot/selfpilot.py`, `fleet/harness.py`, `tools/model_gate.py`, `docs/engine/export_controller_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/shield.cpp`, `cpp/src/compass.cpp`, `cpp/src/hpa.cpp`; held equal by `docs/engine/check_records.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_shield_parity.py`, `tests/test_cpp_hpa_parity.py`.
- **Moves.** The shipped controller's record played decision by decision on the live benchmark engine's clock; a ring opens at each head with its newest value.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v23; an older engine's run is its own record.

### *L.22.2 Monitor 22: Omni-Compass Cluster Trace · Class S*

The same, Omni-Compass's cluster: every write, every trial, every hold.

- **Evidence and state.** Class S; replay.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_recommended`, `omni_writes_total`, `omni_verdict_state`; on a running system: `omni_recommended`; `increase(omni_writes_total[1h])`.
- **Wired.** This page's `drawCluster` → the Python `omni_controller/controller.py`, `pilot/selfpilot.py`, `fleet/harness.py`, `tools/model_gate.py`, `docs/engine/export_controller_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/shield.cpp`, `cpp/src/compass.cpp`, `cpp/src/hpa.cpp`; held equal by `docs/engine/check_records.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_shield_parity.py`, `tests/test_cpp_hpa_parity.py`.
- **Moves.** The shipped controller's record played decision by decision on the live benchmark engine's clock; a ring opens at each head with its newest value.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v23; an older engine's run is its own record.

### *L.22.3 Monitor 23: Machines and Response Time · Class S*

The shipped controller's machines (in service, booting and parked) and the response time over the run, native beside Omni-Compass, replayed from its own audit.

- **Evidence and state.** Class S; replay.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_compass_position`, `omni_compass_force`; on a running system: `omni_compass_position`; `omni_compass_force`.
- **Wired.** This page's `drawCStrip` → the Python `omni_controller/controller.py`, `pilot/selfpilot.py`, `fleet/harness.py`, `tools/model_gate.py`, `docs/engine/export_controller_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/shield.cpp`, `cpp/src/compass.cpp`, `cpp/src/hpa.cpp`; held equal by `docs/engine/check_records.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_shield_parity.py`, `tests/test_cpp_hpa_parity.py`.
- **Moves.** The shipped controller's record played decision by decision on the live benchmark engine's clock; a ring opens at each head with its newest value.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v23; an older engine's run is its own record.

## L.23 Cloud Providers

**Controls.** Recorded data: GitHub's own cloud runs as read at the build. The controller below plays live; its run and speed follow the console's clock.

**The page, in its own words.**

**Cloud Run Ledger**

**Cloud Bill and Gate Monitors**

One monitor a provider, the same harness, mechanism and tuning in all three. Each bar pair is a finished set: native's machine-hours (orange) beside Omni-Compass's (blue), the word the run's own by the one key, the same rule as the bill table below (tools/readings.py run_word: the bill aims lower, a fall of 0.5% or more a GAIN, a rise of 0.5% or more power given, STEADY only under 0.5%), its size with no sign and, above it, whether the change stands beyond the run's own noise; the bill at list price under it. One run is never a set's reading: a set reads GAIN, STEADY or LOSS only from three runs on one engine. A provider with no finished set shows its empty frame and the reason its hourly gate holds, in its own job's words; nothing is drawn for a run that has not run.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

**Azure Bill by Run**

From 19 October: every Azure set runs up to 100 paired repetitions a run, the paid cap, under the preregistered stopping rule (tools/stop_rule.py). After 10, 20, 40 and 70 repetitions the bill's change is read with its 99% interval. The run stops once the bill is read (GAIN, power given, or STEADY within ±2% of native's bill), and never while the speed or the failed requests read a LOSS. Those runs carry "to the cap, stopping rule" in their title and count as their own A, B and C; the 5-repetition runs below stay as their own engines' record.

**Cloud-Setting Controller · Class S**

The controller every cloud set runs from 31 October (omni_controller/controller.py at the max, the whole engine), decision by decision on the repository's modelled cluster (class S; until the live-engine workflow rebuilds this record from the push of 31 October, the record shown is the one read at the tune-down on the machines alone). The cloud's own autoscaler runs in both arms. Omni-Compass moves the target inside the autoscaler's cover and parks a machine the cloud already made, after its trial, waking it on a breach. The bill counts every machine that exists, in service, booting or parked, from the record.

**Native**

**Omni-Compass**

**Its monitors.**

### *L.23.1 Monitor 37: Azure Bill*

Machine-hours billed and the bill at list price, native against Omni-Compass, every finished Azure set with its 95% interval, and the running set's repetitions done.

- **Evidence and state.** Class L (metered by Azure); record.
- **Feed.** OpenCost; Azure's own billing export; series `node_total_hourly_cost`, `kube_node_info`; on a running system: `sum(node_total_hourly_cost)`.
- **Wired.** This page's `drawCloudBill` → the Python `tools/live_reps.py`, `tools/stop_rule.py`, `docs/engine/build_live_engine.py` → the C++ none: the bill as Azure meters it; held equal by `docs/engine/build_live_engine.py`.
- **Moves.** The reading cursor walks the finished sets; the age of GitHub's reading counts up every second.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The bill the cloud meters: machine-hours and the bill at list price, native's bar beside Omni-Compass's, each finished set with its word; the running set's repetitions measured so far; each provider's hourly gate with the reason its own job wrote.
- **What to do with it.** The bill aims lower: a fall is a GAIN, inside the noise STEADY, a rise power given. A provider whose gate holds runs nothing and bills nothing; its frame stays empty until its first set lands.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

### *L.23.2 Monitor 38: Google Cloud Gate and Bill*

Google Cloud's hourly gate reading (the key, the login, the quota) and, once a set runs, its bill native against Omni-Compass; until its first set lands, the mechanism every cloud set runs, live on the cloud setting's model (class S, marked, never this cloud's bill)

- **Evidence and state.** Class L; awaiting its run: the first set starts within the hour after the gate passes: the key stored whole (`docs/GOOGLE_SETUP.md`, section 4).
- **Feed.** OpenCost; Google Cloud Billing export; series `node_total_hourly_cost`; on a running system: `sum(node_total_hourly_cost)`.
- **Wired.** This page's `drawCloudBill` → the Python `tools/cloud_key_check.py`, `omni_controller/controller.py`, `pilot/selfpilot.py`, `fleet/harness.py`, `tools/model_gate.py`, `docs/engine/export_controller_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/shield.cpp`, `cpp/src/compass.cpp`, `cpp/src/hpa.cpp`; held equal by `tests/test_cloud_check.py`, `docs/engine/check_records.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_shield_parity.py`, `tests/test_cpp_hpa_parity.py`.
- **Moves.** Until the first set lands: the modelled machines decision by decision on the live benchmark engine's clock, a ring at each head.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The bill the cloud meters: machine-hours and the bill at list price, native's bar beside Omni-Compass's, each finished set with its word; the running set's repetitions measured so far; each provider's hourly gate with the reason its own job wrote.
- **What to do with it.** The bill aims lower: a fall is a GAIN, inside the noise STEADY, a rise power given. A provider whose gate holds runs nothing and bills nothing; its frame stays empty until its first set lands.

### *L.23.3 Monitor 39: Amazon Web Services Gate and Bill*

Amazon Web Services' hourly gate reading and, once a set runs, its bill native against Omni-Compass; until its first set lands, the mechanism every cloud set runs, live on the cloud setting's model (class S, marked, never this cloud's bill)

- **Evidence and state.** Class L; awaiting its run: the first set starts within the hour after the gate passes: the account on the Paid plan (`docs/AMAZON_SETUP.md`, section 1).
- **Feed.** OpenCost; AWS Cost and Usage Report; series `node_total_hourly_cost`; on a running system: `sum(node_total_hourly_cost)`.
- **Wired.** This page's `drawCloudBill` → the Python `tools/cloud_key_check.py`, `omni_controller/controller.py`, `pilot/selfpilot.py`, `fleet/harness.py`,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`tools/model_gate.py`, `docs/engine/export_controller_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/shield.cpp`, `cpp/src/compass.cpp`, `cpp/src/hpa.cpp`; held equal by `tests/test_cloud_check.py`, `docs/engine/check_records.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_shield_parity.py`, `tests/test_cpp_hpa_parity.py`.

- **Moves.** Until the first set lands: the modelled machines decision by decision on the live benchmark engine's clock, a ring at each head.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The bill the cloud meters: machine-hours and the bill at list price, native's bar beside Omni-Compass's, each finished set with its word; the running set's repetitions measured so far; each provider's hourly gate with the reason its own job wrote.
- **What to do with it.** The bill aims lower: a fall is a GAIN, inside the noise STEADY, a rise power given. A provider whose gate holds runs nothing and bills nothing; its frame stays empty until its first set lands.

### *L.23.4 Monitor 40: Native Cloud Machines · Class S*

The shipped controller's cloud mode on the model cluster: the cloud's own autoscaler alone.

- **Evidence and state.** Class S; replay.
- **Feed.** kube-state-metrics; tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `kube_node_info`, `omni_nodes_observed`; on a running system: `count(kube_node_info)`.
- **Wired.** This page's `drawCluster` → the Python `omni_controller/controller.py`, `pilot/selfpilot.py`, `fleet/harness.py`, `tools/model_gate.py`, `docs/engine/export_controller_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/shield.cpp`, `cpp/src/compass.cpp`, `cpp/src/hpa.cpp`; held equal by `docs/engine/check_records.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_shield_parity.py`, `tests/test_cpp_hpa_parity.py`.
- **Moves.** The shipped controller's record played decision by decision on the live benchmark engine's clock; a ring opens at each head with its newest value.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **How to read it.** The bill the cloud meters: machine-hours and the bill at list price, native's bar beside Omni-Compass's, each finished set with its word; the running set's repetitions measured so far; each provider's hourly gate with the reason its own job wrote.
- **What to do with it.** The bill aims lower: a fall is a GAIN, inside the noise STEADY, a rise power given. A provider whose gate holds runs nothing and bills nothing; its frame stays empty until its first set lands.

### *L.23.5 Monitor 41: Omni-Compass Cloud Machines · Class S*

The same autoscaler with Omni-Compass parking a machine only after its trial, waking it on a breach.

- **Evidence and state.** Class S; replay.
- **Feed.** kube-state-metrics; tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `kube_node_info`, `omni_nodes_observed`, `omni_writes_total`; on a running system: `count(kube_node_info)`; `omni_nodes_observed`.
- **Wired.** This page's `drawCluster` → the Python `omni_controller/controller.py`, `pilot/selfpilot.py`, `fleet/harness.py`, `tools/model_gate.py`, `docs/engine/export_controller_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/shield.cpp`, `cpp/src/compass.cpp`, `cpp/src/hpa.cpp`; held equal by `docs/engine/check_records.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_shield_parity.py`, `tests/test_cpp_hpa_parity.py`.
- **Moves.** The shipped controller's record played decision by decision on the live benchmark engine's clock; a ring opens at each head with its newest value.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The bill the cloud meters: machine-hours and the bill at list price, native's bar beside Omni-Compass's, each finished set with its word; the running set's repetitions measured so far; each provider's hourly gate with the reason its own job wrote.
- **What to do with it.** The bill aims lower: a fall is a GAIN, inside the noise STEADY, a rise power given. A provider whose gate holds runs nothing and bills nothing; its frame stays empty until its first set lands.

## L.24 Orchestrators and Governors

**Controls.** Recorded data: each governor's place as the repository holds it, its runs read from GitHub at the build and played here the moment they land; click a governor (a tile, a row of the road or of the catalog) to hold it.

**The page, in its own words.**

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Each governor and orchestrator runs four ways in every repetition: the organism alone, the orchestrator as shipped, Omni-Compass on the organism alone, and Omni-Compass on top of the orchestrator; only the last two, each against its own native, are readings of Omni-Compass, and the other comparisons are the orchestrator's own cost. Each place fills itself from the repository as its runs land; until then it shows what the repository holds, and nothing more.

**Road to a Result**

**Orchestration Stack**

Where each governor sits in the stack. Omni-Compass reads and writes on top of it once its wire is in the repository. The governor runs native as shipped beneath it, and Kubernetes places the work on the nodes. The work dots move at the live benchmark engine's demand (section L.12): a picture of the flow, not a measurement. Each governor's readings replace this picture the moment its three runs land.

**Governor and Orchestrator Catalog**

**Four-Way Comparison**

**Orchestra Organism · Class S**

**Its monitors.**

### *L.24.1 Monitor 42: Road to a Result*

Every scheduler, queue and multi-cluster governor on its way through trial, A, B and C, on the engine that stands.

- **Evidence and state.** Class L; record.
- **Feed.** GitHub Actions API (read at the build); live: github-actions-exporter.
- **Wired.** This page's `drawOrchRoad` → the Python `tools/governors_page.py`, `tools/confirm_abc.py`, `docs/engine/build_live_engine.py` → the C++ none: the governors are live software on Kubernetes, run as their projects ship them; held equal by `tests/test_governors_page.py`.
- **Moves.** A dot travels each governor's road, a running run pulses, and the reading cursor walks the governors, one a decision.
- **Knobs.** Governor: which orchestrator or governor is shown.
- **How to read it.** Each scheduler, queue and multi-cluster governor four ways: the organism alone, the orchestrator as shipped, Omni-Compass on the organism alone, Omni-Compass on the orchestrator.
- **What to do with it.** Only two of the four comparisons are Omni-Compass against its native (on top of the orchestrator; on the organism alone); the others are the orchestrator's own cost and are never counted as Omni-Compass's.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

### *L.24.2 Monitor 24: Orchestration Stack*

Where the chosen governor sits in the stack: Omni-Compass on top, the governor running native as shipped, Kubernetes and its nodes beneath, the work moving down at the live benchmark engine's demand; a picture of the flow, not a measurement.

- **Evidence and state.** Class L; record.
- **Feed.** kube-state-metrics; each orchestrator's own metrics endpoint (Kueue, Volcano, KAI, Koordinator, Crane, KubeRay); series `kube_pod_status_phase{phase="Pending"}`, `kueue_pending_workloads`; on a running system: `sum(kube_pod_status_phase{phase="Pending"})`.
- **Wired.** This page's `drawOrch` → the Python `tools/governors_page.py`, `tools/run_orchestra.py` → the C++ none: the governors are live software on Kubernetes, run as their projects ship them; held equal by `tests/test_governors_page.py`, `tests/test_orchestra.py`.
- **Moves.** Work flows down the stack at the live benchmark engine's demand.
- **Knobs.** Governor: which orchestrator or governor is shown; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** Each scheduler, queue and multi-cluster governor four ways: the organism alone, the orchestrator as shipped, Omni-Compass on the organism alone, Omni-Compass on the orchestrator.
- **What to do with it.** Only two of the four comparisons are Omni-Compass against its native (on top of the orchestrator; on the organism alone); the others are the orchestrator's own cost and are never counted as Omni-Compass's.

## L.25 GPU and CPU Hardware

Each stack is its own model in the repository, run exactly as the repository runs it and recorded second by second by docs/engine/export_scenarios.py (class S, our own models).

**Controls.** Recorded data: GitHub's chip runs, then the chip models replayed from the repository's own run (class S); pick a setting in the table below.

**The page, in its own words.**

**Metered Hardware Run Slots**

The GPU runs on Lambda (class P: the card's own meter) and the CPU on its own machine (class P: the package's own energy counter, tools/run_cpu_power.py) are the chip's own receipts, read only from their three-run tables: The GPU runs on the engine that stands are counted from GitHub's run records. No CPU package has been measured on real hardware. A place is played back from its raw files like every Kubernetes run once its run is exported for the replay; beside them, the views below other than the metered card run (class

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

P) are the repository's own models of one GPU card and one Linux host (class S), run native and then with Omni-Compass on the same load and seed.

**Metered A100 Run**

The one card run replayed here sample by sample from the card's own meter (class P), native beside Omni-Compass: an NVIDIA A100 (SXM4, 40 GB) on Lambda on 24 October (workflow run 40079645681, omni-v18's card governor at commit e3b05dc), ten paired repetitions of 600 s an arm, native (the card as shipped), Omni-Compass watching only (the switch OFF: it reads and writes nothing) and Omni-Compass governing the power limit and the clock ceiling, every sample of nvidia-smi (200 ms) and every decision of the governor's own audit (results/gpu/run-20261024T091235Z, docs/engine/export_gpu_run.py). On this card native already runs both knobs at their top (the power limit 400 W of 400 W, the clock unlocked to 1,410 MHz), so the factory tune's governor asked for exactly that in its 3,050 decisions and wrote nothing: at its limit on native. The runs on the engine that stands and their three-run tables are counted in the hardware run slots above and listed run by run below, and the 8-card run has not been run (its one launch was a priced dry run). The A10 run of 2 October (the card controller of commit c908054, replaced since) stays as its own record in results/gpu/run-20261002T082232Z. Its own table, re-read by the one key:

**CPU Host Model · Class S**

The CPU power harness (tools/run_cpu_power.py) puts Omni-Compass on top of the Linux kernel's own frequency governor and meters the processor's package energy (RAPL). No GitHub runner exposes that counter or a writable ceiling, so its real run waits for a Linux machine that does (the register's row 18, cpu-power.yml). Until then this monitor plays the harness on its own modelled machine (--plant sim, written for its tests; results/sim/cpu_power_model, three repetitions): the mechanism moving, every decision and write of its own audit, never a measurement. The machine layer enters the whole organism as exactly 1 until the real run lands.

**8-Card GPU Node · Class S**

The 8-card result is one command on a rented machine with eight NVIDIA cards (scripts/gpu_8card.sh, `docs/GPU_RUN_GUIDE.md` section D): every card runs the whole card test at once, its firmware alone against its firmware with Omni-Compass on top, 24 paired repetitions a test pooled across the cards. It has not run. The monitor is built for it: eight card slots, each native lane drawn from the repository's one-card model (class S: AI token generation at the card's shipped 150 W, the same model on every card until the run, never eight measurements), each Omni-Compass lane empty until the run lands. On a running node the same panel reads the NVIDIA DCGM exporter's series card by card.

**Chip Run Ledger**

**Chip Model Settings · Class S**

Read from the replayed record itself, second by second, in each of the three runs (seeds as below). A row with no second held off native is Omni-Compass deciding to stay native: its trial of one step read LOSS and was refused, or the knob is already at its limit on native.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Native**

**Omni-Compass**

**Omni-Compass Audit Trail · Class S**

**Three-Run Score · Class S**

**Its monitors.**

### *L.25.1 Monitor 43: Metered A100 Run*

The NVIDIA A100 (SXM4, 40 GB) run of 24 October on Lambda (omni-v18's card governor, workflow run 40079645681, ten paired repetitions) replayed from the card's own meter: draw, limit, clock and temperature, requests served and the 95th percentile, native beside Omni-Compass, every write of the governor marked. Every service gauge read STEADY: native already ran both knobs at their top, so the governor wrote nothing (at its limit on native).

- **Evidence and state.** Class P; awaiting its run: the stacks with the card inside on omni-v21, A, B and C on Lambda's A10, finished on 28 October (`docs/RUN_LEDGER.md`): their files are on the raw branch and their three-run table is not yet written; the A10 run of 2 October is obsolete and stays as history.
- **Feed.** NVIDIA DCGM exporter; series `DCGM_FI_DEV_POWER_USAGE`, `DCGM_FI_DEV_POWER_MGMT_LIMIT`, `DCGM_FI_DEV_SM_CLOCK`, `DCGM_FI_DEV_GPU_TEMP`; on a running system: `DCGM_FI_DEV_POWER_USAGE`; `DCGM_FI_DEV_POWER_MGMT_LIMIT`; `DCGM_FI_DEV_SM_CLOCK`.
- **Wired.** This page's `drawGpuRun` → the Python `docs/engine/export_gpu_run.py` → the C++ none: omni-v18's card governor, replayed from the card's own meter; today's GPU rules have their twin, cpp/src/gpu_rules.cpp, for the omni-v21 runs on Lambda; held equal by `docs/engine/build_live_engine.py`.
- **Moves.** The card's own meter replayed at ten times the run's seconds.
- **Knobs.** Repetition: which repetition of the run is replayed; Arm: which arm stands on the right: Omni-Compass governing, or watching only; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The card's own meter, second by second: its draw, the power limit Omni-Compass wrote (dashed), its clock, the 95th percentile against the response line, its temperature; every write of the governor a red tick.
- **What to do with it.** Power on a card is a resource: a fall is a cut, a rise power given; the response time is the judge. The one real run on the page is the replaced controller's record and is shown as such until its omni-v23 run lands.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

### *L.25.2 Monitor 44: CPU Host Model · Class S*

The CPU power harness on the host: the frequency ceiling, the clock run at, the package power and the response line, native beside Omni-Compass; today its own modelled machine (class S), the real run the day a machine with its meter runs it.

- **Evidence and state.** Class S (model) until the run; awaiting its run: a Linux machine whose processor exposes its energy counter (RAPL) and a writable frequency ceiling (register row 79, cpu-power.yml).
- **Feed.** node_exporter (rapl, cpufreq collectors); Kepler; series `node_rapl_package_joules_total`, `node_cpu_scaling_frequency_hertz`, `node_cpu_scaling_frequency_max_hertz`, `kepler_node_package_joules_total`; on a running system: `rate(node_rapl_package_joules_total[1m])`; `avg(node_cpu_scaling_frequency_hertz)`.
- **Wired.** This page's `drawCpuHost` → the Python `docs/engine/export_cpu_model.py`, `tools/run_cpu_power.py` → the C++ none: the host's model until its run lands on a machine with its meter; held equal by `tests/test_run_cpu_power.py`.
- **Moves.** The host's model replayed second by second until its run lands.
- **Knobs.** Repetition: which repetition of the run is replayed; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The processor's frequency ceiling (the knob), the clock the kernel's own governor ran at under it, the package power and the busy share, native beside Omni-Compass.
- **What to do with it.** Until a machine with its energy counter runs it, the monitor plays the harness's own modelled machine: read the mechanism, never the numbers.

### *L.25.3 Monitor 45: 8-Card GPU Node · Class S*

Eight card slots for scripts/gpu_8card.sh: each native lane the repository's one-card model (class S), each Omni-Compass lane empty until the 8-card run lands, 24 paired repetitions a test pooled across the cards.

- **Evidence and state.** Class S (native model) until the run; awaiting its run: scripts/gpu_8card.sh on a rented machine with eight NVIDIA cards (`docs/GPU_RUN_GUIDE.md`, section D).
- **Feed.** NVIDIA DCGM exporter, card by card; series `DCGM_FI_DEV_POWER_USAGE`, `DCGM_FI_DEV_POWER_MGMT_LIMIT`, `DCGM_FI_DEV_SM_CLOCK`; on a running system: `sum by (gpu) (DCGM_FI_DEV_POWER_USAGE)`.
- **Wired.** This page's `drawGpu8` → the Python `realms/gpu_card.py`, `omni_controller/gpu_governor.py`, `scripts/gpu_8card.sh` → the C++ `cpp/src/gpu_rules.cpp`; held equal by `docs/engine/check_records.py`, `tests/test_cpp_twins_parity.py`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Moves.** The one-card model's native lanes replayed second by second; the Omni-Compass lanes wait for the 8-card run.
- **Knobs.** Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The card's own meter, second by second: its draw, the power limit Omni-Compass wrote (dashed), its clock, the 95th percentile against the response line, its temperature; every write of the governor a red tick.
- **What to do with it.** Power on a card is a resource: a fall is a cut, a rise power given; the response time is the judge. The one real run on the page is the replaced controller's record and is shown as such until its omni-v23 run lands.

### *L.25.4 Monitor 46: Stack Models · Class S*

The repository's own models of one GPU card and one Linux host, native beside Omni-Compass, second by second (class S).

- **Evidence and state.** Class S; model.
- **Feed.** NVIDIA DCGM exporter; node_exporter; series `DCGM_FI_DEV_POWER_USAGE`, `DCGM_FI_DEV_SM_CLOCK`, `node_cpu_scaling_frequency_hertz`; on a running system: `DCGM_FI_DEV_POWER_USAGE`; `DCGM_FI_DEV_SM_CLOCK`.
- **Wired.** This page's `drawSStrip` → the Python `realms/gpu_card.py`, `omnicompass/compass_law.py`, `tools/model_gate.py`, `omni_controller/muscles.py`, `omnicompass/conveyance.py`, `docs/engine/export_scenarios.py` → the C++ `cpp/src/conveyance.cpp`, `cpp/tools/engine_runs.cpp`; held equal by `docs/engine/check_records.py`, `tests/test_cpp_conveyance_parity.py`, `verify.py`.
- **Moves.** The chip's and the host's models replayed second by second; a ring opens at each head with its newest value.
- **Knobs.** Stack: the GPU card, the Linux host, Kubernetes or the cloud; Run: A, B or C (or the archived run) the monitor plays; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The card's own meter, second by second: its draw, the power limit Omni-Compass wrote (dashed), its clock, the 95th percentile against the response line, its temperature; every write of the governor a red tick.
- **What to do with it.** Power on a card is a resource: a fall is a cut, a rise power given; the response time is the judge. The one real run on the page is the replaced controller's record and is shown as such until its omni-v23 run lands.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# L.26 CI Pipeline

**Controls.** Recorded data: GitHub's runs as read when the page was built.

**The page, in its own words.**

**GitHub Actions in Flight**

Read at the build from GitHub's own API: a workflow's running runs in solid blue, its waiting ones pale; the Azure set to the paid cap with its repetitions measured so far (the table is written when the set ends); each cloud's hourly gate with the reason its own job wrote. Live, the same lives in the open-source github-actions-exporter for Prometheus.

**Running and Queued Workflows**

The page is rebuilt by GitHub itself (.github/workflows/live-engine.yml, docs/engine/build_live_engine.py) whenever the engine, a model or a result changes, and every three hours. Every record is made again by the repository's own code on the engine that stands, checked against the Python and the C++, and committed. This table is GitHub's own list at that moment.

**Its monitors.**

## *L.26.1 Monitor 59: GitHub Actions in Flight*

Every workflow run in progress or queued when the page was built, the Azure set's repetitions done of its cap, and the clouds' gates.

- **Evidence and state.** Class L (GitHub's own API, read at the build); record.
- **Feed.** GitHub Actions API (read at the build); live: github-actions-exporter; series `github_workflow_run_status`, `github_job_duration_seconds`; on a running system: `sum by (workflow) (github_workflow_run_status == 1)`.
- **Wired.** This page's `drawCi` → the Python `docs/engine/build_live_engine.py` → the C++ none: GitHub's own record of its runs, read from its API at the build; held equal by `docs/engine/build_live_engine.py`.
- **Moves.** The reading cursor walks the workflows, one a decision; the age of GitHub's reading counts up every second.
- **Knobs.** Filter: which rows, words, families or organisms are shown.
- **How to read it.** GitHub's runs in flight at the build by workflow, the Azure set's repetitions measured, each cloud's gate with its reason, the chips' runs.
- **What to do with it.** A gate that holds names its fix in its own words; nothing is billed while it holds.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## L.27 Part 5 of the page: Proof and Models

## L.28 Python Runs and Monte Carlo

**Controls.** Live: the runs start on their own when the page opens; the buttons run them again, run a new set from any base, print them and save the receipts.

**The page, in its own words.**

**Seeded Engine Runs and Replayed Scenarios**

Three-Way Run Check. tools/engine_runs.py runs N runs from a base seed: run k's seed is base + k·0x9E3779B9 mod $2^{32}$ (the jump generator), its stream mulberry32 (this page's own generator, written in the Python bit for bit), through the engine that stands: 288 decisions of the brain's state equations integrated by the fourth-order Runge–Kutta step, 80 steps of the compass law and one give-back trial judged by the verdict. Each run's receipt is its final state, its pushes and grants, the law's mean and last force and the trial's answer; the digest is the SHA-256 of every receipt at six decimals. The Python writes every draw of every run, the injection, and the C++ harness (cpp/tools/engine_runs.cpp) reads it as its input, puts the mechanism on through the engine's C++ twins and writes every receipt: its digest must be the Python's. This page runs the same N with its own port of the engine (proven equal to the Python and the C++ to 1e-12 on every build by docs/engine/check_live_engine.py) as it opens and compares every receipt and the digest. GitHub does all three on every set it runs (.github/workflows/engine-runs.yml) and on every push (verify.py); run the Python again with another base and it writes another set, and the three digests must be one.

**Engine Core Monte Carlo**

**Custom System Run**

A system is your own stack written as ranges of the engine's core: a name, and for any of the six starting states (E, energy debt; U, the order parameter, its basins at −1 and +1; I[U], the integral of need; S, stress; B, the bath; Ḃ, its rate; the block below names each by its key) and any of the 22 parameters of omnicompass/core.py a range [low, high] or one number; the rest keep the engine's own. Among the parameters: beta_int and beta_ext, the demand into E from inside and outside; E[max], the headroom; c, ω[B], Q[B] and $\lambda_0$ to $\lambda_2$, the bath's swing; mu, the depth of the double well. The draws keep their one order, so the jump generator's streams are the same; only the ranges they spread over change. The block is read as numbers only, never run as code (`systems/README.md`).

On GitHub (the modelled runs done within GitHub, on its own machines, at no cost): the button checks the block, copies it and opens the monte-carlo workflow. There, press Run workflow, paste the block into system, set runs (500) and base (empty for a new one) and press Run workflow again. The Python (tools/core_runs.py) draws the runs, the C++ twin of the core (cpp/tools/core_runs.cpp on cpp/src/core.cpp) runs the same draws, this page's own port is run on them in node, and the three digests must be one; the record, with the run's link, is committed to results/core_runs/ and joins this monitor's table at the page's next build. A system file committed

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

under systems/ runs there on every push that changes it. GitHub offers Run workflow for a workflow on the repository's main branch, where this one is (.github/workflows/monte-carlo.yml).

Class S: the engine's own mathematics on your ranges, never a measurement of your stack. Wiring Omni-Compass onto the stack itself is the plug of the Operator's Manual.

Every line is one run of omnicompass/core.py: six states (E, U, I[U], S, B, Ḃ), the fourth-order Runge–Kutta step 10 times a macro step (Δt 0.01 s), 20 macro steps, from the starting state and parameters the run drew; the line is its U at step 0 and at the end of every macro step. The left lane is native: the same plant with no command (u = 0), left to its own double well, μU(1 − U²). The right lane is Omni-Compass on top: the bounded feedforward-plus-proportional command (u = −f[U] + 12(σ − U), clipped to ±25 and held across the four stages) toward the nearest basin, or, by the knob, toward the far one, or with the double well switched off (μ = 0). The green bands are the basins, |U ∓ 1| ≤ 0.10. A run is conveyed when it has stayed in one basin for 5 macro steps in a row, certified at 10; the bars under each lane count the macro step each run was conveyed (its colour) and certified (green), and the counters read at the sweeping cursor. The set the page opens on is the Python's own 500 (benchmarks/core_evidence.py, numpy's generator from seed 223387268), whose counts must be its record's (results/core_evidence.json); a set run here from the jump generator is computed by this page's own port of the core, line for line, and its digest must be the one tools/core_runs.py writes for the same base. Class S: the engine's own mathematics, never a measurement.

**Its monitors.**

### *L.28.1 Monitor 50: Three-Way Run Check*

The engine's seeded runs: the Python's receipts (results/engine_runs) beside the same runs computed by this page's own engine, run by run, with one digest each.

- **Evidence and state.** Class S (the engine itself); live.
- **Feed.** tools/engine_runs.py (the Python) and this page.
- **Wired.** This page's `drawProof, engineRuns, proofRecheck` → the Python `tools/engine_runs.py` → the C++ `cpp/tools/engine_runs.cpp`, `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `verify.py`, `tests/test_engine_runs.py`, `.github/workflows/engine-runs.yml`.
- **Moves.** The page recomputes one run of the Python's set from its own seed about twice a second, round after round, and holds its receipt against the Python's.
- **Knobs.** Runs: how many of the Python's runs this page computes again; Print: the receipts of the runs, printable, with the digests and the notice.
- **How to read it.** The Python's seeded runs beside this page's: one cell a run (green agrees to 1e-9, red differs, blue a new set), the two digests under them; and the Python's 500 scenarios a seed replayed with every trace hash.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **What to do with it.** Press the button to run the Python's set here; run a new base to make a new set and run the printed command in the Python to compare digests. Print or download the receipts for the file.

### *L.28.2 Monitor 51: Held-Out Scenario Replay*

benchmarks/stack_benchmark.py's 500 scenarios a seed base (the two held-out seeds), each through every arm, replayed from its record scenario by scenario with every trace hash and the file's SHA-256; verify.py replays them on every push.

- **Evidence and state.** Class S; replay.
- **Feed.** benchmarks/stack_benchmark.py (results/heldout_seed_*/RUNS.csv).
- **Wired.** This page's `drawHeld` → the Python `benchmarks/stack_benchmark.py`, `omnicompass/adapter.py`, `omnicompass/shield.py`, `omnicompass/stack_sim.py`, `docs/engine/export_heldout.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/shield.cpp`; held equal by `verify.py`, `tests/test_cpp_governor_parity.py`, `tests/test_cpp_shield_parity.py`.
- **Moves.** The 500 scenarios replayed one after another from the record, every scenario's trace hash with it.
- **Knobs.** Seed Base: which held-out seed base's 500 scenarios are replayed; Arm: which arm stands on the right: Omni-Compass governing, or watching only; Gauge: which gauge of the run is shown; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The Python's seeded runs beside this page's: one cell a run (green agrees to 1e-9, red differs, blue a new set), the two digests under them; and the Python's 500 scenarios a seed replayed with every trace hash.
- **What to do with it.** Press the button to run the Python's set here; run a new base to make a new set and run the printed command in the Python to compare digests. Print or download the receipts for the file.

### *L.28.3 Monitor 52: Engine Core Monte Carlo*

The engine's core (omnicompass/core.py) over 500 runs, each from step 0: the fourth-order Runge–Kutta step, 20 macro steps of 10 micro steps, native (the plant alone, u = 0) beside Omni-Compass on top (the bounded command toward the nearest basin, the far basin, or with the double well off), every run's U, the basins at ±1, and the macro step each run was conveyed (5 in a row) and certified (10 in a row); the Python's own 500 (benchmarks/core_evidence.py, seed 223387268) and every set the jump generator draws here, each held to tools/core_runs.py's digest; the linked run holds the Python's digest, the C++ twin's on the same draws and this page's beside each other, and a system of your own runs here or on GitHub.

- **Evidence and state.** Class S (the engine itself); live.
- **Feed.** benchmarks/core_evidence.py and tools/core_runs.py (the Python) and this page's port of the core.
- **Wired.** This page's `drawMC, mcRun, mcLinkedRun, mcSysParse, coreSim, coreRk4, jgTick` → the Python `omnicompass/core.py`, `benchmarks/core_evidence.py`,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`tools/core_runs.py`, `systems/README.md`, `docs/engine/build_live_engine.py` → the C++ `cpp/tools/core_runs.cpp`, `cpp/src/core.cpp`; held equal by `tests/test_core_runs.py`, `tests/test_cpp_core_runs_parity.py`, `verify.py`, `.github/workflows/monte-carlo.yml`, `.github/workflows/engine-runs.yml`.

- **Moves.** The cursor sweeps macro step 0 to 20 in 10 s, every run's line drawn up to it, the counters and the bars read at it; with the jump generator ON a new set of 500 arrives set after set.
- **Knobs.** Set: the Python's own 500 (numpy, seed 223387268) or a set the jump generator drew on this page; Omni-Compass Way: the bounded command toward the nearest basin, toward the far basin, or with the double well off (μ = 0); native is the plant alone (u = 0); Custom System: your stack written as ranges of the core's six states and 22 parameters, run in the page or on GitHub through the Python and the C++ (`systems/README.md`); Play / Pause: the cursor's sweep of the 20 macro steps, held or running; Runs: how many of the Python's runs this page computes again; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The Python's seeded runs beside this page's: one cell a run (green agrees to 1e-9, red differs, blue a new set), the two digests under them; and the Python's 500 scenarios a seed replayed with every trace hash.
- **What to do with it.** Press the button to run the Python's set here; run a new base to make a new set and run the printed command in the Python to compare digests. Print or download the receipts for the file.

## L.29 C++ Twin

**The page, in its own words.**

**Twin Parity Checks**

**C++ Engine Run Receipts**

The Python's seeded runs (tools/engine_runs.py: the jump generator's seeds, 288 decisions of the brain, 80 steps of the compass law and one give-back trial a run) are the input: the Python writes every run's draws as the injection, the C++ harness (cpp/tools/engine_runs.cpp) reads it, puts the mechanism on through the C++ twins (the governor, the nervous system, the compass law, the verdict) and writes its own receipts and digest, the output. The proof monitor (section L.28, Seeded Engine Runs and Replayed Scenarios) shows the same column run by run.

**Compass Law Parity on Page Runs**

**Organism Law Parity at the Tune-Down**

**Python-Only Components**

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## L.30 Scenarios and Injection Grid

**Controls.** Recorded data: click an organism and an arm, or hover a cell, for its gauges and its readings. In the injection grid: the organism, the way (Omni-Compass on native, a governor on the organism, Omni-Compass on top of a governor), what it is read against, the runs (1, 10, 100, 500), the organisms stacked (1, 10, 100, 500), the gauge and the injection; exports, an import and a reference to any cell. Both grids also stand in the control panel (section L.2), and the Variation Switchboard's runs, copies and organism drive the injection grid.

**The page, in its own words.**

Twenty cells, each read by the key against native, from the record results/scenarios/SCENARIOS_20.json: cells 1 to 9 our own models (class S), three runs A, B and C on the engine named above the grid, each that engine's own, never read as the engine that stands's until its rerun lands; cell 10 a composition by our own code of measured tables across engines (class S of class L and P measurements), its engines named in the cell, never three runs. Each cell shows every judged gauge, power given beside the work and the time over the line it bought. A cell with no run reads "not yet run", never a number.

**Twenty Scenarios**

**Injection Grid**

Class S, our own models (tools/injection_grid.py, results/injection_grid/).

The governors themselves, as their projects ship them, run on real Kubernetes in section L.24 (class L): fourteen named, twelve with a rig built, eleven run the four ways (KubeEdge and Nephio are modelled only). A LOSS counts against Omni-Compass only on a way with Omni-Compass on top (on native, or on top of a governor); a governor on the organism read against native is the governor's own reading, shown as the governor's. One batch is one run in the key's sense; three batches make a three-run reading.

The injection grid draws as the page opens.

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

## L.31 Whole Organism

The composition is read four ways (the Scenario knob): the two ways a stack is bought, and two more that take its pieces alone, in the device (the bare metal by itself) and as a service (the software by itself). The composition is a model (evidence class S) of class P and class L measurements under the separability assumption; no run has yet stacked every layer at once, and that stacked run is the measurement that would replace it. The chip's newest finished run (the A100 of 24 October on Lambda, omni-v18's card governor, ten paired repetitions: every service gauge STEADY, the governor writing nothing because native already ran both knobs at their top) stands beside the count by default, every row of it shown; one click puts it in, until a three-run table from the stacks with the card inside on omni-v21 replaces it (their runs on Lambda finished; their files are on the raw branch, and no table has been written from them yet). The A10 run of 2 October is

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

obsolete. The machine (the kernel, bare metal, Kepler) is not yet run and enters as exactly 1, named. GAIN is green, LOSS red, STEADY yellow, power given blue, on every word, cell and tile.

**The page, in its own words.**

Composed Reading. —

CLOUD BACK-END METERING DISCLOSURE. Through the cloud, the bare metal at the back end belongs to the provider: no tenant can read its power meter or write its chips' limits, so no run here reaches it. What a tenant can measure is what the cloud meters and bills: the machines it counts every fifteen seconds and the bill at list price (section L.23); that difference, native against Omni-Compass on top of the cloud's own autoscaler, is the cloud piece's benefit, and it is the provider's own health on its own hardware too, fewer machines held for the same work. The bare-metal piece is gauged on hardware we can meter (the card's own meter, the CPU host's energy counter, section L.25) and claimed for the provider's bare metal only as the same mechanism on the same kind of part: the composition multiplies it in as a modelled factor (class S of class P), never as a measurement of the provider's machines. Four ways are composed: on-site bare metal (the owner's own racks and chips: software × bare metal); through the cloud (the cloud's build, its bare metal at the back end and the software; and the cloud as two pieces, the back end left to the provider); in the device (Omni-Compass built into the bare metal alone, the card and the machine, nothing else wired: for an owner who ships hardware); as a service (the software alone, run for an owner with neither bare metal of their own nor a cloud account: the transport of the software only). This whole stack is a composition with two arms, native and with Omni-Compass on top of every piece; it is not counted among the organisms (the seven, and the orchestra the eighth). Each piece is its own newest measurement, on the engine its record names (the cloud piece Azure's three one-run tests, two on omni-v1 and one on the engine of commit 5b2832f of 4 October, the chip piece the A100 run of 24 October under omni-v18's card governor, the software pieces their newest three-run tables): the composition is across those engines, names each, and is remade as each piece's run on the engine that stands lands.

**Stack Composition**

**Composed Benefit by Gauge**

**Composition by Piece and Layer**

**Bills and Receipts**

**Tables Outside the Count**

**Its monitors.**

## *L.31.1 Monitor 2: Stack Composition*

The pieces of a stack, native beside Omni-Compass, each piece's composed factors and the whole, their product, in every way a stack is bought.

- **Evidence and state.** Class S (of P and L); composed.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).
- **Wired.** This page's `drawWhole` → the Python `tools/whole_system.py`, `docs/engine/export_whole.py` → the C++ none: a composition of measured tables by the product law, read as tools/whole_system.py writes it; held equal by `docs/engine/build_live_engine.py`.
- **Moves.** Work flows down both columns; the reading cursor walks the pieces, one a decision.
- **Knobs.** Scenario: on-site bare metal, through the cloud, in the device, as a service; Reading: every measured piece carried, or the index's strict rule; Chip Run: the chip's one run beside the count or in it; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** Bars and tiles in the traffic light of the one key: GAIN green to one side, LOSS red to the other, STEADY yellow on zero, power given blue. A figure is a composition or a table's own word, never a single run's number.
- **What to do with it.** Read the word before the size, and the interval before the word: an interval across zero is STEADY whatever its centre. A LOSS on the board is answered on the status board, with its cause and its knob.

## L.32 Measured Effect

**Controls.** Recorded data: every receipt as the repository holds it. Console controls never modify it.

**The page, in its own words.**

**Seven Organisms Under Three Loads · Class S**

**Catalog Muscle Matrix · Class S**

Click a cell to read the muscle: its name, family, realm, template, knob and its word under each load.

**Open Register Slots**

**Scale-Out Grid**

**Misses · Class S**

**Cluster 95th Percentile · Class L**

Each square runs the real cluster inside the model.

**Standing Three-Run Tables**

From each benchmark's newest engine's table (results/live/V<engine>_*.md, the engine named in each row), read row by row by docs/engine/export_scenarios.py; open a row to read every judged row that is not STEADY. Class L: the real software (Kubernetes on kind, the real databases and broker). Class S: the outside simulators run by our own runner (CityLearn, pandapower on SimBench, MuJoCo Menagerie,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

gym-pybullet-drones). Rows shown and not judged are left out of the counts; the counts take every table of a receipt, its tuning case included where it has one (a receipt's own summary line counts its untouched cases). Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

**Realms Table · Class S**

The eighth organism, the orchestra (every governor's catalog muscles, each governor's a body of its own and all of them one body), has its own runner and its own table: section L.24, under the governors.

**Families by Stack**

From results/realms_v5/REALMS_V5.json, the realms runner at the max, its engine and commit named above (tools/run_realms_v5.py, catalog realms/catalog_v5.csv), ten paired seeds per muscle and per organism, the lease after 80% of every run. Misses: the plant decisions over their line, summed over the ten seeds. A muscle reads GAIN when one of its aims (more work, faster, fewer misses) moved beyond the noise under some load with none moving the other way; STEADY at its limit on native when its knob already sits at the top of its range. Energy is power given where it rose. Evidence class S, our own models: a model's energy is not a meter's, and a plant written by the same people who wrote the law is not an independent test. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

**Its monitors.**

### *L.32.1 Monitor 47: Seven Organisms Under Three Loads · Class S*

Each organism's misses against native under its own load, the climb and the line (the realms table).

- **Evidence and state.** Class S; record.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).
- **Wired.** This page's `drawOrgCv` → the Python `tools/run_realms_v5.py`, `realms/harness.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `tests/test_realms.py`, `tests/test_cpp_governor_parity.py`.
- **Moves.** The reading cursor walks every organism under every load, one a decision.
- **Knobs.** Filter: which rows, words, families or organisms are shown.
- **How to read it.** Each organism or grid square read by the one key: green GAIN, yellow STEADY, red LOSS, grey a single run, dashed not yet run.
- **What to do with it.** A dashed square is pending runner capacity, never rented. A LOSS square names its organism, its size and its runs.

### *L.32.2 Monitor 48: Catalog Muscle Matrix · Class S*

The 1,107 muscles as one matrix, each cell its word against native, by family and realm; a click names the muscle.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Evidence and state.** Class S; record.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).
- **Wired.** This page's `drawMuscleCv` → the Python `tools/run_realms_v5.py`, `realms/harness.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `tests/test_realms.py`, `tests/test_cpp_governor_parity.py`.
- **Moves.** The reading cursor walks the 1,107 muscles, one a decision; a click holds one.
- **Knobs.** Filter: which rows, words, families or organisms are shown.
- **How to read it.** Every muscle of the catalog one cell, its word against native on its own load: green GAIN, yellow STEADY (pale: at its limit on native), red LOSS.
- **What to do with it.** Click a cell to read the muscle: its family, realm, template, knob and its word under each load.

### *L.32.3 Monitor 49: Scale-Out Grid*

Seven organisms × copies × runs: every square's word, modelled and with the real cluster inside.

- **Evidence and state.** Class S/L; record.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).
- **Wired.** This page's `drawGridCv` → the Python `tools/grid_v5.py`, `scripts/dispatch_grid.py`, `docs/engine/build_live_engine.py` → the C++ none: the grid's squares as its runs on GitHub write them; held equal by `tests/test_grid_v5.py`.
- **Moves.** The reading cursor walks every square with runs started, one a decision; the age of GitHub's reading counts up every second.
- **Knobs.** Filter: which rows, words, families or organisms are shown.
- **How to read it.** Each organism or grid square read by the one key: green GAIN, yellow STEADY, red LOSS, grey a single run, dashed not yet run.
- **What to do with it.** A dashed square is pending runner capacity, never rented. A LOSS square names its organism, its size and its runs.

## L.33 Telemetry Analytics

The engine's own record is a tensor: two arms × every decision × sixty muscles, each entry a muscle's compass position (0 calm, 0.5 the center of its band, 1 its line), recorded from realms/harness.py by docs/engine/export_harness_trace.py (class S, our own models). The service level, error budget and gauge panels read the official runs (class L); every GAIN, STEADY, LOSS and power given in this section is a run's own reading.

**Controls.** Recorded data: the engine's harness record (class S), played on the live benchmark engine's clock, and the official run chosen in section L.20 (class L). Console controls never modify either.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

**The page, in its own words.**

**Omni-Compass Narration**

Every line is read from Omni-Compass's own records at the moment on screen (the official run playing in section L.20, its governor's own audit, and the engine's harness on the console's clock) and says what Omni-Compass is doing, beside what native does. Speak aloud reads the lines out on the viewer's own browser voice as the run plays, and reads Omni-Compass's answers below too.

**Telemetry Tensor Cube · Class S**

**Tensor Difference Matrix · Class S**

**Tensor Principal Modes · Class S**

**Service Level and Error Budget**

**Repetition Gauges Against Native**

**Ask Omni-Compass**

Connecting…

Omni-Compass's answers are drawn from this page's records and are not reviewed by any outside party. © 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and `DISCLOSURES.md`. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

**Its monitors.**

### *L.33.1 Monitor 53: Telemetry Tensor Cube · Class S*

The engine's harness record as a tensor (decisions × sixty muscles × two arms), native and Omni-Compass as two slabs, filling in on the live benchmark engine's clock.

- **Evidence and state.** Class S (the harness's record, realms/harness.py); replay.
- **Feed.** kube-state-metrics; the service's own metrics.
- **Wired.** This page's `drawCube` → the Python `realms/harness.py`, `realms/compass_arm_v5.py`, `docs/engine/export_harness_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `docs/engine/check_records.py`.
- **Moves.** Fills in decision by decision on the live benchmark engine's clock, and turns.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.

- **How to read it.** The organism's telemetry as a tensor (muscle by gauge by time), its difference between the arms, its principal modes, the service level against its objective and the error budget burned, native beside Omni-Compass.
- **What to do with it.** The service level and the budget are what an on-call engineer is paged on: watch the burn rate on both arms under the same knobs.

### *L.33.2 Monitor 54: Tensor Difference Matrix · Class S*

The difference of the two sides' tensors, cell by cell.

- **Evidence and state.** Class S (the harness's record, realms/harness.py); replay.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).
- **Wired.** This page's `drawDelta` → the Python `realms/harness.py`, `realms/compass_arm_v5.py`, `docs/engine/export_harness_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `docs/engine/check_records.py`.
- **Moves.** One column a decision on the live benchmark engine's clock.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.
- **How to read it.** The organism's telemetry as a tensor (muscle by gauge by time), its difference between the arms, its principal modes, the service level against its objective and the error budget burned, native beside Omni-Compass.
- **What to do with it.** The service level and the budget are what an on-call engineer is paged on: watch the burn rate on both arms under the same knobs.

### *L.33.3 Monitor 55: Tensor Principal Modes · Class S*

The principal modes of the telemetry (its matrix decomposed).

- **Evidence and state.** Class S (the harness's record, realms/harness.py); replay.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Wired.** This page's `drawModes, tEig` → the Python `realms/harness.py`, `realms/compass_arm_v5.py`, `docs/engine/export_harness_trace.py` → the C++ `cpp/src/governor.cpp`, `cpp/src/core.cpp`, `cpp/src/nervous_system.cpp`; held equal by `docs/engine/check_records.py`.
- **Moves.** Every decision: the line across each bar is that decision's own motion in the mode; the first mode's line takes a point, a ring at its head.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.
- **How to read it.** The organism's telemetry as a tensor (muscle by gauge by time), its difference between the arms, its principal modes, the service level against its objective and the error budget burned, native beside Omni-Compass.
- **What to do with it.** The service level and the budget are what an on-call engineer is paged on: watch the burn rate on both arms under the same knobs.

### *L.33.4 Monitor 56: Service Level*

The share of each half minute's requests answered inside the 500 ms line, native beside Omni-Compass, through the official run chosen in section L.20.

- **Evidence and state.** Class L (the official run's request log, SHA-256-checked); replay.
- **Feed.** SLO recording rules (sloth, pyrra) over the service's histogram; series `slo:sli_error:ratio_rate5m`; on a running system: `1 - slo:sli_error:ratio_rate5m`.
- **Wired.** This page's `drawSankey` → the Python `docs/engine/export_runs.py`, `tools/live_reps.py`, `scripts/raw_archive.sh` → the C++ none: the live software's own measurements, played back from their SHA-256-checked files; held equal by `docs/engine/build_live_engine.py`.
- **Moves.** Half minute by half minute through the official run; its newest value printed at the head.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **How to read it.** The organism's telemetry as a tensor (muscle by gauge by time), its difference between the arms, its principal modes, the service level against its objective and the error budget burned, native beside Omni-Compass.
- **What to do with it.** The service level and the budget are what an on-call engineer is paged on: watch the burn rate on both arms under the same knobs.

### *L.33.5 Monitor 57: Error Budget Burn*

The requests over the 500 ms line counted up since the repetition began, native beside Omni-Compass: each side's error budget burned.

- **Evidence and state.** Class L (the official run's request log, SHA-256-checked); replay.
- **Feed.** SLO recording rules (sloth, pyrra); series `slo:error_budget:remaining`; on a running system: `slo:error_budget:remaining`.
- **Wired.** This page's `drawSankey` → the Python `docs/engine/export_runs.py`, `tools/live_reps.py`, `scripts/raw_archive.sh` → the C++ none: the live software's own measurements, played back from their SHA-256-checked files; held equal by `docs/engine/build_live_engine.py`.
- **Moves.** Half minute by half minute through the official run; its newest value printed at the head.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.
- **How to read it.** The organism's telemetry as a tensor (muscle by gauge by time), its difference between the arms, its principal modes, the service level against its objective and the error budget burned, native beside Omni-Compass.
- **What to do with it.** The service level and the budget are what an on-call engineer is paged on: watch the burn rate on both arms under the same knobs.

### *L.33.6 Monitor 58: Repetition Gauges Against Native*

Every gauge of the repetition replaying in section L.20 against native's 0 line: GAIN to the right in green, LOSS to the left in red, its side the run's own reading.

- **Evidence and state.** Class L (the official run's own gauges); replay.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).
- **Wired.** This page's `drawRadar` → the Python `docs/engine/export_runs.py`, `tools/live_reps.py`, `scripts/raw_archive.sh` → the C++ none: the live software's own

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

measurements, played back from their SHA-256-checked files; held equal by `docs/engine/build_live_engine.py`.

- **Moves.** Each gauge of the repetition replaying in section L.20.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); Tune Dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense Delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One Decision: step the engine one decision while paused.
- **How to read it.** The organism's telemetry as a tensor (muscle by gauge by time), its difference between the arms, its principal modes, the service level against its objective and the error budget burned, native beside Omni-Compass.
- **What to do with it.** The service level and the budget are what an on-call engineer is paged on: watch the burn rate on both arms under the same knobs.

## L.34 Master Switch Wiring

**Controls.** Live: the switch on this monitor is the kill switch of section L.2.

**The page, in its own words.**

**Master Switch and Kill Switch**

One switch for the whole organism, as the repository builds it (omnicompass/master.py, tools/omni_switch.py). It sits at the top and holds the brain's outputs. With it OFF, nothing the brain computes is pushed or pulled into any muscle, and every muscle runs on native's own controllers, which never stop. On a real machine every governor exits at the switch; on this page the brain keeps reading the native body in shadow, so its monitors keep moving, and nothing it computes leaves it. No monitor is switched off: every Omni-Compass side follows native, drawn in native's colour. Every governor on this page is registered with it and obeys it at once. The live organism hands every setting it held back to native's own controller and reads each one back. Every record's Omni-Compass side, and every playback, trace, table, board and grid, reads native's own. Native's own controllers are never switched, so native's side never moves with the switch. Every section carries the same switch in its controls bar; pressing any one of them throws this one.

**Its monitors.**

### *L.34.1 Monitor 12: Master Switch and Kill Switch*

The switch above the brain: ON, Omni-Compass governs every wired muscle; engaged, every setting is handed back to native's own controller at once and every Omni-Compass side on the page follows native.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Evidence and state.** Class S (the page's port of the master switch; the real one is omnicompass/master.py); live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_master_switch`; on a running system: `omni_master_switch`.
- **Wired.** This page's `drawMaster, masterSet` → the Python `omnicompass/master.py`, `tools/omni_switch.py`, `omnicompass/nervous_system.py` → the C++ `cpp/src/nervous_system.cpp`; held equal by `tests/test_master_switch.py`, `tests/test_cpp_twins_parity.py`.
- **Moves.** Every decision: the brain's pulse and the work flowing down to every muscle; the switch acts at once.
- **Knobs.** Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The wiring from the top down: the master switch above the brain, the brain above the organism, the organism above every muscle, and native's own controllers under every muscle, never switched. The switch's state is the word on it: ON in green while Omni-Compass governs; engaged, the switch red and OFF in white.
- **What to do with it.** Press the kill switch only for security: it is instant, every setting goes back to native and is read back. Watch the governors' table beside it: every governor on the page says OFF and what it handed back.

## L.35 Specification and Mathematics

Items 1 to 5 and 8 are the engine’s code and run in every live view (the official-run playback of 4a and 4b replays records and runs none of it). Items 6 and 7 as written (the compass law on each muscle, the plug’s lever) drive the stress-test model. In the harness views the max and the plants’ own overrides take their place. The live controller runs the compass law on the service, as section L.22 shows.

**Controls.** Live: five of the brain's six state variables (E, U, I[U], S, B) and the nerves' calm below are read from the live benchmark engine every decision, beside the equations that give them.

**The page, in its own words.**

**Stress-Test Model Equations · Class S**

**Reads and Sets by Part**

**Engine Code Provenance**

The six-state engine, its constants and RK4 integrator, the assimilation, push and ρ*, the four nerve scalars and the organ thresholds, the compass law and the verdict in words are ported from omnicompass/core.py, adapter.py, nervous_system.py, compass_law.py and verdict.py at commit c8ba7775d. All are inside the fingerprint of the engine that stands, omni-v23: omni-v18 to omni-v23 left core.py, adapter.py,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

compass_law.py and verdict.py as omni-v17's bytes, and omni-v21 changed nervous_system.py and added immune.py for the immune rule, which the page's reflex block ports from those two files; adapter.py carries omni-v16's clocks in seconds, which the page's brain ports (a = 1 − (1 − 0.339)^k and macro_dt = 0.1·k for k = period/60 s; at a 60 s period every value is omni-v15's; checked against the Python at 10, 15, 60 and 120 s with no grant differing) and omni-v15's clear-time gate on the power-cap request, which the port does not compute (it ports the states, the push and the headroom), and the others are omni-v12's bytes. omni-v9 added omnicompass/student_t.py, and the page carries its Student t too, computed by bisection on its own distribution and rounded to three decimals as the printed tables are. omni-v10 changed only the CityLearn wire's glide, omni-v11 only the Kubernetes governor's blind-reading guard, omni-v12 only that governor's hold on a refused raise, and omni-v13 only the governor's conveyance on a shared host at its top, omni-v14 only the governor and its muscles: a muscle left native, the operator's commands without a shell, and omni-v15 the brain's power-cap request through the line's own clear time, with the card governor handing the brain its clear flag. Checked against the Python on seeded inputs (docs/engine/check_live_engine.py): 8,000 decisions of the governor (state, push, ρ*, κ, h, σ, ν, calm), 48,000 organ grants, 4,800 compass forces, 300 verdict intervals and 400 full verdict trials. A build passes only when the largest difference is under $10^{-12}$, no grant and no verdict differs, and the Student t is equal on all 500 degrees of freedom checked. And against the C++ twin of the core (cpp/src/core.cpp, compiled by the check with docs/engine/cpp_core_probe.cpp): 2,000 seeded states, the derivative and the RK4 step within $10^{-12}$ of each other. The check prints its own measured differences in each build's report on GitHub. The harness views are not ported: they are the harness’s own record, made by its code.

**Realms Harness and Stress-Test Model**

Realms Harness. The default body: the sixty muscles are one organism of realms/harness.py. Each is a plant of realms/plants.py: machine pools under their autoscalers, cooled zones, batteries on their lines, motion axes and process loops. They are coupled through their heat and their power, under the climb (1 2 3 2 3 4 …), for 240 decisions of 15 s. Omni-Compass is the max of realms/compass_arm_v5.py with the body’s bone, its twins and the muscles’ cards; at 80% the brain goes silent and the lease walks every setting back to native over 24 decisions. Recorded on seeds 3000, 3001 and 3002 (runs A, B and C) by docs/engine/export_harness_trace.py; the page replays the record. Each muscle alone is the same harness one muscle at a time, as run_muscle_arm runs the preregistered muscle table: under the line (work waiting at the start), the max with the muscle's own twin, and the lease after 80%. Each muscle's decisions are laid on 240 frames. A frame shows the last decision it covers, and reads over the line if any of them was. In that harness the brain’s state is computed from the body’s aggregate, as a governor reads it, and shown; the max itself acts muscle by muscle. Stress-Test Model. A model drawn for this page so the body can be stressed by hand (a surge, a blind sense, a security hold, the kill switch), evidence class S; like every view it runs native and Omni-Compass, side by side, on the same demand. Sixty muscles named from the 1,107-muscle catalog (twelve each from the spine and the four realms), each a capacity lever with its own response time τ and sensing delay, under an HPA-style native controller (target 0.70, 10% tolerance, five-decision scale-down window). Demand climbs 1 2 3 2 3 4, surges, eases and idles over 360 steps of 15 s, 90 minutes of the plant, the same for both arms. The lever rules that turn F into a write are this page's, written out above. Under the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

max: never below native, and lifting off hands back toward native. On the tune-down: never above native, and below it only with the organ's grant and the release gate.

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

## L.36 Failure Modes and Controls

**Controls.** Live: every way below that the live benchmark engine can be put through has its own button, and the response plays on the console (section L.12), native beside Omni-Compass.

**The page, in its own words.**

**Failure-Mode Register**

**Unjudged Resource Costs**

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

## L.37 Self-Governed Rendering

Native is the browser's own loop: every chart on screen drawn on every frame, and when the frame overruns, whole frames dropped. Omni-Compass is the same loop with the engine's own compass law governing it (omnicompass/compass_law.py, ported here and checked against the Python). The reading is the time between frames, the band two frames wide with its center one frame at 60 Hz. The two arms take turns in paired windows of three seconds on this same page, on your own machine, and are read by the three-run rule's interval (class L: this browser, live; a reading of this page, never a benchmark of record).

**The page, in its own words.**

**Native**

**Omni-Compass**

**Inter-Frame Interval by Window**

**Paired Window Measurement**

**Its monitors.**

### *L.37.1 Monitor 62: Inter-Frame Interval by Window*

This page's own render loop governed by the compass law: the frame time against its line, native (no governor) beside Omni-Compass.

- **Evidence and state.** Class S; live.
- **Feed.** this page's own clock (no exporter: the page measures itself).
- **Wired.** This page's `drawSelf` → the Python `omnicompass/compass_law.py` → the C++ `cpp/tools/engine_runs.cpp`; held equal by `docs/engine/check_live_engine.py`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Moves.** Every frame: this page's own frame time under the compass law.
- **Knobs.** Frame Load: +0, +8 or +16 ms burned inside every frame of this page; Kill Switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** This page's own render loop governed by the compass law: the time between frames against its line, native (no governor) beside Omni-Compass.
- **What to do with it.** Load the page with the frame-load knob and watch the governor shed chart frames to keep every frame inside its line.

## L.38 The page's Appendixes

## L.39 A Title Page and Build

**The page, in its own words.**

A live monitor of the Omni-Compass engine and of every benchmark GitHub has run on it: each run played back from its own SHA-256-checked files, each model run by the repository's own code, so the engine can be watched moving instead of only read. Two sides everywhere, and only two: Native, the whole system running by itself, and Omni-Compass and native, the same system with Omni-Compass on top of all of it. On this page Omni-Compass is never run by itself: it always sits on top of a native controller. Beside this title stand the owner's two pictures, each with the engine's live marks over it. The engine that stands is omni-v23 (`docs/OMNI_V23.md`); its road back to omni-v18 is in the disclosures (appendix G), and each earlier engine has its own page (docs/OMNI_V<n>.md). The engine that stands still carries the code omni-v18 and omni-v19 added to run Omni-Compass by itself (its own orchestrator and a pod muscle of its own); it is an opt-in start flag outside the factory tune, and this page shows none of it. Its only modelled record read LOSS: the record of 24 October carried that third arm, Omni-Compass by itself, and it read LOSS in all three of our runs on work and on muscles over the line in eight organisms, 32 rows (on omni-v19, digest 36ee67284c035235, the record's own engine, never read as the engine that stands; class S); the arm is off this page, and its record is kept whole in `docs/history/SCENARIOS_30_2026-10-24.md`.

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

## L.40 B Reading Guide

**The page, in its own words.**

**Monitor Wall Legend**

Each monitor carries its signal and its word (green GAINING, low in its band with room to climb; yellow STEADY, inside its band; red REDLINING, at the top of its band, blinking red and black; red OVER THE LINE, past its band, blinking red and white). The control panel (section L.2) holds every switch and knob: the kill switch, the heartbeat (10, 15, 30, 60, 90 or 120 s), the sweep over every beat, the active profile with

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Restore factory profile and Return to native, the actuation and each route's bill, the mechanism's own knobs, the plant's conditions and faults, the twenty scenarios, and one breaker per monitor. The engine runs itself: its own mathematics decides every beat; the knobs show the mechanism, they never have to drive it.

Five blocks: 1, benefit and organism (twelve monitors, three rows of four on a laptop); 2, real runs on GitHub; 3, brain and mechanism; 4, clouds, chips, orchestrators and scale; 5, proof and telemetry. Each block reads left to right, then the next row. A click on a tile goes to its monitor. The monitor list (appendix E) gives each monitor's source and feed.

The same canvases the sections draw, copied small four times a second; nothing is drawn twice from different data. The page's own governor (section L.37) leaves a monitor resting while it is off screen, so while this wall is in view the resting monitors are drawn in turn, one a frame, each through its own drawing code; every monitor is drawn once as the page opens, so no tile waits for its section to be scrolled to. Two monitors that sit side by side in their section, native beside Omni-Compass, sit side by side on the wall too: a pair never splits across two rows. Each tile's head is its number and its title in capital letters, then its section and its state (live, replay, from the records, a model, or awaiting its run). A tile with the accent border is a monitor on screen now.

**Organism Ring Guide**

**0 Monitor Wall**

The wall's light count is the one count the breakers, the section signals and the dock print.

Redline Reflex. At the top of its band Omni-Compass pushes at once; where a muscle stays over its line three decisions, stuck, and native's twin reads lower by more than 5%, it hands the muscle to native for 10 decisions (doubling for each hand-back in a row, up to 160), and takes it back only when its side is under the line and lower than native's by 5%; where native is not stronger it stays, because redlining less than native is still a benefit against native. Every light still carries its red: if a muscle redlines on Omni-Compass's side and the reflex cannot stop it (a fault, a bug), it is seen at once, and the kill switch hands everything back to native in the same instant. The rule entered the engine with omni-v21 and stands in the engine that stands, omni-v23, in its immune system, knob by knob (omnicompass/immune.py, the same file in both fingerprints, its C++ twin cpp/src/immune.cpp, proven alike on 200 seeded scripts, and the nervous system's native_hold; on Kubernetes each service's autoscaler target, native's twin the operator's own target), beside the SLO reflex and the pod reflex of the Kubernetes governor (omni_controller/controller.py); this page's live model (class S) applies the same rule, with the engine's own numbers, to every muscle it draws.

Green, GAINING: low in its band with room to climb, or a GAIN. Dark, AWAITING: the run that fills it has not landed. Grey, NO LINE: a monitor with no line to cross (a bill, a count of runs, a digest check) or no reading yet; neither is ever counted green. Yellow, STEADY: inside its band, or level with native (a change under 0.5%). Red, REDLINING, blinking red and black: at the top of its band and holding there, or a LOSS row. Red, OVER THE LINE, blinking red and white: past the top of its band. Native is orange everywhere, Omni-Compass blue; red is never a side, only the alarm. A monitor that draws two sides carries three lights in

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

its button: native's, Omni-Compass's, each read from its own side on the same load, and the reading between them now by the key (GAIN green, STEADY yellow, LOSS red), so the moment one side turns a colour the other does not is seen as it happens. REDLINING comes before the line: a gauge within 3% of the top of its band, or whose slope over its last readings reaches it within 3 readings. A needle that falls is not a LOSS and one that rises is not a GAIN: each light reads what its gauge is for (a wait that falls is GAINING; work that falls is not). Click a light for its reason; a light turned red by the knobs goes back with the factory tune. The three-reading projection is the light's own. The brain's law acts on the position inside its band and damps on the position's rate of change (K[D] × v, omnicompass/compass_law.py), with a fail-up at the band's 95% wall; it projects nothing ahead (section 3). Each light reads the series its monitor draws, and a light fed by our own models says so (class S); hover it for the reading.

**1 Control Panel**

Fuse Box. The first row is the master switches: each acts on the whole page at once, through the same code as the console and the dock, both sides alike (the kill switch hands everything back to native; the dial, the body and the time). Under it, seven tabs: Drive (the plant's conditions and faults, the heartbeat, the profile with its actuation, its contract and each route's bill), Mechanism Knobs (the engine's own constants, one group at a time), Scenarios and Injection Grid, Monitor Breakers (one per monitor: its light and word, green GAINING, yellow STEADY, red REDLINING, red OVER THE LINE; ON runs it live, REST stops its drawing; a way to it), Prometheus Metrics (the live model as a Prometheus endpoint, the same series section L.11 exposes), Commands (the request a running governor would take for each press) and All Tabs (the whole panel at once). Every switch is logged with its decision; Back to the factory tune puts all of it back.

Mechanism Knobs. Not the plant's conditions of the Drive tab: these turn the engine's own equations while it runs. Each knob is a constant of the brain (omnicompass/core.py), the compass law (omnicompass/compass_law.py), the governor's law (omnicompass/adapter.py, nervous_system.py), Omni-Compass's push, lift-off and tune-down on each muscle, the model's native controller, or one muscle's body. Turned, the next decision of the live stress-test model is computed with it, on Omni-Compass's side; native's controller knobs move every native controller, the one under Omni-Compass on native too. The harness replays recorded positions, so turning a knob moves the live benchmark engine to the model. A knob turned from the factory turns violet; ↺ puts that one back; Back to the factory tune puts every one back. The monitors (the strip, the rings, the phase plane, the brain's state, the status board) show the mechanism of action as it changes.

The compass law, the push, lift-off and tune-down, native's controller and every muscle's body move the muscles at once. The brain's four pieces move its own state (E, U, I[U], S, B on the brain's monitors) and its grants: at the max every organ may add capacity unless a security hold stands, so the brain's calm acts through the give-backs; at the tune-down the four pieces move the muscles too.

Manuals. The Live Benchmark Engine Manual reads this page top to bottom: how to read, operate, tune and watch it, native beside Omni-Compass, chapter by chapter in the page's own order, and, in its last appendix, the page word for word, every section, every monitor under its number and every knob, written from this page

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

at every build. The Operator's Manual wires and runs Omni-Compass on a stack; the Owner's Manual holds the whole mechanism, its mathematics, its source and every result. Each carries the notice, the LICENSE, the NOTICE and `DISCLOSURES.md`.

**2a Scorecard**

Each row is one way to run a workload, against native alone on the same work.

**2c Benefit Board**

Every benchmark's measured effect in one board, native against Omni-Compass on top of native, by domain and by gauge family, and the Omni index that weighs them together. The whole organism composed from these same tables is in section L.31; every table, every muscle and the grid in full in section L.32.

**2e Status Board**

The tiles, in order: the switch and the engine; the live organism's signals, native beside Omni-Compass, on the four signals of site reliability practice (errors, traffic, latency and saturation), here the muscle-decisions past their line, the work done, and the stand-in each body has (the work waiting in line on the live model, the electrical power drawn on the harness), each named on its tile, with the injected faults beside them; the three-run tables and the verdict guard; GitHub's runs, the clouds and the age of this page. Under the tiles, every alarm and watch item, worst first.

**3a Live Organism**

The system rendered as an organism: each node is a muscle (one controlled actuator) and each ring its operating range. The left instance runs under its native controllers alone; the right runs the same controllers with Omni-Compass's brain, nerves, heart and immune system supervising them.

Every view is run twice on the same body, the same load and the same seed: native (left), then Omni-Compass (right). "One organism" is the sixty muscles in one body: the whole native organism on the left, the same organism with Omni-Compass on the right; "each muscle alone" runs each muscle native and then with Omni-Compass, one at a time. The counters under each ring and the scoreboard (section L.15) give both arms and the difference. Inside one organism the bone, the power and cooling the whole body shares, holds most moves back. A dashed white ring marks every muscle that differs between the two sides at that moment; the counter under each ring counts them.

Every room and every machine pool has a native twin: the same muscle under its own controller, on native's heat and load, the body's memory of healthy. A room that stands warmer than its twin is inflamed. Where its own knob can cool it, it is cured on the spot. Where it cannot, it sends the whole body a signal, its heat debt, like a cytokine calling for help. The other muscles answer: the machine pools give back what Omni-Compass added until the room stands no warmer than native's. The skeleton (the bone: the power and cooling lines the whole body shares) grants nothing it cannot carry. A cooling setting is walked back, never snapped. A pool never loses a machine its queue needs, a battery is timed for its line, and no sense is locked.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Under the max a muscle is moved only where it has room above native. A muscle already at its limit on native (— of these sixty) is left native by design. In these harness views the max acts muscle by muscle and the brain at the center reads the body without steering it, as in the realms harness itself; section L.22 shows the live controller, where the brain's rules run every decision.

**3c Control Console**

Runtime parameter control and fault injection on the live benchmark engine: load scaling, noise injection, sensor latency, capacity loss, a demand surge, a sensor blackout, a security hold, the dial, and the kill switch. Every control is applied to both arms identically; the response is plotted per decision, native beside Omni-Compass.

The engine these knobs turn is the engine's own mathematics running in this page: the brain (omnicompass/core.py, integrated by RK4), the compass law (omnicompass/compass_law.py) and the verdict (omnicompass/verdict.py). docs/engine/check_live_engine.py proves the port equal to the Python and the C++ on every build. It governs the stress-test model's sixty muscles (class S), each on top of its own native controller. Every knob acts on both sides at once, the same demand, the same draw of noise, the same lost pool, so the one difference left between them is Omni-Compass. The knobs move the live benchmark engine here and in sections L.10, L.11, L.14 to L.16, L.18 and L.35, and the kill switch and the frame load also move this page's own renderer (section L.37). A recorded run (the benefit board, the chips' and Kubernetes' replays, the official runs) is a record: no knob rewrites it.

**3d Heartbeat Dial**

Each beat stands beside native with what it buys (the reads of the cluster a day) and what it costs (the governor's own processor, and every service row read against native and against the main engine's 60 s).

**3e Control Cycle**

One control cycle traced end to end at reduced speed: the sensed inputs, the brain's decision, and the actuator (muscle) it wrote.

**3f Phase Plane and Scoreboard**

The phase-plane state of every muscle (its position and velocity in its band) for both arms at the same instant, with the running scoreboard.

The plane is a phase portrait, the picture control engineers use for a system's state. Across is where each muscle sits in its band: p is 0 when calm, 0.5 at the center and 1 at its line. Up and down is how fast it moves there (v, per decision). A muscle at rest sits on the middle line. One climbing toward its line moves up and right, and past the right edge it is past its line. Native is on the left and Omni-Compass on the right: the same muscles at the same instant, one dot each. In the realms harness each muscle answers to its own rule, so no single field is drawn. In the stress-test model the shading on the right is the force the compass law would apply there: the add force gives capacity and the hand-back force returns it. Past the 0.95 wall the law fails up at once. Under each plane the same instant is written in words.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### 3g Brain State

The brain's six state variables rendered on their potential-energy landscapes: a state resting at a minimum is stable; a state driven toward a maximum is under load.

The wells draw the same state the cycle panel (section L.14) prints. Each ball is a live value of the engine's brain, integrated by RK4 every decision. In the stress-test model the brain's push and target feed the body's levers. Native has none of this: no state, no memory of need, no stress, no tide; each controller sees only its own reading, one to three decisions late.

### 3h Verdict

Before a knob moves one step past native, the step runs as a trial: costs at native first, then at the step. The engine that stands reads the change exactly as a report reads a row, on the log of each cost, with its 95% interval. The lab opens wired as the shipped Kubernetes controller is wired, on the noise the real runs measured.

### 3i Dashboards

The dashboards draw the same live benchmark engine the ways the industry draws a system: a board of panels, a host map, a graph of one series, an event stream, a cluster console with a button on every workload, a one-line diagram with its breakers, a wall of numbers, the cost of each body part, and Omni-Compass's own form, the two-way pipeline that reads, decides, writes and reads back. On a phone each drawn form also writes what it shows as plain lines under the drawing.

A muscle is any machine, service or controller Omni-Compass can read and set; native is the system as it runs without Omni-Compass; a write is one setting Omni-Compass changed; read back means it checks the setting landed; the lease walks every setting back to native after a crash; the kill switch stops Omni-Compass at once, for security. Every benchmark puts Omni-Compass on top of a native setup and reads both sides by the same key.

### 4a Kubernetes Run Playback

Playback of the official benchmark runs from their archived raw files: the native cluster and the Omni-Compass cluster under identical load, in 30-second bins, every file verified against its run's SHA-256 manifest.

Native (the whole cluster with everything it ships with, everything but Omni-Compass) against Omni-Compass, the same load, the same second, each file's SHA-256 matched before a byte is drawn. The tiles, the candles, the heatmap, the bars and the governor's ticker all move together with section L.21 (pick a benchmark, a run or a repetition in either place). Every word is the run's own reading, never one drawn from the stream.

### 4b Repetition Telemetry

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Per-repetition telemetry in 30-second bins, with the run's receipt and each repetition's score from the repository's own scorer.

Native and Omni-Compass, repetition by repetition, played in 30-second bins. Every Kubernetes repetition is scored by the repository's own scorer (tools/live_reps.py); where a run holds its own scored summary, the scorer's mean over its repetitions equals it (the count is in section L.21).

**4c Controller Decision Trace**

The shipped controller's decision trace, replayed from its own audit log one decision at a time: its inputs, its choice and its writes.

**4d Cloud Providers**

In a cloud the bill is read beside the work: whether the same bill buys more work with Omni-Compass on top. Each provider's card lists its four test sets and their status.

In a cloud, Omni-Compass shows two things: that the engine holds on the cloud's machines as it does on site, and the work per dollar: more work for the same bill, never a smaller bill as the aim (the founder's words of 31 October). From 31 October every cloud set runs the whole engine at the max on top of the cloud's own autoscaler (the founder's order of that day: no hard rule): the governor senses the service, moves its pod target inside the autoscaler's cover, conveys the processor, and parks a machine the cloud already made, after its own trial, waking it on a breach; it asks the cloud for no machine itself, and the cloud's autoscaler alone adds machines. Every gauge is read against native by the one key, and the cloud's judged number is the work per dollar: the bill may rise where the work rises more (the founder's words of 31 October: "anybody will pay 1% more for 10% more product"), a rise shown beside the work it bought, and a tune whose work per dollar falls against native is the tune falling out wrong, fixed in our models. Azure's queue holds its sets at the max until they carry the load whose work grows with the speed (the work meter), so each receipt reads the work per dollar. Until 31 October every cloud set ran at the tune-down on the machines alone (no pod autoscaler moved, no processor conveyed, no power capped), and those sets stand as their own record. Azure, Google Cloud and Amazon run the same tuning, mechanism and harness.

A cloud is the wire between the software end (the owner's programs, Kubernetes and the open-source orchestrators, managers and autoscalers that carry the work) and the hardware end (the bare metal: the GPU chips, the Linux racks, the data center, on-site or off-site). Omni-Compass can live in all three places: on the software end, on the cloud's wire, and on the bare metal. The wire makes no benefit of its own. On the wire alone (every cloud run on this page so far: Omni-Compass at the tune-down on the machines only, the pods exactly native's, the cloud's rule until 31 October), all it could do was park machines the cloud's own autoscaler already made, after its own trial, so the most those runs can show is a small cut in the bill of the wire. From 31 October every cloud set runs the whole engine at the max on the cloud's cluster, so its receipt shows that the engine holds there as on site and how much of the software end's gain comes through the wire, read as work per dollar beside the bill. It cannot show the energy and throughput gained at the bare metal, because the provider's machines are not a tenant's to meter. Omni-Compass on the software end (the front end)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

and on the bare metal (the back end) reaches gauges a cloud run cannot. A cloud provider that runs Omni-Compass on its own back end would see those gauges on its own GPU chips and Linux racks; no tenant can meter a provider's back end, so this page claims it only as the same mechanism measured on hardware we can meter (section L.25), never as a measurement of the provider's machines. The three places compose into one organism and one bill in section L.31.

The company's own expectation, not a measured result (24 October): about 3 to 4% from the front end, 2 to 3% from the back end, about 5% from the two ends together, and 1 to 2% from the wire alone, as a smaller bill or more work for the same bill; the readings of each place stand in the place table of sections L.23 and L.31; where a measurement differs from the expectation, the measurement stands.

**4e Orchestrators and Governors**

These are the governors and orchestrators that sit between the work and the cluster: batch schedulers and job queues, multi-cluster governors, the default Kubernetes scheduler, GPU and edge orchestrators, and the rightsizing controllers. Each has three runs, and every row is shown.

**4f GPU and CPU Hardware**

The chip is the far end of every stack: there the max would ride each card's own limit, and every watt it takes is power given, shown beside the work it buys.

Each stack is run twice on the same load and the same seed: native first, then Omni-Compass. Left is native, right is Omni-Compass, and every table gives both columns and the difference. The clouds are section L.23 and Kubernetes sections L.20 to L.22; every muscle of the catalog, the seven organisms and the three-run tables are in section L.32.

**4g CI Pipeline**

The canvas and the table list every GitHub Actions workflow run in progress or queued at build time. The hardware and cloud runs, with their terminal status, are in sections L.23 and L.25; every run on the engine that stands that terminated without a measurement is raised on the status board (section L.8).

**5a Python Runs and Monte Carlo**

The Python's runs, the C++ harness's and this page's, run by run: the jump generator's set GitHub ran in the Python (its run linked), the same draws run through the C++ harness on the engine's C++ twins, and the same set recomputed here by this page's own port of the engine as the page opens; every receipt compared three ways, one SHA-256 digest each, and the digest of the draws themselves; and the Python's 500 scenarios a seed replayed from its own record with every trace hash; and the Python's Monte Carlo of the engine's core, 500 runs each from step 0 through the fourth-order Runge–Kutta step, 20 macro steps of 10 micro steps, native beside Omni-Compass, into the basin, held and certified, with every set the jump generator draws here held to the Python's digest.

**5b C++ Twin**

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

The engine is written twice, in Python and in C++, line for line. Every time this page is built, the Python's own runs are fed into the C++ as its input, and the C++ answer is set beside the Python's, number by number. Where they are the same to the last digit, the two are one engine. Where a part runs in Python only, the list Python-Only Components names it.

**5c Scenarios and Injection Grid**

The ten organisms (the spine; the four realms; the four stacked; the whole tower; the orchestra; this page's own sixty-muscle organism; and the whole organism in its three places), each run two ways: native alone, and Omni-Compass on top of native (the factory tune).

Injection Grid. One batch of the Python's Monte Carlo injections (tools/engine_runs.py: base B, injection k drawing seed $B + k \cdot 0x9E3779B9 \bmod 2^{32}$; the C++ engine runs the same seeds to an identical digest, section L.28) runs through every organism the harness runs (the nine organisms of cells 1 to 9, each with a plant; the whole organism is a composition of measured tables and takes no injection), each native and with Omni-Compass on native, and through each of the fourteen governors and orchestrators sitting on the same organism (the third arm): the governor on the organism, and Omni-Compass on top of the governor. Every cell is the one batch cut down: its 500 runs, of which 100 runs are the first 100, 10 the first 10 and 1 the first; and in each run a stack of 500 organisms, of which 100 are the first 100 copies, 10 the first 10 and 1 the first (copy c of run k on injection k + c, mod 500). A click on 100 shows those 100 and no others; a cell whose runs × organisms pass 500 uses an injection more than once: the organism axis is a resampling of the same 500 injections and adds no independent evidence, so the cells at 500 organisms read as the cells at 1 do. A cell's word is one batch's (one run in the key's sense), its injections within that batch, never "our runs". Each run's change against its reference is read over its 24 blocks of ten decisions, and the cell's word by the key over its N runs (tools/readings.py three_run_reading, the 0.5% floor). The page re-hashes the record's cells and recomputes every cell's mean from the runs' means; both checks are printed under the batch.

Governors in the Injection Grid. In our models each governor is the body of the catalog muscles that stand for its knobs (tools/run_orchestra.py, the mapping of tools/governors_page.py), run on the organism's clock on the same injections, each under its own native controllers; Omni-Compass on top of the governor governs both bodies. The governor's body does not reach the organism's own muscles, so its way reads its own muscles beside the organism's.

**5d Whole Organism**

Omni-Compass is wired into every piece of a stack through its own two-way plug: the software (the cluster: Kubernetes, its add-ons, its governors and orchestrators; and the services on it: the pools, the consumers, the caches), the bare metal under it (the chip's power limit and clock; the machine's kernel and its own meter) and, where the stack is rented, the cloud that bills the machines. A stack is bought one of two ways: on-site bare metal, the software on the owner's own bare metal (software × bare metal, no cloud); through the cloud, the cloud's build, its bare metal at the back end and the software (cloud × bare metal × software), with the cloud as two pieces beside it (cloud × software, the back-end bare metal left to the provider).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Every piece's benefit is added into one collective reading, gauge by gauge (work, the 95th percentile response, machines, energy, the bill), with the receipts under it: what native spent, what Omni-Compass spent, and the benefit, in the units each table measured. The repository composes the layers in one place, tools/whole_system.py, which writes `results/WHOLE_SYSTEM.md` from the tables that stand: each layer measured against its own native with the layer under it as native runs it; a gauge's ratio oriented so that above 1 is a GAIN at every layer alike; the layers composed by the product law, $R(g) = \prod_\ell R_\ell(g)$, $\log R = \Sigma_\ell \log R_\ell$ (for small benefits the product is their sum), every interval carried (the exact Student t per run; A, B and C pooled by inverse variance; the layers' variances added); two readings of the same numbers (every measured piece: everything measured enters with its interval; the strict reading, the index's rule: only what held the same way in all three of our runs, everything else exactly 1); and the separability assumption tested on the governors' four ways. Section L.31 recomputes nothing from the tables: it regroups that file's layers into the two ways (docs/engine/export_whole.py), summing the logarithms of the layers a scenario stacks and adding their variances, as the file's rule composes any set of layers; the one number is the mean of the gauge logarithms, its 95% interval under the file's two bounds.

**5e Measured Effect**

Everything the board in section L.6 stands on, from the organisms down to the single muscle: the seven organisms under their three loads, all 1,107 muscles of the catalog one cell each, every registered benchmark still to run, the scale-out grid of copies and runs, and every three-run table in the repository with every row as it is written.

That board holds every benchmark's benefit in one place, as the repository's benefit sheet holds it (results/BENEFIT_SHEET.json), with the Omni index that weighs them together (`results/OMNI_INDEX.md`). The scale-out grid holds every organism at 1, 10, 100 and 1,000 copies, each at 1, 10, 100 and 1,000 runs, modelled and with the real cluster inside.

Seven Organisms Under Three Loads. The seven organisms of the catalog (the spine first, the base every stack runs on; the four realms; the four stacked; the whole tower), each run native and with Omni-Compass on top of native, under its own load, the climb (1 2 3 2 3 4 …) and work waiting in line; and the whole organism, the software, the cloud and the hardware stacked into one body (section L.31), composed from the pieces' own readings, native and with Omni-Compass on top of every piece: a composition with two arms, not an organism of the catalog. The orchestra, every governor's muscles as one body, is the eighth organism (section L.24). The monitor draws the seven from the realms table; the whole stack's two arms are section L.31's composition until the stacked run, preregistered in `docs/WHOLE_SYSTEM_PREREGISTRATION.md`, measures them in one run.

Open Register Slots. Every muscle in them (a legged robot's gait, locomotion_speed; a wind turbine's blade pitch, pitch_angle_limit; a mobile robot's travel speed, agv_speed and trajectory_speed) already runs, modelled, in the 1,107-muscle catalog and the seven organisms of section L.32, native and with Omni-Compass on top. A slot here is the next step for that kind of muscle: the same test on the outside world's

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

own software (MuJoCo's Unitree Go2, NREL's OpenFAST, ROS 2 Nav2), each queued until its three runs land.

The 1,000 tier runs on GitHub's free runners only; paid machines stay at 100. Each bar and square carries its own word with its size and no sign; the raw change keeps its own sign, labelled raw.

**5f Telemetry Analytics**

The same telemetry analysed four ways: a three-dimensional tensor view, a difference matrix, a principal-component decomposition and Omni-Compass's own account of its run; the service level and error budget in SRE terms; and Omni-Compass answering the viewer's questions about itself from the page's own records.

The tensor fills in decision by decision as section L.10 plays, and the same numbers are read three more ways: the difference between the arms, the body's principal motions, and Omni-Compass's own account of what it is doing. The official runs (class L) are read as an SRE reads a service: the service level, the error budget burned and every gauge against native; and Omni-Compass answers questions about itself, reading the page's own records through tools and turning the dashboard to a run.

**5g Master Switch Wiring**

One switch for the whole organism, drawn as the repository wires it: the master switch holds the brain's outputs; the brain sits above the organism; every muscle hangs under it with native's own controller under every muscle, never switched. Every governor of this page is listed with its state at the switch.

**5h Specification and Mathematics**

The component specification: every part, its equations and the source module that implements it, as the repository writes them.

**5i Failure Modes and Controls**

The failure-mode register: every known way Omni-Compass can fall short, the control that mitigates it, that control's coverage, and the residual risk where it does not reach. The second table: the resource costs the three-run tables report but do not judge.

**5j Self-Governed Rendering**

The page's own render loop is governed by the same compass law, measured on the client device, native beside Omni-Compass.

Each chart is a muscle whose knob is how many frames it waits between draws, and the reading is the time between frames. Below the band's center the pedal lifts and every chart is handed back to native, drawn every frame. Toward the line it pushes: the charts farthest from where you are looking wait a frame or more, so the frame itself stays inside its line. Every step is a trial run to its full measurement and kept only when the frame shortens; a step that buys nothing is refused and every chart handed back to native (the reflex rule: where a knob cannot bring a GAIN, it stays native).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

## L.41 C Problem and Answer

**The page, in its own words.**

**Problem, Answer, Proof, Limits and Protection**

**Industry Problem**

Demand Outrunning Capacity. Demand for computing is growing faster than the power, the machines and the budgets built to carry it. The International Energy Agency puts the world's data centres at about 415 TWh of electricity in 2024, around 1.5% of all the electricity the world used, and expects that to more than double, to about 945 TWh, by 2030, with artificial intelligence the largest driver. In the United States, Lawrence Berkeley National Laboratory counts 176 TWh in 2023, 4.4% of the country's electricity, and between 6.7% and 12% by 2028. The usual answer is to build more: more machines, more megawatts, more cooling and a larger bill.

Idle Capacity. Across the Kubernetes clusters Cast AI measured in 2025, the average processor was 8% busy and the average memory 20% used, read before its own automation was switched on. The organizations Flexera surveyed estimate that 29% of what they spend on cloud infrastructure and platforms is wasted.

Coordination Gap. In every realm (compute, AI and the cloud; physics, robotics and autonomy; energy, facilities and industry; distribution and specialized systems) each machine already runs under local feedback loops: horizontal autoscalers, firmware power and clock governors, building-management loops, battery-management systems, connection pools, servo controllers. Each loop is tuned on its own, against its own setpoint, with margins sized for a worst case. Four costs follow, and every operator pays them:

Reaction Latency. Reactive loops act on lagging, averaged signals. A Kubernetes Horizontal Pod Autoscaler recomputes its replicas every 15 seconds by default and damps its scale-downs over a 5-minute stabilization window; new pods then need time to schedule and start. Queues form before capacity arrives.

Static Over-Provisioning. Worst-case margins persist through the far more common nominal regime. The operating point sits well inside the safe region, and the headroom between it and the constraint boundary is paid for and rarely used.

Uncoordinated Coupling. Loops share resources: power, cooling, connections, memory, network. One loop's correction is another loop's disturbance: more web replicas saturate the database's connections; more cooling pushes a power feed toward its limit.

Gaps Closed by Hand. Operators close them with fixed settings, runbooks, alerts and on-call rotations, and the settings go stale as the load moves. That is safe, and it leaves the region between the operating point and the safe boundary unused nearly all of the time.

**Omni-Compass Answer**

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Omni-Compass is a supervisory governor. It sits on top of every native controller and never in place of one; native keeps running underneath, always. It treats each setting a native controller already exposes (a replica floor, a power cap, a clock ceiling, a temperature setpoint, a pool size) as a muscle, and the whole stack as one body, the organism. Through a two-way plug, one wire in (the readings native already reports) and one wire out (the settings native already exposes), it reads every muscle, places each reading in the muscle's own band, from calm to its line (the limit it must not cross: a response time, a temperature, a feed's rating), and moves each setting one bounded, smooth step at a time. By default every muscle is driven up to its own limit, never past it and never below native; every write is read back; every give-back is tried in its own paired trial (the muscle with and without it, under the same load) and undone if the muscle loses by it, by the verdict's own rule: refused and undone where the 95% interval of its change in cost lies wholly above zero, or where the median or the mean of its logarithmic cost rises past the 2% allowance (omnicompass/verdict.py); a raise is held inside the muscle's own bounds and judged afterwards by the three-run tables; and wherever no gain can be brought, the muscle stays native. One brain governs the whole body: one shared calm, read from the brain's state over the whole organism, gates every give-back in every organ (an organ gives capacity back only while that calm stands at or above its own threshold, its service line is clean and its sense is fresh; omnicompass/nervous_system.py), while every raise stays inside the muscle's own band. Every muscle is judged on itself; a trade between muscles, one used less so another carries more, is not in the engine that stands. The aim: more work, faster, on fewer machines, with less energy. Where Omni-Compass takes more energy to buy more work or speed, that energy is shown beside what it bought, as power given, and never called a loss.

**Costs and Responses**

| Industry cost | Omni-Compass response | Judged by |
|---|---|---|
| Reaction Latency: waiting, queues and time over the line before capacity arrives | it reads every muscle on its own clock (on Kubernetes its full read of the state every 60 seconds at the factory, 15 seconds or faster where a person picks it on the dial, and its fast checks every 5 seconds) and moves as a reading nears its line, not after the line is crossed: proactive as well as reactive, like an immune system | time waited, missed or failed: a fall is GAIN, a rise is LOSS |
| Static Over-Provisioning: machines, energy and a bill paid for headroom rarely used | every muscle driven up to its own limit and never past it, so the machines already paid for carry more work; in a public cloud it parks only a machine the cloud already made, after its own trial, | work done: a rise is GAIN; machines, energy and the bill: a fall is a cut, a GAIN, and a rise is power given |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Industry cost | Omni-Compass response | Judged by |
|---|---|---|
| | and wakes it on a breach: by construction it adds no machine and no pod of its own; the bills measured so far are single runs (read from the cloud ledger as the page opens), and no three-run bill exists yet | |
| Uncoordinated Coupling: one loop's correction is another loop's disturbance | one brain for one body: one shared calm over the whole organism gates every give-back in every organ, and every raise stays inside the muscle's own band; a trade between muscles is not in the engine that stands | every muscle judged on itself, and the organism judged whole |
| Gaps Closed by Hand: settings retuned by hand, alert minutes, on-call | an autopilot: every write read back, every give-back on its own trial, every reason written to its audit; an alert minute on Omni-Compass's side is found and fixed, never accepted | the alert minutes, judged; the settings commands it wrote on its own, shown |

Section Map. Reaction latency: the organism live, every muscle's place in its band and the decisions past the line, native beside Omni-Compass (3a), and the shipped controller's decision trace (4c). Static over-provisioning: the benefit board (2c) and the clouds' bills (4d). Uncoordinated coupling: the whole organism (5d) and the organism's immune system (3a). The gaps closed by hand: the status board's alarms (2e) and the golden signals, the event log and the live metrics (3b).

Limits. It replaces no controller, and it creates no capacity a machine does not have: it uses the capacity already there, each muscle to its own limit. It promises no number on a system it has not run on: the only number that applies to a system is the one that system's own paired runs produce. The outside figures above are their authors' own, quoted for scale; none was measured, checked or endorsed by The Omni-Compass LLC, and none is a result of Omni-Compass.

Accountability. Every comparison judged pairs two arms, native and Omni-Compass on top of native, on the same machines with the same work (the governors' tests run four arms and pair them two at a time, section L.24). Every three-run table is run three separate times (A, B and C); the Azure runs read so far and the card's run inside the whole organism's composition are single runs, named as such where they appear. Every reading is one word, GAIN, STEADY or LOSS, decided by what the gauge is for (the key, appendix D); every result carries its evidence class (P a physical meter, L live software, S our own models, never shown as proof of

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

hardware); and every row is shown, LOSS included. The governing rule is do no harm, in its one meaning (`DISCLOSURES.md`, section 3): the aim is an organism never left turning out less than native, nor less healthy; where a row of a three-run table reads LOSS against native (section L.7 lists each, every one; a model's own LOSS rows, class S, stand in their own sections, the heartbeat dial's in 3d), the row stays, its cause is to be found and fixed (the LOSS ledger writes a cause beside those it has; the others read "not yet written"), and the test runs again on the engine that stands. The executive summary (appendix D) reads where the record stands, from the records on this page.

Outside Sources. International Energy Agency, Energy and AI (April 2025), its executive summary. Lawrence Berkeley National Laboratory, 2024 United States Data Center Energy Usage Report (December 2024), prepared for the U.S. Department of Energy. Cast AI, 2026 State of Kubernetes Optimization Report: a seller of Kubernetes optimization, reporting its own customers' clusters as they ran before its automation. Flexera, 2026 State of the Cloud Report (March 2026): a survey of 753 cloud decision-makers and users, the figure their own estimate. The Kubernetes documentation, Horizontal Pod Autoscaling: the controller's defaults.

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

## L.42 D Executive Summary and Reading Key

The Omni index, every real test weighed together, is read from its record as the page opens.

**Controls.** Live: the buttons act on the live benchmark engine; every number below is read from the records on this page.

**The page, in its own words.**

**Summary, Key, Terms and Examples**

**Executive Summary**

Every count and reading in this list is read from the records on this page when it opens (the three-run tables, the Omni index, the cloud ledger, the card run, the models' own readings, the realms table and the harness). The settings it quotes are their sources' own: the native defaults in its last line from `docs/engine/NATIVE_REGISTER.md`, the Azure sets' repetition cap from scripts/azure_queue.txt. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

**Reading Key**

**Defined Terms**

Each term is defined once; where it first appears on the page it carries the same definition, shown when pointed at.

The size is how far the gauge moved from native; the word says whether that move helped the work. Examples, read from the three-run tables on this page, appear here when they load. A miss is a decision where the work went over its line. The key is written once in tools/readings.py and stands at the top of the README,

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

`docs/HOW_TO_READ_THE_RESULTS.md` and the manuals. GAIN, STEADY and LOSS, and the letters behind them, are The Omni-Compass LLC's own convention for reading its own results; the underwriting senses explain the words and are not an actuarial, financial or insurance opinion, and no outside body defined, reviewed or approved them. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

## L.43 E Monitor List

The list that goes with the wall: every monitor by its number, in the same order, with what it shows, the section it lives in, its evidence class and state, the open-source feed that would carry it live and its knobs.

**Controls.** Recorded data: the list is the page's own registry of its monitors, read when the page opens; click a number to go to its monitor, a section to go to its section.

**The page, in its own words.**

**Monitors by Number**

Motion Meter. Every monitor in view is photographed small once a second, as you see it, and its badge says whether it moved.

Every monitor is one canvas on this page; its number is its place in the wall's five blocks (docs/engine/monitors.json), and the wall (section L.1) shows them in that order, left to right, then the next row. The feeds name the open-source exporters an operator's Prometheus and Grafana already run (node_exporter, kube-state-metrics, the NVIDIA DCGM exporter, Kepler, OpenCost, the governor's own tools/omni_exporter.py); the button in the control panel downloads the whole wall as a Grafana dashboard on those series (deploy/grafana/omni-compass-wall.json, written from docs/engine/monitors.json), beside the governor's own screen (deploy/grafana/omni-compass.json). A monitor the page draws from the repository's tables, or from its own clock, says so: no series carries it.

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

## L.44 F Glossary and Index

What every word on this page means, written from the repository's own pages (each definition names its source file), the organisms by their realms and families, and every muscle of the catalog (realms/catalog_v5.csv) with the organisms it sits in; the index sends each term to the sections where it is used. Built by tools/live_glossary.py into docs/engine/glossary.json.

**Controls.** Type to find a term, an organism or a muscle; click a letter to jump; each muscle opens to its family, its realms, its template, its knob and its wiring.

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## L.45 G Disclosures, Law and Contact

**The page, in its own words.**

THIS PAGE IS A LIVE TUNING ENGINE. THE BENCHMARKS ARE ONE TUNED ENGINE. Every knob, button and monitor on this page can be turned in real time: the heartbeat at 10, 15, 30, 60, 90 or 120 s, the dial between the max and the tune-down, the pedal, each muscle driven or left native, up or down, in or out, minus or plus, to see and produce the variations of the mechanism of action with native alone, with Omni-Compass on top of native, or both side by side. Like any engine, where turning the mixture, the intake, the exhaust, the firing or the pistons changes what it delivers, turning any knob can change every metric, either way, in any direction a tuner chooses: that range is the engine's versatility, one engine for many vessels, tuned many ways, for many outputs. What the page shows at any setting other than the factory tune demonstrates that range and is never a benchmark result. Every benchmark result is one engine at its one factory tune: the engine its table names, every benchmark run again on the engine that stands (section L.26). From omni-v17 the factory tune is chosen by a declared rule: of the tunes that harm no gauge against native, the one against which no other tried tune reads GAIN on one service gauge and LOSS on none, searched on development seeds and read again on untouched ones (`docs/THE_TUNE.md`; `DISCLOSURES.md`, section 3). A tune other than the factory tune is the tuner's own, and its results are its own. No profile on this page is locked as the baseline: the factory profile is the one released with the engine, and reading every knob against it, one at a time, is the standing work of the sweeps (`docs/FACTORY_PROFILE.md`).

DISCLOSURE: REAL RUNS, A REAL ENGINE, NOTHING HARDWIRED. Every run played back on this page (sections L.20 and L.21) is read from the raw files of benchmark runs executed on GitHub's own machines, each file checked against its run's SHA-256 list before a single number is drawn; every three-run table is read as the repository holds it, each naming its runs, and the raw branch holds their folders as they are archived (the status board, section L.8, marks each run whose raw files are archived). Every model view runs the engine's own code (the compass law, the brain, the verdict) live in this page, linked through software to the repository's Python and C++ and checked against them on every build. GitHub rebuilds the page from the repository whenever the engine, a model or a result changes, and every three hours: no number on it is typed in by hand, and when the repository changes, the page changes with it.

**Engine Lineage**

**Page Scope**

**Disclosures**

**Contact**

**Its monitors.** This section draws no canvas: its views are tables and text the page builds from its records.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## L.46 Every monitor

| No. | Monitor | Section | Class | State |
|---|---|---|---|---|
| 1 | Benefit by Domain | 2c Benefit Board | L/S | record |
| 2 | Stack Composition | 5d Whole Organism | S (of P and L) | composed |
| 3 | Market Map | 2c Benefit Board | L/S | record |
| 4 | Benefit by Gauge Family | 2c Benefit Board | L/S | record |
| 5 | Native Organism Ring · Class S | 3a Live Organism | S | live |
| 6 | Omni-Compass Organism Ring · Class S | 3a Live Organism | S | live |
| 7 | Errors per Decision · Class S | 3b Golden Signals and Event Log | S | live |
| 8 | Throughput per Decision · Class S | 3b Golden Signals and Event Log | S | live |
| 9 | Latency Stand-In per Decision · Class S | 3b Golden Signals and Event Log | S | live |
| 10 | Saturation Stand-In per Decision · Class S | 3b Golden Signals and Event Log | S | live |
| 11 | Immune System · Class S | 3a Live Organism | S | live |
| 12 | Master Switch and Kill Switch | 5g Master Switch Wiring | S (the page's port of the master switch; the real one is omnicompass/master.py) | live |
| 13 | Data Pipeline | 4a Kubernetes Run Playback | L | replay |
| 14 | Governor Heartbeat and Lease | 4a Kubernetes Run Playback | L | replay |
| 15 | Response Time Candlesticks | 4a Kubernetes Run Playback | L | replay |
| 16 | Latency Heatmap | 4a Kubernetes Run Playback | L | replay |
| 17 | Throughput per 30 s Bin | 4a Kubernetes Run Playback | L | replay |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| No. | Monitor | Section | Class | State |
|---|---|---|---|---|
| 18 | Paired Repetitions | 4a Kubernetes Run Playback | L | replay |
| 19 | Repetition Telemetry per 30 s Bin | 4b Repetition Telemetry | L | replay |
| 20 | Quote Board | 2c Benefit Board | L/S | record |
| 21 | Native Cluster Trace · Class S | 4c Controller Decision Trace | S | replay |
| 22 | Omni-Compass Cluster Trace · Class S | 4c Controller Decision Trace | S | replay |
| 23 | Machines and Response Time · Class S | 4c Controller Decision Trace | S | replay |
| 24 | Orchestration Stack | 4e Orchestrators and Governors | L | record |
| 25 | Compass Band of the Followed Muscle | 3e Control Cycle | S | live |
| 26 | Scoreboard Strip · Class S | 3f Phase Plane and Scoreboard | S | live |
| 27 | Native Phase Plane | 3f Phase Plane and Scoreboard | S | live |
| 28 | Omni-Compass Phase Plane | 3f Phase Plane and Scoreboard | S | live |
| 29 | Double-Well Potential of U | 3g Brain State | S | live |
| 30 | Cubic Potential of Stress S | 3g Brain State | S | live |
| 31 | Bath Oscillator B | 3g Brain State | S | live |
| 32 | Native Controller Trace · Class S | 3g Brain State | S | live |
| 33 | Trial Costs and 95% Interval | 3h Verdict | S | live |
| 34 | Share of Trials Allowed | 3h Verdict | S | live |
| 35 | Body Parts on the Band · Class S | 3a Live Organism | S | live |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| No. | Monitor | Section | Class | State |
|---|---|---|---|---|
| 36 | Heartbeat Sweep · Class S | 4a Kubernetes Run Playback | S | model |
| 37 | Azure Bill | 4d Cloud Providers | L (metered by Azure) | record |
| 38 | Google Cloud Gate and Bill | 4d Cloud Providers | L | awaiting its run |
| 39 | Amazon Web Services Gate and Bill | 4d Cloud Providers | L | awaiting its run |
| 40 | Native Cloud Machines · Class S | 4d Cloud Providers | S | replay |
| 41 | Omni-Compass Cloud Machines · Class S | 4d Cloud Providers | S | replay |
| 42 | Road to a Result | 4e Orchestrators and Governors | L | record |
| 43 | Metered A100 Run | 4f GPU and CPU Hardware | P | awaiting its run |
| 44 | CPU Host Model · Class S | 4f GPU and CPU Hardware | S (model) until the run | awaiting its run |
| 45 | 8-Card GPU Node · Class S | 4f GPU and CPU Hardware | S (native model) until the run | awaiting its run |
| 46 | Stack Models · Class S | 4f GPU and CPU Hardware | S | model |
| 47 | Seven Organisms Under Three Loads · Class S | 5e Measured Effect | S | record |
| 48 | Catalog Muscle Matrix · Class S | 5e Measured Effect | S | record |
| 49 | Scale-Out Grid | 5e Measured Effect | S/L | record |
| 50 | Three-Way Run Check | 5a Python Runs and Monte Carlo | S (the engine itself) | live |
| 51 | Held-Out Scenario Replay | 5a Python Runs and Monte Carlo | S | replay |
| 52 | Engine Core Monte Carlo | 5a Python Runs and Monte Carlo | S (the engine itself) | live |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| No. | Monitor | Section | Class | State |
|---|---|---|---|---|
| 53 | Telemetry Tensor Cube · Class S | 5f Telemetry Analytics | S (the harness's record, realms/harness.py) | replay |
| 54 | Tensor Difference Matrix · Class S | 5f Telemetry Analytics | S (the harness's record, realms/harness.py) | replay |
| 55 | Tensor Principal Modes · Class S | 5f Telemetry Analytics | S (the harness's record, realms/harness.py) | replay |
| 56 | Service Level | 5f Telemetry Analytics | L (the official run's request log, SHA-256-checked) | replay |
| 57 | Error Budget Burn | 5f Telemetry Analytics | L (the official run's request log, SHA-256-checked) | replay |
| 58 | Repetition Gauges Against Native | 5f Telemetry Analytics | L (the official run's own gauges) | replay |
| 59 | GitHub Actions in Flight | 4g CI Pipeline | L (GitHub's own API, read at the build) | record |
| 60 | System Response per Decision · Class S | 3c Control Console | S | live |
| 61 | Inter-Frame Interval | 3c Control Console | S | live |
| 62 | Inter-Frame Interval by Window | 5j Self-Governed Rendering | S | live |

## L.47 Every knob

| Knob | What it does | Where it is |
|---|---|---|
| Kill Switch | Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white) | the kill switch (section L.2; its wiring, section L.34) |
| Tune Dial | the max (every muscle to its own limit) or the tune-down (never more than native) | the console's dial |
| Demand | ×0.25 to ×2 of the day's own demand, both arms alike | the console's demand slider |
| Noise | σ of the demand's noise, one draw a muscle a decision, both arms alike | the console's noise slider |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Knob | What it does | Where it is |
|---|---|---|
| Sense Delay | decisions added between what a muscle does and what its controller reads, both arms alike | the console's delay slider |
| Faults | a surge, half the machines lost, a sense blind, a security hold, on the chosen body part | the console's fault buttons |
| Speed | 1×, 4×, 16× or 60× of the engine's clock | the console's time knob |
| One Decision | step the engine one decision while paused | the console's time knob |
| Pause | hold the engine's clock | the console's time knob |
| Frame Load | +0, +8 or +16 ms burned inside every frame of this page | the console's frame-load knob |
| Run | A, B or C (or the archived run) the monitor plays | the monitor's own buttons |
| Repetition | which repetition of the run is replayed | the monitor's own buttons |
| Arm | which arm stands on the right: Omni-Compass governing, or watching only | the monitor's own buttons |
| Stack | the GPU card, the Linux host, Kubernetes or the cloud | the monitor's own buttons |
| Governor | which orchestrator or governor is shown | the monitor's own table |
| Scenario | on-site bare metal, through the cloud, in the device, as a service | the monitor's own buttons |
| Reading | every measured piece carried, or the index's strict rule | the monitor's own buttons |
| Chip Run | the chip's one run beside the count or in it | the monitor's own buttons |
| Beat | the governor's full read of the state and its decision: every 5, 10, 15, 20, 30, 60, 75, 90 or 120 s in the heartbeat sweep's record; on a real governor its --interval (60 s at the factory since omni-v17, 15 s on omni-v16) | the monitor's own buttons |
| Engine | which engine's record the monitor reads: the engine that stands or the one before it | the monitor's own buttons |
| Baseline | what each reading is set against: native, or the same engine at another setting | the monitor's own buttons |
| Gauge | which gauge of the run is shown | the monitor's own buttons |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Knob | What it does | Where it is |
|---|---|---|
| Filter | which rows, words, families or organisms are shown | the monitor's own buttons |
| Body | which body the organism runs: the harness, the organism, every muscle alone | the organism's own buttons |
| Muscle Switch | pick one muscle on either ring: its card, and its own switch, which hands that one muscle back to native (Omni-Compass OFF on it) or puts Omni-Compass on it again; the kill switch hands every muscle back at once | the organism's monitors and the muscle's card (section L.10) |
| Runs | how many of the Python's runs this page computes again | the monitor's own buttons |
| Print | the receipts of the runs, printable, with the digests and the notice | the monitor's own button |
| Seed Base | which held-out seed base's 500 scenarios are replayed | the monitor's own buttons |
| Set | the Python's own 500 (numpy, seed 223387268) or a set the jump generator drew on this page | the Monte Carlo monitor's set knob (section L.28) |
| Omni-Compass Way | the bounded command toward the nearest basin, toward the far basin, or with the double well off (μ = 0); native is the plant alone (u = 0) | the Monte Carlo monitor's way knob (section L.28) |
| Play / Pause | the cursor's sweep of the 20 macro steps, held or running | the Monte Carlo monitor (section L.28) |
| Custom System | your stack written as ranges of the core's six states and 22 parameters, run in the page or on GitHub through the Python and the C++ (`systems/README.md`) | the Monte Carlo monitor's own-system panel (section L.28) |

## L.48 Every LOSS row, whose it is, its cause and the knob that answers it

163 LOSS rows are Omni-Compass against its native; 171 are an orchestrator's own cost against the organism alone, with no Omni-Compass in either arm. Every one is shown as its table writes it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Omni-Compass against native.** All four in one run (omni-v21, `results/live/V21_ALL_FOUR.md`): pod start wait, total (s), A B C 8.9 → 10.3; raw +15.7% (95%: −19.6% to +51.1%); LOSS · 9 → 12.8; raw +42.2% (95%: −16.2% to +100.6%); LOSS · 7.8 → 9.3; raw +19.2% (95%: −23.0% to +61.4%); LOSS. Cause: The pod start wait on one shared host: where it goes, why it rose, and the rig amendment of 18 October (`docs/POD_START_WAIT.md`). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_BARE.md`): response time (ms), 95th percentile, A B C 2452 → 2510; raw +2.4% (95%: −2.1% to +6.9%); LOSS · 2697 → 2871; raw +6.5% (95%: −3.8% to +16.7%); LOSS · 1422 → 1533; raw +7.7% (95%: +1.4% to +14.1%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_BARE.md`): response time (ms), 99th percentile, A B C 3425 → 3535; raw +3.2% (95%: −1.9% to +8.3%); LOSS · 3778 → 4014; raw +6.3% (95%: −5.8% to +18.3%); LOSS · 1972 → 2162; raw +9.7% (95%: +3.3% to +16.0%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_BARE.md`): failed requests (%), A B C 0.006817 → 0.007637; raw +12.0% (95%: −355.1% to +379.2%), +0.0008 in the gauge's unit; LOSS · 0 → 0; raw ±0.0000 in the gauge's unit (95%: ±0.0000 to ±0.0000); STEADY · 0 → 0; raw ±0.0000 in the gauge's unit (95%: ±0.0000 to ±0.0000); STEADY. The knob: the muscle's floor and the trial's tolerance; a muscle that fails a request native served stays native until the gate reads no LOSS.
- **Omni-Compass against native.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_BARE.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 12 → 12.33; raw +2.8% (95%: −0.4% to +6.0%); LOSS · 13 → 14; raw +7.7% (95%: −2.4% to +17.7%); LOSS · 6.8 → 8.2; raw +20.6% (95%: ±0.0% to +41.2%); LOSS. The knob: the muscle's floor and the trial's tolerance; a muscle that fails a request native served stays native until the gate reads no LOSS.
- **Omni-Compass against native.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_BARE.md`): pod start wait, total (s), A B C 0.4444 → 1; raw +125.0% (95%: −50.3% to +300.3%), +0.5556 in the gauge's unit; LOSS · 0.3 → 2.3; raw +666.7% (95%: +151.6% to +1181.7%), +2.0000 in the gauge's unit; LOSS · 0.3 → 1; raw +233.3% (95%: −257.2% to +723.9%), +0.7000 in the gauge's unit; LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Omni-Compass against native.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_BARE.md`): pod start wait, mean (s), A B C 0.2222 → 0.8333; raw +275.0% (95%: −46.0% to +596.0%), +0.6111 in the gauge's unit; LOSS · 0.3 → 1.083; raw +261.1% (95%: +52.5% to +469.8%), +0.7833 in the gauge's unit; LOSS · 0.15 → 0.65; raw +333.3% (95%: −217.3% to +884.0%), +0.5000 in the gauge's unit; LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_BARE.md`): batch: queue finished (s), A B C 872.8 → 885.1; raw +1.4% (95%: +0.4% to +2.4%); LOSS · 892.4 → 912.5; raw +2.3% (95%: +1.0% to +3.5%); LOSS · 806 → 818.8; raw +1.6% (95%: −0.1% to +3.2%); LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_BARE.md`): queue: job turnaround, submitted to done, mean (s), A B C 184.4 → 193.8; raw +5.1% (95%: +1.9% to +8.3%); LOSS · 203.2 → 221.3; raw +8.9% (95%: +4.9% to +12.9%); LOSS · 115.3 → 125.8; raw +9.1% (95%: +0.9% to +17.3%); LOSS. Cause: Job turnaround under Armada, Omni-Compass on the organism alone (`V12_ARMADA_4W_BARE.md`): under investigation since 21 October 2026 (`docs/LOSS_ROWS_UNDER_INVESTIGATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_TOP.md`): pod start wait, total (s), A B C 0.4444 → 2.222; raw +400.0% (95%: −79.7% to +879.7%), +1.7778 in the gauge's unit; LOSS · 0 → 2.3; raw +2.3000 in the gauge's unit (95%: +1.0361 to +3.5639); LOSS · 1.5 → 1.6; raw +6.7% (95%: −168.1% to +181.4%); LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Batch queue (omni-v21, `results/live/V21_BATCH.md`): failed requests (%), A B C 0 → 0.01036; raw +0.0104 in the gauge's unit (95%: −0.0131 to +0.0338); LOSS · 0 → 0; raw ±0.0000 in the gauge's unit (95%: ±0.0000 to ±0.0000); STEADY · 0 → 0; raw ±0.0000 in the gauge's unit (95%: ±0.0000 to ±0.0000); STEADY. Cause: 1. The pod start wait: thirteen rows in eight tables (`docs/LOSS_CAUSES_2026-10-31.md`). The knob: the muscle's floor and the trial's tolerance; a muscle that fails a request native served stays native until the gate reads no LOSS.
- **Omni-Compass against native.** Batch queue (omni-v21, `results/live/V21_BATCH.md`): pod start wait, total (s), A B C 0 → 0.9; raw +0.9000 in the gauge's unit (95%: +0.1887 to +1.6113); LOSS · 0 → 4; raw +4.0000 in the gauge's unit (95%: −0.2519 to +8.2519); LOSS · 0 → 1.5; raw +1.5000 in the gauge's unit (95%: −0.7176 to +3.7176); LOSS. Cause: The pod start wait on one shared host: where it

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

goes, why it rose, and the rig amendment of 18 October (`docs/POD_START_WAIT.md`). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.

- **Omni-Compass against native.** Batch queue (omni-v21, `results/live/V21_BATCH.md`): pod start wait, mean (s), A B C 0 → 0.9; raw +0.9000 in the gauge's unit (95%: +0.1887 to +1.6113); LOSS · 0 → 2.2; raw +2.2000 in the gauge's unit (95%: +0.1475 to +4.2525); LOSS · 0 → 0.8333; raw +0.8333 in the gauge's unit (95%: −0.0319 to +1.6985); LOSS. Cause: The pod start wait on one shared host: where it goes, why it rose, and the rig amendment of 18 October (`docs/POD_START_WAIT.md`). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream (omni-v12, `results/live/V12_BINPACK.md`): response time (ms), 99th percentile, A B C 427.2 → 439.5; raw +2.9% (95%: −1.2% to +7.0%); LOSS · 415.6 → 425.7; raw +2.4% (95%: −3.4% to +8.3%); LOSS · 419.3 → 430.5; raw +2.7% (95%: −0.4% to +5.7%); LOSS. Cause: Amendment 5: the governor's own core (written before any run on it, 20 October 2026) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream (omni-v12, `results/live/V12_BINPACK.md`): time over the response line (% of samples), A B C 0.7443 → 1.042; raw +40.0% (95%: −9.4% to +89.3%), +0.2976 in the gauge's unit; LOSS · 0.5461 → 0.6976; raw +27.7% (95%: −62.9% to +118.3%), +0.1515 in the gauge's unit; LOSS · 0.7367 → 0.8459; raw +14.8% (95%: −23.2% to +52.8%), +0.1093 in the gauge's unit; LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream (omni-v12, `results/live/V12_BINPACK.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 1.2 → 2.1; raw +75.0% (95%: −11.4% to +161.4%); LOSS · 0.9 → 1.3; raw +44.4% (95%: −123.9% to +212.8%), +0.4000 in the gauge's unit; LOSS · 1.7 → 1.7; raw ±0.0% (95%: −44.4% to +44.4%); STEADY. The knob: the muscle's floor and the trial's tolerance; a muscle that fails a request native served stays native until the gate reads no LOSS.
- **Omni-Compass against native.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream (omni-v12, `results/live/V12_BINPACK.md`): pod start wait, total (s), A B C 0.1 → 0.6; raw +500.0% (95%: −272.6% to +1272.6%), +0.5000 in the gauge's unit; LOSS · 0.2 → 1; raw +400.0% (95%: −179.2% to +979.2%), +0.8000 in the gauge's unit; LOSS · 0.1 → 0.1; raw ±0.0% (95%: −337.2% to +337.2%), ±0.0000 in the gauge's unit; STEADY. Cause: Amendment 5: the governor's own core (written before any run on it, 20 October 2026) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the test machine, not the governor: the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.

- **Omni-Compass against native.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream (omni-v12, `results/live/V12_BINPACK.md`): pod start wait, mean (s), A B C 0.1 → 0.5; raw +400.0% (95%: −291.1% to +1091.1%), +0.4000 in the gauge's unit; LOSS · 0.2 → 0.75; raw +275.0% (95%: −158.6% to +708.5%), +0.5500 in the gauge's unit; LOSS · 0.1 → 0.1; raw ±0.0% (95%: −337.2% to +337.2%), ±0.0000 in the gauge's unit; STEADY. Cause: Amendment 5: the governor's own core (written before any run on it, 20 October 2026) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream (omni-v12, `results/live/V12_BINPACK.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.1 → 0.1; raw ±0.0% (95%: −337.2% to +337.2%), ±0.0000 in the gauge's unit; STEADY · 0.2 → 0.2; raw ±0.0% (95%: ±0.0% to ±0.0%), ±0.0000 in the gauge's unit; STEADY · 0 → 0.1; raw +0.1000 in the gauge's unit (95%: −0.1262 to +0.3262); LOSS. Cause: Amendment 5: the governor's own core (written before any run on it, 20 October 2026) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream (omni-v12, `results/live/V12_BINPACK.md`): queue: job turnaround, submitted to done, mean (s), A B C 26.6 → 26.97; raw +1.4% (95%: +0.8% to +1.9%); LOSS · 26.39 → 26.66; raw +1.0% (95%: +0.4% to +1.7%); LOSS · 26.58 → 26.89; raw +1.2% (95%: +0.7% to +1.6%); LOSS. Cause: Amendment 5: the governor's own core (written before any run on it, 20 October 2026) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream, the four ways (omni-v12, `results/live/V12_BINPACK_4W_BARE.md`): response time (ms), 95th percentile, A B C 1397 → 1463; raw +4.7% (95%: −1.7% to +11.2%); LOSS · 866.2 → 923.9; raw +6.7% (95%: −1.7% to +15.0%); LOSS · 827.2 → 872.2; raw +5.4% (95%: −1.5% to +12.4%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream, the four ways (omni-v12, `results/live/V12_BINPACK_4W_BARE.md`): response time (ms), 99th percentile, A B C 2014 → 2059; raw +2.3% (95%: −6.1% to +10.6%); LOSS · 1230 → 1274; raw +3.5% (95%: −2.6% to +9.6%); LOSS · 1250 → 1246; raw −0.3% (95%: −11.9% to

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

+11.3%); STEADY. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.

- **Omni-Compass against native.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream, the four ways (omni-v12, `results/live/V12_BINPACK_4W_BARE.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 10.8 → 10.9; raw +0.9% (95%: −2.8% to +4.7%); LOSS · 6.9 → 7.7; raw +11.6% (95%: −2.1% to +25.2%); LOSS · 8.3 → 8.6; raw +3.6% (95%: −4.6% to +11.8%); LOSS. The knob: the muscle's floor and the trial's tolerance; a muscle that fails a request native served stays native until the gate reads no LOSS.
- **Omni-Compass against native.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream, the four ways (omni-v12, `results/live/V12_BINPACK_4W_BARE.md`): queue: job turnaround, submitted to done, mean (s), A B C 102.4 → 106.9; raw +4.4% (95%: −1.7% to +10.5%); LOSS · 63.56 → 65.83; raw +3.6% (95%: −3.2% to +10.3%); LOSS · 58.77 → 61.3; raw +4.3% (95%: −2.3% to +10.9%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream, the four ways (omni-v12, `results/live/V12_BINPACK_4W_TOP.md`): pod start wait, total (s), A B C 0.5 → 0.9; raw +80.0% (95%: −124.6% to +284.6%), +0.4000 in the gauge's unit; LOSS · 0.5 → 1; raw +100.0% (95%: −294.7% to +494.7%), +0.5000 in the gauge's unit; LOSS · 0 → 0.6; raw +0.6000 in the gauge's unit (95%: −0.0032 to +1.2032); LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream, the four ways (omni-v12, `results/live/V12_BINPACK_4W_TOP.md`): pod start wait, mean (s), A B C 0.25 → 0.9; raw +260.0% (95%: +7.2% to +512.8%), +0.6500 in the gauge's unit; LOSS · 0.4 → 0.65; raw +62.5% (95%: −267.4% to +392.4%), +0.2500 in the gauge's unit; LOSS · 0 → 0.6; raw +0.6000 in the gauge's unit (95%: −0.0032 to +1.2032); LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream, the four ways (omni-v12, `results/live/V12_BINPACK_4W_TOP.md`): queue: job turnaround, submitted to done, mean (s), A B C 106.1 → 108.4; raw +2.2% (95%: −2.3% to +6.7%); LOSS · 63.75 → 65.27; raw +2.4% (95%: −1.2% to +6.0%); LOSS · 58.35 → 61.38; raw +5.2% (95%: +0.9% to +9.5%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Omni-Compass against native.** The capacity test, native and Omni-Compass on top at the same resource cap (omni-v21, `results/live/V21_CAPACITY.md`): cap 3 cores: completed work (requests answered within the line over the window), A B C −0.6% (−1.4 to +0.2) · −1.1% (−3.6 to +1.4) · −0.2% (−0.7 to +0.4). Cause: 5. The fixed-capacity test at its top cap: 3 cores, the completed work (`docs/LOSS_CAUSES_2026-10-31.md`). The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** Crane (Effective HPA, on Kubernetes 1.25) under the batch stream, the four ways (omni-v12, `results/live/V12_CRANE_4W_BARE.md`): pod start wait, total (s), A B C 6.8 → 10.9; raw +60.3% (95%: +15.0% to +105.6%); LOSS · 8 → 10.3; raw +28.7% (95%: −22.5% to +80.0%); LOSS · 7.1 → 12; raw +69.0% (95%: +30.9% to +107.1%); LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Crane (Effective HPA, on Kubernetes 1.25) under the batch stream, the four ways (omni-v12, `results/live/V12_CRANE_4W_TOP.md`): pod start wait, total (s), A B C 6.8 → 12.3; raw +80.9% (95%: +37.3% to +124.5%); LOSS · 6.6 → 13.5; raw +104.5% (95%: +46.9% to +162.2%); LOSS · 7.9 → 12.2; raw +54.4% (95%: +20.0% to +88.9%); LOSS. Cause: Amendment 10: what the counted runs of 20 October found (written after those runs, before the Gödel runs start again) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Crane (Effective HPA, on Kubernetes 1.25) under the batch stream, the four ways (omni-v12, `results/live/V12_CRANE_4W_TOP.md`): pod start wait, mean (s), A B C 1.492 → 1.62; raw +8.6% (95%: −9.5% to +26.8%); LOSS · 1.325 → 1.607; raw +21.3% (95%: +1.4% to +41.2%); LOSS · 1.58 → 1.594; raw +0.9% (95%: −15.0% to +16.8%); LOSS. Cause: Amendment 10: what the counted runs of 20 October found (written after those runs, before the Gödel runs start again) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** CruiseKube (omni-v21, `results/live/V21_CRUISEKUBE.md`): response time (ms), mean, A B C 83.61 → 84.39; raw +0.9% (95%: −0.8% to +2.7%); LOSS · 89.5 → 90.76; raw +1.4% (95%: +0.1% to +2.7%); LOSS · 85.69 → 87.79; raw +2.4% (95%: +0.9% to +4.0%); LOSS. Cause: 2. CruiseKube: three rows, two arms that are one system (`docs/LOSS_CAUSES_2026-10-31.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** CruiseKube (omni-v21, `results/live/V21_CRUISEKUBE.md`): response time (ms), 95th percentile, A B C 127.6 → 127.6; raw ±0.0% (95%: −1.6% to +1.5%); STEADY · 137.9 → 140.4; raw +1.8% (95%: +0.5% to +3.1%); LOSS · 136.4 → 137.8; raw +1.0% (95%: −0.8% to +2.9%); LOSS. Cause: 2. CruiseKube: three rows, two arms that are one system

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(docs/LOSS_CAUSES_2026-10-31.md). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.

- **Omni-Compass against native.** CruiseKube (omni-v21, results/live/V21_CRUISEKUBE.md): time over the response line (% of samples), A B C 0.01176 → 0.01227; raw +4.3% (95%: −340.2% to +348.8%), +0.0005 in the gauge's unit; LOSS · 0 → 0; raw ±0.0000 in the gauge's unit (95%: ±0.0000 to ±0.0000); STEADY · 0 → 0; raw ±0.0000 in the gauge's unit (95%: ±0.0000 to ±0.0000); STEADY. Cause: 2. CruiseKube: three rows, two arms that are one system (docs/LOSS_CAUSES_2026-10-31.md). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Fairness (omni-v21, results/live/V21_FAIRNESS.md): pod start wait, mean (s), A B C 1.6 → 1.678; raw +4.9% (95%: −28.0% to +37.8%); LOSS · 1.606 → 1.769; raw +10.1% (95%: −14.4% to +34.7%); LOSS · 1.203 → 1.817; raw +51.0% (95%: −7.9% to +109.9%); LOSS. Cause: 1. The pod start wait: thirteen rows in eight tables (docs/LOSS_CAUSES_2026-10-31.md). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (docs/POD_START_WAIT.md); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Fairness (omni-v21, results/live/V21_FAIRNESS.md): second app: failed requests (%), A B C 13.65 → 13.8; raw +1.1% (95%: −4.9% to +7.0%); LOSS · 11.61 → 11.73; raw +1.1% (95%: −3.5% to +5.6%); LOSS · 13.67 → 14.18; raw +3.7% (95%: −1.6% to +9.0%); LOSS. Cause: 1. The pod start wait: thirteen rows in eight tables (docs/LOSS_CAUSES_2026-10-31.md). The knob: the muscle's floor and the trial's tolerance; a muscle that fails a request native served stays native until the gate reads no LOSS.
- **Omni-Compass against native.** Goldilocks (omni-v21, results/live/V21_GOLDILOCKS.md): pod start wait, total (s), A B C 4.8 → 8.4; raw +75.0% (95%: +27.8% to +122.2%); LOSS · 4.3 → 8.7; raw +102.3% (95%: +38.5% to +166.1%); LOSS · 4.6 → 10.9; raw +137.0% (95%: +32.0% to +241.9%); LOSS. Cause: The pod start wait on one shared host: where it goes, why it rose, and the rig amendment of 18 October (docs/POD_START_WAIT.md). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (docs/POD_START_WAIT.md); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Goldilocks (omni-v21, results/live/V21_GOLDILOCKS.md): pod start wait, mean (s), A B C 1.408 → 1.681; raw +19.4% (95%: −6.1% to +44.9%); LOSS · 1.267 → 1.673; raw +32.1% (95%: +7.0% to +57.2%); LOSS · 1.425 → 1.584; raw +11.2% (95%: −27.5% to +49.8%); LOSS. Cause: The pod start wait on one shared host: where it goes, why it rose, and the rig amendment of 18 October (docs/POD_START_WAIT.md). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (docs/POD_START_WAIT.md); the rig pre-pulls the image in both arms.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Omni-Compass against native.** The KAI scheduler under the batch stream (omni-v12, `results/live/V12_KAI.md`): response time (ms), 99th percentile, A B C 484.3 → 487.6; raw +0.7% (95%: −3.4% to +4.7%); LOSS · 519.6 → 569.8; raw +9.7% (95%: −2.4% to +21.7%); LOSS · 565.6 → 598.1; raw +5.7% (95%: −11.8% to +23.3%); LOSS. Cause: Amendment 5: the governor's own core (written before any run on it, 20 October 2026) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** The KAI scheduler under the batch stream (omni-v12, `results/live/V12_KAI.md`): time over the response line (% of samples), A B C 1.003 → 1.091; raw +8.8% (95%: −24.7% to +42.4%); LOSS · 1.527 → 2.172; raw +42.3% (95%: −9.2% to +93.8%); LOSS · 2.335 → 2.487; raw +6.5% (95%: −47.1% to +60.1%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** The KAI scheduler under the batch stream (omni-v12, `results/live/V12_KAI.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 1.4 → 1.8; raw +28.6% (95%: −74.2% to +131.3%); LOSS · 1.8 → 2.9; raw +61.1% (95%: −21.5% to +143.7%); LOSS · 3.5 → 3.5; raw ±0.0% (95%: −46.2% to +46.2%); STEADY. The knob: the muscle's floor and the trial's tolerance; a muscle that fails a request native served stays native until the gate reads no LOSS.
- **Omni-Compass against native.** The KAI scheduler under the batch stream (omni-v12, `results/live/V12_KAI.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 1 → 1; raw ±0.0% (95%: ±0.0% to ±0.0%); STEADY · 1 → 1; raw ±0.0% (95%: ±0.0% to ±0.0%); STEADY · 1 → 1.2; raw +20.0% (95%: −10.2% to +50.2%); LOSS. Cause: Amendment 5: the governor's own core (written before any run on it, 20 October 2026) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** The KAI scheduler under the batch stream (omni-v12, `results/live/V12_KAI.md`): queue: job turnaround, submitted to done, mean (s), A B C 29.85 → 30.29; raw +1.5% (95%: +0.4% to +2.5%); LOSS · 30.8 → 32.45; raw +5.3% (95%: −2.4% to +13.1%); LOSS · 32.25 → 32.97; raw +2.2% (95%: −5.4% to +9.9%); LOSS. Cause: Amendment 5: the governor's own core (written before any run on it, 20 October 2026) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BARE.md`): response time (ms), 95th percentile, A B C 1070 → 1095; raw +2.3% (95%: −4.4% to +9.1%); LOSS · 1366 → 1450; raw +6.2% (95%: −4.2% to +16.5%);

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

LOSS · 1135 → 1150; raw +1.3% (95%: −8.6% to +11.2%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.

- **Omni-Compass against native.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BARE.md`): batch: queue finished (s), A B C 771.5 → 775.5; raw +0.5% (95%: +0.1% to +0.9%); LOSS · 790 → 797.8; raw +1.0% (95%: +0.2% to +1.8%); LOSS · 772.7 → 773.3; raw +0.1% (95%: −0.9% to +1.0%); STEADY. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BARE.md`): queue: job turnaround, submitted to done, mean (s), A B C 78.3 → 81.36; raw +3.9% (95%: +0.6% to +7.2%); LOSS · 98.16 → 106.2; raw +8.2% (95%: +2.6% to +13.8%); LOSS · 82.27 → 82.29; raw ±0.0% (95%: −11.1% to +11.2%); STEADY. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_TOP.md`): response time (ms), 95th percentile, A B C 1478 → 1519; raw +2.8% (95%: −8.6% to +14.1%); LOSS · 1927 → 1992; raw +3.4% (95%: −2.4% to +9.1%); LOSS · 1712 → 1788; raw +4.4% (95%: −7.1% to +16.0%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_TOP.md`): response time (ms), 99th percentile, A B C 2206 → 2288; raw +3.7% (95%: −4.5% to +12.0%); LOSS · 2841 → 2937; raw +3.4% (95%: −3.5% to +10.3%); LOSS · 2761 → 2775; raw +0.5% (95%: −9.2% to +10.2%); STEADY. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_TOP.md`): pod start wait, mean (s), A B C 0.15 → 0.8; raw +433.3% (95%: −117.9% to +984.6%), +0.6500 in the gauge's unit; LOSS · 0.15 → 0.4; raw +166.7% (95%: −36.0% to +369.3%), +0.2500 in the gauge's unit; LOSS · 0.4 → 0.5; raw +25.0% (95%: −171.8% to +221.8%), +0.1000 in the gauge's unit; LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_TOP.md`): batch: queue finished (s), A B C 804.2 → 810.9; raw +0.8% (95%: +0.1% to +1.6%); LOSS · 835 → 844.2; raw +1.1% (95%: +0.3% to +1.9%); LOSS · 807.9 → 812.4; raw +0.6% (95%: −0.3% to +1.4%); LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Omni-Compass against native.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_TOP.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 1.6 → 2.6; raw +62.5% (95%: −21.8% to +146.8%); LOSS · 2.6 → 3; raw +15.4% (95%: −60.5% to +91.2%); LOSS · 1.4 → 1.6; raw +14.3% (95%: −18.0% to +46.6%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_TOP.md`): queue: job turnaround, submitted to done, mean (s), A B C 109.7 → 115.9; raw +5.6% (95%: −0.4% to +11.7%); LOSS · 139.5 → 148.6; raw +6.5% (95%: +0.8% to +12.2%); LOSS · 119.4 → 125.4; raw +5.0% (95%: +0.2% to +9.9%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BARE.md`): response time (ms), 95th percentile, A B C 2164 → 2439; raw +12.7% (95%: +5.9% to +19.5%); LOSS · 2260 → 2366; raw +4.6% (95%: −3.6% to +12.9%); LOSS · 2707 → 2815; raw +4.0% (95%: −2.0% to +10.0%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BARE.md`): response time (ms), 99th percentile, A B C 3077 → 3538; raw +15.0% (95%: +3.5% to +26.5%); LOSS · 3117 → 3349; raw +7.4% (95%: −0.4% to +15.3%); LOSS · 3712 → 3925; raw +5.7% (95%: −5.3% to +16.8%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BARE.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 11.33 → 13; raw +14.7% (95%: −2.6% to +32.0%); LOSS · 11.3 → 12.3; raw +8.8% (95%: −2.7% to +20.4%); LOSS · 10.4 → 10.9; raw +4.8% (95%: +1.2% to +8.4%); LOSS. The knob: the muscle's floor and the trial's tolerance; a muscle that fails a request native served stays native until the gate reads no LOSS.
- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BARE.md`): pod start wait, total (s), A B C 0.5556 → 2.222; raw +300.0% (95%: −72.5% to +672.5%), +1.6667 in the gauge's unit; LOSS · 0 → 1.2; raw +1.2000 in the gauge's unit (95%: +0.3879 to +2.0121); LOSS · 0 → 1.9; raw +1.9000 in the gauge's unit (95%: −0.1077 to +3.9077); LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BARE.md`): pod start wait, mean (s), A B C 0.5556 → 1.389; raw +150.0% (95%: −99.4% to +399.4%), +0.8333 in the gauge's unit; LOSS · 0 → 0.95; raw +0.9500 in the gauge's unit (95%: +0.3091 to +1.5909); LOSS · 0 → 1.25; raw +1.2500 in the gauge's unit (95%: +0.2210 to +2.2790); LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BARE.md`): batch: queue finished (s), A B C 852.7 → 875.9; raw +2.7% (95%: +1.0% to +4.5%); LOSS · 856.4 → 866.9; raw +1.2% (95%: −0.1% to +2.6%); LOSS · 909.9 → 928.2; raw +2.0% (95%: +0.7% to +3.3%); LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BARE.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.07407 → 0.1019; raw +37.5% (95%: −30.9% to +105.9%), +0.0278 in the gauge's unit; LOSS · 0.1042 → 0.1083; raw +4.0% (95%: −58.5% to +66.5%), +0.0042 in the gauge's unit; LOSS · 0.0875 → 0.1; raw +14.3% (95%: −43.7% to +72.3%), +0.0125 in the gauge's unit; LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BARE.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.5556 → 0.6667; raw +20.0% (95%: −63.1% to +103.1%), +0.1111 in the gauge's unit; LOSS · 0.6 → 0.8; raw +33.3% (95%: −16.9% to +83.6%), +0.2000 in the gauge's unit; LOSS · 0.5 → 0.5; raw ±0.0% (95%: ±0.0% to ±0.0%), ±0.0000 in the gauge's unit; STEADY. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BARE.md`): queue: job turnaround, submitted to done, mean (s), A B C 163.2 → 182; raw +11.6% (95%: +4.9% to +18.2%); LOSS · 167.6 → 171.2; raw +2.2% (95%: −1.7% to +6.0%); LOSS · 216.2 → 228.7; raw +5.8% (95%: +1.3% to +10.4%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_TOP.md`): response time (ms), mean, A B C 709.8 → 808.5; raw +13.9% (95%: −0.4% to +28.2%); LOSS · 691.8 → 764.6; raw +10.5% (95%: −0.5% to +21.6%); LOSS · 844.3 → 867.4; raw +2.7% (95%: −8.4% to +13.9%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_TOP.md`): response time (ms), 95th percentile, A B C 3269 → 3923; raw +20.0% (95%: +4.1% to +35.9%); LOSS · 3140 → 3665; raw +16.7% (95%: +1.6% to +31.8%); LOSS · 3666 → 3759; raw +2.5% (95%: −6.2% to +11.3%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_TOP.md`): response time (ms), 99th percentile, A B C 4767 → 5337; raw +12.0% (95%: +1.8% to +22.1%); LOSS · 4602 → 5114; raw +11.1% (95%: +1.2% to +21.0%); LOSS · 4876 → 5069; raw +4.0% (95%: −4.2% to +12.2%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_TOP.md`): time over the response line (% of samples), A B C 24.64 → 25.85; raw +4.9% (95%: −0.1% to +10.0%); LOSS · 23.99 → 25.28; raw +5.4% (95%: +1.7% to +9.1%); LOSS · 25.85 → 28; raw +8.3% (95%: −7.7% to +24.3%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_TOP.md`): failed requests (%), A B C 0.08233 → 0.159; raw +93.2% (95%: −3.5% to +189.8%), +0.0767 in the gauge's unit; LOSS · 0.0524 → 0.1452; raw +177.2% (95%: −58.9% to +413.2%), +0.0928 in the gauge's unit; LOSS · 0.06374 → 0.1831; raw +187.3% (95%: −70.3% to +444.8%), +0.1194 in the gauge's unit; LOSS. The knob: the muscle's floor and the trial's tolerance; a muscle that fails a request native served stays native until the gate reads no LOSS.
- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_TOP.md`): pod start wait, total (s), A B C 1.111 → 3.889; raw +250.0% (95%: −212.5% to +712.5%); LOSS · 1.1 → 1.9; raw +72.7% (95%: −145.8% to +291.2%); LOSS · 1.1 → 2.2; raw +100.0% (95%: −164.5% to +364.5%); LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_TOP.md`): pod start wait, mean (s), A B C 0.8333 → 1.101; raw +32.1% (95%: −147.0% to +211.2%), +0.2672 in the gauge's unit; LOSS · 0.65 → 1.033; raw +59.0% (95%: −121.0% to +238.9%), +0.3833 in the gauge's unit; LOSS · 0.85 → 1.225; raw +44.1% (95%: −131.4% to +219.6%), +0.3750 in the gauge's unit; LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_TOP.md`): batch: queue finished (s), A B C 952.2 → 993.1; raw +4.3% (95%: +1.8% to +6.8%); LOSS · 952.2 → 991; raw +4.1% (95%: +1.8% to +6.3%); LOSS · 1010 → 1040; raw +3.0% (95%: −0.3% to +6.4%); LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_TOP.md`): queue: job turnaround, submitted to done, mean (s), A B C 258.1 → 298.7; raw +15.7% (95%: +5.9% to +25.6%); LOSS · 261.5 → 298.5; raw +14.2% (95%: +6.1% to +22.2%); LOSS · 311.6 → 341.4; raw +9.6% (95%: −1.7% to +20.8%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Kubernetes with KEDA on the CPU target and the live requests in flight (omni-v21, `results/live/V21_KEDA_CPU_REQUESTS.md`): pod start wait, total (s), A B C 6.3 → 11.6; raw +84.1% (95%: +40.6% to +127.6%); LOSS · 8.1 → 10.6; raw +30.9% (95%: −7.6% to +69.3%); LOSS · 8 → 10.2; raw +27.5% (95%: −14.0% to +69.0%); LOSS. Cause: The pod start wait on one shared host: where it goes, why it rose, and the rig amendment of 18 October (`docs/POD_START_WAIT.md`). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Koordinator under the batch stream (omni-v12, `results/live/V12_KOORDINATOR.md`): pod start wait, total (s), A B C 0.2 → 0.8; raw +300.0% (95%: −264.2% to +864.2%), +0.6000 in the gauge's unit; LOSS · 0.2 → 0.8; raw +300.0% (95%: −45.5% to +645.5%), +0.6000 in the gauge's unit; LOSS · 0.2 → 0.9; raw +350.0% (95%: −304.1% to +1004.1%), +0.7000 in the gauge's unit; LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Koordinator under the batch stream (omni-v12, `results/live/V12_KOORDINATOR.md`): pod start wait, mean (s), A B C 0.2 → 0.6; raw +200.0% (95%: −219.8% to +619.8%), +0.4000 in the gauge's unit; LOSS · 0.2 → 0.8; raw +300.0% (95%: −45.5% to +645.5%), +0.6000 in the gauge's unit; LOSS · 0.2 → 0.65; raw +225.0% (95%: −224.6% to +674.6%), +0.4500 in the gauge's unit; LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_BARE.md`): response time (ms), 95th percentile, A B C 536.2 → 567.1; raw +5.8% (95%: +1.6% to +9.9%); LOSS · 1283 → 1372; raw +6.9% (95%: −1.2% to +15.1%); LOSS · 863.2 → 913.5; raw +5.8% (95%: −1.2% to +12.8%); LOSS. The knob: the compass

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.

- **Omni-Compass against native.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_BARE.md`): response time (ms), 99th percentile, A B C 767.9 → 818.7; raw +6.6% (95%: −4.7% to +17.9%); LOSS · 1874 → 2010; raw +7.3% (95%: −1.6% to +16.1%); LOSS · 1252 → 1296; raw +3.6% (95%: −1.3% to +8.4%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_BARE.md`): queue: job turnaround, submitted to done, mean (s), A B C 38.07 → 40.1; raw +5.3% (95%: −4.7% to +15.3%); LOSS · 85.79 → 89.12; raw +3.9% (95%: −0.1% to +7.9%); LOSS · 55.45 → 59.44; raw +7.2% (95%: +0.2% to +14.2%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_TOP.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 0 → 0; raw ±0.0000 in the gauge's unit (95%: ±0.0000 to ±0.0000); STEADY · 0.1 → 0.2; raw +100.0% (95%: −306.0% to +506.0%), +0.1000 in the gauge's unit; LOSS · 0.1 → 0.1; raw ±0.0% (95%: −337.2% to +337.2%), ±0.0000 in the gauge's unit; STEADY. The knob: the muscle's floor and the trial's tolerance; a muscle that fails a request native served stays native until the gate reads no LOSS.
- **Omni-Compass against native.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_TOP.md`): batch: queue finished (s), A B C 1290 → 1313; raw +1.8% (95%: −3.7% to +7.2%); LOSS · 1421 → 1421; raw ±0.0% (95%: −0.8% to +0.8%); STEADY · 1396 → 1421; raw +1.8% (95%: −1.3% to +4.8%); LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_TOP.md`): queue: job turnaround, submitted to done, mean (s), A B C 614 → 650.2; raw +5.9% (95%: −2.3% to +14.1%); LOSS · 799.3 → 814.5; raw +1.9% (95%: −4.7% to +8.5%); LOSS · 763.3 → 789; raw +3.4% (95%: −1.1% to +7.8%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BARE.md`): response time (ms), 95th percentile, A B C 1132 → 1204; raw +6.3% (95%: −0.8% to +13.4%); LOSS · 1090 → 1166; raw +7.0% (95%: +2.5% to +11.4%); LOSS · 1496 → 1547; raw +3.4% (95%: −0.7% to +7.4%); LOSS. Cause: Amendment 10: what the counted runs of 20 October found (written after those runs, before the Gödel runs start again)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(docs/BATCH_QUEUE_PREREGISTRATION.md). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.

- **Omni-Compass against native.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, results/live/V12_KUBERAY_4W_BARE.md): response time (ms), 99th percentile, A B C 1583 → 1738; raw +9.8% (95%: +1.5% to +18.0%); LOSS · 1617 → 1688; raw +4.4% (95%: +0.1% to +8.7%); LOSS · 2030 → 2294; raw +13.0% (95%: +4.7% to +21.2%); LOSS. Cause: Amendment 10: what the counted runs of 20 October found (written after those runs, before the Gödel runs start again) (docs/BATCH_QUEUE_PREREGISTRATION.md). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, results/live/V12_KUBERAY_4W_BARE.md): pod start wait, total (s), A B C 0 → 0.5556; raw +0.5556 in the gauge's unit (95%: −0.3133 to +1.4244); LOSS · 0 → 0.5; raw +0.5000 in the gauge's unit (95%: −0.1952 to +1.1952); LOSS · 0 → 0.6; raw +0.6000 in the gauge's unit (95%: −0.1689 to +1.3689); LOSS. Cause: Amendment 10: what the counted runs of 20 October found (written after those runs, before the Gödel runs start again) (docs/BATCH_QUEUE_PREREGISTRATION.md). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (docs/POD_START_WAIT.md); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, results/live/V12_KUBERAY_4W_BARE.md): pod start wait, mean (s), A B C 0 → 0.5556; raw +0.5556 in the gauge's unit (95%: −0.3133 to +1.4244); LOSS · 0 → 0.35; raw +0.3500 in the gauge's unit (95%: −0.0647 to +0.7647); LOSS · 0 → 0.6; raw +0.6000 in the gauge's unit (95%: −0.1689 to +1.3689); LOSS. Cause: Amendment 10: what the counted runs of 20 October found (written after those runs, before the Gödel runs start again) (docs/BATCH_QUEUE_PREREGISTRATION.md). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (docs/POD_START_WAIT.md); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, results/live/V12_KUBERAY_4W_BARE.md): batch: queue finished (s), A B C 774.1 → 780.1; raw +0.8% (95%: +0.1% to +1.5%); LOSS · 775.3 → 780.5; raw +0.7% (95%: +0.2% to +1.2%); LOSS · 802.6 → 807.7; raw +0.6% (95%: +0.2% to +1.1%); LOSS. Cause: Amendment 10: what the counted runs of 20 October found (written after those runs, before the Gödel runs start again) (docs/BATCH_QUEUE_PREREGISTRATION.md). The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, results/live/V12_KUBERAY_4W_BARE.md): queue: job wait, submitted to its first pod running, mean (s), A B C 0.03241 → 0.05093; raw +57.1% (95%: −54.6% to +168.9%), +0.0185 in the gauge's unit; LOSS · 0.04167 → 0.09167; raw +120.0% (95%: −61.0% to +301.0%), +0.0500 in the gauge's unit; LOSS · 0.05417 → 0.09583; raw +76.9% (95%: −65.1% to +219.0%), +0.0417 in the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

gauge's unit; LOSS. Cause: Amendment 10: what the counted runs of 20 October found (written after those runs, before the Gödel runs start again) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.

- **Omni-Compass against native.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BARE.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.1111 → 0.3333; raw +200.0% (95%: −105.1% to +505.1%), +0.2222 in the gauge's unit; LOSS · 0.4 → 0.4; raw ±0.0% (95%: −119.2% to +119.2%), ±0.0000 in the gauge's unit; STEADY · 0.4 → 0.6; raw +50.0% (95%: −25.4% to +125.4%), +0.2000 in the gauge's unit; LOSS. Cause: Amendment 10: what the counted runs of 20 October found (written after those runs, before the Gödel runs start again) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BARE.md`): queue: job turnaround, submitted to done, mean (s), A B C 81.79 → 87.35; raw +6.8% (95%: −0.6% to +14.2%); LOSS · 83.3 → 87.46; raw +5.0% (95%: −0.2% to +10.2%); LOSS · 107.6 → 114.2; raw +6.1% (95%: +1.9% to +10.4%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_TOP.md`): response time (ms), 95th percentile, A B C 1305 → 1356; raw +3.9% (95%: −6.9% to +14.7%); LOSS · 1271 → 1343; raw +5.6% (95%: −4.2% to +15.5%); LOSS · 1432 → 1440; raw +0.5% (95%: −5.9% to +7.0%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_TOP.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 635 → 637.5; raw +0.4% (95%: −8.5% to +9.2%); STEADY · 533.6 → 536.6; raw +0.6% (95%: −7.9% to +9.1%); LOSS · 670.7 → 677.6; raw +1.0% (95%: −3.7% to +5.7%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BARE.md`): response time (ms), 95th percentile, A B C 1006 → 1044; raw +3.8% (95%: −2.2% to +9.8%); LOSS · 1151 → 1169; raw +1.6% (95%: −5.0% to +8.2%); LOSS · 1278 → 1334; raw +4.4% (95%: −5.1% to +13.8%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Omni-Compass against native.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BARE.md`): pod start wait, total (s), A B C 0 → 0; raw ±0.0000 in the gauge's unit (95%: ±0.0000 to ±0.0000); STEADY · 0 → 0.3; raw +0.3000 in the gauge's unit (95%: −0.3786 to +0.9786); LOSS · 0 → 0; raw ±0.0000 in the gauge's unit (95%: ±0.0000 to ±0.0000); STEADY. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BARE.md`): pod start wait, mean (s), A B C 0 → 0; raw ±0.0000 in the gauge's unit (95%: ±0.0000 to ±0.0000); STEADY · 0 → 0.3; raw +0.3000 in the gauge's unit (95%: −0.3786 to +0.9786); LOSS · 0 → 0; raw ±0.0000 in the gauge's unit (95%: ±0.0000 to ±0.0000); STEADY. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BARE.md`): queue: job turnaround, submitted to done, mean (s), A B C 73.89 → 79.93; raw +8.2% (95%: +0.2% to +16.1%); LOSS · 81.23 → 83.89; raw +3.3% (95%: −1.2% to +7.8%); LOSS · 87.61 → 92.22; raw +5.3% (95%: −2.3% to +12.8%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_TOP.md`): failed requests (%), A B C 0 → 0; raw ±0.0000 in the gauge's unit (95%: ±0.0000 to ±0.0000); STEADY · 0 → 0.009302; raw +0.0093 in the gauge's unit (95%: −0.0117 to +0.0303); LOSS · 0 → 0; raw ±0.0000 in the gauge's unit (95%: ±0.0000 to ±0.0000); STEADY. The knob: the muscle's floor and the trial's tolerance; a muscle that fails a request native served stays native until the gate reads no LOSS.
- **Omni-Compass against native.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_TOP.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 7.8 → 8; raw +2.6% (95%: −8.7% to +13.8%); LOSS · 10.3 → 10.6; raw +2.9% (95%: −1.8% to +7.6%); LOSS · 11.11 → 11.33; raw +2.0% (95%: −1.1% to +5.1%); LOSS. The knob: the muscle's floor and the trial's tolerance; a muscle that fails a request native served stays native until the gate reads no LOSS.
- **Omni-Compass against native.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_TOP.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.1625 → 0.1667; raw +2.6% (95%: −75.0% to +80.1%), +0.0042 in the gauge's unit; LOSS · 0.2 → 0.2; raw ±0.0% (95%: −63.2% to +63.2%), ±0.0000 in the gauge's unit; STEADY · 0.2361 → 0.2361; raw ±0.0% (95%: −17.9% to +17.9%), ±0.0000 in the gauge's unit; STEADY. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Omni-Compass against native.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_TOP.md`): queue: job turnaround, submitted to done, mean (s), A B C 82.38 → 83.16; raw +0.9% (95%: −5.4% to +7.3%); LOSS · 88.75 → 92.45; raw +4.2% (95%: −2.2% to +10.6%); LOSS · 98.69 → 102.1; raw +3.5% (95%: −2.7% to +9.6%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BARE.md`): response time (ms), 95th percentile, A B C 2969 → 3098; raw +4.3% (95%: −0.7% to +9.4%); LOSS · 2682 → 2784; raw +3.8% (95%: −4.2% to +11.8%); LOSS · 2493 → 2538; raw +1.8% (95%: −8.0% to +11.6%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BARE.md`): response time (ms), 99th percentile, A B C 4041 → 4247; raw +5.1% (95%: −6.1% to +16.3%); LOSS · 3704 → 3891; raw +5.1% (95%: −3.0% to +13.1%); LOSS · 3409 → 3512; raw +3.0% (95%: −4.7% to +10.7%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BARE.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 14.9 → 15.3; raw +2.7% (95%: +0.2% to +5.2%); LOSS · 12.4 → 12.7; raw +2.4% (95%: −0.4% to +5.2%); LOSS · 10.1 → 11.2; raw +10.9% (95%: −0.9% to +22.7%); LOSS. The knob: the muscle's floor and the trial's tolerance; a muscle that fails a request native served stays native until the gate reads no LOSS.
- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BARE.md`): pod start wait, total (s), A B C 0.8 → 1.7; raw +112.5% (95%: −108.3% to +333.3%), +0.9000 in the gauge's unit; LOSS · 0.2 → 2.4; raw +1100.0% (95%: +243.6% to +1956.4%), +2.2000 in the gauge's unit; LOSS · 0.6 → 1.3; raw +116.7% (95%: −70.2% to +303.5%), +0.7000 in the gauge's unit; LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BARE.md`): pod start wait, mean (s), A B C 0.55 → 1.1; raw +100.0% (95%: −102.1% to +302.1%), +0.5500 in the gauge's unit; LOSS · 0.2 → 1.25; raw +525.0% (95%: +52.2% to +997.8%), +1.0500 in the gauge's unit; LOSS · 0.4 → 0.8; raw +100.0% (95%: −44.8% to +244.8%), +0.4000 in the gauge's unit; LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BARE.md`): batch: queue finished (s), A B C 911.9 → 932.9; raw +2.3% (95%: +0.9% to +3.7%); LOSS · 885.4 → 898.2; raw +1.4% (95%: +0.3% to +2.6%); LOSS · 881.5 → 890.5; raw +1.0% (95%: −0.3% to +2.4%); LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BARE.md`): queue: job turnaround, submitted to done, mean (s), A B C 222.3 → 236.9; raw +6.5% (95%: +3.2% to +9.9%); LOSS · 194.8 → 204.3; raw +4.9% (95%: −0.2% to +9.9%); LOSS · 190.6 → 195; raw +2.3% (95%: −2.9% to +7.5%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_TOP.md`): response time (ms), 95th percentile, A B C 4119 → 4499; raw +9.2% (95%: −1.6% to +20.0%); LOSS · 4095 → 4162; raw +1.6% (95%: −12.0% to +15.2%); LOSS · 2738 → 3866; raw +41.2% (95%: +1.6% to +80.7%); LOSS. Cause: Alert minutes under Open Cluster Management, Omni-Compass on top (`V12_OCM_4W_TOP.md`): under investigation since 21 October 2026 (`docs/LOSS_ROWS_UNDER_INVESTIGATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_TOP.md`): response time (ms), 99th percentile, A B C 5429 → 6036; raw +11.2% (95%: −2.0% to +24.3%); LOSS · 5491 → 5867; raw +6.9% (95%: −7.7% to +21.4%); LOSS · 3776 → 5213; raw +38.0% (95%: +0.1% to +76.0%); LOSS. Cause: Alert minutes under Open Cluster Management, Omni-Compass on top (`V12_OCM_4W_TOP.md`): under investigation since 21 October 2026 (`docs/LOSS_ROWS_UNDER_INVESTIGATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_TOP.md`): failed requests (%), A B C 0.3539 → 0.4937; raw +39.5% (95%: −46.7% to +125.8%), +0.1399 in the gauge's unit; LOSS · 0.05059 → 0.1132; raw +123.7% (95%: −203.3% to +450.7%), +0.0626 in the gauge's unit; LOSS · 0.01653 → 0.05082; raw +207.5% (95%: −260.7% to +675.6%), +0.0343 in the gauge's unit; LOSS. The knob: the muscle's floor and the trial's tolerance; a muscle that fails a request native served stays native until the gate reads no LOSS.
- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_TOP.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 17.6 → 18.6; raw

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

+5.7% (95%: +1.4% to +10.0%); LOSS · 14.8 → 15.7; raw +6.1% (95%: +0.8% to +11.4%); LOSS · 12.1 → 17.6; raw +45.5% (95%: +5.7% to +85.2%); LOSS. Cause: Alert minutes under Open Cluster Management, Omni-Compass on top (`V12_OCM_4W_TOP.md`): under investigation since 21 October 2026 (`docs/LOSS_ROWS_UNDER_INVESTIGATION.md`). The knob: the muscle's floor and the trial's tolerance; a muscle that fails a request native served stays native until the gate reads no LOSS.

- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_TOP.md`): pod start wait, total (s), A B C 0.3 → 2.9; raw +866.7% (95%: +127.9% to +1605.4%), +2.6000 in the gauge's unit; LOSS · 0 → 2.9; raw +2.9000 in the gauge's unit (95%: +1.1014 to +4.6986); LOSS · 0 → 5; raw +5.0000 in the gauge's unit (95%: −0.4892 to +10.4892); LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_TOP.md`): pod start wait, mean (s), A B C 0.3 → 1.55; raw +416.7% (95%: +69.1% to +764.2%), +1.2500 in the gauge's unit; LOSS · 0 → 1.617; raw +1.6167 in the gauge's unit (95%: +0.8358 to +2.3975); LOSS · 0 → 2.467; raw +2.4667 in the gauge's unit (95%: −0.2772 to +5.2106); LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_TOP.md`): batch: queue finished (s), A B C 1049 → 1107; raw +5.5% (95%: +2.2% to +8.9%); LOSS · 1026 → 1058; raw +3.0% (95%: +1.0% to +5.0%); LOSS · 1023 → 1050; raw +2.6% (95%: −4.4% to +9.5%); LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_TOP.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 7.229 → 22.47; raw +210.8% (95%: −13.9% to +435.5%); LOSS · 6.929 → 18.62; raw +168.7% (95%: −49.3% to +386.7%); LOSS · 3.13 → 14.34; raw +358.1% (95%: −20.3% to +736.6%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_TOP.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 39.4 → 159.5; raw +304.8% (95%: −8.1% to +617.8%); LOSS · 55.9 → 105.6; raw +88.9% (95%: −32.2% to +210.0%); LOSS · 6.333 → 94.33; raw +1389.5% (95%: −119.4% to +2898.4%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Omni-Compass against native.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_TOP.md`): queue: job turnaround, submitted to done, mean (s), A B C 328.8 → 380; raw +15.6% (95%: +2.8% to +28.4%); LOSS · 305.8 → 326.8; raw +6.9% (95%: −5.0% to +18.7%); LOSS · 258.9 → 309.6; raw +19.6% (95%: +8.1% to +31.0%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Robustness (omni-v21, `results/live/V21_ROBUST_KILL.md`): pod start wait, total (s), A B C 6.9 → 7.6; raw +10.1% (95%: −37.0% to +57.3%); LOSS · 8.2 → 10.2; raw +24.4% (95%: −63.1% to +111.9%); LOSS · 8.4 → 15; raw +78.6% (95%: +16.2% to +140.9%); LOSS. Cause: 1. The pod start wait: thirteen rows in eight tables (`docs/LOSS_CAUSES_2026-10-31.md`). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Robustness (omni-v21, `results/live/V21_ROBUST_LONG.md`): pod start wait, total (s), A B C 10 → 19; raw +90.0% (95%: −18.3% to +198.3%); LOSS · 7.667 → 24.67; raw +221.7% (95%: −317.6% to +761.1%); LOSS · 8.333 → 35.33; raw +324.0% (95%: −244.8% to +892.8%); LOSS. Cause: 1. The pod start wait: thirteen rows in eight tables (`docs/LOSS_CAUSES_2026-10-31.md`). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Robustness (omni-v21, `results/live/V21_ROBUST_LONG.md`): pod start wait, mean (s), A B C 1.279 → 1.591; raw +24.5% (95%: −69.7% to +118.6%); LOSS · 1.444 → 1.834; raw +27.0% (95%: −100.1% to +154.0%); LOSS · 1.38 → 1.732; raw +25.6% (95%: −32.6% to +83.7%); LOSS. Cause: 1. The pod start wait: thirteen rows in eight tables (`docs/LOSS_CAUSES_2026-10-31.md`). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** A public day of demand (omni-v21, `results/live/V21_TRACE_GOOGLE2011.md`): pod start wait, total (s), A B C 10 → 20.9; raw +109.0% (95%: +31.2% to +186.8%); LOSS · 10.7 → 20.1; raw +87.9% (95%: +13.7% to +162.0%); LOSS · 7.7 → 29.2; raw +279.2% (95%: +180.7% to +377.8%); LOSS. Cause: The pod start wait on one shared host: where it goes, why it rose, and the rig amendment of 18 October (`docs/POD_START_WAIT.md`). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** The Kubernetes scheduler as shipped under the batch stream (omni-v12, `results/live/V12_VANILLA.md`): response time (ms), 95th percentile, A B C 523.3 → 522.3; raw −0.2% (95%: −5.3% to +4.9%); STEADY · 417.2 → 455; raw +9.1% (95%: +3.9% to +14.2%);

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

LOSS · 469.7 → 494.8; raw +5.3% (95%: −1.2% to +11.9%); LOSS. Cause: Amendment 5: the governor's own core (written before any run on it, 20 October 2026) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.

- **Omni-Compass against native.** The Kubernetes scheduler as shipped under the batch stream (omni-v12, `results/live/V12_VANILLA.md`): response time (ms), 99th percentile, A B C 672.1 → 678.9; raw +1.0% (95%: −2.6% to +4.6%); LOSS · 553.2 → 584.8; raw +5.7% (95%: +1.0% to +10.4%); LOSS · 621.8 → 650.1; raw +4.5% (95%: +1.1% to +8.0%); LOSS. Cause: Amendment 5: the governor's own core (written before any run on it, 20 October 2026) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** The Kubernetes scheduler as shipped under the batch stream (omni-v12, `results/live/V12_VANILLA.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 9.7 → 9.7; raw ±0.0% (95%: −4.9% to +4.9%); STEADY · 4.5 → 5.8; raw +28.9% (95%: −3.8% to +61.6%); LOSS · 6.8 → 8.3; raw +22.1% (95%: −5.7% to +49.8%); LOSS. The knob: the muscle's floor and the trial's tolerance; a muscle that fails a request native served stays native until the gate reads no LOSS.
- **Omni-Compass against native.** The Kubernetes scheduler as shipped under the batch stream (omni-v12, `results/live/V12_VANILLA.md`): queue: job turnaround, submitted to done, mean (s), A B C 28.22 → 28.48; raw +0.9% (95%: +0.2% to +1.7%); LOSS · 25.72 → 25.95; raw +0.9% (95%: +0.1% to +1.6%); LOSS · 27.32 → 27.65; raw +1.2% (95%: +0.6% to +1.8%); LOSS. Cause: Amendment 5: the governor's own core (written before any run on it, 20 October 2026) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_BARE.md`): response time (ms), 95th percentile, A B C 926.1 → 996.5; raw +7.6% (95%: +0.8% to +14.5%); LOSS · 1133 → 1167; raw +2.9% (95%: −2.9% to +8.8%); LOSS · 1065 → 1145; raw +7.5% (95%: −0.1% to +15.1%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_BARE.md`): response time (ms), 99th percentile, A B C 1337 → 1467; raw +9.7% (95%: +1.9% to +17.5%); LOSS · 1627 → 1620; raw −0.4% (95%: −10.6% to +9.7%); STEADY · 1483 → 1638; raw +10.4% (95%: −0.9% to +21.7%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Omni-Compass against native.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_BARE.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.3 → 0.3; raw ±0.0% (95%: −159.0% to +159.0%), ±0.0000 in the gauge's unit; STEADY · 0.4 → 0.5; raw +25.0% (95%: −76.5% to +126.5%), +0.1000 in the gauge's unit; LOSS · 0.2 → 0.5; raw +150.0% (95%: −91.4% to +391.4%), +0.3000 in the gauge's unit; LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_BARE.md`): queue: job turnaround, submitted to done, mean (s), A B C 65.63 → 70.4; raw +7.3% (95%: +1.1% to +13.4%); LOSS · 85.9 → 87.21; raw +1.5% (95%: −1.8% to +4.8%); LOSS · 75.65 → 82.43; raw +9.0% (95%: +1.5% to +16.5%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_TOP.md`): response time (ms), 95th percentile, A B C 517.4 → 518.9; raw +0.3% (95%: −21.7% to +22.3%); STEADY · 884.6 → 1003; raw +13.4% (95%: −8.4% to +35.1%); LOSS · 758.9 → 839.4; raw +10.6% (95%: −7.0% to +28.2%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_TOP.md`): pod start wait, total (s), A B C 0.2 → 0.4; raw +100.0% (95%: −306.0% to +506.0%), +0.2000 in the gauge's unit; LOSS · 0.1 → 1.4; raw +1300.0% (95%: −1085.5% to +3685.5%), +1.3000 in the gauge's unit; LOSS · 0 → 1.2; raw +1.2000 in the gauge's unit (95%: −0.3004 to +2.7004); LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_TOP.md`): pod start wait, mean (s), A B C 0.2 → 0.4; raw +100.0% (95%: −306.0% to +506.0%), +0.2000 in the gauge's unit; LOSS · 0.1 → 0.7; raw +600.0% (95%: −625.1% to +1825.1%), +0.6000 in the gauge's unit; LOSS · 0 → 1.2; raw +1.2000 in the gauge's unit (95%: −0.3004 to +2.7004); LOSS. The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_TOP.md`): queue: job turnaround, submitted to done, mean (s), A B C 81.51 → 86.72; raw +6.4% (95%: +0.7% to +12.1%); LOSS · 101 → 102.8; raw +1.8% (95%: −4.5% to +8.1%); LOSS · 93.17 → 95.34; raw +2.3% (95%: −2.1% to +6.8%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Omni-Compass against native.** Kafka, a consumer group's operator-set size, resource objective (omni-v21, `results/live/V21_KAFKA.md`): end-to-end latency, mean (ms), A B C 341 → 339.6; raw −0.4% (95%: −8.4% to +7.6%); STEADY · 284.4 → 293.1; raw +3.0% (95%: −6.0% to +12.0%); LOSS · 214.2 → 220.5; raw +2.9% (95%: −20.8% to +26.7%); LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** Kafka, a consumer group's operator-set size, resource objective (omni-v21, `results/live/V21_KAFKA.md`): consumer lag, mean messages waiting, A B C 62.6 → 62.55; raw −0.1% (95%: −6.1% to +5.9%); STEADY · 52.12 → 53.75; raw +3.1% (95%: −5.2% to +11.5%); LOSS · 39.89 → 41.44; raw +3.9% (95%: −23.7% to +31.5%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Kafka, a consumer group's operator-set size, resource objective (omni-v21, `results/live/V21_KAFKA.md`): end-to-end latency, 99th percentile (ms), A B C 3736 → 3759; raw +0.6% (95%: −4.3% to +5.5%); LOSS · 3649 → 3679; raw +0.8% (95%: −3.8% to +5.4%); LOSS · 2394 → 2611; raw +9.1% (95%: −14.1% to +32.2%); LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** Kafka, a consumer group's operator-set size, resource objective (omni-v21, `results/live/V21_KAFKA.md`): end-to-end latency, mean (ms), A B C 318.8 → 323.5; raw +1.5% (95%: −3.7% to +6.6%); LOSS · 326.7 → 326.9; raw +0.1% (95%: −10.7% to +10.8%); STEADY · 210.7 → 235.5; raw +11.8% (95%: −13.9% to +37.4%); LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** Kafka, a consumer group's operator-set size, resource objective (omni-v21, `results/live/V21_KAFKA.md`): consumer lag, mean messages waiting, A B C 277.4 → 281.1; raw +1.3% (95%: −4.4% to +7.1%); LOSS · 287.4 → 287.7; raw +0.1% (95%: −10.3% to +10.5%); STEADY · 184.6 → 205.4; raw +11.2% (95%: −13.1% to +35.6%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Kafka, a consumer group's operator-set size, service objective (omni-v21, `results/live/V21_KAFKA_SERVICE.md`): end-to-end latency, mean (ms), A B C 362.2 → 371.4; raw +2.5% (95%: −4.5% to +9.6%); LOSS · 385.8 → 384.7; raw −0.3% (95%: −3.4% to +2.8%); STEADY · 405.4 → 418.6; raw +3.3% (95%: −3.3% to +9.8%); LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** Kafka, a consumer group's operator-set size, service objective (omni-v21, `results/live/V21_KAFKA_SERVICE.md`): consumer lag, most messages waiting at

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

once, A B C 1565 → 1574; raw +0.5% (95%: −6.5% to +7.5%); LOSS · 1656 → 1678; raw +1.3% (95%: −4.8% to +7.4%); LOSS · 1737 → 1788; raw +2.9% (95%: +1.9% to +3.9%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.

- **Omni-Compass against native.** Kafka, a consumer group's operator-set size, service objective (omni-v21, `results/live/V21_KAFKA_SERVICE.md`): consumer lag, mean messages waiting, A B C 158 → 162.6; raw +2.9% (95%: −4.1% to +9.9%); LOSS · 167.4 → 167; raw −0.2% (95%: −5.9% to +5.4%); STEADY · 180.1 → 184.5; raw +2.4% (95%: −4.4% to +9.3%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Kafka, a consumer group's operator-set size, service objective (omni-v21, `results/live/V21_KAFKA_SERVICE.md`): end-to-end latency, mean (ms), A B C 289.1 → 287.8; raw −0.4% (95%: −2.4% to +1.5%); STEADY · 347.2 → 354; raw +1.9% (95%: −9.9% to +13.8%); LOSS · 328.8 → 337.7; raw +2.7% (95%: −4.3% to +9.8%); LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** Kafka, a consumer group's operator-set size, service objective (omni-v21, `results/live/V21_KAFKA_SERVICE.md`): consumer lag, most messages waiting at once, A B C 646.3 → 654; raw +1.2% (95%: −14.0% to +16.4%); LOSS · 731.7 → 741.3; raw +1.3% (95%: −4.3% to +6.9%); LOSS · 695.7 → 706; raw +1.5% (95%: −7.4% to +10.4%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Kafka, a consumer group's operator-set size, service objective (omni-v21, `results/live/V21_KAFKA_SERVICE.md`): consumer lag, mean messages waiting, A B C 52.71 → 52.45; raw −0.5% (95%: −8.2% to +7.2%); STEADY · 63.84 → 65.04; raw +1.9% (95%: −10.3% to +14.0%); LOSS · 60.45 → 61.5; raw +1.7% (95%: −2.4% to +5.9%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Kafka, a consumer group's operator-set size, service objective (omni-v21, `results/live/V21_KAFKA_SERVICE.md`): work inside the response line (messages a second handled within the line), A B C 724.8 → 715.5; raw −1.3% (95%: −3.1% to +0.5%); LOSS · 725.4 → 720.8; raw −0.6% (95%: −2.8% to +1.5%); LOSS · 723.4 → 720.3; raw −0.4% (95%: −2.2% to +1.3%); STEADY. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** Kafka, a consumer group's operator-set size, service objective (omni-v21, `results/live/V21_KAFKA_SERVICE.md`): end-to-end latency, 95th percentile (ms), A B C 2272 → 2293; raw +0.9% (95%: −6.0% to +7.9%); LOSS · 2181 → 2238; raw +2.6% (95%: −0.7% to +5.9%); LOSS · 2100 → 2098; raw −0.1% (95%: −2.6% to +2.4%); STEADY. The knob: the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.

- **Omni-Compass against native.** Kafka, a consumer group's operator-set size, service objective (omni-v21, `results/live/V21_KAFKA_SERVICE.md`): end-to-end latency, mean (ms), A B C 304.5 → 320.9; raw +5.4% (95%: −9.7% to +20.5%); LOSS · 298.5 → 310.1; raw +3.9% (95%: −3.9% to +11.7%); LOSS · 274.6 → 279.5; raw +1.8% (95%: −7.6% to +11.2%); LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** Kafka, a consumer group's operator-set size, service objective (omni-v21, `results/live/V21_KAFKA_SERVICE.md`): consumer lag, mean messages waiting, A B C 266.5 → 277.3; raw +4.0% (95%: −11.0% to +19.1%); LOSS · 259.2 → 268.7; raw +3.7% (95%: −4.4% to +11.7%); LOSS · 235.9 → 239.9; raw +1.7% (95%: −7.3% to +10.7%); LOSS. The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row STEADY in the model gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Redis, a cache's operator-set memory ceiling, resource objective (omni-v21, `results/live/V21_REDIS.md`): latency, 99th percentile (ms), A B C 5.74 → 5.75; raw +0.2% (95%: −0.3% to +0.6%); STEADY · 5.739 → 5.749; raw +0.2% (95%: +0.1% to +0.3%); STEADY · 5.366 → 5.458; raw +1.7% (95%: +0.4% to +3.1%); LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** Redis, a cache's operator-set memory ceiling, service objective (omni-v21, `results/live/V21_REDIS_SERVICE.md`): latency, 99th percentile (ms), A B C 5.691 → 5.713; raw +0.4% (95%: −0.7% to +1.5%); STEADY · 5.339 → 5.393; raw +1.0% (95%: −2.2% to +4.2%); LOSS · 5.754 → 5.768; raw +0.2% (95%: ±0.0% to +0.5%); STEADY. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** sysbench on MySQL, a database's buffer pool as shipped, resource objective (omni-v21, `results/live/V21_SYSBENCH_SHIPPED.md`): latency, 95th percentile (ms), A B C 5.57 → 5.57; raw ±0.0% (95%: ±0.0% to ±0.0%); STEADY · 15.38 → 54.62; raw +255.1% (95%: −554.4% to +1064.6%); LOSS · 5.438 → 5.471; raw +0.6% (95%: −2.0% to +3.3%); LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** sysbench on MySQL, a database's buffer pool as shipped, resource objective (omni-v21, `results/live/V21_SYSBENCH_SHIPPED.md`): latency, mean (ms), A B C 3.052 → 3.041; raw −0.4% (95%: −4.6% to +3.9%); STEADY · 2.3 → 2.499; raw +8.7% (95%: −40.0% to +57.4%); LOSS · 3.347 → 3.344; raw −0.1% (95%: −1.7% to +1.5%); STEADY. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Omni-Compass against native.** sysbench on MySQL, a database's buffer pool as shipped, resource objective (omni-v21, `results/live/V21_SYSBENCH_SHIPPED.md`): latency, mean (ms), A B C 1.046 → 1.064; raw +1.6% (95%: −17.8% to +21.1%); LOSS · 1.143 → 1.175; raw +2.8% (95%: −22.6% to +28.2%); LOSS · 0.886 → 0.9053; raw +2.2% (95%: −4.1% to +8.4%), +0.0192 in the gauge's unit; LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** YCSB on MongoDB, a database's operator-set cache size, resource objective (omni-v21, `results/live/V21_YCSB.md`): latency, 95th percentile (ms), A B C 0.4083 → 0.409; raw +0.2% (95%: −2.9% to +3.2%), +0.0007 in the gauge's unit; STEADY · 0.3047 → 0.321; raw +5.4% (95%: −7.8% to +18.6%), +0.0163 in the gauge's unit; LOSS · 0.2703 → 0.2723; raw +0.7% (95%: −1.1% to +2.6%), +0.0020 in the gauge's unit; LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** YCSB on MongoDB, a database's operator-set cache size, resource objective (omni-v21, `results/live/V21_YCSB.md`): latency, 99th percentile (ms), A B C 0.577 → 0.5793; raw +0.4% (95%: −4.4% to +5.2%), +0.0023 in the gauge's unit; STEADY · 0.462 → 0.4737; raw +2.5% (95%: −0.4% to +5.5%), +0.0117 in the gauge's unit; LOSS · 0.4513 → 0.459; raw +1.7% (95%: −2.5% to +5.9%), +0.0077 in the gauge's unit; LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** YCSB on MongoDB, a database's operator-set cache size, service objective (omni-v21, `results/live/V21_YCSB_SERVICE.md`): latency, 95th percentile (ms), A B C 0.38 → 0.3807; raw +0.2% (95%: −7.1% to +7.5%), +0.0007 in the gauge's unit; STEADY · 0.3177 → 0.321; raw +1.0% (95%: −1.2% to +3.3%), +0.0033 in the gauge's unit; LOSS · 0.1807 → 0.1817; raw +0.6% (95%: −11.7% to +12.8%), +0.0010 in the gauge's unit; LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** YCSB on MongoDB, a database's operator-set cache size, service objective (omni-v21, `results/live/V21_YCSB_SERVICE.md`): latency, mean (ms), A B C 0.2119 → 0.2128; raw +0.4% (95%: −6.9% to +7.8%), +0.0009 in the gauge's unit; STEADY · 0.1874 → 0.1905; raw +1.7% (95%: −1.8% to +5.1%), +0.0031 in the gauge's unit; LOSS · 0.09107 → 0.09106; raw ±0.0% (95%: −9.7% to +9.7%), ±0.0000 in the gauge's unit; STEADY. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** YCSB on MongoDB, a database's operator-set cache size, native as shipped (omni-v21, `results/live/V21_YCSB_SHIPPED.md`): latency, 95th percentile (ms), A B C 0.3097 → 0.3277; raw +5.8% (95%: −8.1% to +19.7%), +0.0180 in the gauge's unit; LOSS · 0.1267 → 0.1553; raw +22.6% (95%: −26.5% to +71.7%), +0.0287 in the gauge's unit; LOSS · 0.2133 → 0.2173; raw +1.9% (95%: −9.9% to +13.7%), +0.0040 in the gauge's unit; LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Omni-Compass against native.** YCSB on MongoDB, a database's operator-set cache size, native as shipped (omni-v21, `results/live/V21_YCSB_SHIPPED.md`): latency, 99th percentile (ms), A B C 0.5047 → 0.515; raw +2.0% (95%: −5.1% to +9.2%), +0.0103 in the gauge's unit; LOSS · 0.3333 → 0.3507; raw +5.2% (95%: −2.2% to +12.6%), +0.0173 in the gauge's unit; LOSS · 0.4087 → 0.4177; raw +2.2% (95%: −10.4% to +14.9%), +0.0090 in the gauge's unit; LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** YCSB on MongoDB, a database's operator-set cache size, native as shipped (omni-v21, `results/live/V21_YCSB_SHIPPED.md`): latency, mean (ms), A B C 0.1866 → 0.1895; raw +1.6% (95%: −2.4% to +5.5%), +0.0029 in the gauge's unit; LOSS · 0.06213 → 0.06473; raw +4.2% (95%: −17.6% to +26.0%), +0.0026 in the gauge's unit; LOSS · 0.1527 → 0.1626; raw +6.5% (95%: −8.3% to +21.3%), +0.0099 in the gauge's unit; LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** YCSB on MongoDB, a database's operator-set cache size, native as shipped (omni-v21, `results/live/V21_YCSB_SHIPPED.md`): latency, 95th percentile (ms), A B C 0.378 → 0.385; raw +1.9% (95%: −2.7% to +6.5%), +0.0070 in the gauge's unit; LOSS · 0.4893 → 0.49; raw +0.1% (95%: −17.5% to +17.7%), +0.0007 in the gauge's unit; STEADY · 0.2417 → 0.252; raw +4.3% (95%: −6.4% to +15.0%), +0.0103 in the gauge's unit; LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** YCSB on MongoDB, a database's operator-set cache size, native as shipped (omni-v21, `results/live/V21_YCSB_SHIPPED.md`): latency, 99th percentile (ms), A B C 0.5347 → 0.559; raw +4.6% (95%: −6.6% to +15.7%), +0.0243 in the gauge's unit; LOSS · 0.7627 → 0.767; raw +0.6% (95%: −49.8% to +50.9%), +0.0043 in the gauge's unit; LOSS · 0.3813 → 0.3997; raw +4.8% (95%: −2.5% to +12.1%), +0.0183 in the gauge's unit; LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** YCSB on MongoDB, a database's operator-set cache size, native as shipped (omni-v21, `results/live/V21_YCSB_SHIPPED.md`): latency, mean (ms), A B C 0.1386 → 0.142; raw +2.5% (95%: −1.5% to +6.5%), +0.0034 in the gauge's unit; LOSS · 0.2326 → 0.234; raw +0.6% (95%: −16.5% to +17.7%), +0.0014 in the gauge's unit; LOSS · 0.07182 → 0.0752; raw +4.7% (95%: −3.9% to +13.3%), +0.0034 in the gauge's unit; LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** YCSB on MongoDB, a database's operator-set cache size, native as shipped (omni-v21, `results/live/V21_YCSB_SHIPPED.md`): latency, 95th percentile (ms), A B C 0.178 → 0.1793; raw +0.7% (95%: −2.2% to +3.7%), +0.0013 in the gauge's unit; LOSS · 0.294 → 0.2947; raw +0.2% (95%: −2.2% to +2.7%), +0.0007 in the gauge's unit; STEADY · 0.075 → 0.077; raw +2.7% (95%: −8.8% to +14.1%), +0.0020 in the gauge's unit; LOSS. The knob: the muscle's own card (its

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.

- **Omni-Compass against native.** YCSB on MongoDB, a database's operator-set cache size, native as shipped (omni-v21, `results/live/V21_YCSB_SHIPPED.md`): latency, mean (ms), A B C 0.1313 → 0.1317; raw +0.3% (95%: −2.5% to +3.1%), +0.0004 in the gauge's unit; STEADY · 0.1818 → 0.1839; raw +1.2% (95%: −3.7% to +6.0%), +0.0021 in the gauge's unit; LOSS · 0.04938 → 0.04998; raw +1.2% (95%: −2.3% to +4.8%), +0.0006 in the gauge's unit; LOSS. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **Omni-Compass against native.** YCSB on MongoDB, a database's operator-set cache size, native as shipped (omni-v21, `results/live/V21_YCSB_SHIPPED.md`): latency, 95th percentile (ms), A B C 0.4157 → 0.418; raw +0.6% (95%: −2.2% to +3.3%), +0.0023 in the gauge's unit; LOSS · 0.162 → 0.1657; raw +2.3% (95%: −1.6% to +6.1%), +0.0037 in the gauge's unit; LOSS · 0.3997 → 0.4007; raw +0.3% (95%: −1.0% to +1.5%), +0.0010 in the gauge's unit; STEADY. The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the model gate is refused and the muscle stays native.
- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_BOTH.md`): pod start wait, total (s), A B C 0.4444 → 2.222; raw +400.0% (95%: +25.3% to +774.7%), +1.7778 in the gauge's unit; LOSS · 0.3 → 2.3; raw +666.7% (95%: +176.7% to +1156.6%), +2.0000 in the gauge's unit; LOSS · 0.3 → 1.6; raw +433.3% (95%: −140.3% to +1007.0%), +1.3000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_BOTH.md`): pod start wait, mean (s), A B C 0.2222 → 1.167; raw +425.0% (95%: +53.6% to +796.4%), +0.9444 in the gauge's unit; LOSS · 0.3 → 1.4; raw +366.7% (95%: +136.3% to +597.0%), +1.1000 in the gauge's unit; LOSS · 0.15 → 0.8917; raw +494.4% (95%: −34.9% to +1023.8%), +0.7417 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_BOTH.md`): batch: queue finished (s), A B C 872.8 → 1504; raw +72.4% (95%: +61.8% to +82.9%); LOSS · 892.4 → 1505; raw +68.6% (95%: +58.6% to +78.6%); LOSS · 806 → 1505; raw +86.7% (95%: +75.6% to +97.9%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_BOTH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.09722 → 1158; raw +1190995.2% (95%: +1185985.7% to +1196004.8%), +1157.9120 in the gauge's unit; LOSS · 0.07083 → 1158; raw +1634917.6% (95%: +1630334.8% to +1639500.5%), +1158.0667 in the gauge's unit; LOSS · 0.0875 → 1159; raw

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

+1324419.0% (95%: +1319896.0% to +1328942.1%), +1158.8667 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_BOTH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.6667 → 1474; raw +221050.0% (95%: +220287.4% to +221812.6%), +1473.6667 in the gauge's unit; LOSS · 0.6 → 1474; raw +245633.3% (95%: +245069.1% to +246197.6%), +1473.8000 in the gauge's unit; LOSS · 0.6 → 1475; raw +245750.0% (95%: +245056.6% to +246443.4%), +1474.5000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_BOTH.md`): queue: job turnaround, submitted to done, mean (s), A B C 184.4 → 1158; raw +528.0% (95%: +478.0% to +578.0%); LOSS · 203.2 → 1158; raw +469.9% (95%: +426.8% to +513.0%); LOSS · 115.3 → 1159; raw +904.9% (95%: +825.1% to +984.6%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_BOTH.md`): queue: jobs unfinished at the window's end, A B C 0 → 24; raw +24.0000 in the gauge's unit (95%: +24.0000 to +24.0000); LOSS · 0 → 24; raw +24.0000 in the gauge's unit (95%: +24.0000 to +24.0000); LOSS · 0 → 24; raw +24.0000 in the gauge's unit (95%: +24.0000 to +24.0000); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_ORCH.md`): batch: queue finished (s), A B C 872.8 → 1507; raw +72.6% (95%: +62.2% to +83.1%); LOSS · 892.4 → 1503; raw +68.4% (95%: +58.3% to +78.6%); LOSS · 806 → 1505; raw +86.7% (95%: +75.4% to +98.1%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_ORCH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.09722 → 1160; raw +1193142.9% (95%: +1189052.7% to +1197233.0%), +1160.0000 in the gauge's unit; LOSS · 0.07083 → 1157; raw +1632935.3% (95%: +1629797.0% to +1636073.6%), +1156.6625 in the gauge's unit; LOSS · 0.0875 → 1158; raw +1323895.2% (95%: +1320577.4% to +1327213.0%), +1158.4083 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_ORCH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.6667 → 1476; raw +221366.7% (95%: +220793.7% to +221939.6%), +1475.7778 in the gauge's unit; LOSS · 0.6 → 1473; raw +245383.3% (95%: +244977.9% to +245788.8%), +1472.3000 in the gauge's unit; LOSS · 0.6 → 1475; raw +245666.7% (95%:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

+245189.8% to +246143.5%), +1474.0000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_ORCH.md`): queue: job turnaround, submitted to done, mean (s), A B C 184.4 → 1160; raw +529.1% (95%: +479.5% to +578.7%); LOSS · 203.2 → 1157; raw +469.3% (95%: +425.4% to +513.1%); LOSS · 115.3 → 1158; raw +904.5% (95%: +823.7% to +985.2%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_ORCH.md`): queue: jobs unfinished at the window's end, A B C 0 → 24; raw +24.0000 in the gauge's unit (95%: +24.0000 to +24.0000); LOSS · 0 → 24; raw +24.0000 in the gauge's unit (95%: +24.0000 to +24.0000); LOSS · 0 → 24; raw +24.0000 in the gauge's unit (95%: +24.0000 to +24.0000); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_VS.md`): response time (ms), mean, A B C 125.9 → 586.4; raw +365.9% (95%: +157.4% to +574.3%); LOSS · 137.3 → 669.1; raw +387.2% (95%: +211.6% to +562.8%); LOSS · 102.2 → 379.7; raw +271.6% (95%: +1.7% to +541.5%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_VS.md`): response time (ms), 95th percentile, A B C 171.6 → 2510; raw +1362.7% (95%: +633.9% to +2091.4%); LOSS · 186.5 → 2871; raw +1439.3% (95%: +847.3% to +2031.3%); LOSS · 141 → 1533; raw +986.8% (95%: +75.2% to +1898.5%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_VS.md`): response time (ms), 99th percentile, A B C 197.3 → 3535; raw +1691.7% (95%: +817.5% to +2565.8%); LOSS · 207.1 → 4014; raw +1837.8% (95%: +1123.2% to +2552.5%); LOSS · 162.2 → 2162; raw +1233.4% (95%: +145.9% to +2320.8%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_VS.md`): time over the response line (% of samples), A B C 0.004241 → 24.76; raw +583748.7% (95%: +317248.1% to +850249.4%), +24.7561 in the gauge's unit; LOSS · 0 → 27.76; raw +27.7590 in the gauge's unit (95%: +17.8729 to +37.6451); LOSS · 0 → 13.96; raw +13.9600 in the gauge's unit (95%: +2.2913 to +25.6287); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_VS.md`): failed requests (%), A B C 0 → 0.007637; raw +0.0076 in the gauge's unit (95%: −0.0100 to +0.0252); LOSS · 0 → 0; raw ±0.0000 in the gauge's

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

unit (95%: ±0.0000 to ±0.0000); STEADY · 0 → 0; raw ±0.0000 in the gauge's unit (95%: ±0.0000 to ±0.0000); STEADY. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Armada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_ARMADA_4W_VS.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 0 → 12.33; raw +12.3333 in the gauge's unit (95%: +7.0357 to +17.6310); LOSS · 0 → 14; raw +14.0000 in the gauge's unit (95%: +10.0676 to +17.9324); LOSS · 0 → 8.2; raw +8.2000 in the gauge's unit (95%: +2.5094 to +13.8906); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream, the four ways (omni-v12, `results/live/V12_BINPACK_4W_BOTH.md`): response time (ms), 95th percentile, A B C 1397 → 1543; raw +10.5% (95%: −0.8% to +21.8%); LOSS · 866.2 → 874.5; raw +1.0% (95%: −10.2% to +12.1%); LOSS · 827.2 → 887.2; raw +7.2% (95%: −3.3% to +17.7%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream, the four ways (omni-v12, `results/live/V12_BINPACK_4W_BOTH.md`): response time (ms), 99th percentile, A B C 2014 → 2300; raw +14.2% (95%: −2.7% to +31.2%); LOSS · 1230 → 1327; raw +7.9% (95%: −3.3% to +19.0%); LOSS · 1250 → 1410; raw +12.7% (95%: −3.6% to +29.1%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream, the four ways (omni-v12, `results/live/V12_BINPACK_4W_BOTH.md`): queue: job turnaround, submitted to done, mean (s), A B C 102.4 → 108.4; raw +5.9% (95%: −0.1% to +12.0%); LOSS · 63.56 → 65.27; raw +2.7% (95%: −0.5% to +5.8%); LOSS · 58.77 → 61.38; raw +4.4% (95%: −1.1% to +10.0%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream, the four ways (omni-v12, `results/live/V12_BINPACK_4W_ORCH.md`): response time (ms), 99th percentile, A B C 2014 → 2292; raw +13.8% (95%: −0.5% to +28.2%); LOSS · 1230 → 1412; raw +14.7% (95%: −3.2% to +32.7%); LOSS · 1250 → 1375; raw +9.9% (95%: −7.4% to +27.3%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream, the four ways (omni-v12, `results/live/V12_BINPACK_4W_VS.md`): time over the response line (% of samples), A B C 16.92 → 20.49; raw +21.1% (95%: +5.3% to +36.8%); LOSS · 8.854 → 12.15; raw +37.2% (95%: +12.4% to +61.9%); LOSS · 10.38 → 13.47; raw +29.8% (95%: +1.0% to +58.6%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream, the four ways (omni-v12, `results/live/V12_BINPACK_4W_VS.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 10.7 → 10.9; raw +1.9% (95%: −2.4% to +6.1%); LOSS · 6 → 7.7; raw +28.3% (95%: −3.0% to +59.6%); LOSS

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

· 7.9 → 8.6; raw +8.9% (95%: −4.7% to +22.4%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream, the four ways (omni-v12, `results/live/V12_BINPACK_4W_VS.md`): batch: queue finished (s), A B C 796.8 → 799.8; raw +0.4% (95%: −0.7% to +1.4%); STEADY · 756.2 → 761.1; raw +0.6% (95%: −0.2% to +1.4%); LOSS · 748.4 → 754.2; raw +0.8% (95%: −0.2% to +1.7%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream, the four ways (omni-v12, `results/live/V12_BINPACK_4W_VS.md`): queue: job turnaround, submitted to done, mean (s), A B C 106.1 → 106.9; raw +0.7% (95%: −6.2% to +7.7%); LOSS · 63.75 → 65.83; raw +3.3% (95%: −2.8% to +9.3%); LOSS · 58.35 → 61.3; raw +5.0% (95%: −4.9% to +15.0%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Crane (Effective HPA, on Kubernetes 1.25) under the batch stream, the four ways (omni-v12, `results/live/V12_CRANE_4W_BOTH.md`): response time (ms), mean, A B C 146.9 → 150; raw +2.2% (95%: −0.1% to +4.4%); LOSS · 132.1 → 135.3; raw +2.4% (95%: −0.2% to +5.1%); LOSS · 150.4 → 153; raw +1.7% (95%: −1.0% to +4.4%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Crane (Effective HPA, on Kubernetes 1.25) under the batch stream, the four ways (omni-v12, `results/live/V12_CRANE_4W_BOTH.md`): pod start wait, total (s), A B C 6.8 → 12.3; raw +80.9% (95%: +32.7% to +129.0%); LOSS · 8 → 13.5; raw +68.8% (95%: +12.8% to +124.7%); LOSS · 7.1 → 12.2; raw +71.8% (95%: +31.7% to +112.0%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Crane (Effective HPA, on Kubernetes 1.25) under the batch stream, the four ways (omni-v12, `results/live/V12_CRANE_4W_ORCH.md`): response time (ms), mean, A B C 146.9 → 150.6; raw +2.6% (95%: +0.7% to +4.4%); LOSS · 132.1 → 136.3; raw +3.2% (95%: +0.5% to +5.9%); LOSS · 150.4 → 154.8; raw +2.9% (95%: +0.9% to +5.0%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Crane (Effective HPA, on Kubernetes 1.25) under the batch stream, the four ways (omni-v12, `results/live/V12_CRANE_4W_VS.md`): pod start wait, total (s), A B C 6.8 → 10.9; raw +60.3% (95%: +28.4% to +92.2%); LOSS · 6.6 → 10.3; raw +56.1% (95%: +0.6% to +111.6%); LOSS · 7.9 → 12; raw +51.9% (95%: +20.4% to +83.4%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Crane (Effective HPA, on Kubernetes 1.25) under the batch stream, the four ways (omni-v12, `results/live/V12_CRANE_4W_VS.md`): pod start wait, mean (s), A B C 1.492 → 1.515; raw +1.6% (95%: −21.9% to +25.0%); LOSS · 1.325 → 1.547; raw +16.8% (95%: −11.2% to +44.7%); LOSS · 1.58 → 1.704; raw +7.8% (95%: −8.6% to +24.3%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BOTH.md`): response time (ms), mean, A B C 316.4 → 318.3; raw +0.6% (95%: −8.8% to +10.0%); LOSS · 396.8 → 425; raw +7.1% (95%: −2.9% to +17.1%); LOSS · 347.3 → 371.1; raw +6.9% (95%: −2.3% to +16.0%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BOTH.md`): response time (ms), 95th percentile, A B C 1070 → 1519; raw +41.9% (95%: +11.7% to +72.1%); LOSS · 1366 → 1992; raw +45.9% (95%: +20.6% to +71.1%); LOSS · 1135 → 1788; raw +57.5% (95%: +32.2% to +82.7%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BOTH.md`): response time (ms), 99th percentile, A B C 1577 → 2288; raw +45.2% (95%: +12.1% to +78.2%); LOSS · 1972 → 2937; raw +48.9% (95%: +22.4% to +75.5%); LOSS · 1712 → 2775; raw +62.1% (95%: +33.1% to +91.1%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BOTH.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 8.6 → 8.8; raw +2.3% (95%: −13.7% to +18.4%); LOSS · 11.6 → 12.1; raw +4.3% (95%: −3.0% to +11.6%); LOSS · 10.9 → 11.5; raw +5.5% (95%: +0.9% to +10.1%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BOTH.md`): pod start wait, total (s), A B C 0.4 → 0.8; raw +100.0% (95%: −169.2% to +369.2%), +0.4000 in the gauge's unit; LOSS · 0 → 0.4; raw +0.4000 in the gauge's unit (95%: −0.1001 to +0.9001); LOSS · 0 → 0.5; raw +0.5000 in the gauge's unit (95%: −0.0058 to +1.0058); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BOTH.md`): pod start wait, mean (s), A B C 0.2 → 0.8; raw +300.0% (95%: −84.5% to +684.5%), +0.6000 in the gauge's unit; LOSS · 0 → 0.4; raw +0.4000 in the gauge's unit (95%: −0.1001 to +0.9001); LOSS · 0 → 0.5; raw +0.5000 in the gauge's unit (95%: −0.0058 to +1.0058); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BOTH.md`): batch: queue finished (s), A B C 771.5 → 810.9; raw +5.1% (95%: +1.7% to +8.5%); LOSS · 790 → 844.2; raw +6.9% (95%: +3.4% to +10.3%); LOSS · 772.7 → 812.4; raw +5.1% (95%: +3.0% to +7.3%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BOTH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.05417 → 1.004; raw +1753.8% (95%: +1040.8% to +2466.9%), +0.9500 in the gauge's unit;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

LOSS · 0.0625 → 1.079; raw +1626.7% (95%: +910.1% to +2343.3%), +1.0167 in the gauge's unit; LOSS · 0.07917 → 0.8042; raw +915.8% (95%: +807.5% to +1024.0%), +0.7250 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BOTH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.4 → 2.6; raw +550.0% (95%: +38.6% to +1061.4%), +2.2000 in the gauge's unit; LOSS · 0.4 → 3; raw +650.0% (95%: +129.0% to +1171.0%), +2.6000 in the gauge's unit; LOSS · 0.7 → 1.6; raw +128.6% (95%: +70.6% to +186.6%), +0.9000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BOTH.md`): queue: job turnaround, submitted to done, mean (s), A B C 78.3 → 115.9; raw +48.0% (95%: +17.8% to +78.1%); LOSS · 98.16 → 148.6; raw +51.3% (95%: +28.0% to +74.7%); LOSS · 82.27 → 125.4; raw +52.5% (95%: +31.0% to +73.9%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_ORCH.md`): response time (ms), mean, A B C 316.4 → 358.1; raw +13.2% (95%: +1.6% to +24.8%); LOSS · 396.8 → 459.4; raw +15.8% (95%: +8.1% to +23.5%); LOSS · 347.3 → 410.9; raw +18.3% (95%: +7.5% to +29.1%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_ORCH.md`): response time (ms), 95th percentile, A B C 1070 → 1478; raw +38.1% (95%: +7.2% to +69.0%); LOSS · 1366 → 1927; raw +41.1% (95%: +19.5% to +62.6%); LOSS · 1135 → 1712; raw +50.8% (95%: +22.6% to +79.0%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_ORCH.md`): response time (ms), 99th percentile, A B C 1577 → 2206; raw +39.9% (95%: +7.8% to +72.1%); LOSS · 1972 → 2841; raw +44.0% (95%: +23.2% to +64.9%); LOSS · 1712 → 2761; raw +61.3% (95%: +30.8% to +91.7%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_ORCH.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 8.6 → 8.7; raw +1.2% (95%: −11.5% to +13.8%); LOSS · 11.6 → 12.4; raw +6.9% (95%: +4.3% to +9.5%); LOSS · 10.9 → 11.6; raw +6.4% (95%: +2.0% to +10.9%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_ORCH.md`): batch: queue finished (s), A B C 771.5 → 804.2; raw +4.2% (95%: +1.6% to +6.9%); LOSS · 790 → 835; raw +5.7% (95%: +2.8% to +8.6%); LOSS · 772.7

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

→ 807.9; raw +4.6% (95%: +2.7% to +6.4%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_ORCH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.05417 → 0.8458; raw +1461.5% (95%: +1162.4% to +1760.7%), +0.7917 in the gauge's unit; LOSS · 0.0625 → 1.071; raw +1613.3% (95%: +1157.7% to +2069.0%), +1.0083 in the gauge's unit; LOSS · 0.07917 → 0.8458; raw +968.4% (95%: +828.5% to +1108.4%), +0.7667 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_ORCH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.4 → 1.6; raw +300.0% (95%: +115.3% to +484.7%), +1.2000 in the gauge's unit; LOSS · 0.4 → 2.6; raw +550.0% (95%: +225.7% to +874.3%), +2.2000 in the gauge's unit; LOSS · 0.7 → 1.4; raw +100.0% (95%: +31.0% to +169.0%), +0.7000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_ORCH.md`): queue: job turnaround, submitted to done, mean (s), A B C 78.3 → 109.7; raw +40.1% (95%: +16.1% to +64.0%); LOSS · 98.16 → 139.5; raw +42.1% (95%: +24.1% to +60.0%); LOSS · 82.27 → 119.4; raw +45.2% (95%: +26.8% to +63.6%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_VS.md`): time over the response line (% of samples), A B C 13.22 → 15.37; raw +16.2% (95%: +5.8% to +26.6%); LOSS · 18.89 → 19.84; raw +5.0% (95%: −12.2% to +22.3%); LOSS · 17.63 → 19.09; raw +8.2% (95%: −2.0% to +18.5%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): response time (ms), mean, A B C 555 → 808.5; raw +45.7% (95%: +15.8% to +75.5%); LOSS · 574.4 → 764.6; raw +33.1% (95%: +14.0% to +52.3%); LOSS · 694 → 867.4; raw +25.0% (95%: +7.6% to +42.4%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): response time (ms), 95th percentile, A B C 2164 → 3923; raw +81.3% (95%: +33.4% to +129.1%); LOSS · 2260 → 3665; raw +62.1% (95%: +27.0% to +97.3%); LOSS · 2707 → 3759; raw +38.8% (95%: +13.8% to +63.9%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): response time (ms), 99th percentile, A B C 3077 → 5337; raw +73.4% (95%: +34.4% to +112.5%); LOSS · 3117 → 5114; raw +64.1% (95%:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

+30.5% to +97.7%); LOSS · 3712 → 5069; raw +36.5% (95%: +13.7% to +59.4%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): time over the response line (% of samples), A B C 21.93 → 25.85; raw +17.9% (95%: +8.8% to +27.1%); LOSS · 22.45 → 25.28; raw +12.6% (95%: +0.3% to +25.0%); LOSS · 22.87 → 28; raw +22.4% (95%: +7.1% to +37.8%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): failed requests (%), A B C 0.0157 → 0.159; raw +912.6% (95%: +76.0% to +1749.3%), +0.1433 in the gauge's unit; LOSS · 0 → 0.1452; raw +0.1452 in the gauge's unit (95%: −0.0085 to +0.2990); LOSS · 0.01501 → 0.1831; raw +1119.8% (95%: −207.1% to +2446.7%), +0.1681 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 11.33 → 16.44; raw +45.1% (95%: +21.3% to +68.9%); LOSS · 11.3 → 15.2; raw +34.5% (95%: +21.0% to +48.0%); LOSS · 10.4 → 13.6; raw +30.8% (95%: +18.3% to +43.2%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): pod start wait, total (s), A B C 0.5556 → 3.889; raw +600.0% (95%: −258.5% to +1458.5%), +3.3333 in the gauge's unit; LOSS · 0 → 1.9; raw +1.9000 in the gauge's unit (95%: +0.4914 to +3.3086); LOSS · 0 → 2.2; raw +2.2000 in the gauge's unit (95%: −0.0820 to +4.4820); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): pod start wait, mean (s), A B C 0.5556 → 1.101; raw +98.1% (95%: −145.9% to +342.1%), +0.5450 in the gauge's unit; LOSS · 0 → 1.033; raw +1.0333 in the gauge's unit (95%: +0.3341 to +1.7326); LOSS · 0 → 1.225; raw +1.2250 in the gauge's unit (95%: +0.2048 to +2.2452); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): batch: queue finished (s), A B C 852.7 → 993.1; raw +16.5% (95%: +7.4% to +25.6%); LOSS · 856.4 → 991; raw +15.7% (95%: +8.5% to +23.0%); LOSS · 909.9 → 1040; raw +14.3% (95%: +5.7% to +22.9%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): queue: job wait, submitted to its first

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

pod running, mean (s), A B C 0.07407 → 1.963; raw +2550.0% (95%: −2197.6% to +7297.6%), +1.8889 in the gauge's unit; LOSS · 0.1042 → 3.271; raw +3040.0% (95%: −1048.9% to +7128.9%), +3.1667 in the gauge's unit; LOSS · 0.0875 → 16.2; raw +18419.0% (95%: +3305.6% to +33532.5%), +16.1167 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.5556 → 1.556; raw +180.0% (95%: +82.2% to +277.8%), +1.0000 in the gauge's unit; LOSS · 0.6 → 1.7; raw +183.3% (95%: +52.1% to +314.5%), +1.1000 in the gauge's unit; LOSS · 0.5 → 122.3; raw +24360.0% (95%: +1893.1% to +46826.9%), +121.8000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): queue: job turnaround, submitted to done, mean (s), A B C 163.2 → 298.7; raw +83.0% (95%: +40.0% to +126.1%); LOSS · 167.6 → 298.5; raw +78.2% (95%: +42.6% to +113.8%); LOSS · 216.2 → 341.4; raw +57.9% (95%: +24.1% to +91.8%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): response time (ms), mean, A B C 555 → 709.8; raw +27.9% (95%: +13.5% to +42.3%); LOSS · 574.4 → 691.8; raw +20.4% (95%: +10.8% to +30.0%); LOSS · 694 → 844.3; raw +21.6% (95%: +7.9% to +35.4%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): response time (ms), 95th percentile, A B C 2164 → 3269; raw +51.1% (95%: +21.9% to +80.2%); LOSS · 2260 → 3140; raw +38.9% (95%: +18.4% to +59.5%); LOSS · 2707 → 3666; raw +35.4% (95%: +13.9% to +56.9%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): response time (ms), 99th percentile, A B C 3077 → 4767; raw +54.9% (95%: +23.6% to +86.2%); LOSS · 3117 → 4602; raw +47.7% (95%: +25.1% to +70.2%); LOSS · 3712 → 4876; raw +31.3% (95%: +13.1% to +49.6%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): time over the response line (% of samples), A B C 21.93 → 24.64; raw +12.4% (95%: +0.5% to +24.3%); LOSS · 22.45 → 23.99; raw +6.9% (95%: −6.2% to +19.9%); LOSS · 22.87 → 25.85; raw +13.0% (95%: +4.3% to +21.8%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): failed requests (%), A B C 0.0157 → 0.08233; raw +424.2% (95%: −104.6% to +953.0%), +0.0666 in the gauge's unit; LOSS · 0 → 0.0524;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

raw +0.0524 in the gauge's unit (95%: −0.0355 to +0.1403); LOSS · 0.01501 → 0.06374; raw +324.7% (95%: −281.1% to +930.4%), +0.0487 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 11.33 → 15.22; raw +34.3% (95%: +13.3% to +55.3%); LOSS · 11.3 → 13.8; raw +22.1% (95%: +12.1% to +32.1%); LOSS · 10.4 → 13.7; raw +31.7% (95%: +8.1% to +55.3%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): pod start wait, total (s), A B C 0.5556 → 1.111; raw +100.0% (95%: −239.7% to +439.7%), +0.5556 in the gauge's unit; LOSS · 0 → 1.1; raw +1.1000 in the gauge's unit (95%: −0.2676 to +2.4676); LOSS · 0 → 1.1; raw +1.1000 in the gauge's unit (95%: −0.2254 to +2.4254); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): pod start wait, mean (s), A B C 0.5556 → 0.8333; raw +50.0% (95%: −224.8% to +324.8%), +0.2778 in the gauge's unit; LOSS · 0 → 0.65; raw +0.6500 in the gauge's unit (95%: −0.1049 to +1.4049); LOSS · 0 → 0.85; raw +0.8500 in the gauge's unit (95%: −0.1338 to +1.8338); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): batch: queue finished (s), A B C 852.7 → 952.2; raw +11.7% (95%: +5.1% to +18.3%); LOSS · 856.4 → 952.2; raw +11.2% (95%: +6.1% to +16.2%); LOSS · 909.9 → 1010; raw +11.0% (95%: +5.0% to +16.9%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.07407 → 3.431; raw +4531.3% (95%: −4930.2% to +13992.7%), +3.3565 in the gauge's unit; LOSS · 0.1042 → 1.867; raw +1692.0% (95%: −1263.7% to +4647.7%), +1.7625 in the gauge's unit; LOSS · 0.0875 → 9.812; raw +11114.3% (95%: −373.7% to +22602.3%), +9.7250 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.5556 → 36.89; raw +6540.0% (95%: −8333.9% to +21413.9%), +36.3333 in the gauge's unit; LOSS · 0.6 → 1.5; raw +150.0% (95%: +31.4% to +268.6%), +0.9000 in

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

the gauge's unit; LOSS · 0.5 → 64.4; raw +12780.0% (95%: −6059.7% to +31619.7%), +63.9000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): queue: job turnaround, submitted to done, mean (s), A B C 163.2 → 258.1; raw +58.2% (95%: +28.9% to +87.5%); LOSS · 167.6 → 261.5; raw +56.1% (95%: +30.6% to +81.6%); LOSS · 216.2 → 311.6; raw +44.2% (95%: +20.7% to +67.7%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_VS.md`): pod start wait, total (s), A B C 1.111 → 2.222; raw +100.0% (95%: −48.6% to +248.6%); LOSS · 1.1 → 1.2; raw +9.1% (95%: −162.8% to +181.0%); LOSS · 1.1 → 1.9; raw +72.7% (95%: −105.5% to +250.9%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_VS.md`): pod start wait, mean (s), A B C 0.8333 → 1.389; raw +66.7% (95%: −35.0% to +168.4%), +0.5556 in the gauge's unit; LOSS · 0.65 → 0.95; raw +46.2% (95%: −124.3% to +216.6%), +0.3000 in the gauge's unit; LOSS · 0.85 → 1.25; raw +47.1% (95%: −53.7% to +147.8%), +0.4000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_BOTH.md`): pod start wait, total (s), A B C 0.2 → 0.3; raw +50.0% (95%: −63.1% to +163.1%), +0.1000 in the gauge's unit; LOSS · 0 → 0.7; raw +0.7000 in the gauge's unit (95%: −0.1294 to +1.5294); LOSS · 0.2 → 0.2; raw ±0.0% (95%: −337.2% to +337.2%), ±0.0000 in the gauge's unit; STEADY. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_BOTH.md`): batch: queue finished (s), A B C 731 → 1313; raw +79.6% (95%: +57.4% to +101.8%); LOSS · 778.8 → 1421; raw +82.5% (95%: +72.2% to +92.7%); LOSS · 747.2 → 1421; raw +90.2% (95%: +71.9% to +108.5%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_BOTH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.03333 → 110.9; raw +332575.0% (95%: +204178.2% to +460971.8%), +110.8583 in the gauge's unit; LOSS · 0.025 → 175.4; raw +701433.3% (95%: +502619.2% to +900247.5%), +175.3583 in the gauge's unit; LOSS · 0.0625 → 142.5; raw +227980.0% (95%: +149811.1% to +306148.9%), +142.4875 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_BOTH.md`): queue: job wait, submitted to its first pod

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

running, 95th percentile (s), A B C 0.3 → 518.5; raw +172733.3% (95%: +104746.8% to +240719.9%), +518.2000 in the gauge's unit; LOSS · 0.1 → 763.9; raw +763800.0% (95%: +530043.0% to +997557.0%), +763.8000 in the gauge's unit; LOSS · 0.5 → 684.4; raw +136780.0% (95%: +86044.3% to +187515.7%), +683.9000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_BOTH.md`): queue: job turnaround, submitted to done, mean (s), A B C 38.07 → 650.2; raw +1607.8% (95%: +1081.3% to +2134.4%); LOSS · 85.79 → 814.5; raw +849.4% (95%: +675.6% to +1023.2%); LOSS · 55.45 → 789; raw +1322.9% (95%: +1027.8% to +1618.1%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_BOTH.md`): queue: jobs unfinished at the window's end, A B C 0 → 5.3; raw +5.3000 in the gauge's unit (95%: −0.8906 to +11.4906); LOSS · 0 → 10.7; raw +10.7000 in the gauge's unit (95%: +4.0490 to +17.3510); LOSS · 0 → 8.8; raw +8.8000 in the gauge's unit (95%: +3.2613 to +14.3387); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_ORCH.md`): batch: queue finished (s), A B C 731 → 1290; raw +76.5% (95%: +54.1% to +98.9%); LOSS · 778.8 → 1421; raw +82.5% (95%: +72.8% to +92.1%); LOSS · 747.2 → 1396; raw +86.9% (95%: +68.4% to +105.4%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_ORCH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.03333 → 103.6; raw +310637.5% (95%: +166814.0% to +454461.0%), +103.5458 in the gauge's unit; LOSS · 0.025 → 171.5; raw +686083.3% (95%: +439006.5% to +933160.2%), +171.5208 in the gauge's unit; LOSS · 0.0625 → 146.3; raw +233933.3% (95%: +162122.7% to +305743.9%), +146.2083 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_ORCH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.3 → 513.9; raw +171200.0% (95%: +91149.9% to +251250.1%), +513.6000 in the gauge's unit; LOSS · 0.1 → 798; raw +797900.0% (95%: +545467.5% to +1050332.5%), +797.9000 in the gauge's unit; LOSS · 0.5 → 698.5; raw +139600.0% (95%: +101748.5% to +177451.5%), +698.0000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_ORCH.md`): queue: job turnaround, submitted to done, mean (s), A B C 38.07 → 614; raw +1512.7% (95%: +1011.0% to +2014.4%); LOSS · 85.79 → 799.3;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

raw +831.8% (95%: +691.5% to +972.0%); LOSS · 55.45 → 763.3; raw +1276.6% (95%: +979.1% to +1574.2%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_ORCH.md`): queue: jobs unfinished at the window's end, A B C 0 → 4.9; raw +4.9000 in the gauge's unit (95%: −0.7971 to +10.5971); LOSS · 0 → 9.4; raw +9.4000 in the gauge's unit (95%: +3.5091 to +15.2909); LOSS · 0 → 9.6; raw +9.6000 in the gauge's unit (95%: +3.5005 to +15.6995); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_VS.md`): response time (ms), mean, A B C 113 → 158.7; raw +40.4% (95%: −3.0% to +83.9%); LOSS · 131.2 → 325.7; raw +148.2% (95%: +55.6% to +240.8%); LOSS · 141.2 → 242; raw +71.4% (95%: +13.0% to +129.9%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_VS.md`): response time (ms), 95th percentile, A B C 194.7 → 567.1; raw +191.2% (95%: +63.6% to +318.8%); LOSS · 234.3 → 1372; raw +485.5% (95%: +212.9% to +758.0%); LOSS · 236.9 → 913.5; raw +285.6% (95%: +106.4% to +464.8%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_VS.md`): response time (ms), 99th percentile, A B C 291 → 818.7; raw +181.3% (95%: +56.9% to +305.8%); LOSS · 345.6 → 2010; raw +481.6% (95%: +212.0% to +751.1%); LOSS · 336.7 → 1296; raw +285.0% (95%: +116.9% to +453.1%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_VS.md`): time over the response line (% of samples), A B C 0.1615 → 7.237; raw +4381.5% (95%: −131.3% to +8894.3%), +7.0757 in the gauge's unit; LOSS · 0.2311 → 17.38; raw +7419.5% (95%: +3065.0% to +11774.0%), +17.1481 in the gauge's unit; LOSS · 0.1697 → 13.87; raw +8073.6% (95%: +3500.6% to +12646.7%), +13.6994 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_VS.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 0 → 4.9; raw +4.9000 in the gauge's unit (95%: +0.7503 to +9.0497); LOSS · 0.1 → 9; raw +8900.0% (95%: +4446.3% to +13353.7%), +8.9000 in the gauge's unit; LOSS · 0.1 → 9.6; raw +9500.0% (95%: +6140.7% to +12859.3%), +9.5000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): response time (ms), mean, A B C 330.3 → 613.4; raw +85.7% (95%: +68.2% to +103.3%); LOSS · 317.1 → 587; raw +85.1% (95%:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

+68.5% to +101.7%); LOSS · 405.7 → 677.4; raw +67.0% (95%: +49.1% to +84.9%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): time over the response line (% of samples), A B C 16.77 → 52.45; raw +212.8% (95%: +175.6% to +250.1%); LOSS · 15.06 → 48.66; raw +223.2% (95%: +172.9% to +273.6%); LOSS · 19.25 → 59.29; raw +207.9% (95%: +172.9% to +243.0%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 8 → 24.22; raw +202.8% (95%: +146.0% to +259.6%); LOSS · 7.3 → 23.1; raw +216.4% (95%: +168.3% to +264.6%); LOSS · 9.3 → 23.9; raw +157.0% (95%: +115.5% to +198.5%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): pod start wait, total (s), A B C 0 → 1.111; raw +1.1111 in the gauge's unit (95%: −0.2981 to +2.5203); LOSS · 0 → 0.2; raw +0.2000 in the gauge's unit (95%: −0.2524 to +0.6524); LOSS · 0 → 0.8; raw +0.8000 in the gauge's unit (95%: −0.0121 to +1.6121); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): pod start wait, mean (s), A B C 0 → 0.7407; raw +0.7407 in the gauge's unit (95%: −0.1540 to +1.6355); LOSS · 0 → 0.2; raw +0.2000 in the gauge's unit (95%: −0.2524 to +0.6524); LOSS · 0 → 0.8; raw +0.8000 in the gauge's unit (95%: −0.0121 to +1.6121); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): batch: queue finished (s), A B C 774.1 → 1466; raw +89.3% (95%: +79.7% to +98.9%); LOSS · 775.3 → 1396; raw +80.1% (95%: +69.7% to +90.5%); LOSS · 802.6 → 1451; raw +80.8% (95%: +71.6% to +90.1%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.03241 → 637.5; raw +1967119.6% (95%: +1111934.3% to +2822304.9%), +637.4925 in the gauge's unit; LOSS · 0.04167 → 536.6; raw +1287745.8% (95%: +521467.2% to +2054024.4%), +536.5607 in the gauge's unit; LOSS · 0.05417 → 677.6; raw +1250833.4% (95%: +747221.5% to +1754445.3%), +677.5348 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.1111 → 1113; raw +1001696.3% (95%: +623421.0% to

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

+1379971.7%), +1112.9959 in the gauge's unit; LOSS · 0.4 → 924.3; raw +230987.0% (95%: +116467.0% to +345506.9%), +923.9479 in the gauge's unit; LOSS · 0.4 → 1141; raw +285052.5% (95%: +193399.9% to +376705.2%), +1140.2102 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): queue: job turnaround, submitted to done, mean (s), A B C 81.79 → 955; raw +1067.6% (95%: +891.6% to +1243.7%); LOSS · 83.3 → 841.3; raw +910.0% (95%: +690.2% to +1129.8%); LOSS · 107.6 → 941.7; raw +774.9% (95%: +649.7% to +900.2%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): queue: jobs unfinished at the window's end, A B C 0 → 13.44; raw +13.4444 in the gauge's unit (95%: +7.1631 to +19.7258); LOSS · 0 → 11.3; raw +11.3000 in the gauge's unit (95%: +3.8583 to +18.7417); LOSS · 0 → 14.5; raw +14.5000 in the gauge's unit (95%: +8.3259 to +20.6741); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_ORCH.md`): response time (ms), mean, A B C 330.3 → 620.7; raw +87.9% (95%: +61.0% to +114.8%); LOSS · 317.1 → 591.8; raw +86.6% (95%: +61.9% to +111.4%); LOSS · 405.7 → 692.2; raw +70.6% (95%: +49.8% to +91.5%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_ORCH.md`): time over the response line (% of samples), A B C 16.77 → 53.85; raw +221.2% (95%: +183.5% to +258.9%); LOSS · 15.06 → 50.32; raw +234.2% (95%: +179.7% to +288.8%); LOSS · 19.25 → 61.12; raw +217.5% (95%: +182.6% to +252.4%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_ORCH.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 8 → 24.33; raw +204.2% (95%: +145.9% to +262.4%); LOSS · 7.3 → 23.3; raw +219.2% (95%: +169.2% to +269.1%); LOSS · 9.3 → 24.1; raw +159.1% (95%: +116.0% to +202.3%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_ORCH.md`): pod start wait, total (s), A B C 0 → 0.2222; raw +0.2222 in the gauge's unit (95%: −0.2902 to +0.7347); LOSS · 0 → 0.8; raw +0.8000 in the gauge's unit (95%: −0.4526 to +2.0526); LOSS · 0 → 0.3; raw +0.3000 in the gauge's unit (95%: −0.1828 to +0.7828); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_ORCH.md`): pod start wait, mean (s), A B C 0 →

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

0.2222; raw +0.2222 in the gauge's unit (95%: −0.2902 to +0.7347); LOSS · 0 → 0.55; raw +0.5500 in the gauge's unit (95%: −0.2837 to +1.3837); LOSS · 0 → 0.3; raw +0.3000 in the gauge's unit (95%: −0.1828 to +0.7828); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_ORCH.md`): batch: queue finished (s), A B C 774.1 → 1474; raw +90.4% (95%: +80.4% to +100.4%); LOSS · 775.3 → 1400; raw +80.6% (95%: +70.9% to +90.3%); LOSS · 802.6 → 1456; raw +81.4% (95%: +72.4% to +90.3%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_ORCH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.03241 → 635; raw +1959438.6% (95%: +1137492.1% to +2781385.0%), +635.0032 in the gauge's unit; LOSS · 0.04167 → 533.6; raw +1280430.0% (95%: +511805.8% to +2049054.2%), +533.5125 in the gauge's unit; LOSS · 0.05417 → 670.7; raw +1238207.0% (95%: +751232.1% to +1725181.9%), +670.6955 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_ORCH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.1111 → 1129; raw +1015791.5% (95%: +684899.6% to +1346683.4%), +1128.6572 in the gauge's unit; LOSS · 0.4 → 914.7; raw +228574.2% (95%: +114225.9% to +342922.5%), +914.2970 in the gauge's unit; LOSS · 0.4 → 1145; raw +286211.2% (95%: +190609.6% to +381812.7%), +1144.8447 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_ORCH.md`): queue: job turnaround, submitted to done, mean (s), A B C 81.79 → 954.8; raw +1067.4% (95%: +882.4% to +1252.3%); LOSS · 83.3 → 855.5; raw +927.0% (95%: +727.3% to +1126.7%); LOSS · 107.6 → 946.3; raw +779.3% (95%: +655.6% to +902.9%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_ORCH.md`): queue: jobs unfinished at the window's end, A B C 0 → 14.33; raw +14.3333 in the gauge's unit (95%: +8.2201 to +20.4465); LOSS · 0 → 11.1; raw +11.1000 in the gauge's unit (95%: +3.7552 to +18.4448); LOSS · 0 → 14.6; raw +14.6000 in the gauge's unit (95%: +8.6134 to +20.5866); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BOTH.md`): response time (ms), 95th percentile, A B C 1006 → 1101; raw +9.5% (95%: +1.9% to +17.1%); LOSS · 1151 → 1305; raw +13.4% (95%: +3.4% to +23.4%); LOSS · 1278 → 1474; raw +15.4% (95%: +7.8% to +22.9%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BOTH.md`): response time (ms), 99th percentile, A B C 1451 → 1605; raw +10.6% (95%: −0.7% to +22.0%); LOSS · 1720 → 1946; raw +13.1% (95%: +4.1% to +22.1%); LOSS · 1825 → 2040; raw +11.8% (95%: +3.7% to +19.8%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BOTH.md`): failed requests (%), A B C 0 → 0; raw ±0.0000 in the gauge's unit (95%: ±0.0000 to ±0.0000); STEADY · 0 → 0.009302; raw +0.0093 in the gauge's unit (95%: −0.0117 to +0.0303); LOSS · 0 → 0; raw ±0.0000 in the gauge's unit (95%: ±0.0000 to ±0.0000); STEADY. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BOTH.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 7.8 → 8; raw +2.6% (95%: −5.9% to +11.0%); LOSS · 10.2 → 10.6; raw +3.9% (95%: −1.0% to +8.8%); LOSS · 11 → 11.33; raw +3.0% (95%: −0.5% to +6.5%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BOTH.md`): pod start wait, total (s), A B C 0 → 0.1; raw +0.1000 in the gauge's unit (95%: −0.1262 to +0.3262); LOSS · 0 → 0.3; raw +0.3000 in the gauge's unit (95%: −0.1828 to +0.7828); LOSS · 0 → 1.222; raw +1.2222 in the gauge's unit (95%: −0.3983 to +2.8427); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BOTH.md`): pod start wait, mean (s), A B C 0 → 0.1; raw +0.1000 in the gauge's unit (95%: −0.1262 to +0.3262); LOSS · 0 → 0.3; raw +0.3000 in the gauge's unit (95%: −0.1828 to +0.7828); LOSS · 0 → 0.7222; raw +0.7222 in the gauge's unit (95%: −0.1607 to +1.6052); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BOTH.md`): batch: queue finished (s), A B C 768.1 → 775.8; raw +1.0% (95%: +0.3% to +1.7%); LOSS · 772.9 → 785.5; raw +1.6% (95%: +0.8% to +2.5%); LOSS · 779.2 → 794.2; raw +1.9% (95%: +1.1% to +2.8%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BOTH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.05833 → 0.1667; raw +185.7% (95%: −48.1% to +419.5%), +0.1083 in the gauge's unit; LOSS · 0.09167 → 0.2; raw +118.2% (95%: +27.2% to +209.1%), +0.1083 in the gauge's unit; LOSS · 0.03704 → 0.2361; raw +537.5% (95%: +360.4% to +714.6%), +0.1991 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BOTH.md`): queue: job wait, submitted to its first pod running,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

95th percentile (s), A B C 0.3 → 0.6; raw +100.0% (95%: −15.2% to +215.2%), +0.3000 in the gauge's unit; LOSS · 0.8 → 0.8; raw ±0.0% (95%: −42.1% to +42.1%), ±0.0000 in the gauge's unit; STEADY · 0.1111 → 1; raw +800.0% (95%: +569.4% to +1030.6%), +0.8889 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BOTH.md`): queue: job turnaround, submitted to done, mean (s), A B C 73.89 → 83.16; raw +12.5% (95%: +3.2% to +21.8%); LOSS · 81.23 → 92.45; raw +13.8% (95%: +6.4% to +21.3%); LOSS · 87.61 → 102.1; raw +16.6% (95%: +8.6% to +24.5%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_ORCH.md`): response time (ms), mean, A B C 307 → 329.3; raw +7.3% (95%: ±0.0% to +14.5%); LOSS · 346.2 → 368.3; raw +6.4% (95%: +1.8% to +11.0%); LOSS · 371.1 → 404; raw +8.9% (95%: +3.7% to +14.1%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_ORCH.md`): response time (ms), 95th percentile, A B C 1006 → 1134; raw +12.8% (95%: +0.3% to +25.2%); LOSS · 1151 → 1266; raw +10.0% (95%: +1.4% to +18.7%); LOSS · 1278 → 1409; raw +10.2% (95%: +2.9% to +17.5%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_ORCH.md`): response time (ms), 99th percentile, A B C 1451 → 1643; raw +13.2% (95%: −1.3% to +27.7%); LOSS · 1720 → 1841; raw +7.0% (95%: +0.4% to +13.6%); LOSS · 1825 → 2029; raw +11.1% (95%: +5.0% to +17.3%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_ORCH.md`): time over the response line (% of samples), A B C 14.95 → 15.73; raw +5.2% (95%: −0.2% to +10.5%); LOSS · 20.05 → 20.83; raw +3.9% (95%: −0.7% to +8.5%); LOSS · 21.68 → 23.23; raw +7.2% (95%: +2.9% to +11.4%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_ORCH.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 7.8 → 7.8; raw ±0.0% (95%: −6.1% to +6.1%); STEADY · 10.2 → 10.3; raw +1.0% (95%: −6.0% to +8.0%); LOSS · 11 → 11.11; raw +1.0% (95%: −1.3% to +3.3%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_ORCH.md`): pod start wait, total (s), A B C 0 → 0.4; raw +0.4000 in the gauge's unit (95%: −0.5048 to +1.3048); LOSS · 0 → 0.4; raw +0.4000 in the gauge's unit (95%:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

−0.5048 to +1.3048); LOSS · 0 → 0.3333; raw +0.3333 in the gauge's unit (95%: −0.4353 to +1.1020); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_ORCH.md`): pod start wait, mean (s), A B C 0 → 0.4; raw +0.4000 in the gauge's unit (95%: −0.5048 to +1.3048); LOSS · 0 → 0.4; raw +0.4000 in the gauge's unit (95%: −0.5048 to +1.3048); LOSS · 0 → 0.3333; raw +0.3333 in the gauge's unit (95%: −0.4353 to +1.1020); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_ORCH.md`): batch: queue finished (s), A B C 768.1 → 775.1; raw +0.9% (95%: −0.1% to +1.9%); LOSS · 772.9 → 781.3; raw +1.1% (95%: +0.3% to +1.8%); LOSS · 779.2 → 790.4; raw +1.4% (95%: +0.6% to +2.3%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_ORCH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.05833 → 0.1625; raw +178.6% (95%: +75.7% to +281.5%), +0.1042 in the gauge's unit; LOSS · 0.09167 → 0.2; raw +118.2% (95%: +22.2% to +214.1%), +0.1083 in the gauge's unit; LOSS · 0.03704 → 0.2361; raw +537.5% (95%: +313.8% to +761.2%), +0.1991 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_ORCH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.3 → 1; raw +233.3% (95%: +118.2% to +348.5%), +0.7000 in the gauge's unit; LOSS · 0.8 → 1; raw +25.0% (95%: −12.7% to +62.7%), +0.2000 in the gauge's unit; LOSS · 0.1111 → 0.8889; raw +700.0% (95%: +394.9% to +1005.1%), +0.7778 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_ORCH.md`): queue: job turnaround, submitted to done, mean (s), A B C 73.89 → 82.38; raw +11.5% (95%: +0.3% to +22.7%); LOSS · 81.23 → 88.75; raw +9.3% (95%: +2.0% to +16.5%); LOSS · 87.61 → 98.69; raw +12.7% (95%: +5.4% to +19.9%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BOTH.md`): response time (ms), mean, A B C 752.2 → 1203; raw +59.9% (95%: +38.6% to +81.3%); LOSS · 665.7 → 1010; raw +51.7% (95%: +23.3% to +80.0%); LOSS · 630.1 → 1000; raw +58.7% (95%: +37.1% to +80.3%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BOTH.md`): response time (ms), 95th percentile, A B C 2969 → 4499; raw +51.5% (95%: +36.6% to +66.4%); LOSS · 2682 → 4162; raw

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

+55.2% (95%: +22.4% to +88.0%); LOSS · 2493 → 3866; raw +55.0% (95%: +29.0% to +81.1%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BOTH.md`): response time (ms), 99th percentile, A B C 4041 → 6036; raw +49.4% (95%: +31.0% to +67.8%); LOSS · 3704 → 5867; raw +58.4% (95%: +25.9% to +90.9%); LOSS · 3409 → 5213; raw +52.9% (95%: +30.2% to +75.6%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BOTH.md`): time over the response line (% of samples), A B C 28.08 → 43.77; raw +55.9% (95%: +30.2% to +81.5%); LOSS · 26.74 → 33.12; raw +23.9% (95%: +11.2% to +36.5%); LOSS · 21.13 → 37.09; raw +75.5% (95%: +53.5% to +97.5%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BOTH.md`): failed requests (%), A B C 0.04202 → 0.4937; raw +1075.1% (95%: −954.5% to +3104.7%), +0.4517 in the gauge's unit; LOSS · 0.007491 → 0.1132; raw +1410.9% (95%: −485.9% to +3307.7%), +0.1057 in the gauge's unit; LOSS · 0.03597 → 0.05082; raw +41.3% (95%: −219.2% to +301.7%), +0.0148 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BOTH.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 14.9 → 18.6; raw +24.8% (95%: +18.0% to +31.6%); LOSS · 12.4 → 15.7; raw +26.6% (95%: +20.5% to +32.7%); LOSS · 10.1 → 17.6; raw +74.3% (95%: +40.2% to +108.3%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BOTH.md`): pod start wait, total (s), A B C 0.8 → 2.9; raw +262.5% (95%: −113.2% to +638.2%), +2.1000 in the gauge's unit; LOSS · 0.2 → 2.9; raw +1350.0% (95%: +337.7% to +2362.3%), +2.7000 in the gauge's unit; LOSS · 0.6 → 5; raw +733.3% (95%: −39.7% to +1506.4%), +4.4000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BOTH.md`): pod start wait, mean (s), A B C 0.55 → 1.55; raw +181.8% (95%: −92.4% to +456.0%), +1.0000 in the gauge's unit; LOSS · 0.2 → 1.617; raw +708.3% (95%: +164.7% to +1251.9%), +1.4167 in the gauge's unit; LOSS · 0.4 → 2.467; raw +516.7% (95%: −69.1% to +1102.4%), +2.0667 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BOTH.md`): batch: queue finished

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(s), A B C 911.9 → 1107; raw +21.4% (95%: +16.0% to +26.8%); LOSS · 885.4 → 1058; raw +19.4% (95%: +11.5% to +27.4%); LOSS · 881.5 → 1050; raw +19.1% (95%: +13.1% to +25.0%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BOTH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.1542 → 22.47; raw +14475.7% (95%: +1985.4% to +26966.0%), +22.3167 in the gauge's unit; LOSS · 0.1167 → 18.62; raw +15860.7% (95%: +1447.0% to +30274.5%), +18.5042 in the gauge's unit; LOSS · 0.075 → 13.05; raw +17305.6% (95%: +349.2% to +34262.0%), +12.9792 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BOTH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.7 → 159.5; raw +22685.7% (95%: +4894.5% to +40476.9%), +158.8000 in the gauge's unit; LOSS · 0.8 → 105.6; raw +13100.0% (95%: +1240.5% to +24959.5%), +104.8000 in the gauge's unit; LOSS · 0.5 → 85.4; raw +16980.0% (95%: +170.1% to +33789.9%), +84.9000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_BOTH.md`): queue: job turnaround, submitted to done, mean (s), A B C 222.3 → 380; raw +70.9% (95%: +54.6% to +87.2%); LOSS · 194.8 → 326.8; raw +67.7% (95%: +38.0% to +97.5%); LOSS · 190.6 → 337.1; raw +76.8% (95%: +55.5% to +98.1%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_ORCH.md`): response time (ms), mean, A B C 752.2 → 1080; raw +43.6% (95%: +24.2% to +62.9%); LOSS · 665.7 → 1041; raw +56.4% (95%: +31.6% to +81.2%); LOSS · 630.1 → 729.1; raw +15.7% (95%: −32.1% to +63.5%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_ORCH.md`): response time (ms), 95th percentile, A B C 2969 → 4119; raw +38.7% (95%: +19.4% to +58.1%); LOSS · 2682 → 4095; raw +52.7% (95%: +28.2% to +77.2%); LOSS · 2493 → 2738; raw +9.8% (95%: −42.8% to +62.5%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_ORCH.md`): response time (ms), 99th percentile, A B C 4041 → 5429; raw +34.4% (95%: +13.7% to +55.0%); LOSS · 3704 → 5491; raw +48.2% (95%: +27.5% to +69.0%); LOSS · 3409 → 3776; raw +10.8% (95%: −40.6% to +62.1%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_ORCH.md`): time over the response line (% of samples), A B C 28.08 → 38.86; raw +38.4% (95%: +26.1% to +50.6%); LOSS · 26.74 → 33.85; raw +26.6% (95%: +12.2% to +41.0%); LOSS · 21.13 → 24.85; raw +17.6% (95%: −34.5% to +69.8%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_ORCH.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 14.9 → 17.6; raw +18.1% (95%: +12.6% to +23.7%); LOSS · 12.4 → 14.8; raw +19.4% (95%: +10.7% to +28.0%); LOSS · 10.1 → 12.1; raw +19.8% (95%: −34.2% to +73.8%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_ORCH.md`): batch: queue finished (s), A B C 911.9 → 1049; raw +15.0% (95%: +9.3% to +20.7%); LOSS · 885.4 → 1026; raw +15.9% (95%: +9.3% to +22.6%); LOSS · 881.5 → 1023; raw +16.1% (95%: +4.7% to +27.4%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_ORCH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.1542 → 7.229; raw +4589.2% (95%: −3377.6% to +12556.0%), +7.0750 in the gauge's unit; LOSS · 0.1167 → 6.929; raw +5839.3% (95%: +343.3% to +11335.2%), +6.8125 in the gauge's unit; LOSS · 0.075 → 3.135; raw +4080.2% (95%: −2342.8% to +10503.3%), +3.0602 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_ORCH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.7 → 39.4; raw +5528.6% (95%: −3642.5% to +14699.6%), +38.7000 in the gauge's unit; LOSS · 0.8 → 55.9; raw +6887.5% (95%: −217.2% to +13992.2%), +55.1000 in the gauge's unit; LOSS · 0.5 → 6.389; raw +1177.8% (95%: −860.9% to +3216.5%), +5.8889 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_ORCH.md`): queue: job turnaround, submitted to done, mean (s), A B C 222.3 → 328.8; raw +47.9% (95%: +27.4% to +68.3%); LOSS · 194.8 → 305.8; raw +57.0% (95%: +30.7% to +83.2%); LOSS · 190.6 → 278.3; raw +46.0% (95%: +18.7% to +73.3%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_VS.md`): pod start wait, total (s), A B C 0.3 → 1.7; raw +466.7% (95%: +186.8% to +746.5%), +1.4000 in the gauge's unit; LOSS · 0 → 2.4;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

raw +2.4000 in the gauge's unit (95%: +0.9225 to +3.8775); LOSS · 0 → 1.3; raw +1.3000 in the gauge's unit (95%: +0.4047 to +2.1953); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Open Cluster Management on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_OCM_4W_VS.md`): pod start wait, mean (s), A B C 0.3 → 1.1; raw +266.7% (95%: +26.9% to +506.4%), +0.8000 in the gauge's unit; LOSS · 0 → 1.25; raw +1.2500 in the gauge's unit (95%: +0.6136 to +1.8864); LOSS · 0 → 0.8; raw +0.8000 in the gauge's unit (95%: +0.2615 to +1.3385); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_BOTH.md`): pod start wait, total (s), A B C 0.4 → 0.4; raw ±0.0% (95%: −168.6% to +168.6%), ±0.0000 in the gauge's unit; STEADY · 0.3 → 1.4; raw +366.7% (95%: −477.8% to +1211.2%), +1.1000 in the gauge's unit; LOSS · 0.2 → 1.2; raw +500.0% (95%: −326.0% to +1326.0%), +1.0000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_BOTH.md`): pod start wait, mean (s), A B C 0.2 → 0.4; raw +100.0% (95%: −126.2% to +326.2%), +0.2000 in the gauge's unit; LOSS · 0.15 → 0.7; raw +366.7% (95%: −477.8% to +1211.2%), +0.5500 in the gauge's unit; LOSS · 0.2 → 1.2; raw +500.0% (95%: −326.0% to +1326.0%), +1.0000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_BOTH.md`): batch: queue finished (s), A B C 758.7 → 772.3; raw +1.8% (95%: +0.8% to +2.8%); LOSS · 780.2 → 793.3; raw +1.7% (95%: +0.4% to +2.9%); LOSS · 770.9 → 784.7; raw +1.8% (95%: +0.4% to +3.2%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_BOTH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.0625 → 0.7667; raw +1126.7% (95%: +1020.2% to +1233.2%), +0.7042 in the gauge's unit; LOSS · 0.075 → 0.8917; raw +1088.9% (95%: +690.1% to +1487.7%), +0.8167 in the gauge's unit; LOSS · 0.05 → 0.7583; raw +1416.7% (95%: +986.8% to +1846.5%), +0.7083 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_BOTH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.3 → 1.3; raw +333.3% (95%: +174.4% to +492.3%), +1.0000 in the gauge's unit; LOSS · 0.4 → 2.2; raw +450.0% (95%: +38.7% to +861.3%), +1.8000 in the gauge's unit; LOSS · 0.2 → 1.3; raw +550.0% (95%: +436.9% to +663.1%), +1.1000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_BOTH.md`): queue: job turnaround, submitted to done, mean (s), A B C 65.63 → 86.72; raw +32.1% (95%: +13.4% to +50.8%); LOSS · 85.9 → 102.8; raw +19.7% (95%: +5.5% to +33.9%); LOSS · 75.65 → 95.34; raw +26.0% (95%: +8.1% to +43.9%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_ORCH.md`): batch: queue finished (s), A B C 758.7 → 767.6; raw +1.2% (95%: +0.6% to +1.8%); LOSS · 780.2 → 791.2; raw +1.4% (95%: +0.4% to +2.5%); LOSS · 770.9 → 781.9; raw +1.4% (95%: +0.3% to +2.6%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_ORCH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C 0.0625 → 0.7208; raw +1053.3% (95%: +883.9% to +1222.8%), +0.6583 in the gauge's unit; LOSS · 0.075 → 0.725; raw +866.7% (95%: +664.7% to +1068.6%), +0.6500 in the gauge's unit; LOSS · 0.05 → 0.7958; raw +1491.7% (95%: +1085.0% to +1898.3%), +0.7458 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_ORCH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C 0.3 → 1.4; raw +366.7% (95%: +231.3% to +502.0%), +1.1000 in the gauge's unit; LOSS · 0.4 → 1.5; raw +275.0% (95%: +78.2% to +471.8%), +1.1000 in the gauge's unit; LOSS · 0.2 → 1.2; raw +500.0% (95%: +331.4% to +668.6%), +1.0000 in the gauge's unit; LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_ORCH.md`): queue: job turnaround, submitted to done, mean (s), A B C 65.63 → 81.51; raw +24.2% (95%: +10.9% to +37.5%); LOSS · 85.9 → 101; raw +17.6% (95%: +5.6% to +29.5%); LOSS · 75.65 → 93.17; raw +23.2% (95%: +6.9% to +39.4%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_VS.md`): response time (ms), mean, A B C 222.2 → 266.7; raw +20.0% (95%: +0.7% to +39.3%); LOSS · 260.4 → 290.2; raw +11.4% (95%: −6.3% to +29.2%); LOSS · 253.5 → 286.6; raw +13.1% (95%: −1.5% to +27.6%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_VS.md`): response time (ms), 95th percentile, A B C 517.4 → 996.5; raw +92.6% (95%: +46.7% to +138.5%); LOSS · 884.6 → 1167; raw +31.9% (95%: +10.8% to +53.0%); LOSS · 758.9 → 1145; raw +50.9% (95%: +23.2% to +78.6%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_VS.md`): response time (ms), 99th percentile, A B C 846.3 → 1467; raw +73.3% (95%: +31.5% to +115.1%); LOSS · 1478 → 1620; raw +9.6% (95%: −8.3% to +27.4%); LOSS · 1326 → 1638; raw +23.6% (95%: +1.1% to +46.0%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_VS.md`): time over the response line (% of samples), A B C 7.226 → 16.19; raw +124.1% (95%: +51.4% to +196.7%); LOSS · 8.862 → 13.31; raw +50.2% (95%: +5.7% to +94.8%); LOSS · 9.369 → 14.5; raw +54.8% (95%: +19.1% to +90.4%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_VS.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C 7.6 → 9.9; raw +30.3% (95%: +6.8% to +53.8%); LOSS · 7.1 → 7.9; raw +11.3% (95%: −3.6% to +26.1%); LOSS · 6.9 → 8.5; raw +23.2% (95%: −1.8% to +48.2%); LOSS. The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

BACK MATTER

# Index

Every term of the glossary where this book defines or discusses it, every muscle family where it is named, every file of the repository and every command-line setting this book names, with the pages they stand on. A plain mention of a common word is not an entry: the index points to where a thing is explained. A term whose pages would run past its line keeps the pages that fit, where it is defined and first used; after those it is throughout the book.

-

--check . . . . . x, 81
--give-back-only . . . . . 58
--interval . . . . . 6, 40, 52
--profile . . . . . 18
--steel . . . . . 84
--tuned . . . . . 84, 100
--verdict-samples . . . . . 46

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

--web . . . . . 17

.

.github/workflows/cloud-check.yml . . . . . 59
.github/workflows/engine-runs.yml . . . . . 180, 182
.github/workflows/live-engine.yml . . . . . 63, 109
.github/workflows/monte-carlo.yml . . . . . 65, 182
.github/workflows/peek.yml . . . . . 59

B

benchmarks/core_evidence.py . . . . . 64, 181
benchmarks/stack_benchmark.py . . . . . 181

C

cpp/src/compass.cpp . . . . . 165, 166, 167, 169, 170, 171
cpp/src/compass_law.cpp . . . . . 13
cpp/src/conveyance.cpp . . . . . 177
cpp/src/core.cpp . . . . . 11, 65, 139, 140, 141, 143, 144, 145, 146, 150, 151, 153
cpp/src/governor.cpp . . . . . 13, 139, 140, 141, 143, 144, 145, 146, 150, 151, 153, 154
cpp/src/gpu_rules.cpp . . . . . 176
cpp/src/hpa.cpp . . . . . 165, 166, 167, 169, 170, 171
cpp/src/immune.cpp . . . . . 15, 86
cpp/src/nervous_system.cpp . . . . . 139, 140, 141, 143, 144, 145, 146, 150, 151, 180, 186, 187
cpp/src/shield.cpp . . . . . 131, 165, 166, 167, 169, 170, 171, 181
cpp/src/verdict.cpp . . . . . 77
cpp/tools/core_runs.cpp . . . . . 65, 182
cpp/tools/engine_runs.cpp . . . . . 64, 66, 148, 156, 157, 177, 180, 195

D

deploy/grafana/omni-compass-wall.json . . . . . 2, 17, 22, 88
deploy/grafana/omni-compass.json . . . . . 40, 88

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

deploy/kind/rbac-omni.yaml . . . . . 131
deploy/prometheus/omni-compass-rules.yaml . . . . . 40
deploy/wall . . . . . ix, 17
docs/AMAZON_SETUP.md . . . . . 59, 169
docs/AZURE_SETUP.md . . . . . 59
docs/BATCH_QUEUE_PREREGISTRATION.md . . . . . 60, 93, 221, 222, 224, 226, 233, 234, 240
docs/BENEFIT_SHEET.md . . . . . 28
docs/book/build_live_manual.py . . . . . 132
docs/DOSSIER.md . . . . . xxii
docs/engine/APPRAISAL.md . . . . . 83, 97
docs/engine/build_live_engine.py . . . . . xv, 63, 82, 136, 137, 159, 160, 161, 162, 163, 164, 168, 172
docs/engine/check_live_engine.py . . . . . ix, xxiv, 3, 83, 139, 140, 141, 143, 144, 145, 146, 148, 150
docs/engine/check_records.py . . . . . 3, 83, 139, 140, 141, 143, 144, 145, 146, 150, 151, 155, 165
docs/engine/cpp_core_probe.cpp . . . . . 97
docs/engine/export_controller_trace.py . . . . . xi, 55, 82, 165, 166, 167, 169, 170, 171
docs/engine/export_cpu_model.py . . . . . 176
docs/engine/export_gpu_run.py . . . . . 61, 175
docs/engine/export_harness_trace.py . . . . . xi, 82, 139, 140, 141, 143, 144, 145, 146, 150, 151, 154
docs/engine/export_heldout.py . . . . . 181
docs/engine/export_runs.py . . . . . xi, 82, 160, 161, 162, 163, 164, 190, 191
docs/engine/export_scenarios.py . . . . . xi, 82, 177
docs/engine/export_whole.py . . . . . xxiii, 77, 185
docs/engine/github_ledger.json . . . . . xi, 82
docs/engine/glossary.json . . . . . 83, 89
docs/engine/monitors.json . . . . . xxii, 3, 17, 88
docs/engine/NATIVE_REGISTER.md . . . . . 210
docs/engine/OMNI_COMPASS_LIVE_ENGINE.html . . . . . xiv, xxii, xxiv, 132
docs/engine/PAGE_WORD_FOR_WORD.md . . . . . xxii
docs/EVIDENCE_LEDGER.md . . . . . xviii, xxii

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

docs/FACTORY_PROFILE.md . . . . . xiv, xvi, xx, 2, 5, 8, 9, 35, 44, 49, 81, 90, 95, 97, 212
docs/GOOGLE_SETUP.md . . . . . 59, 169
docs/GPU_PREREGISTRATION.md . . . . . xviii, xxi, xxii
docs/GPU_RUN_GUIDE.md . . . . . 62, 174, 176
docs/history/SCENARIOS_30_2026-10-24.md . . . . . 67, 196
docs/HOW_TO_READ_THE_RESULTS.md . . . . . 211
docs/K8S_COMPASS_PREREGISTRATION.md . . . . . xviii, xxii
docs/LOSS_CAUSES_2026-10-31.md . . . . . 220, 224, 225, 239
docs/LOSS_LEDGER.md . . . . . 27, 29, 137
docs/LOSS_ROWS_UNDER_INVESTIGATION.md . . . . . 220, 237, 238
docs/MUSCLE_CATALOG.md . . . . . xxii
docs/OMNI_COMPASS_LIVE_ENGINE_MANUAL.md . . . . . xxii
docs/OMNI_COMPASS_MANUAL.md . . . . . xxii
docs/OMNI_COMPASS_MANUAL.pdf . . . . . xxii
docs/OMNI_V16.md . . . . . 158
docs/OMNI_V23.md . . . . . 196
docs/PACE_PREREGISTRATION.md . . . . . 7, 40, 147
docs/POD_START_WAIT.md . . . . . x, 32, 93, 219, 220, 221, 222, 223, 224, 225, 227, 228, 229
docs/REALMS_PREREGISTRATION.md . . . . . xviii, xxii
docs/REGISTER.md . . . . . 73
docs/RUN_LEDGER.md . . . . . 175
docs/SAFETY_CASE.md . . . . . 78
docs/STATE_OF_PLAY.md . . . . . xii, xxii
docs/TAIL_LATENCY.md . . . . . 32, 93
docs/THE_FIELD.md . . . . . 49, 157
docs/THE_TUNE.md . . . . . xix, 27, 81, 90, 212
docs/THREAT_MODEL.md . . . . . 78, 132
docs/TRACES_PREREGISTRATION.md . . . . . 125
docs/WHOLE_SYSTEM_PREREGISTRATION.md . . . . . xxiii, 205

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

F

fleet/capture/kube_capture.sh . . . . . 132
fleet/harness.py . . . . . 165, 166, 167, 169, 170, 171

O

omnicompass/adapter.py . . . . . 10, 13, 14, 139, 140, 141, 143, 144, 145, 146, 150, 151, 153
omnicompass/compass_law.py . . . . . xi, 10, 13, 83, 139, 140, 141, 143, 144, 145, 146, 148, 150
omnicompass/conveyance.py . . . . . 177
omnicompass/core.py . . . . . xi, 10, 11, 12, 64, 83, 139, 140, 141, 143, 144, 145, 146
omnicompass/immune.py . . . . . 10, 15, 86
omnicompass/master.py . . . . . 20, 50, 75, 193
omnicompass/nervous_system.py . . . . . 10, 13, 15, 35, 139, 140, 141, 143, 144, 145, 146, 150, 151
omnicompass/shield.py . . . . . 131, 181
omnicompass/stack_sim.py . . . . . 181
omnicompass/student_t.py . . . . . 156
omnicompass/verdict.py . . . . . xi, 83, 156
omni_controller/controller.py . . . . . 50, 55, 82, 86, 160, 165, 166, 167, 169, 170, 171
omni_controller/gpu_governor.py . . . . . 176
omni_controller/muscles.py . . . . . 177

P

pilot/selfpilot.py . . . . . 55, 165, 166, 167, 169, 170, 171

R

realms/catalog_v5.csv . . . . . 73, 88
realms/compass_arm_v5.py . . . . . 82, 140, 188, 189, 190
realms/gpu_card.py . . . . . 82, 176, 177
realms/harness.py . . . . . 3, 74, 82, 99, 139, 140, 141, 143, 144, 145, 146, 150, 151
realms/muscle_cards_v5.csv . . . . . xix, 81
results/BENEFIT_SHEET.json . . . . . 28

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

results/core_evidence.json . . . 64
results/gpu/run-20261024T091235Z . . . 61
results/live/raw . . . 54
results/live/V12_ARMADA_4W_BARE.md . . . 219, 220
results/live/V12_ARMADA_4W_BOTH.md . . . 247, 248
results/live/V12_ARMADA_4W_ORCH.md . . . 248, 249
results/live/V12_ARMADA_4W_TOP.md . . . 220
results/live/V12_ARMADA_4W_VS.md . . . 249, 250
results/live/V12_BINPACK.md . . . 221, 222
results/live/V12_BINPACK_4W_BARE.md . . . 222, 223
results/live/V12_BINPACK_4W_BOTH.md . . . 250
results/live/V12_BINPACK_4W_ORCH.md . . . 250
results/live/V12_BINPACK_4W_TOP.md . . . 223
results/live/V12_BINPACK_4W_VS.md . . . 250, 251
results/live/V12_CRANE_4W_BARE.md . . . 224
results/live/V12_CRANE_4W_BOTH.md . . . 251
results/live/V12_CRANE_4W_ORCH.md . . . 251
results/live/V12_CRANE_4W_TOP.md . . . 224
results/live/V12_CRANE_4W_VS.md . . . 251
results/live/V12_KAI.md . . . 226
results/live/V12_KAI_4W_BARE.md . . . 226, 227
results/live/V12_KAI_4W_BOTH.md . . . 252, 253
results/live/V12_KAI_4W_ORCH.md . . . 253, 254
results/live/V12_KAI_4W_TOP.md . . . 227, 228
results/live/V12_KAI_4W_VS.md . . . 254
results/live/V12_KARMADA_4W_BARE.md . . . 228, 229
results/live/V12_KARMADA_4W_BOTH.md . . . 254, 255, 256
results/live/V12_KARMADA_4W_ORCH.md . . . 256, 257, 258
results/live/V12_KARMADA_4W_TOP.md . . . 229, 230, 231

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

results/live/V12_KARMADA_4W_VS.md . . . 258
results/live/V12_KOORDINATOR.md . . . 231
results/live/V12_KOORDINATOR_4W_BARE.md . . . 231, 232
results/live/V12_KOORDINATOR_4W_BOTH.md . . . 258, 259
results/live/V12_KOORDINATOR_4W_ORCH.md . . . 259, 260
results/live/V12_KOORDINATOR_4W_TOP.md . . . 232
results/live/V12_KOORDINATOR_4W_VS.md . . . 260
results/live/V12_KUBERAY_4W_BARE.md . . . 232, 233, 234
results/live/V12_KUBERAY_4W_BOTH.md . . . 260, 261, 262
results/live/V12_KUBERAY_4W_ORCH.md . . . 262, 263
results/live/V12_KUBERAY_4W_TOP.md . . . 234
results/live/V12_KUEUE_4W_BARE.md . . . 234, 235
results/live/V12_KUEUE_4W_BOTH.md . . . 263, 264, 265
results/live/V12_KUEUE_4W_ORCH.md . . . 265, 266
results/live/V12_KUEUE_4W_TOP.md . . . 235, 236
results/live/V12_OCM_4W_BARE.md . . . 236, 237
results/live/V12_OCM_4W_BOTH.md . . . 266, 267, 268
results/live/V12_OCM_4W_ORCH.md . . . 268, 269
results/live/V12_OCM_4W_TOP.md . . . 237, 238, 239
results/live/V12_OCM_4W_VS.md . . . 269, 270
results/live/V12_VANILLA.md . . . 239, 240
results/live/V12_VOLCANO_4W_BARE.md . . . 240, 241
results/live/V12_VOLCANO_4W_BOTH.md . . . 270, 271
results/live/V12_VOLCANO_4W_ORCH.md . . . 271
results/live/V12_VOLCANO_4W_TOP.md . . . 241
results/live/V12_VOLCANO_4W_VS.md . . . 271, 272
results/live/V21_ALL_FOUR.md . . . 219
results/live/V21_BATCH.md . . . 220, 221
results/live/V21_CAPACITY.md . . . 224

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

results/live/V21_CRUISEKUBE.md . . . 224, 225
results/live/V21_FAIRNESS.md . . . 225
results/live/V21_GOLDILOCKS.md . . . 225
results/live/V21_KAFKA.md . . . 242
results/live/V21_KAFKA_SERVICE.md . . . 242, 243, 244
results/live/V21_KEDA_CPU_REQUESTS.md . . . 231
results/live/V21_REDIS.md . . . 244
results/live/V21_REDIS_SERVICE.md . . . 244
results/live/V21_ROBUST_KILL.md . . . 239
results/live/V21_ROBUST_LONG.md . . . 239
results/live/V21_SYSBENCH_SHIPPED.md . . . 244, 245
results/live/V21_TRACE_GOOGLE2011.md . . . 239
results/live/V21_YCSB.md . . . 245
results/live/V21_YCSB_SERVICE.md . . . 245
results/live/V21_YCSB_SHIPPED.md . . . 245, 246, 247
results/model_gate/HEARTBEAT_SWEEP.md . . . 52, 158, 161
results/model_gate/V17_LADDER.json . . . 40
results/OMNI_INDEX.md . . . 27, 29, 205
results/orchestra/ORCHESTRA.md . . . 61
results/orchestrator/V21_THREE_ARMS_ABC.json . . . 26, 27
results/realms_v5 . . . 82
results/realms_v5/REALMS_V5.json . . . 73
results/scenarios/SCENARIOS_20.json . . . xxiv, 67, 83
results/scenarios/SCENARIOS_20.md . . . 67
results/SEAL.json . . . xxii, 132
results/STEERING.json . . . 57
results/WHOLE_SYSTEM.md . . . xxiii, 28, 205

S

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

scripts/dispatch_grid.py . . . . . 63, 187
scripts/gpu_8card.sh . . . . . 62, 176
scripts/raw_archive.sh . . . . . 159, 161, 162, 163, 164, 190, 191

T

tests/test_benefit_sheet.py . . . . . 135, 136
tests/test_cloud_check.py . . . . . 169, 170
tests/test_confirm_abc.py . . . . . 136, 137
tests/test_core_runs.py . . . . . 182
tests/test_cpp_conveyance_parity.py . . . . . 177
tests/test_cpp_core_runs_parity.py . . . . . 182
tests/test_cpp_governor_parity.py . . . . . 139, 140, 141, 143, 144, 145, 146, 150, 151, 153, 154, 165
tests/test_cpp_hpa_parity.py . . . . . 165, 166, 167, 169, 170, 171
tests/test_cpp_shield_parity.py . . . . . 165, 166, 167, 169, 170, 171, 181
tests/test_cpp_twins_parity.py . . . . . 139, 140, 141, 143, 144, 145, 146, 150, 151, 176, 193
tests/test_engine_runs.py . . . . . 180
tests/test_governors_page.py . . . . . 172, 173
tests/test_grid_v5.py . . . . . 187
tests/test_master_switch.py . . . . . 193
tests/test_orchestra.py . . . . . 173
tests/test_readings.py . . . . . 135, 136, 137
tests/test_realms.py . . . . . 186, 187
tests/test_run_cpu_power.py . . . . . 176
tests/test_workflow_triggers.py . . . . . 63
tests/third_party/hpa_independent.py . . . . . 124
tools/beat_ladder.py . . . . . 40
tools/benefit_sheet.py . . . . . 135, 136
tools/cloud_key_check.py . . . . . 59, 169
tools/confirm_abc.py . . . . . 136, 137, 159, 172

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

tools/core_runs.py . . . . . 21, 65, 182
tools/engine_runs.py . . . . . xxiii, 21, 64, 180
tools/governors_page.py . . . . . 172, 173
tools/grid_v5.py . . . . . 187
tools/heartbeat_sweep.py . . . . . 52, 160
tools/injection_grid.py . . . . . 68, 70
tools/live_glossary.py . . . . . 89
tools/live_reps.py . . . . . xi, 54, 82, 92, 159, 161, 162, 163, 164, 168, 190, 191
tools/loss_ledger.py . . . . . 30
tools/model_gate.py . . . . . 82, 160, 165, 166, 167, 169, 170, 171, 177
tools/omni_control.py . . . . . 5, 7, 40
tools/omni_exporter.py . . . . . 17, 52, 88
tools/omni_index.py . . . . . 135
tools/omni_switch.py . . . . . 20, 75, 131, 193
tools/omni_version.py . . . . . 126
tools/readings.py . . . . . xv, 83, 84, 87, 135, 136, 137
tools/run_cpu_power.py . . . . . 61, 62, 176
tools/run_orchestra.py . . . . . 61, 173
tools/run_realms_v5.py . . . . . 73, 186, 187
tools/scenarios_20.py . . . . . 67
tools/seal.py . . . . . 65
tools/stop_rule.py . . . . . 59, 100, 168
tools/tune_study.py . . . . . xix
tools/verdict_audit.py . . . . . 29
tools/whole_system.py . . . . . xxiii, 26, 28, 77, 83, 185

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

BACK MATTER

# Contact

Licensing, pilots, evaluations and the Omni-Compass Enterprise License:

**The Omni-Compass LLC**

Owner and developer: AJ Dubra

Main office infrastructure: toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1

Landline, registered in Florida: 321-746-4411

www.omni-compass.com

Repository: github.com/The-Omni-Compass-LLC/The-Omni-Compass-LLC

Any use beyond evaluation and simulation requires a signed, paid Omni-Compass Enterprise License. Write to The Omni-Compass LLC before any production use.

© 2026 The Omni-Compass LLC. All rights reserved. **Evaluation and Simulation use only.** Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with `LICENSE`, `NOTICE` and `DISCLOSURES.md`. `SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0` All patents, copyrights and trademarks filed in the USA. www.omni-compass.com